Thus the nonzero carrier exists on the strict partial-controller branch; the limit is a one-sided selected branch-matrix limit, not a nonzero two-sided tied fiber or unique classical Jacobian.
Exact controller-fiber theorem package

Created:

text

AIEF_1_4_CONTROLLER_FIBER_THEOREM_PACKAGE.md
AIEF_1_4_CONTROLLER_FIBER_CLAIM_REGISTRY.csv
AIEF_1_4_CONTROLLER_FIBER_FREEZE_MANIFEST.json
AIEF_1_4_freeze_controller_fiber_theorem.py

Integrated into:

text

AIEF_1_4_OPERATOR_THEORY_MANUSCRIPT.md
AIEF_1_4_OPERATOR_COCYCLE_CLOSURE_THEOREM_NOTE.md

Exact results:

polyhedral lineality;
safeguarded output interiority;
tie-safe multiplier-controller fiber affinity;
invariance/intertwining factorization;
fixed/moving dual closure;
exceptional K3 controller kernel;
pointwise/fixed left covectors;
one-sided tie-boundary interpretation.

Freeze generator:

Bash

python3 AIEF_1_4_freeze_controller_fiber_theorem.py

produces 12 claims and additive manifest. Earlier lineage freeze is preserved as parent.
Proposal-equalization bridge

Created:

text

AIEF_1_4_CONDITIONAL_PROPOSAL_EQUALIZATION_THEOREM.md
AIEF_1_4_proposal_equalization_bridge_analysis.py
AIEF_1_4_PROPOSAL_EQUALIZATION_BRIDGE_INTERPRETATION.md

Exact recurrence for channels j∈{L,R}j∈{L,R}:
dn+1j=
dn+1j=

\lambda_n^jd_n^j
+
dt(\eta+\mu)r_{n+1}^j,
\qquad
0<\lambda_n^j<1-\gamma dt<1.
]

V4 bridge audit:

text

final replay exact: True
trajectory rows: 64000
all exact identity errors ~1e-16 to 1e-15
gate_all=True
positive_all=True
controllers_clipped_all=True
winner_map_all=True
isolation_max=0
minimum winner margin=4.44e-14

Late fixed-left:

text

lambda_L ~0.998738246680
intrinsic rate ~0.1262550001
r_EMA ~0.0571333334
gap ~0.0691216667 = prior kappa~0.06913

Late right mode:

text

lambda_R ~0.499369154879
intrinsic rate ~69.44097

Both proposal/resonance channels acquire the EMA rate late.

Sign audit:

text

fixed_left:
delta_x min=5.4226660384237846e-09
delta_R min=5.352774001821103e-09
sign_changes=0
all_positive=True

right_mode:
delta_x min=2.8199664825478976e-14
delta_R min=2.015421163292785e-11
sign_changes=0
all_positive=True

Resonance ordering:
R25>R59>R29
R25>R59>R29

at every saved step.

Asymptotic transfer gain:
$dnrn \to G*=dt(\eta+\mu)\alpha\alpha-\lambda*.$
$rndn \to G*=\alpha-\lambda*dt(\eta+\mu)\alpha.$

Predicted/final:

text

fixed-left predicted 1.0130571021
final observed 1.0130571619

right-mode predicted 0.0013990334
final observed 0.0014019490

V4 two-channel alignment closure

Successful V4 report:

text

Final replay exact: True
All-step R25 > R59 > R29: PASS
minimum left gap 5.35277400182e-09
minimum right gap 2.01542116329e-11
winner map all steps true
gate all steps true
controllers clipped all steps true
isolation error 0
minimum winner margin 4.440892e-14

Exact identity max errors:

text

e=T1+T2 6.95e-16
radial recurrence 7.29e-16
symmetrized R 3.89e-16
phase separation balance 2.34e-15

phase mean balance 1.25e-15

For all four directed orientations:
$T1/T2 \to -1$
$T1/T2 \to -1$

over resolved late windows. Cancellation indices decrease to approximately $10^{-8}10^{-8}$, and alignment residual rates are approximately $0.1050.105–0.1080.108$, faster than $rEMA=0.057133rEMA=0.057133$.

ReinfFrac=1 while resolved: the small alignment forcing has the same sign as $\Delta M\Delta M$, reinforces it, and decays faster, leaving the homogeneous $\alpha\alpha$ envelope.

Finite-horizon chain:
rotating lock/history equalization $\to$ source increment envelope $\to$ $T1+T2=e$, $T1/T2 \approx -1$ $\to$ ΔM inherits $\alpha \to \Delta R \to \Delta x \to$ one-sided tie approach.
rotating lock/history equalization $\to$ source increment envelope $\to$ $T1+T2=e$, $T1/T2 \approx -1$ $\to$ ΔM inherits $\alpha \to \Delta R \to \Delta x \to$ one-sided tie approach.
Radial-filter lemma

Sharpened conditional lemma:

If:
$\Delta Mn+1=$
$\Delta Mn+1=$

\alpha\Delta M_n+
\alpha(1-\alpha)e_n,
\qquad0<\alpha<1,
]

and there is fixed $\sigma \in \{-1,+1\}\sigma \in \{-1,+1\}$ such that:
$\sigma \Delta Mn0>0, \sigma en \geq 0,$
$\sigma \Delta Mn0>0, \sigma en \geq 0,$

with weighted summability:
$\sum n=n0 \infty |en| \alpha n < \infty,$
$n=n0 \sum \infty \alpha n |en| < \infty,$

then:
$\sigma\Delta Mn>0, \Delta Mn\alpha n \to A, \sigma A>0,$
$\sigma\Delta Mn>0, \alpha n\Delta Mn \to A, \sigma A>0,$

and:
$\Delta Mn=A\alpha n+o(\alpha n).$
$\Delta Mn=A\alpha n+o(\alpha n).$

Geometric decay $en=O(\beta n)en=O(\beta n)$, $\beta<\alpha\beta<\alpha$, is a convenient sufficient condition. At $\beta=\alpha\beta=\alpha$, additional summability is required; a nonzero normalized forcing limit generically gives $n\alpha nn\alpha n$.

V4 supplies finite-horizon signatures only; infinite-time continuation remains open.
Rotating frequency lock and phase balance

Corrected locking result:
$\omega i+Gi\approx 1.062609$
$\omega i+Gi\approx 1.062609$

for nodes $2,7,112,7,11$, so the K3 is frequency locked in a rotating frame. The residual is:
$\epsilon i=(\omega i+Gi)-\Omega,$
$\epsilon i=(\omega i+Gi)-\Omega,$

not $\omega i+Gi \to 0\omega i+Gi \to 0$.

Two phase-increment mechanisms:

Three directed comparisons:
$dt\ \Delta\omega+dt\ \Delta G \to 0$
$dt\Delta\omega+dt\Delta G \to 0$ by cancellation.
Right-reverse:
$\Delta\omega=\Delta G=0$
$\Delta\omega=\Delta G=0$ identically; delayed-history separation alone decays.

Frozen weighted-Laplacian rate $\mu phase\approx 0.69\mu phase\approx 0.69$ is only a quasi-static/frozen-history surrogate, not the full delayed tangent.
Controller margins

Final:

text

m25=s2-s7 ~4.440892e-14
m29=s11-s2 ~8.543074e-09
m59=s11-s7 ~8.543119e-09

Exact:
m59=m25+m29.
m59=m25+m29.

The strict branch approaches a triple tie one-sidedly, hierarchically. Time-to-tie extrapolations near numerical floor should be retired.
Spectral material quarantine

The following claims should not be used without explicit exploratory qualification:

text

18 variables parameterize the true augmented K3 manifold
alpha is the unique slow pole of the full dynamics
seven independent slow mechanisms
effective dimension four from eigenvector-stack SVD
individual near-degenerate eigenvector participation factors
current operator leakage percentages
expanding compressed multipliers as physical modes

Reason:

the 18D state omits theta/full delay/external coordinates;
it is a selected-coordinate compression PDΦEPDΦE, not validated full spectrum;
reported J_EMA compressed norm ~44.9 violates the orthonormal bound α7≈2.64α7
≈2.64;
rotational symmetry supplies a neutral common-phase direction;
T≈αIT≈αI implies individual eigenvectors are nonidentifiable.

Allowed wording:

A selected-coordinate response map exhibits a provisional CC-dominated near-αα cluster. Its relationship to a full invariant slow bundle is unvalidated.

The V4 agent wrote the successful V4 two-channel script. A different post-V4 coding agent wrote many flawed exploratory spectral/prevalence scripts. That agent was assessed approximately:

text

3.5/10 overall
strong rapid prototyper
unreliable autonomous computational scientist

The V4 coding agent is substantially more reliable and wrote the matched-phase near-final design.
K3 prevalence scan correction

A flawed report claimed:

text

40/40 seeds with K3 components
856 K3 components
755 partial-controller K3s
0 full-controller K3s

This counted overlapping triangle subgraphs inside K4,K5,K6K4,K5,K6, not actual connected K3 components, and ignored active ties.

The existing frozen component audit implies approximately:

text

26 actual isolated K3 connected components:
25 ordinary tied/full-controller K3s
1 exceptional strict/partial-controller K3

A correct component scan must:

restrict to connected support components with exactly three nodes/three edges;
use positive proposal support/isolation;
count active ties as both controller endpoints;
canonicalize local winner maps under node relabeling;
not call exact global labels a geometric class.

The valid conclusion, if confirmed:

The frozen 40-seed cohort contains one observed isolated partial-controller K3 component; cross-seed variation for that class cannot be estimated.

T-window sensitivity scientific estimands

Baseline:
$T*=25\omega^{-}=17.502917152858814\ldots$
$T*=\omega^{-}25=17.502917152858814\ldots$

Grid:
$\rho\in\{0.5,0.75,1,1.5,2\}, Twindow=\rho T*.$
$\rho\in\{0.5,0.75,1,1.5,2\}, Twindow=\rho T*.$

All (seed,ρ)(seed,ρ) trajectories must regenerate from initialization; no split reuse.

Study branches:

text

continue
reset_ema
reset_B
future_shuffle

Primary regime contrast:
$\Delta s,\rho regime(Y)=$
$\Delta s,\rho regime(Y)=$
Y_{s,\rho,\rm continue}

Y_{s,1,\rm continue}.

]

Branch DID:
Is,ρ,b(Y)=
Is,ρ,b(Y)=
[
Y_{s,\rho,b}-Y_{s,\rho,\rm continue}
]

[
Y_{s,1,b}-Y_{s,1,\rm continue}
].
]

Primary fixed-physical horizons:

text

immediate 0–17.5
recovery 17.5–87.5
late 87.5–320

Secondary normalized horizons:

text

immediate 0–1*T_window
recovery 1–5*T_window
late after 5*T_window

Primary fields:

text

Weighted_similarity_late_fixed_physical
Retention_contrast_late_fixed_physical
P_mean_late_fixed_physical

Secondary fields:

text

Weighted_similarity_late_normalized_EMA
Retention_contrast_late_normalized_EMA
P_mean_late_normalized_EMA

Cross-TT topology for same seed:
cos(Bs,ρ,cfinal,Bs,1,cfinal),
cos(Bs,ρ,cfinal,Bs,1,cfinal),

threshold Jaccard, normalized edge-space distance.

Recovery:

condition-specific continue reference;
first crossing;
sustained rest-of-run recovery;
held for one current TwindowTwindow;
recovery probability;
restricted physical time;
restricted time divided by TwindowTwindow;
AUC deficit;
both horizon systems.

User-Pasted Content
Latest three-file stack state
1. aief_experiment_engine_v1.py

Latest reviewed engine includes:

exact frozen constants;
corrected prefix RNG:

Python

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)
rng = np.random.default_rng(init_sequence)

full branch RNG namespace;
exact step map;
projection diagnostics;
dual horizon accumulators;
fixed and normalized scalar/matrix outputs;
NaN-aware exact baseline equality;
separate frozen and extended schemas;
frozen scalar CSV comparisons.

Key latest correction:

historical keys compare to frozen artifacts;
new fixed/normalized keys compare internally to legacy fields at ρ=1ρ=1.

Remaining small engine concerns before tests:

run_seed still loops over branch_order rather than protocol.branches; the study driver bypasses it correctly.
actual frozen manifest file hash should be distinguished from canonical JSON hash.
baseline certificate must be saved by driver.
state validation should check config.N.
Jaccard threshold should be explicit.
no test output yet.

Latest engine review rating:

text

Core model fidelity ~9/10
RNG fidelity ~9.5/10
Dual-horizon implementation~9.5/10
Baseline test readiness ~9/10
Overall candidate engine ~9/10

2. AIEF_1_4_Twindow_sensitivity.py

Latest reviewed driver includes:

engine/driver hashes in config hash;
mandatory frozen manifest/NPZ/scalar;
all-40/all-seven-branch baseline equivalence;
saved detailed baseline certificate;
fixed-primary and normalized-secondary fields;
full frozen branch RNG namespace with four study branches;
200 condition status rows;
per-condition diagnostic snapshots;
exact output schema checks;
atomic CSV/NPZ;
final output hashes;
explicit completion/freezeability.

Latest review said:

ready for a fresh baseline test;
add/retain --baseline-only before first run so a passing baseline does not immediately launch the full grid;
do not use the fake/removed resume path;
dry-run must not contaminate production directory;
first run should be baseline-only.

3. AIEF_1_4_Twindow_sensitivity_analysis.py

Latest pasted version includes:

fixed-primary and normalized-secondary regime contrasts/DIDs;
exact cross-TT keys;
condition-status/baseline/freezeability validation;
recovery rows for continue and all branches;
3200 recovery rows expected;
restricted physical and restricted normalized times;
recovery probabilities and CIs;
recovery DID;
raw/summary outputs separated;
stale-output --overwrite.

Final static review identified only small remaining corrections:

Similarity rows incorrectly set:

Python

"target_horizon_dependence": "horizon_independent"

Change to:

Python

"horizon_dependent"

Validate all SECONDARY_NORMALIZED_OUTCOMES as finite in Stage 1.
Add to stale-output list:

text

twindow_recovery_interactions.csv
twindow_recovery_interactions_summary.csv

Validate expected recovery DID count:
40×4×3×2×2×13=24,960.
40×4×3×2×2×13=24,960.
Require every restricted-time recovery summary group to have 40 seeds.
Density rows are duplicated across horizon systems by design; keep
target_horizon_dependence=horizon_independent and never pool them.
Optional: add CIs for raw sustained/held conditional times.
Remove unused bootstrap arguments from compute_recovery_did.
Prefer np.trapezoid.
CSV writer fsync detail is nonblocking.

Latest readiness rating:

text

Overall analysis ~9/10
GO for compile/staged testing after small fixes

Accomplished
Completed

Corrected and productionized AIEF-1.3.
Two independent 50-seed cohorts and closure.
Frozen AIEF-1.4 memory ablation.
Recovery/survival/equivalence analyses.
Entropy geometry and active-boundary interpretation.
Fixed-face tangent spectra and component reconstruction.
Exceptional K3 branch audit.
Exact finite-step factorization.
Frozen and augmented cocycles.
Duhamel feedback decomposition.
Rank-one, geometric, triangular, and biorthogonal reductions.
Nested operator families and analytic defect.
Radial observable, source locking, resonant tracking, phase balance.
Existing K4/K5/K6 invariant subspaces.
Controller-fiber theorem package and additive freeze.
Conditional proposal-equalization theorem.
V4 two-channel bridge and alignment closure.
Matched-phase memory-ablation correction, 40/40 exact and freezeable.
Corrected rotating-frequency-lock interpretation.
Spectral claims quarantined.
K3 prevalence scan conceptual error diagnosed.
Scientific estimand and architecture for regenerated TwindowTwindow study defined.
Reusable-engine architecture selected:

text

validated shared kernel
+
separately versioned study drivers

Files created in workspace during this continuation:

text

AIEF_1_4_CONTROLLER_FIBER_THEOREM_PACKAGE.md
AIEF_1_4_CONTROLLER_FIBER_CLAIM_REGISTRY.csv
AIEF_1_4_CONTROLLER_FIBER_FREEZE_MANIFEST.json
AIEF_1_4_freeze_controller_fiber_theorem.py
AIEF_1_4_CONDITIONAL_PROPOSAL_EQUALIZATION_THEOREM.md
AIEF_1_4_PROPOSAL_EQUALIZATION_BRIDGE_INTERPRETATION.md
AIEF_1_4_proposal_equalization_bridge_analysis.py
AIEF_MATCHED_PHASE_CODING_AGENT_PROMPT.md

Updated:

text

AIEF_1_4_OPERATOR_THEORY_MANUSCRIPT.md
AIEF_1_4_OPERATOR_COCYCLE_CLOSURE_THEOREM_NOTE.md

Not Solved

No actual baseline-equivalence output has yet been shown for:

text

aief_experiment_engine_v1.py
AIEF_1_4_Twindow_sensitivity.py

No actual TwindowTwindow grid has yet been run.
No actual sensitivity report exists.
Infinite-time continuation of the exceptional alignment/proposal/tie mechanism remains open.
Formal Hadamard directional differentiability of the ideal/global safeguarded map remains open.
Strong trajectory proof for branch-cone invariance remains open.
Full delayed-state spectrum/invariant manifold remains unvalidated.
Timestep/safeguard regenerated study remains downstream.
Literature references still require direct verification before publication.

Next Steps

Apply the few remaining analysis-script fixes:

text

similarity target_horizon_dependence
secondary finite validation
stale recovery interaction filenames
recovery DID expected count 24,960
40-seed recovery-group validation

Ensure driver has --baseline-only.
Compile:

Bash

python -m py_compile \
aief_experiment_engine_v1.py \
AIEF_1_4_Twindow_sensitivity.py \
AIEF_1_4_Twindow_sensitivity_analysis.py

Run engine import:

Bash

python aief_experiment_engine_v1.py

Run baseline-only in a fresh run root:

Bash

python AIEF_1_4_Twindow_sensitivity.py \
--frozen-manifest runs/5a9f093a2fe0/manifest.json \
--frozen-npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--frozen-scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--baseline-only \
--run-root twindow_baseline_tests

Required baseline result:

text

40/40 seeds passed
7/7 branches passed
all frozen prefix arrays exact
all historical matrix/trajectory arrays exact
all historical scalar fields exact
all fixed/normalized baseline extensions internally exact

Only then run the full 200-condition grid in a fresh production root.
Analyze with:

Bash

python AIEF_1_4_Twindow_sensitivity_analysis.py \
<run-directory> \
--bootstrap-seed 20260808 \
--n-bootstrap 5000

Freeze successful outputs and update lineage.
Then design regenerated timestep/safeguard study.

Relevant files / directories
Frozen production/results

text

runs/c3da1ac707bf/
runs/73dca37cf93f/
runs/5a9f093a2fe0/

Frozen AIEF-1.4 authorities:

text

runs/5a9f093a2fe0/manifest.json
runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz
runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv
AIEF_1_4_memory_ablation_updated.py

Main production and analyses

text

AIEF_1_3_falsification_updated.py
AIEF_1_3_paired_statistics_updated.py
AIEF_1_3_closure_diagnostics_updated.py

AIEF_1_4_memory_ablation_updated.py
AIEF_1_4_results_analysis.py
AIEF_1_4_trajectory_recovery_equivalence.py
AIEF_1_4_survival_outlier_spectral.py
AIEF_1_4_entropy_geometry_diagnostics.py
AIEF_1_4_local_update_comparison.py
AIEF_1_4_active_boundary_analysis.py
AIEF_1_4_stationarity_perturbation_analysis_updated.py

Operator/cocycle/reduction

text

AIEF_1_4_actual_admissibility_branch_audit.py
AIEF_1_4_existing_clique_invariant_subspaces.py
AIEF_1_4_frozen_driver_cocycle_analysis.py
AIEF_1_4_augmented_tangent_cocycle_analysis.py
AIEF_1_4_augmented_duhamel_feedback_analysis.py
AIEF_1_4_biorthogonal_triangular_closure.py
AIEF_1_4_nested_operator_family_audit.py
AIEF_1_4_defect_recurrence_analytic_mode.py
AIEF_1_4_defect_causal_ema_trace.py
AIEF_1_4_radial_observable_subsystem_analysis.py
AIEF_1_4_phase_source_locking_analysis.py
AIEF_1_4_resonant_tracking_angular_analysis.py
AIEF_1_4_phase_balance_source_dynamics.py

Formalization/administration

text

AIEF_1_4_OPERATOR_THEORY_MANUSCRIPT.md
AIEF_1_4_OPERATOR_COCYCLE_CLOSURE_THEOREM_NOTE.md
AIEF_1_4_OPERATOR_COCYCLE_THOUGHT_TREE.md
AIEF_1_4_FORWARD_PLAN.md
AIEF_1_4_analysis_lineage.csv
AIEF_1_4_superseded_claims.csv
AIEF_1_4_operator_cocycle_closure_note.md
AIEF_1_4_lineage_freeze_manifest.json
AIEF_1_4_freeze_analysis_lineage.py
AIEF_CODING_AGENT_PROMPT.md
AIEF_MATCHED_PHASE_CODING_AGENT_PROMPT.md
AIEF_HISTORIC_UNFOLDING.md
AI_AGENT_HANDOFF_PROMPT.md

New TwindowTwindow stack, currently user-pasted/not yet test-validated

text

aief_experiment_engine_v1.py
AIEF_1_4_Twindow_sensitivity.py
AIEF_1_4_Twindow_sensitivity_analysis.py

The exact current source files may exist in the user's environment but were pasted in chat rather than created by this assistant. Actual compile/baseline outputs have not yet been shown.

And finally the file we have not touched in a while the engine:

#!/usr/bin/env python3

"""AIEF Experiment Engine v1 - Parameterized simulation kernel.

This is a reusable, validated simulation engine that reproduces the frozen

AIEF-1.4 production run exactly and supports parameterized studies.

FROZEN PRODUCTION VALUES (from AIEF_1_4_memory_ablation_updated_REAL.py):

N = 12

DT = 0.01

TAU = 0.5

OMEGA = [1.00, 1.02, 1.50, 2.00, 2.01, 1.01, 1.03, 1.51, 2.02, 2.00, 1.00, 1.04]

A = 1.0

Q = 1.5

DELTA = 0.05

ETA = 0.04

GAMMA = 0.015

MU = 0.03

T_WINDOW = 25.0 / mean(OMEGA) = 17.5

B_C = 0.12

B_INIT_SCALE = 0.0005

PROTOCOL (from frozen run):

T_SIM = 800.0

N_STEPS = int(round(T_SIM / DT)) = 80000

BURN_END = int(0.20 * N_STEPS) = 16000 steps = 160.0 time units

HIST_END = int(0.60 * N_STEPS) = 48000 steps = 480.0 time units

FUTURE_STEPS = N_STEPS - HIST_END = 32000 steps = 320.0 time units

DELAY_STEPS = int(round(TAU / DT)) = 50

WINDOW_STEPS = int(round(T_WINDOW / DT)) = 1750

RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS) = 8750

Architecture:

ModelConfig: frozen dataclass for all model parameters

ProtocolConfig: frozen dataclass for protocol parameters

DynamicState: state container (immutable where possible)

Core functions: initialize, advance, simulate, interventions

Validation: baseline equivalence test

NOTE: This is a candidate reference implementation. After passing 40/40 baseline

text

equivalence tests, it will be frozen as the reference implementation.

"""

from future import annotations

```
import copy

import hashlib

import json

import math

import os

import tempfile

import time

from dataclasses import dataclass, field

from pathlib import Path

from typing import Any

import numpy as np
================================================================================
Frozen Production Constants (DO NOT CHANGE)
================================================================================

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text
```

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005

FROZEN_ZERO_TOL = 1e-12

FROZEN_PROJECTION_NORM_TOL = 1e-14

FROZEN_GAP_TOL = 1e-9
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA)) # 1.428333...

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.502917152858814

FROZEN_T_WINDOW = FROZEN_T_STAR # 17.502917152858814 (exact, NOT rounded)

FROZEN_ALPHA = math.exp(-FROZEN_DT / FROZEN_T_WINDOW)

FROZEN_CAPACITY = (FROZEN_A - FROZEN_DELTA) / FROZEN_Q
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_WINDOW / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
Fixed physical horizon boundaries (in time units, NOT T_window dependent)
These are the PRIMARY estimand horizons

FIXED_PHYSICAL_IMMEDIATE_END = 17.5

FIXED_PHYSICAL_RECOVERY_END = 87.5

FIXED_PHYSICAL_FUTURE_END = 320.0
Derived fixed physical horizon steps

FIXED_PHYSICAL_IMMEDIATE_STEPS = int(round(FIXED_PHYSICAL_IMMEDIATE_END /
FROZEN_DT)) # 1750

FIXED_PHYSICAL_RECOVERY_STEPS = int(round(FIXED_PHYSICAL_RECOVERY_END / FROZEN_DT)) # 8750

==========

Model Configuration

==========

@dataclass(frozen=True)

class ModelConfig:

text

"""Immutable model configuration.

All model-changing quantities must come from here. No hidden module globals.

"""

Core parameters

N: int

dt: float

tau: float

omega: tuple[float, ...] # Immutable tuple

Coupling parameters

a: float

q: float

delta: float

Dynamics parameters

eta: float

mu: float

gamma: float

EMA/threshold parameters

T_window: float

B_c: float

B_init_scale: float

scale_epsilon: float = 1e-12

Derived properties

@property

```
def mean_omega(self) -&gt; float:

"""Mean of omega frequencies."""

return float(np.mean([self.omega](http://self.omega)))



@property

def T_star(self) -&gt; float:

"""T_* = 25 / mean(omega)"""

return 25.0 / self.mean_omega



@property

def alpha(self) -&gt; float:

"""EMA smoothing factor: exp(-dt / T_window)"""

return math.exp(-self.dt / self.T_window)



@property

def capacity(self) -&gt; float:

"""Projection capacity: (a - delta) / q"""
```

```
return (self.a - [self.delta](http://self.delta)) / self.q



@property

def delay_steps(self) -&gt; int:

"""Delay steps: round(tau / dt)"""

return int(round(self.tau / self.dt))



def validate(self) -&gt; None:

"""Validate model configuration."""

if self.N &lt;= 0:

raise ValueError("N must be positive")

if self.dt &lt;= 0:

raise ValueError("dt must be positive")

if self.tau &lt;= 0:

raise ValueError("tau must be positive")

if len([self.omega](http://self.omega)) != self.N:

raise ValueError(f"omega must have {self.N} elements")
```

if self.a <= 0 or self.q <= 0 or self.delta < 0:

raise ValueError("a, q must be positive; delta must be nonnegative")

if self.eta < 0 or self.mu < 0 or self.gamma < 0:

raise ValueError("eta, mu, gamma must be nonnegative")

if self.T_window <= 0:

raise ValueError("T_window must be positive")

if self.B_c < 0:

raise ValueError("B_c must be nonnegative")

if self.B_init_scale < 0:

raise ValueError("B_init_scale must be nonnegative")

if self.scale_epsilon <= 0:

raise ValueError("scale_epsilon must be positive")

Validate derived values

if not (0 < self.alpha <= 1):

raise ValueError(f"alpha={self.alpha} must be in (0, 1]")

if self.capacity < 0:

raise ValueError(f"capacity={self.capacity} must be nonnegative")

@dataclass(frozen=True)

class ProtocolConfig:

text

"""Protocol configuration for experiment execution."""

Simulation time parameters (in time units, not steps)

T_sim: float

burn_end_time: float

split_time: float

future_duration: float

Diagnostics

diagnostic_stride: int

Branches to simulate

branches: tuple[str, ...]

```
def n_steps(self, model: ModelConfig) -&gt; int:

"""Total steps."""

return int(round(self.T_sim / model.dt))



def burn_end(self, model: ModelConfig) -&gt; int:

"""Burn-in end step."""

return int(round(self.burn_end_time / model.dt))



def hist_end(self, model: ModelConfig) -&gt; int:

"""History end step (split point)."""

return int(round(self.split_time / model.dt))



def future_steps(self, model: ModelConfig) -&gt; int:

"""Future simulation steps."""

return int(round(self.future_duration / model.dt))



def validate(self, model: ModelConfig) -&gt; None:
```

"""Validate protocol against model."""

if self.T_sim <= 0:

raise ValueError("T_sim must be positive")

if not (0 < self.burn_end_time < self.split_time < self.T_sim):

raise ValueError("Invalid time parameters: must have 0 < burn_end < split < T_sim")

if self.future_duration <= 0:

raise ValueError("future_duration must be positive")

if self.diagnostic_stride <= 0:

raise ValueError("diagnostic_stride must be positive")

Validate that all time parameters are compatible with dt

if not np.isclose(self.T_sim / model.dt, round(self.T_sim / model.dt), rtol=1e-10):

raise ValueError(f"T_sim ({self.T_sim}) must be divisible by dt ({model.dt})")

if not np.isclose(self.burn_end_time / model.dt, round(self.burn_end_time / model.dt), rtol=1e-10):

raise ValueError(f"burn_end_time ({self.burn_end_time}) must be divisible by dt ({model.dt})")

if not np.isclose(self.split_time / model.dt, round(self.split_time / model.dt), rtol=1e-10):

raise ValueError(f"split_time ({self.split_time}) must be divisible by dt ({model.dt})")

if not np.isclose(self.future_duration / model.dt, round(self.future_duration / model.dt), rtol=1e-10):

raise ValueError(f"future_duration ({self.future_duration}) must be divisible by dt ({model.dt})")

Default frozen production configuration
NOTE: T_window must be the EXACT value 25.0/mean(OMEGA), not rounded

DEFAULT_MODEL_CONFIG = ModelConfig(

text

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=FROZEN_T_WINDOW, # 17.502917152858814 (exact)

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=FROZEN_ZERO_TOL,

)
Default protocol matching frozen run

DEFAULT_PROTOCOL_CONFIG = ProtocolConfig(

text

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=("continue", "reset_ema", "reset_B", "reset_B_ema", "B_carrier_only", "full_reset", "future_shuffle"),

)
==========
Utility Functions
==========

def compute_edges(N: int) -> tuple[np.ndarray, np.ndarray]:

text

"""Get upper triangle indices for edges."""

return np.triu_indices(N, k=1)

```
def validate_B_matrix(B: np.ndarray, N: int, eps: float = 1e-12) -> None:

text

"""Validate B matrix: symmetric, nonnegative, zero diagonal."""

if B.shape != (N, N):

raise ValueError(f"B shape {B.shape} != ({N}, {N})")

if not np.allclose(B, B.T, atol=eps):

raise ValueError("B is not symmetric")

if not np.allclose(np.diag(B), 0, atol=eps):

raise ValueError("B diagonal is not zero")

if np.any(B &lt; -eps):

raise ValueError("B has negative entries")

def file_sha256(path: str | os.PathLike[str]) -> str:

text

"""Compute SHA-256 hash of a file."""

digest = hashlib.sha256()

with open(path, "rb") as handle:

for block in iter(lambda: [handle.read](http://handle.read)(1024 * 1024), b""):

digest.update(block)
```

```
return digest.hexdigest()

def array_sha256(arr: np.ndarray) -> str:

text

"""Compute SHA-256 hash of a numpy array."""

digest = hashlib.sha256()

shape_bytes = np.asarray(arr.shape, dtype=np.int64).tobytes()

digest.update(shape_bytes)

digest.update(arr.dtype.str.encode())

contiguous = np.ascontiguousarray(arr)

if contiguous.size &gt; 0:

digest.update(contiguous.tobytes())

return digest.hexdigest()

def atomic_write_text(path: Path, text: str) -> None:

text

"""Write text atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{[path.name](http://path.name)}.",
suffix=".tmp")
```

```
try:
with os.fdopen(fd, "w", encoding="utf-8") as handle:
handle.write(text)
handle.flush()
os.fsync(handle.fileno())
os.replace(str(tmp_path), str(path))
finally:
if os.path.exists(tmp_path):
os.remove(tmp_path)
def atomic_write_json(path: Path, data: dict[str, Any]) -> None:
text
"""Write JSON atomically with fsync."""
text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"
atomic_write_text(path, text)
def atomic_write_npz(path: Path, **kwargs: Any) -> None:
text
"""Write NPZ atomically with fsync."""
```

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{path.name}.", suffix=".tmp.npz")

try:

with os.fdopen(fd, "wb") as handle:

np.savez(handle, **kwargs)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))

finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

==========
RNG Utilities (matching frozen production exactly)
==========

def make_seed_sequence(seed: int, child_index: int) -> int:

text

"""Generate a child seed from SeedSequence(seed).spawn(...)[child_index].

This matches the frozen production RNG hierarchy exactly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

children = root.spawn(child_index + 1)

return int(children[child_index].generate_state(1, dtype=np.uint64)[0])

def make_prefix_rng(seed: int) -> np.random.Generator:

text

"""Create prefix RNG matching frozen production exactly.

Frozen production does:

root = np.random.SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = np.random.default_rng(init_sequence)

NOTE: default_rng(SeedSequence_object) is NOT the same as

default_rng(SeedSequence_object.generate_state(...))

The production uses the SeedSequence object directly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

return np.random.default_rng(init_sequence)

def make_branch_rngs(seed: int, branch_order: tuple[str, ...]) -> dict[str, np.random.Generator]:

text

"""Create branch RNGs matching frozen production algorithm.

Frozen: *, branch*root = SeedSequence(seed).spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))

branch_rng_seeds = {branch: int(sequence.generate_state(1, dtype=np.uint64)[0])

for branch, sequence in zip(branch_order, branch_sequences)}

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

*, branch*root = root.spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))

return {

branch: np.random.default_rng(int(sequence.generate_state(1, dtype=np.uint64)[0]))

for branch, sequence in zip(branch_order, branch_sequences)

}

==========
Projection Accumulator (matching production)
==========

@dataclass

class ProjectionTotals:

text

"""Accumulator for projection diagnostics - exact match to production."""

steps: int = 0

violations: int = 0

active: int = 0

```
norm_sum: float = 0.0

norm_max: float = 0.0

relative_state_sum: float = 0.0

relative_state_max: float = 0.0

relative_update_sum: float = 0.0

relative_update_max: float = 0.0

g_min: float = np.inf

g_min_pre: float = np.inf

def update(self, info: dict[str, Any]) -&gt; None:

"""Update accumulator with step info."""

self.steps += 1

self.violations += int(info["pre_violation"])

self.g_min = min(self.g_min, float(info["g_min"]))

self.g_min_pre = min(self.g_min_pre, float(info["g_min_pre"]))

if info["projection_active"]:

self.active += 1

norm = float(info["projection_norm"])

rel_state = float(info["projection_relative_state"])
```

```
rel_update = float(info["projection_relative_update"])

self.norm_sum += norm

self.norm_max = max(self.norm_max, norm)

self.relative_state_sum += rel_state

self.relative_state_max = max(self.relative_state_max, rel_state)

self.relative_update_sum += rel_update

self.relative_update_max = max(self.relative_update_max, rel_update)

def metrics(self) -&gt; dict[str, float]:

"""Compute final metrics from accumulated values."""

active_denominator = max(self.active, 1)

return {

"G_min_future": float(self.g_min),

"G_min_pre_future": float(self.g_min_pre),

"Pre_gap_violation_fraction": float(self.violations / self.steps),

"Projection_active_fraction": float(self.active / self.steps),

"Projection_active_given_violation": float(self.active / self.violations)

if self.violations &gt; 0
```

else float('nan'),

"Projection_norm_mean_active": float(

self.norm_sum / active_denominator

),

"Projection_norm_max": float(self.norm_max),

"Projection_relative_state_mean_active": float(

self.relative_state_sum / active_denominator

),

"Projection_relative_state_max": float(self.relative_state_max),

"Projection_relative_update_mean_active": float(

self.relative_update_sum / active_denominator

),

"Projection_relative_update_max": float(self.relative_update_max),

}

==========
Dynamic State
==========

class DynamicState:

text

"""State container for AIEF-1.4 dynamics.

Matches the frozen production implementation exactly.

"""

def **init**(

self,

theta: np.ndarray,

B: np.ndarray,

coherence_acc: np.ndarray,

history: list[np.ndarray] | np.ndarray,

):

self.theta = np.asarray(theta, dtype=float).copy()

self.B = np.asarray(B, dtype=float).copy()

self.coherence_acc = np.asarray(coherence_acc, dtype=complex).copy()

Handle history input - ensure it's a list of arrays

```
if isinstance(history, np.ndarray):

# If it's a 2D array, convert to list of 1D arrays

self.history = [np.asarray(h, dtype=float).copy() for h in history]

else:

self.history = [np.asarray(h, dtype=float).copy() for h in history]



def clone(self) -&gt; "DynamicState":

"""Create a deep copy of the state."""

return DynamicState(

theta=self.theta.copy(),

B=self.B.copy(),

coherence_acc=self.coherence_acc.copy(),

history=[h.copy() for h in self.history],

)



@property

def N(self) -&gt; int:
```

```
return len(self.theta)



def validate(self, config: ModelConfig, step: int | None = None) -&gt; None:

"""Validate state consistency."""

# Check history length

expected_history_len = config.delay_steps + 1

if len(self.history) != expected_history_len:

raise RuntimeError(

f"History length {len(self.history)} != expected {expected_history_len} "

f"(delay_steps={config.delay_steps}) at step {step}"

)

# Check theta shape

if self.theta.shape != (self.N,):

raise RuntimeError(f"theta shape {self.theta.shape} != ({self.N},)")

# Check B shape

if self.B.shape != (self.N, self.N):

raise RuntimeError(f"B shape {self.B.shape} != ({self.N}, {self.N})")
```

```
# Check coherence_acc shape

if self.coherence_acc.shape != (self.N, self.N):

raise RuntimeError(f"coherence_acc shape {self.coherence_acc.shape} != ({self.N}, {self.N})")

# Check B symmetry

if not np.allclose(self.B, self.B.T, atol=1e-12):

raise RuntimeError("B matrix is not symmetric")

# Check B diagonal

if not np.allclose(np.diag(self.B), 0, atol=1e-12):

raise RuntimeError("B matrix diagonal is not zero")

# Check B non-negativity

if np.any(self.B &lt; -1e-12):

raise RuntimeError("B matrix has negative entries")

============================================================================
====
Simulation Core (exact reproduction of frozen production)
============================================================================
====

def initialize_state(

text
```

seed: int,

config: ModelConfig,

) -> DynamicState:

text

"""Initialize state from seed using frozen algorithm exactly.

Matches production: simulate_prefix() initialization.

Uses: root = SeedSequence(seed); init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence)

"""

N = config.N

rng = make_prefix_rng(seed)

Initialize theta: uniform in [0, 2*pi) - matching production

theta = rng.random(N) *2.0* np.pi

Initialize B: small random symmetric matrix

```
B = config.B_init_scale * rng.random((N, N))

B = 0.5 * (B + B.T)

np.fill_diagonal(B, 0.0)



# Initialize coherence accumulator

coherence_acc = np.zeros((N, N), dtype=complex)



# Initialize history: DELAY_STEPS + 1 copies of theta

history = [theta.copy() for *in range(config.delay*steps + 1)]



return DynamicState(

theta=theta,

B=B,

coherence_acc=coherence_acc,

history=history,

)

def compute_R(
```

```
text

state: DynamicState,

config: ModelConfig,

) -> np.ndarray:

text

"""Compute coherence matrix R from current state.



Exact reproduction of production compute_R().

"""

N = state.N

delayed = state.history[0]



# Convert omega to numpy array for indexing

omega_arr = np.asarray([config.omega](http://config.omega), dtype=float)



dphi = (

state.theta[:, None]
```

```
- delayed[None, :]

- ((omega_arr[:, None] + omega_arr[None, :]) / 2.0) * config.tau

)

z_inst = np.exp(1j * dphi)



# Update EMA

state.coherence_acc[:] = (

config.alpha *state.coherence_acc + (1.0 - config.alpha)* z_inst

)



# Compute magnitude and symmetrize

result = np.abs(state.coherence_acc)

result = 0.5 * (result + result.T)

np.fill_diagonal(result, 0.0)



return result

def advance_state(
```

text

state: DynamicState,

config: ModelConfig,

rng: np.random.Generator,

shuffle_driver: bool = False,

) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:

text

"""Advance state by one time step.

Exact reproduction of production advance_state().

Returns (R_physical, R_driver, info).

Production also returns projection diagnostics:

- projection_active: whether correction was applied

- projection_norm: norm of correction

- projection_relative_state: correction_norm / ||B_unprojected||

- projection_relative_update: correction_norm / ||B_unprojected - B_previous||

"""

N = state.N

edges = compute_edges(N)

Store previous B for projection

B_previous = state.B.copy()

Compute R matrices

R_physical = compute_R(state, config)

R_driver = R_physical

if shuffle_driver:

values = R_physical[edges].copy()

rng.shuffle(values)

R_driver = np.zeros_like(R_physical)

R_driver[edges] = values

R_driver[(edges[1], edges[0])] = values

```
# Update B
B_unprojected = state.B + config.dt * (
(config.eta + [config.mu](http://config.mu)) * R_driver
- [config.mu](http://config.mu)
- config.gamma * state.B
)
B_unprojected = np.maximum(B_unprojected, 0.0)
B_unprojected = 0.5 * (B_unprojected + B_unprojected.T)
np.fill_diagonal(B_unprojected, 0.0)


# Projection diagnostics (matching production exactly)
proposed_update_norm = float(np.linalg.norm(B_unprojected - B_previous))


# Check gap constraint
row_sum_pre = np.sum(B_unprojected, axis=1)
g_min_pre = float(np.min(config.a - config.q * row_sum_pre))
```

pre_violation = bool(g_min_pre < config.delta)

Project if needed

B_projected = B_unprojected

correction_norm = 0.0

if pre_violation:

scale = np.minimum(1.0, config.capacity / (row_sum_pre + config.scale_epsilon))

B_projected = B_unprojected * np.minimum.outer(scale, scale)

np.fill_diagonal(B_projected, 0.0)

correction_norm = float(np.linalg.norm(B_projected - B_unprojected))

projection_active = bool(correction_norm > FROZEN_PROJECTION_NORM_TOL)

relative_state = correction_norm / max(

float(np.linalg.norm(B_unprojected)), FROZEN_ZERO_TOL

)

relative_update = correction_norm / max(proposed_update_norm, FROZEN_ZERO_TOL)

```
# Validate projection

state.B = B_projected

row_sum = np.sum(state.B, axis=1)

g_min = float(np.min(config.a - config.q * row_sum))

if g_min &lt; [config.delta](http://config.delta) - 1e-9:

raise RuntimeError(f"Gap projection failed: g_min={g_min}")



# Update theta

delayed = state.history[0]

omega_arr = np.asarray([config.omega](http://config.omega), dtype=float)

coupling = np.sum(

state.B * np.sin(delayed[None, :] - state.theta[:, None]), axis=1

)

state.theta = np.mod(

state.theta + config.dt * (omega_arr + coupling),

2.0 * np.pi

)
```

Update history

state.history.append(state.theta.copy())

if len(state.history) > config.delay_steps + 1:

state.history.pop(0)

info = {

"pre_violation": pre_violation,

"projection_active": projection_active,

"projection_norm": correction_norm,

"projection_relative_state": relative_state,

"projection_relative_update": relative_update,

"g_min_pre": g_min_pre,

"g_min": g_min,

}

return R_physical, R_driver, info

def simulate_prefix(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

) -> tuple[DynamicState, dict[str, Any]]:

text

"""Simulate prefix up to split_time.

Exact reproduction of production simulate_prefix().

Returns (state, prefix_dict).

Production uses:

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence) # for initialization

Then in the loop, uses:

```
unused_rng = default_rng(0) # for advance_state when shuffle_driver=False

"""

N = config.N

edges = compute_edges(N)



# Initialize state (uses init_sequence via make_prefix_rng)

state = initialize_state(seed, config)



# Get RNG for prefix simulation loop

# Production uses unused_rng = default_rng(0) for the advance_state calls

# when shuffle_driver=False

unused_rng = np.random.default_rng(0)



# Simulate to split point

R_past_sum = np.zeros((N, N), dtype=float)

R_past_count = 0
```

```
burn_end = protocol.burn_end(config)

hist_end = protocol.hist_end(config)



for step in range(hist_end):

R_physical, *,* = advance_state(state, config, unused_rng, shuffle_driver=False)


if burn_end &lt;= step &lt; hist_end:

R_past_sum += R_physical

R_past_count += 1



prefix = {

"B_at_split": state.B.copy(),

"R_physical_past": R_past_sum / R_past_count if R_past_count &gt; 0 else np.zeros((N, N)),

"theta_at_split": state.theta.copy(),

"coherence_acc_at_split": state.coherence_acc.copy(),

"history_at_split": np.stack(state.history),

"selected_at_split": (state.B &gt; config.B_c).astype(np.uint8),
```

```
}



return state, prefix

def apply_intervention(

text

state: DynamicState,

branch: str,

config: ModelConfig,

rng: np.random.Generator,

) -> None:

text

"""Apply intervention to state. Mutates in place.



Exact reproduction of production apply_intervention().

"""

N = state.N
```

```
if branch in {"reset_ema", "reset_B_ema", "B_carrier_only", "full_reset"}:

# Zero coherence accumulator

state.coherence_acc.fill(0.0)



if branch in {"reset_B", "reset_B_ema", "full_reset"}:

# Zero B

state.B.fill(0.0)



if branch in {"B_carrier_only", "full_reset"}:

# Randomize theta and history

new_theta = rng.random(N) *2.0* np.pi

state.theta = new_theta

state.history = [new_theta.copy() for *in range(config.delay*steps + 1)]



# Ensure symmetry

state.B = 0.5 * (state.B + state.B.T)

np.fill_diagonal(state.B, 0.0)
```

```
def simulate_future(
text
split_state: DynamicState,
prefix: dict[str, Any],
branch: str,
config: ModelConfig,
protocol: ProtocolConfig,
rng: np.random.Generator,
) -> dict[str, Any]:
text
"""Simulate future trajectory from split state.


Reproduces production simulate_future() with ALL required outputs.


CRITICAL: Now accumulates BOTH horizon systems:


Fixed Physical Horizons (time-based, NOT T_window dependent):
```

- immediate: 0 to 17.5 time units

- recovery: 17.5 to 87.5 time units

- late: 87.5 to 320 time units

EMA-Normalized Horizons (T_window dependent):

- immediate: 0 to 1*T_window

- recovery: 1*T_window to 5*T_window

- late: 5*T_window to end

At rho=1 (T_window=17.502917...), the fixed physical and normalized horizons

are approximately aligned but use exact step boundaries.

Production accumulates:

- P_full, P_immediate, P_recovery, P_late (incidence matrices)

- B_mean_full, B_mean_immediate, B_mean_recovery, B_mean_late

- R_physical_future, R_driver_future

- Projection diagnostics via ProjectionTotals

- Various scalar metrics

Uses horizon system from production for normalized:

- WINDOW_STEPS = round(T_window / dt)

- RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS)

For fixed physical:

- FIXED_IMMEDIATE_END = round(17.5 / dt)

- FIXED_RECOVERY_END = round(87.5 / dt)

"""

state = split_state.clone()

apply_intervention(state, branch, config, rng)

N = state.N

edges = compute_edges(N)

future_steps = protocol.future_steps(config)

```
# Compute horizon step boundaries

# Normalized EMA horizons (T_window dependent)

window_steps = int(round(config.T_window / config.dt))

recovery_end_ema = min(5 * window_steps, future_steps)



# Fixed physical horizons (NOT T_window dependent)

# These use the production's physical time boundaries: 17.5, 87.5, 320

fixed_immediate_end = int(round(17.5 / config.dt)) # 17.5 time units

fixed_recovery_end = int(round(87.5 / config.dt)) # 87.5 time units

# Late goes from 87.5 to end (320 time units)



# Clamp all boundaries to future_steps

recovery_end_ema = min(recovery_end_ema, future_steps)

fixed_immediate_end = min(fixed_immediate_end, future_steps)

fixed_recovery_end = min(fixed_recovery_end, future_steps)



# Initialize accumulators for BOTH horizon systems
```

P matrices (incidence: B > B_c as 0/1)

P_sums_ema = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}

P_sums_fixed = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}

B matrices (mean B values)

B_sums_ema = {

"full": np.zeros((N, N), dtype=float),

```
"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}

B_sums_fixed = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}



# Counts for each horizon in both systems

counts_ema = {"full": 0, "immediate": 0, "recovery": 0, "late": 0}

counts_fixed = {"full": 0, "immediate": 0, "recovery": 0, "late": 0}



# R matrices

R_physical_sum = np.zeros((N, N), dtype=float)
```

```
R_driver_sum = np.zeros((N, N), dtype=float)



# Projection accumulator (matching production)

projection = ProjectionTotals()



# Trajectory accumulators

sample_times = []

density_trajectory = []

similarity_trajectory = []



# For retention/emergence calculations

r_sum = 0.0

r_count = 0

r_late_ema_sum = 0.0

r_late_ema_count = 0

r_late_fixed_sum = 0.0

r_late_fixed_count = 0
```

B_reference = prefix["B_at_split"]

shuffle_driver = (branch == "future_shuffle")

for step in range(future_steps):

R_physical, R_driver, info = advance_state(

state, config, rng, shuffle_driver=shuffle_driver

)

Update projection accumulator

projection.update(info)

Accumulate R matrices

R_physical_sum += R_physical

R_driver_sum += R_driver

Determine which horizons this step belongs to for BOTH systems

selected = (state.B > config.B_c).astype(float)

```
# === EMA-Normalized Horizons ===

# Full: all steps

P_sums_ema["full"] += selected

B_sums_ema["full"] += state.B

counts_ema["full"] += 1


# immediate: 0 to window_steps (exclusive)

if step &lt; window_steps:

P_sums_ema["immediate"] += selected

B_sums_ema["immediate"] += state.B

counts_ema["immediate"] += 1


# recovery: window_steps to recovery_end_ema (exclusive)

if window_steps &lt;= step &lt; recovery_end_ema:

P_sums_ema["recovery"] += selected

B_sums_ema["recovery"] += state.B

counts_ema["recovery"] += 1


# late: recovery_end_ema to end
```

if recovery_end_ema <= step:

P_sums_ema["late"] += selected

B_sums_ema["late"] += state.B

counts_ema["late"] += 1

=== Fixed Physical Horizons ===

Full: all steps (same as EMA)

P_sums_fixed["full"] += selected

B_sums_fixed["full"] += state.B

counts_fixed["full"] += 1

immediate: 0 to fixed_immediate_end (exclusive)

if step < fixed_immediate_end:

P_sums_fixed["immediate"] += selected

B_sums_fixed["immediate"] += state.B

counts_fixed["immediate"] += 1

recovery: fixed_immediate_end to fixed_recovery_end (exclusive)

```
if fixed_immediate_end &lt;= step &lt; fixed_recovery_end:

P_sums_fixed["recovery"] += selected

B_sums_fixed["recovery"] += state.B

counts_fixed["recovery"] += 1


# late: fixed_recovery_end to end

if fixed_recovery_end &lt;= step:

P_sums_fixed["late"] += selected

B_sums_fixed["late"] += state.B

counts_fixed["late"] += 1


# Trajectory sampling - CRITICAL: r_mean_late is ONLY accumulated here

if step % protocol.diagnostic_stride == 0:

t = (step + 1) * config.dt

sample_times.append(t)


# Density: fraction of edges above B_c

density = float(np.count_nonzero(state.B[edges] &gt; config.B_c) / len(edges[0]))

density_trajectory.append(density)
```

```
# Similarity: cosine between current and reference B

B_vec = state.B[edges]

B_ref_vec = B_reference[edges]

norm_B = np.linalg.norm(B_vec)

norm_ref = np.linalg.norm(B_ref_vec)

if norm_B &gt; 0 and norm_ref &gt; 0:

similarity = float([np.dot](http://np.dot)(B_vec, B_ref_vec) / (norm_B * norm_ref))

else:

similarity = float('nan')

similarity_trajectory.append(similarity)


# Track overall r

r_value = float(np.abs(np.mean(np.exp(1j * state.theta))))

r_sum += r_value

r_count += 1


# Track late-phase r for BOTH horizon systems
```

if recovery_end_ema <= step:

r_late_ema_sum += r_value

r_late_ema_count += 1

if fixed_recovery_end <= step:

r_late_fixed_sum += r_value

r_late_fixed_count += 1

Compute mean matrices for BOTH horizon systems

P_ema = {name: P_sums_ema[name] / counts_ema[name] if counts_ema[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

P_fixed = {name: P_sums_fixed[name] / counts_fixed[name] if counts_fixed[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

B_mean_ema = {name: B_sums_ema[name] / counts_ema[name] if counts_ema[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

B_mean_fixed = {name: B_sums_fixed[name] / counts_fixed[name] if counts_fixed[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

Compute R means

R_physical_mean = R_physical_sum / future_steps if future_steps > 0 else np.zeros((N, N))

R_driver_mean = R_driver_sum / future_steps if future_steps > 0 else np.zeros((N, N))

Final state

B_final = state.B.copy()

final_weights = state.B[edges]

final_rows = np.sum(state.B, axis=1)

Retention and emergence for BOTH horizon systems

split_selected = prefix["selected_at_split"][edges].astype(bool)

EMA-normalized

p_late_edges_ema = P_ema["late"][edges]

```
retention_ema = (

float(np.mean(p_late_edges_ema[split_selected]))

if np.any(split_selected)

else float('nan')

)

emergence_ema = (

float(np.mean(p_late_edges_ema[~split_selected]))

if np.any(~split_selected)

else float('nan')

)



# Fixed physical

p_late_edges_fixed = P_fixed["late"][edges]

retention_fixed = (

float(np.mean(p_late_edges_fixed[split_selected]))

if np.any(split_selected)

else float('nan')
```

```
)

emergence_fixed = (

float(np.mean(p_late_edges_fixed[~split_selected]))

if np.any(~split_selected)

else float('nan')

)



# Minimum similarity from trajectory

similarity_array = np.array(similarity_trajectory, dtype=float)

finite_similarity = similarity_array[np.isfinite(similarity_array)]

minimum_similarity = (

float(np.min(finite_similarity)) if len(finite_similarity) &gt; 0

else float('nan')

)



# Build result with BOTH horizon systems

result = {
```

```
# === Matrix outputs for EMA-normalized horizons (backward compatible) ===

"B_final": B_final,

"B_mean_late": B_mean_ema["late"],

"P_full": P_ema["full"],

"P_immediate": P_ema["immediate"],

"P_recovery": P_ema["recovery"],

"P_late": P_ema["late"],

"R_physical_future": R_physical_mean,

"R_driver_future": R_driver_mean,


# === Matrix outputs for FIXED-PHYSICAL horizons (NEW) ===

"B_mean_late_fixed_physical": B_mean_fixed["late"],

"P_full_fixed_physical": P_fixed["full"],

"P_immediate_fixed_physical": P_fixed["immediate"],

"P_recovery_fixed_physical": P_fixed["recovery"],

"P_late_fixed_physical": P_fixed["late"],


# === Matrix outputs for NORMALIZED-EMA horizons (NEW - explicit) ===
```

"B_mean_late_normalized_EMA": B_mean_ema["late"],

"P_full_normalized_EMA": P_ema["full"],

"P_immediate_normalized_EMA": P_ema["immediate"],

"P_recovery_normalized_EMA": P_ema["recovery"],

"P_late_normalized_EMA": P_ema["late"],

Trajectory outputs

"sample_times": np.array(sample_times, dtype=float),

"density_trajectory": np.array(density_trajectory, dtype=float),

"similarity_trajectory": np.array(similarity_trajectory, dtype=float),

Scalar outcomes - EMA-normalized (backward compatible names)

"D_B_final": float(np.count_nonzero(final_weights > config.B_c) / len(edges[0])),

"P_mean_full": float(np.mean(P_ema["full"][edges])),

"P_mean_immediate": float(np.mean(P_ema["immediate"][edges])),

"P_mean_recovery": float(np.mean(P_ema["recovery"][edges])),

"P_mean_late": float(np.mean(P_ema["late"][edges])),

```
# Scalar outcomes - FIXED-PHYSICAL (NEW - primary estimands)

"P_mean_full_fixed_physical": float(np.mean(P_fixed["full"][edges])),

"P_mean_immediate_fixed_physical": float(np.mean(P_fixed["immediate"][edges])),

"P_mean_recovery_fixed_physical": float(np.mean(P_fixed["recovery"][edges])),

"P_mean_late_fixed_physical": float(np.mean(P_fixed["late"][edges])),


# Scalar outcomes - NORMALIZED-EMA (NEW - explicit)

"P_mean_full_normalized_EMA": float(np.mean(P_ema["full"][edges])),

"P_mean_immediate_normalized_EMA": float(np.mean(P_ema["immediate"][edges])),

"P_mean_recovery_normalized_EMA": float(np.mean(P_ema["recovery"][edges])),

"P_mean_late_normalized_EMA": float(np.mean(P_ema["late"][edges])),


# Similarity metrics - BOTH systems

"Weighted_similarity_final": cosine_edges(B_reference, B_final, edges),

"Weighted_similarity_late": cosine_edges(B_reference, B_mean_ema["late"], edges),

"Weighted_similarity_late_fixed_physical": cosine_edges(B_reference, B_mean_fixed["late"], edges),

"Weighted_similarity_late_normalized_EMA": cosine_edges(B_reference, B_mean_ema["late"],
edges),

"Jaccard_final": jaccard_edges(B_reference, B_final, edges, config.B_c),
```

"Jaccard_final_fixed_physical": jaccard_edges(B_reference, B_final, edges, config.B_c),

Retention/emergence - BOTH systems

"Retention_selected_late": retention_ema,

"Emergence_unselected_late": emergence_ema,

"Retention_contrast_late": retention_ema - emergence_ema,

"Retention_selected_late_fixed_physical": retention_fixed,

"Emergence_unselected_late_fixed_physical": emergence_fixed,

"Retention_contrast_late_fixed_physical": retention_fixed - emergence_fixed,

"Retention_selected_late_normalized_EMA": retention_ema,

"Emergence_unselected_late_normalized_EMA": emergence_ema,

"Retention_contrast_late_normalized_EMA": retention_ema - emergence_ema,

Correlations with past R - BOTH systems

"Corr_Rpast_Pfull": safe_corr(prefix["R_physical_past"][edges], P_ema["full"][edges]),

"Corr_Rpast_Plate": safe_corr(prefix["R_physical_past"][edges], P_ema["late"][edges]),

"Corr_Rpast_Pfull_fixed_physical": safe_corr(prefix["R_physical_past"][edges], P_fixed["full"][edges]),

"Corr_Rpast_Plate_fixed_physical": safe_corr(prefix["R_physical_past"][edges], P_fixed["late"][edges]),

"Corr_Rpast_Pfull_normalized_EMA": safe_corr(prefix["R_physical_past"][edges], P_ema["full"][edges]),

"Corr_Rpast_Plate_normalized_EMA": safe_corr(prefix["R_physical_past"][edges], P_ema["late"][edges]),

Weight statistics (same for both - based on final state)

"Total_edge_weight": float(np.sum(final_weights)),

"Mean_row_sum": float(np.mean(final_rows)),

"Max_row_sum": float(np.max(final_rows)),

"Nonzero_edge_fraction": float(np.mean(final_weights > config.scale_epsilon)),

"Effective_edges": effective_number(final_weights),

"Weight_gini": gini_coefficient(final_weights),

"Weight_entropy": normalized_entropy(final_weights),

Order parameter - BOTH systems

"r_mean_future": float(r_sum / r_count) if r_count > 0 else float('nan'),

"r_mean_late": float(r_late_ema_sum / r_late_ema_count) if r_late_ema_count > 0 else float('nan'),

"r_mean_late_fixed_physical": float(r_late_fixed_sum / r_late_fixed_count) if r_late_fixed_count > 0 else float('nan'),

"r_mean_late_normalized_EMA": float(r_late_ema_sum / r_late_ema_count) if r_late_ema_count > 0 else float('nan'),

Minimum similarity

"Minimum_similarity": minimum_similarity,

Projection diagnostics (from ProjectionTotals)

**projection.metrics(),

Horizon metadata

"window_steps": window_steps,

"recovery_end_ema": recovery_end_ema,

"fixed_immediate_end": fixed_immediate_end,

"fixed_recovery_end": fixed_recovery_end,

Time-based metrics (placeholders - will be NaN for now)

"Time_to_50pct_continue_final_density": float('nan'),

"Time_to_90pct_continue_final_density": float('nan'),

"Time_to_50pct_continue_late_similarity": float('nan'),

"Time_to_90pct_continue_late_similarity": float('nan'),

"Time_to_similarity_below_0_9": float('nan'),

"Time_to_similarity_below_0_5": float('nan'),

"Late_persistence_ratio_to_continue": float('nan'),

"Late_similarity_ratio_to_continue": float('nan'),

}

return result

==
Metric Helpers
==

def cosine_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None) -> float:

text

"""Compute cosine similarity between reference and current B matrices over edges.

If edges is None, computes over all upper triangle edges.

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

x = np.asarray(reference[edges], dtype=float)

y = np.asarray(current[edges], dtype=float)

denominator = float(np.linalg.norm(x) * np.linalg.norm(y))

if denominator <= 1e-12:

return float('nan')

return float(np.dot(x, y) / denominator)

def jaccard_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None, threshold: float = FROZEN_ZERO_TOL) -> float:

text

"""Compute Jaccard similarity between reference and current B matrices over edges.

Uses B > threshold as the threshold for edge presence.

If edges is None, computes over all upper triangle edges.

Matches production: jaccard_edges(B_reference, state.B) uses B > B_C

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

x = reference[edges] > threshold

y = current[edges] > threshold

union = np.count_nonzero(x | y)

if union == 0:

return float('nan')

return float(np.count_nonzero(x & y) / union)

def safe_corr(x: np.ndarray, y: np.ndarray) -> float:

text

"""Compute correlation coefficient safely."""

x = np.asarray(x, dtype=float)

y = np.asarray(y, dtype=float)

```
valid = np.isfinite(x) &amp;amp; np.isfinite(y)

x = x[valid]

y = y[valid]

if len(x) &amp;lt; 3 or np.std(x) &amp;lt; 1e-12 or np.std(y) &amp;lt; 1e-12:

return float('nan')

return float(np.corrcoef(x, y)[0, 1])

def effective_number(weights: np.ndarray) -> float:

text

"""Compute effective number of edges."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

sum_sq = float(np.sum(weights**2))

if total &amp;lt;= 1e-12 or sum_sq &amp;lt;= 1e-12:

return 0.0

return total**2 / sum_sq

def gini_coefficient(weights: np.ndarray) -> float:

text

"""Compute Gini coefficient."""
```

```
x = np.sort(np.asarray(weights, dtype=float))

total = float(np.sum(x))

if total &lt;= 1e-12:

return 0.0

n = len(x)

ranks = np.arange(1, n + 1, dtype=float)

return float(2.0 *[np.dot](http://np.dot)(ranks, x) / (n* total) - (n + 1.0) / n)

def normalized_entropy(weights: np.ndarray) -> float:

text

"""Compute normalized entropy."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

if total &lt;= 1e-12:

return 0.0

probabilities = weights / total

positive = probabilities &gt; 0.0

entropy = -float(np.sum(probabilities[positive] * np.log(probabilities[positive])))
```

```
return entropy / float(np.log(len(weights)))


================================================================================
Required Output Schema for Baseline Equivalence
================================================================================
IMPORTANT: We separate FROZEN keys (must exist in historical NPZ) from
EXTENDED keys (new in v1, only validated internally at rho=1).
The frozen NPZ does NOT contain dual-horizon keys, so baseline equivalence
must only check FROZEN keys against historical data.
Required prefix keys for baseline equivalence

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)
These are the keys that MUST exist in the frozen NPZ for baseline equivalence

FROZEN_BRANCH_MATRIX_KEYS = (

text

"B_final",
```

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
These are the NEW keys introduced in v1 for dual-horizon support
They will NOT exist in frozen NPZ, so baseline equivalence skips them

EXTENDED_BRANCH_MATRIX_KEYS = (

text

"B_mean_late_fixed_physical",

"P_full_fixed_physical",

"P_immediate_fixed_physical",

"P_recovery_fixed_physical",

"P_late_fixed_physical",

"B_mean_late_normalized_EMA",

"P_full_normalized_EMA",

"P_immediate_normalized_EMA",

"P_recovery_normalized_EMA",

"P_late_normalized_EMA",

)
All matrix keys (frozen + extended)

REQUIRED_BRANCH_MATRIX_KEYS = FROZEN_BRANCH_MATRIX_KEYS + EXTENDED_BRANCH_MATRIX_KEYS

REQUIRED_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

Fixed physical horizon scalars (NEW - PRIMARY ESTIMANDS)

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

Normalized EMA horizon scalars (NEW - explicit)

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

Projection diagnostics (from ProjectionTotals)

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

)
==========
Baseline Equivalence Test
==========

def exact_array_equal_with_nan(a: np.ndarray, b: np.ndarray) -> bool:

text

"""Compare arrays with NaN-aware exact equality."""

a = np.asarray(a)

b = np.asarray(b)

if a.shape != b.shape or a.dtype != b.dtype:

return False

if np.issubdtype(a.dtype, np.floating) or np.issubdtype(a.dtype, np.complexfloating):

```
a_nan = np.isnan(a)

b_nan = np.isnan(b)

if not np.array_equal(a_nan, b_nan):

return False

finite = ~a_nan

return np.array_equal(a[finite], b[finite])

return np.array_equal(a, b)

def run_baseline_equivalence(

text

seeds: list[int],

config: ModelConfig,

protocol: ProtocolConfig,

frozen_manifest_path: str,

frozen_npz_path: str,

branch_order: tuple[str, ...],

frozen_csv_path: str | None = None,

) -> dict[str, Any]:

text
```

"""Run baseline equivalence test at rho=1 (T_window = T_*).

MANDATORY: Must reproduce frozen run exactly for all seeds.

This function validates that the engine produces EXACTLY the same outputs

as the frozen production run for all required keys.

Requires NaN-aware exact equality for ALL required schema keys:

- All prefix keys

- All branch matrix keys

- All branch scalar keys

Returns:

status: "passed" | "failed" | "not_run"

seed_results: per-seed validation details

engine_source_sha256: hash of this engine source

```
frozen_manifest_hash: hash of frozen manifest

frozen_npz_hash: hash of frozen NPZ

"""

import numpy as np

from pathlib import Path

import hashlib



# Helper function to compute source hash

def *source*sha256() -&gt; str | None:

try:

return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

return None



# Early failure: frozen scalar CSV is required for scalar equivalence

if frozen_csv_path is None:

return {
```

"status": "failed",

"error": "frozen scalar CSV is required for scalar equivalence",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": None,

}

Record engine source hash

engine_source_hash = *source*sha256()

results = {

"status": "not_run",

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"mean_omega": config.mean_omega,

"T_window": config.T_window,

"T_star": config.T_star,

"alpha": config.alpha,

"a": config.a,

"q": config.q,

"delta": config.delta,

"eta": config.eta,

"mu": config.mu,

"gamma": config.gamma,

"B_c": config.B_c,

"B_init_scale": config.B_init_scale,

"scale_epsilon": config.scale_epsilon,

"capacity": config.capacity,

"delay_steps": config.delay_steps,

},

"protocol": {

"T_sim": protocol.T_sim,

"burn_end_time": protocol.burn_end_time,

```
"split_time": protocol.split_time,
"future_duration": protocol.future_duration,
"diagnostic_stride": protocol.diagnostic_stride,
"branches": list(protocol.branches),
},
"n_seeds": len(seeds),
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"engine_source_sha256": engine_source_hash,
"seed_results": {},
}


# Load frozen manifest
with open(frozen_manifest_path, "r") as f:
frozen_manifest = json.load(f)


# Load frozen data
```

```
with np.load(frozen_npz_path, allow_pickle=False) as frozen_data:

frozen_files = dict(frozen_data)



# Load frozen scalar CSV if provided

frozen_scalar_csv: dict[str, dict[str, Any]] = {}

if frozen_csv_path is not None and os.path.exists(frozen_csv_path):

try:

import csv

with open(frozen_csv_path, "r", newline="", encoding="utf-8") as f:

reader = csv.DictReader(f)

for row in reader:

seed_id = row.get("seed", "")

branch = row.get("branch", "")

if seed_id and branch:

key = f"{branch}_seed_{seed_id}"

if key not in frozen_scalar_csv:

frozen_scalar_csv[key] = {}
```

```
for k, v in row.items():

if k not in ("seed", "seed_index", "branch", "rho", "T_window"):

try:

frozen_scalar_csv[key][k] = float(v)

except (ValueError, TypeError):

frozen_scalar_csv[key][k] = v

except Exception as e:

print(f"Warning: Could not load frozen CSV {frozen_csv_path}: {e}")



# Record frozen hashes - use actual file hashes, not canonical JSON hashes

# Record frozen hashes - use actual file hashes, not canonical JSON hashes

results["frozen_manifest_file_sha256"] = file_sha256(frozen_manifest_path)

results["frozen_npz_file_sha256"] = file_sha256(frozen_npz_path)

if frozen_csv_path is not None and os.path.exists(frozen_csv_path):

results["frozen_scalar_csv_file_sha256"] = file_sha256(frozen_csv_path)



# Also record engine source hash
```

```
results["engine_source_sha256"] = engine_source_hash



# Also keep canonical JSON hash for reference

results["frozen_manifest_hash"] = hashlib.sha256(

json.dumps(frozen_manifest, sort_keys=True).encode()

).hexdigest()



# Validate config matches frozen manifest model - use EXACT equality

frozen_model = frozen_manifest["model"]



# Check N

if frozen_model["N"] != config.N:

results["status"] = "failed"

results["error"] = f"N mismatch: config={config.N}, frozen={frozen_model['N']}"

return results
```

```
# Check omega - use exact equality

frozen_omega_arr = np.array(frozen_model["omega"], dtype=float)

config_omega_arr = np.array(list([config.omega](http://config.omega)), dtype=float)

if not np.array_equal(frozen_omega_arr, config_omega_arr):

results["status"] = "failed"

results["error"] = f"omega mismatch: frozen={frozen_model['omega']}, config={list([config.omega]
(http://config.omega))}"

return results



# Check all other model params with EXACT equality (rtol=0, atol=0)

model_checks = [

("dt", frozen_model["dt"], config.dt),

("tau", frozen_model["tau"], config.tau),

("a", frozen_model["a"], config.a),

("q", frozen_model["q"], config.q),

("delta", frozen_model["delta"], [config.delta](http://config.delta)),

("eta", frozen_model["eta"], config.eta),

("mu", frozen_model["mu"], [config.mu](http://config.mu)),
```

```
("gamma", frozen_model["gamma"], config.gamma),

("B_c", frozen_model["B_c"], config.B_c),

("B_init_scale", frozen_model.get("B_init_scale", 0.0), config.B_init_scale),

("T_window", frozen_model["T_window"], config.T_window),

]



for name, frozen_val, config_val in model_checks:

if not np.isclose(frozen_val, config_val, rtol=0.0, atol=0.0):

results["status"] = "failed"

results["error"] = f"{name} mismatch: frozen={frozen_val}, config={config_val}"

return results



# Check derived values

if not np.isclose(frozen_model.get("alpha", 0.0), config.alpha, rtol=0.0, atol=1e-15):

results["status"] = "failed"

results["error"] = f"alpha mismatch: frozen={frozen_model.get('alpha')}, config={config.alpha}"

return results
```

```
# Now run actual equivalence test for each seed

print("Running baseline equivalence test...")

all_passed = True



for seed in seeds:

seed_str = str(seed)

seed_results = {"status": "passed", "checks": {}, "max_diffs": {}}


try:

# Simulate with our engine

split_state, prefix = simulate_prefix(seed, config, protocol)

branch_rngs = make_branch_rngs(seed, branch_order)


# Simulate each branch

engine_results = {}

for branch in protocol.branches:

rng = branch_rngs[branch]
```

```
engine_results[branch] = simulate_future(

split_state.clone(),

prefix,

branch,

config,

protocol,

rng,

)


# Check prefix state - iterate over REQUIRED_PREFIX_KEYS

for key in REQUIRED_PREFIX_KEYS:

engine_val = prefix.get(key)

frozen_key = f"prefix_seed_{seed_str}_{key}"

frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"prefix_{key}_missing"] = True
```

```
all_passed = False

continue


if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"prefix_{key}_exact"] = True

else:

seed_results["checks"][f"prefix_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(frozen_val), 0.0, frozen_val)

seed_results["max_diffs"][f"prefix_{key}"] = float(np.max(np.abs(e - f)))


# Check branch outputs - separate FROZEN vs EXTENDED keys

for branch in protocol.branches:

engine_branch = engine_results[branch]

frozen_prefix = f"{branch}_seed_{seed_str}"
```

```
# === FROZEN MATRIX KEYS: Must exist in historical NPZ ===
for key in FROZEN_BRANCH_MATRIX_KEYS:
engine_val = engine_branch.get(key)
frozen_key = f"{frozen_prefix}_{key}"
frozen_val = frozen_files.get(frozen_key)

if frozen_val is None:
seed_results["status"] = "failed"
seed_results["checks"][f"{branch}_{key}_missing_in_frozen"] = True
all_passed = False
continue

if engine_val is None:
seed_results["status"] = "failed"
seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True
all_passed = False
continue
```

```
if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"{branch}_{key}_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(frozen_val), 0.0, frozen_val)

seed_results["max_diffs"][f"{branch}_{key}"] = float(np.max(np.abs(e - f)))


# === EXTENDED MATRIX KEYS: Validate internally at rho=1 ===

# These do NOT exist in frozen NPZ. At rho=1, both horizon systems

# should produce identical results (1750 and 8750 steps match).

if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):

for key in EXTENDED_BRANCH_MATRIX_KEYS:

engine_val = engine_branch.get(key)

if engine_val is None:
```

```
seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True

all_passed = False

continue


# Map extended key to its frozen equivalent

frozen_key_map = {

"B_mean_late_fixed_physical": "B_mean_late",

"P_full_fixed_physical": "P_full",

"P_immediate_fixed_physical": "P_immediate",

"P_recovery_fixed_physical": "P_recovery",

"P_late_fixed_physical": "P_late",

"B_mean_late_normalized_EMA": "B_mean_late",

"P_full_normalized_EMA": "P_full",

"P_immediate_normalized_EMA": "P_immediate",

"P_recovery_normalized_EMA": "P_recovery",

"P_late_normalized_EMA": "P_late",

}
```

```
frozen_equiv_key = frozen_key_map.get(key)

if frozen_equiv_key:

engine_frozen_equiv = engine_branch.get(frozen_equiv_key)

if engine_frozen_equiv is not None:

if exact_array_equal_with_nan(engine_val, engine_frozen_equiv):

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = True

else:

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(engine_frozen_equiv, np.ndarray):

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(engine_frozen_equiv), 0.0, engine_frozen_equiv)

seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = float(np.max(np.abs(e - f)))


# === SCALAR KEYS ===

# Separate FROZEN vs EXTENDED scalars
```

```
# First, define which scalars are frozen (exist in historical data)

FROZEN_BRANCH_SCALAR_KEYS = (

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",
```

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

)

EXTENDED_BRANCH_SCALAR_KEYS = (

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

)

for key in FROZEN_BRANCH_SCALAR_KEYS:

engine_val = engine_branch.get(key)

```
# Get from CSV first (production stores scalars in CSV)

frozen_val = None

csv_key = f"{branch}_seed_{seed_str}"

if csv_key in frozen_scalar_csv and key in frozen_scalar_csv[csv_key]:

frozen_val = frozen_scalar_csv[csv_key][key]

else:

# Fallback to NPZ

frozen_key = f"{frozen_prefix}_{key}"

frozen_val = frozen_files.get(frozen_key)


# FROZEN scalars MUST exist in historical data

if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_frozen"] = True

all_passed = False

continue


if engine_val is None:
```

```
seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True

all_passed = False

continue


# For scalars, compare directly

if isinstance(engine_val, (int, float, np.number)):

engine_is_nan = bool(np.isnan(engine_val))

frozen_is_nan = bool(np.isnan(frozen_val)) if isinstance(frozen_val, (int, float, np.number)) else False


if engine_is_nan and frozen_is_nan:

seed_results["checks"][f"{branch}_{key}_exact"] = True

elif np.isclose(engine_val, frozen_val, rtol=0.0, atol=0.0):

seed_results["checks"][f"{branch}_{key}_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False
```

```
seed_results["max_diffs"][f"{branch}_{key}"] = float(abs(engine_val - frozen_val)) if not (engine_is_nan or frozen_is_nan) else float('nan')


# === EXTENDED SCALAR KEYS: Validate internally at rho=1 ===

if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):

for key in EXTENDED_BRANCH_SCALAR_KEYS:

engine_val = engine_branch.get(key)

if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True

all_passed = False

continue


# Map to frozen equivalent

frozen_scalar_map = {

"P_mean_late_fixed_physical": "P_mean_late",

"Weighted_similarity_late_fixed_physical": "Weighted_similarity_late",

"Retention_contrast_late_fixed_physical": "Retention_contrast_late",
```

"Corr_Rpast_Plate_fixed_physical": "Corr_Rpast_Plate",

"r_mean_late_fixed_physical": "r_mean_late",

"P_mean_late_normalized_EMA": "P_mean_late",

"Weighted_similarity_late_normalized_EMA": "Weighted_similarity_late",

"Retention_contrast_late_normalized_EMA": "Retention_contrast_late",

"Corr_Rpast_Plate_normalized_EMA": "Corr_Rpast_Plate",

"r_mean_late_normalized_EMA": "r_mean_late",

}

frozen_equiv = frozen_scalar_map.get(key)

if frozen_equiv:

Get the frozen equivalent value from CSV

csv_key = f"{branch}_seed_{seed_str}"

if csv_key in frozen_scalar_csv and frozen_equiv in frozen_scalar_csv[csv_key]:

frozen_equiv_val = frozen_scalar_csv[csv_key][frozen_equiv]

if isinstance(engine_val, (int, float, np.number)) and isinstance(frozen_equiv_val, (int, float, np.number)):

engine_is_nan = bool(np.isnan(engine_val))

frozen_is_nan = bool(np.isnan(frozen_equiv_val))

```
if engine_is_nan and frozen_is_nan:

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = True

elif np.isclose(engine_val, frozen_equiv_val, rtol=0.0, atol=0.0):

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = True

else:

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = False

seed_results["status"] = "failed"

all_passed = False

seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = float(abs(engine_val - frozen_equiv_val))


except Exception as e:

seed_results["status"] = "failed"

seed_results["error"] = str(e)

import traceback

seed_results["traceback"] = traceback.format_exc()

all_passed = False
```

```
results["seed_results"][seed_str] = seed_results

print(f" Seed {seed_str}: {seed_results['status']}")



if all_passed:

results["status"] = "passed"

print("Baseline equivalence: PASSED")

else:

results["status"] = "failed"

print("Baseline equivalence: FAILED")

# Print details of failures

for seed_str, seed_results in results["seed_results"].items():

if seed_results.get("status") != "passed":

print(f" Seed {seed_str} failures:")

for check, val in seed_results.get("checks", {}).items():

if not val:

print(f" CHECK FAILED: {check}")

for key, val in seed_results.get("max_diffs", {}).items():
```

```
print(f" MAX DIFF: {key} = {val}")



return results

# ============================================================================
# Main API
# ============================================================================

def run_seed(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

branch_order: tuple[str, ...],

) -> dict[str, Any]:

text

"""Run a single seed through prefix + all branches.



Returns complete seed results with all branches.

"""
```

```
# Simulate prefix

split_state, prefix = simulate_prefix(seed, config, protocol)



# Create branch RNGs using frozen algorithm

branch_rngs = make_branch_rngs(seed, branch_order)



# Use branch_order to determine which branches to run

# All branches in branch_order must have RNGs

branch_results = {}

for branch in branch_order:

if branch not in branch_rngs:

raise ValueError(

f"Branch '{branch}' not found in branch_rngs. "

f"All branches must be registered in branch_order for explicit RNG namespace. "

f"Registered branches: {list(branch_rngs.keys())}"

)

rng = branch_rngs[branch]
```

```
branch_results[branch] = simulate_future(

split_state.clone(),

prefix,

branch,

config,

protocol,

rng,

)



return {

"seed": int(seed),

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"T_window": config.T_window,
```

"alpha": config.alpha,

"mean_omega": config.mean_omega,

"T_star": config.T_star,

},

"protocol": {

"T_sim": protocol.T_sim,

"burn_end_time": protocol.burn_end_time,

"split_time": protocol.split_time,

"future_duration": protocol.future_duration,

},

"prefix": prefix,

"branches": branch_results,

}

if name == "main":

text

Quick validation

config = DEFAULT_MODEL_CONFIG

config.validate()

```
protocol = DEFAULT_PROTOCOL_CONFIG

protocol.validate(config)



print(f"Engine v1 loaded successfully")

print(f" N={config.N}, dt={config.dt}, tau={config.tau}")

print(f" T_window={config.T_window}, alpha={config.alpha:.6f}")

print(f" mean_omega={config.mean_omega:.6f}, T_*={config.T_star:.6f}")

print(f" delay_steps={config.delay_steps}")

print(f" Protocol: T_sim={protocol.T_sim}, burn={protocol.burn_end_time}, "

f"split={protocol.split_time}, future={protocol.future_duration}")
```

Ignoring the Markdown/HTML corruption, I compared this engine against the frozen . The numerical step map and RNG construction are very close, but I found one material baseline-gate defect that was missed previously.
Verdict

Core simulation: strong candidate, approximately 9–9.5/10.
Current baseline-equivalence gate: not yet sufficient to certify exact frozen reproduction.
Recommendation: compile tests are fine, but do not accept or freeze a baseline certificate until the issues below are corrected.

No finite replay has been run here; this is a static source comparison.
1. Blocker: eight historical scalar fields are not tested

The frozen production CSV contains 42 branch scalar fields. The local FROZEN_BRANCH_SCALAR_KEYS inside run_baseline_equivalence() checks only 34.

These eight frozen fields are omitted:

Python

```
Time_to_50pct_continue_final_density
Time_to_90pct_continue_final_density
Time_to_50pct_continue_late_similarity
Time_to_90pct_continue_late_similarity
Time_to_similarity_below_0_9
Time_to_similarity_below_0_5
Late_persistence_ratio_to_continue
Late_similarity_ratio_to_continue
```

Worse, simulate_future() initializes all eight to NaN, while frozen production calculates them in run_seed() after all branches are available. Because the baseline schema excludes them, the certificate could report “passed” while those historical outputs have not been reproduced.
Required correction

Add the frozen time_to_at_least() and time_to_below() helpers and a cross-branch finalizer equivalent to the frozen postprocessing:

Python

```
def finalize_historical_branch_metrics(
branch_results: dict[str, dict[str, Any]],
) -> None:
continue_result = branch_results["continue"]

reference_density = float(continue_result["D_B_final"])
reference_late_similarity = float(
continue_result["Weighted_similarity_late"]
)
reference_late_persistence = float(
continue_result["P_mean_late"]
)

for result in branch_results.values():
```

Reproduce the frozen calculations exactly.
...

Call it:

in run_baseline_equivalence() after all seven branches have run;
in run_seed() after its requested branches have run, provided continue is present.

Then place the full 42-key tuple at module scope and compare every key.

Until this is done, the requested statement

all historical scalar fields exact

would not be true.
2. Blocker: scalar comparison can silently skip a check

The current logic only compares when:

Python

isinstance(engine_val, (int, float, np.number))

If the NPZ fallback returns a zero-dimensional or one-element np.ndarray, that condition is false and the code reaches the end without marking either success or failure.

The historical scalar authority is the CSV, so the cleanest policy is:

require each historical scalar in the CSV;
do not silently fall back to the matrix NPZ;
normalize scalar-like values explicitly;
fail on non-scalar values.

For example:

Python

```
def normalize_scalar(value: Any) -> float:
arr = np.asarray(value)
if arr.ndim != 0:
raise TypeError(f"Expected scalar, got shape {arr.shape}")
return float(arr.item())
```

Use a dedicated NaN-aware exact scalar comparison rather than a type-conditional branch.

Also validate:

no duplicate (seed, branch) rows;
exactly 40 × 7 expected frozen rows;
all 42 required columns exist.

3. Blocker: missing checks are recorded as True

Several failure paths do this:

Python

```
seed_results["checks"][f"{branch}_{key}_missing_in_frozen"] = True
```

The seed is marked failed, but the detailed check itself appears true. The failure printer only prints checks whose values are false, so the actual missing field is hidden.

Use positive check names:

Python

```
seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = False
```

The same applies to missing prefix arrays, engine outputs, and extended keys.

This matters because the saved certificate is intended to be auditable, not merely a top-level Boolean.
4. Extended scalar validation is not actually internal

The matrices are correctly compared as:

text

extended engine field versus legacy engine field

But the extended scalars are compared against the frozen CSV. That is transitively adequate only if every historical check was definitely executed—which currently is not guaranteed.

At ρ=1ρ=1, compare directly:

Python

```
exact_scalar_equal(
engine_branch[extended_key],
engine_branch[legacy_key],
)
```

Then separately compare the legacy key against the frozen CSV.

Also expand the internal checks beyond only the five late quantities. The engine creates additional extensions for:

full/immediate/recovery/late P_mean;
selected retention and emergence;
full and late correlations;
both explicit horizon aliases;
horizon metadata and effective step boundaries.

If the certificate says all fixed/normalized baseline extensions internally exact, all such fields need validation.
5. run_seed() confuses RNG namespace with execution set

For this study:

branch_order should define the full frozen seven-branch RNG namespace;

protocol.branches should define the branches actually executed.

The current run_seed() loops over branch_order. This has two undesirable alternatives:

Passing all seven names preserves RNG positions but runs unwanted branches.
Passing only the four study branches changes future_shuffle from frozen child index 6 to child index 3.

The correct structure is:

Python

```
branch_rngs = make_branch_rngs(seed, branch_order)

for branch in protocol.branches:
if branch not in branch_rngs:
raise ValueError(...)
branch_results[branch] = simulate_future(
split_state,
prefix,
branch,
config,
protocol,
branch_rngs[branch],
)
```

Validate uniqueness and:

Python

```
set(protocol.branches) <= set(branch_order)
```

The sensitivity driver reportedly bypasses run_seed(), so this does not presently alter that driver, but it must be fixed before freezing the engine as a reusable API.
6. Unknown branch names silently behave like continue

A typo such as "reset_EMA" currently receives no intervention and no shuffle, effectively becoming continue.

Add a strict allowlist:

Python

```
KNOWN_BRANCHES = {
"continue",
"reset_ema",
"reset_B",
"reset_B_ema",
"B_carrier_only",
"full_reset",
"future_shuffle",
}
```

Reject unknown names before simulation.
7. Important performance corrections before the 10.88-million-step baseline

The mandatory 40-seed, seven-branch baseline invokes approximately:
$40(48{,}000+7\cdot 32{,}000)=10.88$ million
$40(48{,}000+7\cdot 32{,}000)=10.88$ million

calls to advance_state(). The full grid invokes approximately 35.2 million.

Three avoidable operations occur on every step:
compute_edges(N) on every step

advance_state() calls np.triu_indices() every time, even when shuffling is disabled. Cache it by NN, preferably with read-only returned arrays.
Reconstructing omega_arr twice per step

Both compute_R() and advance_state() call:

Python

```
np.asarray(config.omega, dtype=float)
```

Cache a read-only array per omega tuple.
list.pop(0) for delay history

Frozen production uses:

Python

deque(..., maxlen=delay_steps + 1)

The engine uses a list and shifts it every step. Restore a bounded deque; it is behaviorally equivalent here and matches production more directly.

These are not mathematical defects, but they should be fixed before the successful baseline, so the frozen engine is also the version used for the full grid.
8. Validation hardening

Add the following before simulation:
DynamicState.validate()

It currently derives self.N from theta, so it never verifies the state dimension against the model.

Add:

Python

if self.N != config.N:
raise RuntimeError(...)

Also validate:

every history vector has shape (config.N,);
all required arrays are finite;
coherence_acc has complex-compatible shape;
history length is exactly delay_steps + 1.

ModelConfig.validate()

Require:

Python

delay_steps * dt == tau

within a strict absolute tolerance. The frozen driver explicitly requires integral delay indexing.

Also check all omega and scalar parameters are finite.
ProtocolConfig.validate()

Require the exact consistency relation:

Python

split_time + future_duration == T_sim

and equivalently:

Python

hist_end(model) + future_steps(model) == n_steps(model)

Currently T_sim is mostly metadata; inconsistent split/future values could still be executed.

Use strict rtol=0 compatibility checks rather than relying on the default np.isclose(..., atol=1e-8).
9. Preserve the frozen alpha construction exactly

Frozen production uses:

Python

ALPHA = float(np.exp(-DT / T_WINDOW))

The engine uses math.exp. On the present platform these likely produce the same binary float, but exact-source fidelity should use:

Python

return float(np.exp(-self.dt / self.T_window))

This is a source-level consistency correction, not evidence of an observed numerical mismatch.
10. Baseline cohort and authority checks

A mandatory “passed” gate should establish that it ran the intended authority set, not merely whatever subset the caller supplied.

Require:

Python

seeds == frozen_manifest["randomization"]["paired_seeds"]
branch_order == tuple(frozen_manifest["protocol"]["branches"])
protocol.branches == tuple(frozen_manifest["protocol"]["branches"])

and validate the manifest protocol fields:

text

n_steps
hist_end
future_steps
window_steps
recovery_end
diagnostic_stride

If subset testing is useful, label it explicitly as a smoke/subset result, never "passed" for the mandatory baseline.

Recording input hashes is good. For a freezeable result, the driver should also verify them against the previously frozen lineage authority rather than merely recording whichever files were supplied.
What is already strong

The following look correctly aligned with frozen production:

prefix SeedSequence use passes the child object directly to default_rng;
branch RNG namespace construction matches the frozen hierarchy;
phase, EMA, weight update, positivity, symmetrization, and global scaling order;
fixed +1e-12 projection safeguard;
delayed-history indexing;
physical versus shuffled resonance separation;
threshold uses B_c explicitly for Jaccard_final;
fixed-physical and normalized-EMA accumulators;
NaN-aware exact array equality;
actual manifest/NPZ/CSV file hashes are distinguished from canonical JSON hash;
at ρ=1ρ=1, rounded normalized boundaries are 1750 and 8750 steps, matching the fixed physical boundaries.

The duplicate “Record frozen hashes” comment and unused imports are cosmetic.
Recommended sequence

Fix the eight omitted historical metrics and all certificate-integrity issues.
Fix run_seed() namespace/execution separation and reject unknown branches.
Apply the performance and validation changes.
Compile all three scripts.
Run engine import/self-test.
Run a clearly labeled one-seed smoke equivalence test if desired.
Run the mandatory fresh 40-seed, seven-branch baseline-only gate.
Inspect the saved certificate for:
40/40 seeds;
7/7 branches per seed;
all six prefix arrays;
all eleven historical matrix/trajectory arrays;
all 42 historical scalar fields;
all declared dual-horizon internal identities.
Only then launch the five-level grid.

So my current judgment is: the dynamics are probably ready to test, but the gate is not yet trustworthy enough to authorize the full study.

Evaluate this before our first run:

#!/usr/bin/env python3

"""AIEF Experiment Engine v1 - Parameterized simulation kernel.

This is a reusable, validated simulation engine that reproduces the frozen

AIEF-1.4 production run exactly and supports parameterized studies.

FROZEN PRODUCTION VALUES (from AIEF_1_4_memory_ablation_updated_REAL.py):

N = 12

DT = 0.01

TAU = 0.5

OMEGA = [1.00, 1.02, 1.50, 2.00, 2.01, 1.01, 1.03, 1.51, 2.02, 2.00, 1.00, 1.04]

A = 1.0

Q = 1.5

DELTA = 0.05

ETA = 0.04

GAMMA = 0.015

MU = 0.03

T_WINDOW = 25.0 / mean(OMEGA) = 17.5

B_C = 0.12

B_INIT_SCALE = 0.0005

PROTOCOL (from frozen run):

T_SIM = 800.0

N_STEPS = int(round(T_SIM / DT)) = 80000

BURN_END = int(0.20 * N_STEPS) = 16000 steps = 160.0 time units

HIST_END = int(0.60 * N_STEPS) = 48000 steps = 480.0 time units

FUTURE_STEPS = N_STEPS - HIST_END = 32000 steps = 320.0 time units

DELAY_STEPS = int(round(TAU / DT)) = 50

WINDOW_STEPS = int(round(T_WINDOW / DT)) = 1750

RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS) = 8750

Architecture:

ModelConfig: frozen dataclass for all model parameters

ProtocolConfig: frozen dataclass for protocol parameters

DynamicState: state container (immutable where possible)

Core functions: initialize, advance, simulate, interventions

Validation: baseline equivalence test

NOTE: This is a candidate reference implementation. After passing 40/40 baseline

text

equivalence tests, it will be frozen as the reference implementation.

"""

from future import annotations

import copy

import hashlib

import json

import math

import os

import tempfile

import time

from dataclasses import dataclass, field

from pathlib import Path

from typing import Any

import numpy as np
==
Frozen Production Constants (DO NOT CHANGE)
==

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005

FROZEN_ZERO_TOL = 1e-12

FROZEN_PROJECTION_NORM_TOL = 1e-14

FROZEN_GAP_TOL = 1e-9
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA)) # 1.428333...

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.502917152858814

FROZEN_T_WINDOW = FROZEN_T_STAR # 17.502917152858814 (exact, NOT rounded)

FROZEN_ALPHA = float(np.exp(-FROZEN_DT / FROZEN_T_WINDOW))

FROZEN_CAPACITY = (FROZEN_A - FROZEN_DELTA) / FROZEN_Q
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_WINDOW / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
Fixed physical horizon boundaries (in time units, NOT T_window dependent)
These are the PRIMARY estimand horizons

FIXED_PHYSICAL_IMMEDIATE_END = 17.5

FIXED_PHYSICAL_RECOVERY_END = 87.5

FIXED_PHYSICAL_FUTURE_END = 320.0
Derived fixed physical horizon steps

FIXED_PHYSICAL_IMMEDIATE_STEPS = int(round(FIXED_PHYSICAL_IMMEDIATE_END / FROZEN_DT)) # 1750

FIXED_PHYSICAL_RECOVERY_STEPS = int(round(FIXED_PHYSICAL_RECOVERY_END / FROZEN_DT)) # 8750
==========
Model Configuration
==========

@dataclass(frozen=True)

class ModelConfig:

text

"""Immutable model configuration.

All model-changing quantities must come from here. No hidden module globals.

"""

Core parameters

N: int

dt: float

tau: float

omega: tuple[float, ...] # Immutable tuple

Coupling parameters

a: float

q: float

delta: float

Dynamics parameters

eta: float

mu: float

gamma: float

EMA/threshold parameters

T_window: float

B_c: float

B_init_scale: float

scale_epsilon: float = 1e-12

```
# Derived properties

@property

def mean_omega(self) -&gt; float:

"""Mean of omega frequencies."""

return float(np.mean([self.omega](http://self.omega)))



@property

def T_star(self) -&gt; float:

"""T_* = 25 / mean(omega)"""

return 25.0 / self.mean_omega



@property

def alpha(self) -&gt; float:

"""EMA smoothing factor: exp(-dt / T_window)"""

return float(np.exp(-self.dt / self.T_window))



@property
```

```
def capacity(self) -&gt; float:

"""Projection capacity: (a - delta) / q"""

return (self.a - [self.delta](http://self.delta)) / self.q



@property

def delay_steps(self) -&gt; int:

"""Delay steps: round(tau / dt)"""

return int(round(self.tau / self.dt))



def validate(self) -&gt; None:

"""Validate model configuration with strict consistency checks.


Uses rtol=0, atol=0 for exact compatibility with frozen production.

"""

if self.N &lt;= 0:

raise ValueError("N must be positive")

if self.dt &lt;= 0:

raise ValueError("dt must be positive")
```

if self.tau <= 0:

raise ValueError("tau must be positive")

if len(self.omega) != self.N:

raise ValueError(f"omega must have {self.N} elements")

if self.a <= 0 or self.q <= 0 or self.delta < 0:

raise ValueError("a, q must be positive; delta must be nonnegative")

if self.eta < 0 or self.mu < 0 or self.gamma < 0:

raise ValueError("eta, mu, gamma must be nonnegative")

if self.T_window <= 0:

raise ValueError("T_window must be positive")

if self.B_c < 0:

raise ValueError("B_c must be nonnegative")

if self.B_init_scale < 0:

raise ValueError("B_init_scale must be nonnegative")

if self.scale_epsilon <= 0:

raise ValueError("scale_epsilon must be positive")

```
# Validate all omega values are finite

if not np.all(np.isfinite([self.omega](http://self.omega))):

raise ValueError("omega must contain only finite values")


# Validate derived values

if not (0 &lt; self.alpha &lt;= 1):

raise ValueError(f"alpha={self.alpha} must be in (0, 1]")

if self.capacity &lt; 0:

raise ValueError(f"capacity={self.capacity} must be nonnegative")


# Strict consistency: delay_steps * dt must equal tau (exact)

if not np.isclose(self.delay_steps * self.dt, self.tau, rtol=0.0, atol=0.0):

raise ValueError(

f"delay_steps *dt ({self.delay_steps}* {self.dt} = {self.delay_steps * self.dt}) "

f"must equal tau ({self.tau}) exactly"

)

@dataclass(frozen=True)

class ProtocolConfig:
```

text

"""Protocol configuration for experiment execution."""

Simulation time parameters (in time units, not steps)

T_sim: float

burn_end_time: float

split_time: float

future_duration: float

Diagnostics

diagnostic_stride: int

Branches to simulate

branches: tuple[str, ...]

def n_steps(self, model: ModelConfig) -> int:

"""Total steps."""

return int(round(self.T_sim / model.dt))

def burn_end(self, model: ModelConfig) -> int:

"""Burn-in end step."""

return int(round(self.burn_end_time / model.dt))

def hist_end(self, model: ModelConfig) -> int:

"""History end step (split point)."""

return int(round(self.split_time / model.dt))

def future_steps(self, model: ModelConfig) -> int:

"""Future simulation steps."""

return int(round(self.future_duration / model.dt))

def validate(self, model: ModelConfig) -> None:

"""Validate protocol against model with strict consistency checks.

Uses rtol=0, atol=0 for exact compatibility with frozen production.

"""

if self.T_sim <= 0:

raise ValueError("T_sim must be positive")

if not (0 < self.burn_end_time < self.split_time < self.T_sim):

raise ValueError("Invalid time parameters: must have 0 < burn_end < split < T_sim")

if self.future_duration <= 0:

raise ValueError("future_duration must be positive")

if self.diagnostic_stride <= 0:

raise ValueError("diagnostic_stride must be positive")

Strict consistency: split_time + future_duration must equal T_sim

if not np.isclose(self.split_time + self.future_duration, self.T_sim, rtol=0.0, atol=0.0):

raise ValueError(

f"split_time ({self.split_time}) + future_duration ({self.future_duration}) "

f"= {self.split_time + self.future_duration} must equal T_sim ({self.T_sim}) exactly"

)

Validate that all time parameters are compatible with dt (exact divisibility)

if not np.isclose(self.T_sim / model.dt, round(self.T_sim / model.dt), rtol=0.0, atol=0.0):

raise ValueError(f"T_sim ({self.T_sim}) must be divisible by dt ({model.dt}) exactly")

if not np.isclose(self.burn_end_time / model.dt, round(self.burn_end_time / model.dt), rtol=0.0, atol=0.0):

raise ValueError(f"burn_end_time ({self.burn_end_time}) must be divisible by dt ({model.dt}) exactly")

if not np.isclose(self.split_time / model.dt, round(self.split_time / model.dt), rtol=0.0, atol=0.0):

raise ValueError(f"split_time ({self.split_time}) must be divisible by dt ({model.dt}) exactly")

if not np.isclose(self.future_duration / model.dt, round(self.future_duration / model.dt), rtol=0.0, atol=0.0):

raise ValueError(f"future_duration ({self.future_duration}) must be divisible by dt ({model.dt}) exactly")

Default frozen production configuration
NOTE: T_window must be the EXACT value 25.0/mean(OMEGA), not rounded

DEFAULT_MODEL_CONFIG = ModelConfig(

text

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=FROZEN_T_WINDOW, # 17.502917152858814 (exact)

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=FROZEN_ZERO_TOL,

)
Default protocol matching frozen run

DEFAULT_PROTOCOL_CONFIG = ProtocolConfig(

text

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=("continue", "reset_ema", "reset_B", "reset_B_ema", "B_carrier_only", "full_reset", "future_shuffle"),

)
==========
Cached Utility Values
==========
Cache for compute_edges to avoid recomputing on every step

EDGESCACHE: dict[int, tuple[np.ndarray, np.ndarray]] = {}
Cache for omega array conversion

OMEGAARRAY_CACHE: dict[tuple[float, ...], np.ndarray] = {}
==========
Utility Functions
==========

def compute_edges(N: int) -> tuple[np.ndarray, np.ndarray]:

text

"""Get upper triangle indices for edges.

Cached to avoid recomputing on every step.

"""

if N not in *EDGES*CACHE:

*EDGES*CACHE[N] = np.triu_indices(N, k=1)

return *EDGES*CACHE[N]

def get_omega_array(omega_tuple: tuple[float, ...]) -> np.ndarray:

text

"""Get omega as numpy array with caching.

Cached to avoid recomputing on every step.

"""

if omega_tuple not in *OMEGA*ARRAY_CACHE:

*OMEGA*ARRAY_CACHE[omega_tuple] = np.asarray(omega_tuple, dtype=float)

return *OMEGA*ARRAY_CACHE[omega_tuple]

def validate_B_matrix(B: np.ndarray, N: int, eps: float = 1e-12) -> None:

text

"""Validate B matrix: symmetric, nonnegative, zero diagonal."""

if B.shape != (N, N):

raise ValueError(f"B shape {B.shape} != ({N}, {N})")

if not np.allclose(B, B.T, atol=eps):

raise ValueError("B is not symmetric")

```
if not np.allclose(np.diag(B), 0, atol=eps):

raise ValueError("B diagonal is not zero")

if np.any(B &lt; -eps):

raise ValueError("B has negative entries")

def file_sha256(path: str | os.PathLike[str]) -> str:

text

"""Compute SHA-256 hash of a file."""

digest = hashlib.sha256()

with open(path, "rb") as handle:

for block in iter(lambda: [handle.read](http://handle.read)(1024 * 1024), b""):

digest.update(block)

return digest.hexdigest()

def array_sha256(arr: np.ndarray) -> str:

text

"""Compute SHA-256 hash of a numpy array."""

digest = hashlib.sha256()

shape_bytes = np.asarray(arr.shape, dtype=np.int64).tobytes()
```

```
digest.update(shape_bytes)

digest.update(arr.dtype.str.encode())

contiguous = np.ascontiguousarray(arr)

if contiguous.size &gt; 0:

digest.update(contiguous.tobytes())

return digest.hexdigest()

def atomic_write_text(path: Path, text: str) -> None:

text

"""Write text atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{[path.name](http://path.name)}.",
suffix=".tmp")

try:

with os.fdopen(fd, "w", encoding="utf-8") as handle:

handle.write(text)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))
```

```
finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

def atomic_write_json(path: Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically with fsync."""

text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"

atomic_write_text(path, text)

def atomic_write_npz(path: Path, **kwargs: Any) -> None:

text

"""Write NPZ atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{[path.name](http://path.name)}.",
suffix=".tmp.npz")

try:

with os.fdopen(fd, "wb") as handle:

np.savez(handle, **kwargs)

handle.flush()
```

```
os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))

finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

================================================================================
RNG Utilities (matching frozen production exactly)
================================================================================

def make_seed_sequence(seed: int, child_index: int) -> int:

text

"""Generate a child seed from SeedSequence(seed).spawn(...)[child_index].



This matches the frozen production RNG hierarchy exactly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

children = root.spawn(child_index + 1)

return int(children[child_index].generate_state(1, dtype=np.uint64)[0])
```

```
def make_prefix_rng(seed: int) -> np.random.Generator:

text

"""Create prefix RNG matching frozen production exactly.



Frozen production does:

root = np.random.SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = np.random.default_rng(init_sequence)



NOTE: default_rng(SeedSequence_object) is NOT the same as

default_rng(SeedSequence_object.generate_state(...))

The production uses the SeedSequence object directly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

return np.random.default_rng(init_sequence)
```

```
def make_branch_rngs(seed: int, branch_order: tuple[str, ...]) -> dict[str, np.random.Generator]:

text

"""Create branch RNGs matching frozen production algorithm.



Frozen: *, branch*root = SeedSequence(seed).spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))

branch_rng_seeds = {branch: int(sequence.generate_state(1, dtype=np.uint64)[0])

for branch, sequence in zip(branch_order, branch_sequences)}

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

*, branch*root = root.spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))



return {

branch: np.random.default_rng(int(sequence.generate_state(1, dtype=np.uint64)[0]))

for branch, sequence in zip(branch_order, branch_sequences)
```

```
}

================================================================================
Projection Accumulator (matching production)
================================================================================

@dataclass

class ProjectionTotals:

text

"""Accumulator for projection diagnostics - exact match to production."""

steps: int = 0

violations: int = 0

active: int = 0

norm_sum: float = 0.0

norm_max: float = 0.0

relative_state_sum: float = 0.0

relative_state_max: float = 0.0

relative_update_sum: float = 0.0

relative_update_max: float = 0.0
```

```
g_min: float = np.inf

g_min_pre: float = np.inf

def update(self, info: dict[str, Any]) -&gt; None:

"""Update accumulator with step info."""

self.steps += 1

self.violations += int(info["pre_violation"])

self.g_min = min(self.g_min, float(info["g_min"]))

self.g_min_pre = min(self.g_min_pre, float(info["g_min_pre"]))

if info["projection_active"]:

self.active += 1

norm = float(info["projection_norm"])

rel_state = float(info["projection_relative_state"])

rel_update = float(info["projection_relative_update"])

self.norm_sum += norm

self.norm_max = max(self.norm_max, norm)

self.relative_state_sum += rel_state

self.relative_state_max = max(self.relative_state_max, rel_state)

self.relative_update_sum += rel_update
```

self.relative_update_max = max(self.relative_update_max, rel_update)

def metrics(self) -> dict[str, float]:

"""Compute final metrics from accumulated values."""

active_denominator = max(self.active, 1)

return {

"G_min_future": float(self.g_min),

"G_min_pre_future": float(self.g_min_pre),

"Pre_gap_violation_fraction": float(self.violations / self.steps),

"Projection_active_fraction": float(self.active / self.steps),

"Projection_active_given_violation": float(self.active / self.violations)

if self.violations > 0

else float('nan'),

"Projection_norm_mean_active": float(

self.norm_sum / active_denominator

),

"Projection_norm_max": float(self.norm_max),

"Projection_relative_state_mean_active": float(

self.relative_state_sum / active_denominator

),

"Projection_relative_state_max": float(self.relative_state_max),

"Projection_relative_update_mean_active": float(

self.relative_update_sum / active_denominator

),

"Projection_relative_update_max": float(self.relative_update_max),

}

==========
Dynamic State
==========

class DynamicState:

text

"""State container for AIEF-1.4 dynamics.

Matches the frozen production implementation exactly.

"""

```
def **init**(

self,

theta: np.ndarray,

B: np.ndarray,

coherence_acc: np.ndarray,

history: list[np.ndarray] | np.ndarray,

):

self.theta = np.asarray(theta, dtype=float).copy()

self.B = np.asarray(B, dtype=float).copy()

self.coherence_acc = np.asarray(coherence_acc, dtype=complex).copy()


# Handle history input - ensure it's a list of arrays

if isinstance(history, np.ndarray):

# If it's a 2D array, convert to list of 1D arrays

self.history = [np.asarray(h, dtype=float).copy() for h in history]

else:

self.history = [np.asarray(h, dtype=float).copy() for h in history]
```

def clone(self) -> "DynamicState":

"""Create a deep copy of the state."""

return DynamicState(

theta=self.theta.copy(),

B=self.B.copy(),

coherence_acc=self.coherence_acc.copy(),

history=[h.copy() for h in self.history],

)

@property

def N(self) -> int:

return len(self.theta)

def validate(self, config: ModelConfig, step: int | None = None) -> None:

"""Validate state consistency.

Checks:

- State dimension matches config.N

- History length is exactly delay_steps + 1

- All arrays have correct shapes

- B matrix is symmetric, zero-diagonal, non-negative

- All history vectors have shape (N,)

- All arrays are finite

"""

Check state dimension matches config

if self.N != config.N:

raise RuntimeError(

f"State N ({self.N}) does not match config N ({config.N})"

)

Check history length

expected_history_len = config.delay_steps + 1

if len(self.history) != expected_history_len:

raise RuntimeError(

```
f"History length {len(self.history)} != expected {expected_history_len} "

f"(delay_steps={config.delay_steps}) at step {step}"

)


# Check all history vectors have correct shape

for i, h in enumerate(self.history):

if h.shape != (self.N,):

raise RuntimeError(

f"History[{i}] shape {h.shape} != ({self.N},)"

)


# Check theta shape

if self.theta.shape != (self.N,):

raise RuntimeError(f"theta shape {self.theta.shape} != ({self.N},)")


# Check B shape

if self.B.shape != (self.N, self.N):

raise RuntimeError(f"B shape {self.B.shape} != ({self.N}, {self.N})")
```

```
# Check coherence_acc shape

if self.coherence_acc.shape != (self.N, self.N):

raise RuntimeError(f"coherence_acc shape {self.coherence_acc.shape} != ({self.N}, {self.N})")


# Check all arrays are finite

if not np.all(np.isfinite(self.theta)):

raise RuntimeError("theta contains non-finite values")

if not np.all(np.isfinite(self.B)):

raise RuntimeError("B contains non-finite values")

if not np.all(np.isfinite(self.coherence_acc)):

raise RuntimeError("coherence_acc contains non-finite values")

for i, h in enumerate(self.history):

if not np.all(np.isfinite(h)):

raise RuntimeError(f"History[{i}] contains non-finite values")


# Check B symmetry

if not np.allclose(self.B, self.B.T, atol=1e-12):

raise RuntimeError("B matrix is not symmetric")
```

```
# Check B diagonal

if not np.allclose(np.diag(self.B), 0, atol=1e-12):

raise RuntimeError("B matrix diagonal is not zero")



# Check B non-negativity

if np.any(self.B &lt; -1e-12):

raise RuntimeError("B matrix has negative entries")

============================================================================
====
Simulation Core (exact reproduction of frozen production)
============================================================================
====

def initialize_state(

text

seed: int,

config: ModelConfig,

) -> DynamicState:

text

"""Initialize state from seed using frozen algorithm exactly.
```

```
Matches production: simulate_prefix() initialization.

Uses: root = SeedSequence(seed); init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence)

"""

N = config.N

rng = make_prefix_rng(seed)



# Initialize theta: uniform in [0, 2*pi) - matching production

theta = rng.random(N) *2.0* np.pi



# Initialize B: small random symmetric matrix

B = config.B_init_scale * rng.random((N, N))

B = 0.5 * (B + B.T)

np.fill_diagonal(B, 0.0)



# Initialize coherence accumulator

coherence_acc = np.zeros((N, N), dtype=complex)
```

```
# Initialize history: DELAY_STEPS + 1 copies of theta

history = [theta.copy() for *in range(config.delay*steps + 1)]



return DynamicState(

theta=theta,

B=B,

coherence_acc=coherence_acc,

history=history,

)

def compute_R(

text

state: DynamicState,

config: ModelConfig,

) -> np.ndarray:

text

"""Compute coherence matrix R from current state.
```

Exact reproduction of production compute_R().

"""

N = state.N

delayed = state.history[0]

Convert omega to numpy array for indexing (cached)

omega_arr = get_omega_array(config.omega)

dphi = (

state.theta[:, None]

- delayed[None, :]

- ((omega_arr[:, None] + omega_arr[None, :]) / 2.0) * config.tau

)

z_inst = np.exp(1j * dphi)

```
# Update EMA

state.coherence_acc[:] = (

config.alpha *state.coherence_acc + (1.0 - config.alpha)* z_inst

)



# Compute magnitude and symmetrize

result = np.abs(state.coherence_acc)

result = 0.5 * (result + result.T)

np.fill_diagonal(result, 0.0)



return result

def advance_state(

text

state: DynamicState,

config: ModelConfig,

rng: np.random.Generator,

shuffle_driver: bool = False,
```

) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:

text

"""Advance state by one time step.

Exact reproduction of production advance_state().

Returns (R_physical, R_driver, info).

Production also returns projection diagnostics:

- projection_active: whether correction was applied

- projection_norm: norm of correction

- projection_relative_state: correction_norm / ||B_unprojected||

- projection_relative_update: correction_norm / ||B_unprojected - B_previous||

"""

N = state.N

edges = compute_edges(N)

Store previous B for projection

```
B_previous = state.B.copy()



# Compute R matrices

R_physical = compute_R(state, config)

R_driver = R_physical



if shuffle_driver:

values = R_physical[edges].copy()

rng.shuffle(values)

R_driver = np.zeros_like(R_physical)

R_driver[edges] = values

R_driver[(edges[1], edges[0])] = values



# Update B

B_unprojected = state.B + config.dt * (

(config.eta + [config.mu](http://config.mu)) * R_driver

- [config.mu](http://config.mu)
```

```
- config.gamma * state.B

)

B_unprojected = np.maximum(B_unprojected, 0.0)

B_unprojected = 0.5 * (B_unprojected + B_unprojected.T)

np.fill_diagonal(B_unprojected, 0.0)



# Projection diagnostics (matching production exactly)

proposed_update_norm = float(np.linalg.norm(B_unprojected - B_previous))



# Check gap constraint

row_sum_pre = np.sum(B_unprojected, axis=1)

g_min_pre = float(np.min(config.a - config.q * row_sum_pre))

pre_violation = bool(g_min_pre &lt; [config.delta](http://config.delta))



# Project if needed

B_projected = B_unprojected

correction_norm = 0.0
```

```
if pre_violation:

scale = np.minimum(1.0, config.capacity / (row_sum_pre + config.scale_epsilon))

B_projected = B_unprojected * np.minimum.outer(scale, scale)

np.fill_diagonal(B_projected, 0.0)

correction_norm = float(np.linalg.norm(B_projected - B_unprojected))



projection_active = bool(correction_norm &gt; FROZEN_PROJECTION_NORM_TOL)

relative_state = correction_norm / max(

float(np.linalg.norm(B_unprojected)), FROZEN_ZERO_TOL

)

relative_update = correction_norm / max(proposed_update_norm, FROZEN_ZERO_TOL)



# Validate projection

state.B = B_projected

row_sum = np.sum(state.B, axis=1)

g_min = float(np.min(config.a - config.q * row_sum))

if g_min &lt; [config.delta](http://config.delta) - 1e-9:
```

```
raise RuntimeError(f"Gap projection failed: g_min={g_min}")



# Update theta

delayed = state.history[0]

omega_arr = get_omega_array([config.omega](http://config.omega))

coupling = np.sum(

state.B * np.sin(delayed[None, :] - state.theta[:, None]), axis=1

)

state.theta = np.mod(

state.theta + config.dt * (omega_arr + coupling),

2.0 * np.pi

)



# Update history

state.history.append(state.theta.copy())

if len(state.history) &gt; config.delay_steps + 1:

state.history.pop(0)
```

```
info = {

"pre_violation": pre_violation,

"projection_active": projection_active,

"projection_norm": correction_norm,

"projection_relative_state": relative_state,

"projection_relative_update": relative_update,

"g_min_pre": g_min_pre,

"g_min": g_min,

}



return R_physical, R_driver, info

def simulate_prefix(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,
```

) -> tuple[DynamicState, dict[str, Any]]:

text

"""Simulate prefix up to split_time.

Exact reproduction of production simulate_prefix().

Returns (state, prefix_dict).

Production uses:

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence) # for initialization

Then in the loop, uses:

unused_rng = default_rng(0) # for advance_state when shuffle_driver=False

"""

N = config.N

edges = compute_edges(N)

```
# Initialize state (uses init_sequence via make_prefix_rng)

state = initialize_state(seed, config)



# Get RNG for prefix simulation loop

# Production uses unused_rng = default_rng(0) for the advance_state calls

# when shuffle_driver=False

unused_rng = np.random.default_rng(0)



# Simulate to split point

R_past_sum = np.zeros((N, N), dtype=float)

R_past_count = 0



burn_end = protocol.burn_end(config)

hist_end = protocol.hist_end(config)



for step in range(hist_end):
```

R_physical, *,* = advance_state(state, config, unused_rng, shuffle_driver=False)

if burn_end <= step < hist_end:

R_past_sum += R_physical

R_past_count += 1

prefix = {

"B_at_split": state.B.copy(),

"R_physical_past": R_past_sum / R_past_count if R_past_count > 0 else np.zeros((N, N)),

"theta_at_split": state.theta.copy(),

"coherence_acc_at_split": state.coherence_acc.copy(),

"history_at_split": np.stack(state.history),

"selected_at_split": (state.B > config.B_c).astype(np.uint8),

}

return state, prefix

def apply_intervention(

text

```
state: DynamicState,

branch: str,

config: ModelConfig,

rng: np.random.Generator,

) -> None:

text

"""Apply intervention to state. Mutates in place.



Exact reproduction of production apply_intervention().

"""

N = state.N



if branch in {"reset_ema", "reset_B_ema", "B_carrier_only", "full_reset"}:

# Zero coherence accumulator

state.coherence_acc.fill(0.0)



if branch in {"reset_B", "reset_B_ema", "full_reset"}:
```

```
# Zero B

state.B.fill(0.0)



if branch in {"B_carrier_only", "full_reset"}:

# Randomize theta and history

new_theta = rng.random(N) *2.0* np.pi

state.theta = new_theta

state.history = [new_theta.copy() for *in range(config.delay*steps + 1)]



# Ensure symmetry

state.B = 0.5 * (state.B + state.B.T)

np.fill_diagonal(state.B, 0.0)

def simulate_future(

text

split_state: DynamicState,

prefix: dict[str, Any],

branch: str,
```

config: ModelConfig,

protocol: ProtocolConfig,

rng: np.random.Generator,

) -> dict[str, Any]:

text

"""Simulate future trajectory from split state.

Reproduces production simulate_future() with ALL required outputs.

CRITICAL: Now accumulates BOTH horizon systems:

Fixed Physical Horizons (time-based, NOT T_window dependent):

- immediate: 0 to 17.5 time units

- recovery: 17.5 to 87.5 time units

- late: 87.5 to 320 time units

EMA-Normalized Horizons (T_window dependent):

- immediate: 0 to 1*T_window

- recovery: 1*T_window to 5*T_window

- late: 5*T_window to end

At rho=1 (T_window=17.502917...), the fixed physical and normalized horizons

are approximately aligned but use exact step boundaries.

Production accumulates:

- P_full, P_immediate, P_recovery, P_late (incidence matrices)

- B_mean_full, B_mean_immediate, B_mean_recovery, B_mean_late

- R_physical_future, R_driver_future

- Projection diagnostics via ProjectionTotals

- Various scalar metrics

Uses horizon system from production for normalized:

- WINDOW_STEPS = round(T_window / dt)

- RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS)

For fixed physical:

- FIXED_IMMEDIATE_END = round(17.5 / dt)

- FIXED_RECOVERY_END = round(87.5 / dt)

"""

state = split_state.clone()

apply_intervention(state, branch, config, rng)

N = state.N

edges = compute_edges(N)

future_steps = protocol.future_steps(config)

Compute horizon step boundaries

Normalized EMA horizons (T_window dependent)

window_steps = int(round(config.T_window / config.dt))

recovery_end_ema = min(5 * window_steps, future_steps)

```
# Fixed physical horizons (NOT T_window dependent)

# These use the production's physical time boundaries: 17.5, 87.5, 320

fixed_immediate_end = int(round(17.5 / config.dt)) # 17.5 time units

fixed_recovery_end = int(round(87.5 / config.dt)) # 87.5 time units

# Late goes from 87.5 to end (320 time units)



# Clamp all boundaries to future_steps

recovery_end_ema = min(recovery_end_ema, future_steps)

fixed_immediate_end = min(fixed_immediate_end, future_steps)

fixed_recovery_end = min(fixed_recovery_end, future_steps)



# Initialize accumulators for BOTH horizon systems

# P matrices (incidence: B &gt; B_c as 0/1)

P_sums_ema = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),
```

```
"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}

P_sums_fixed = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}



# B matrices (mean B values)

B_sums_ema = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}
```

```
B_sums_fixed = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}



# Counts for each horizon in both systems

counts_ema = {"full": 0, "immediate": 0, "recovery": 0, "late": 0}

counts_fixed = {"full": 0, "immediate": 0, "recovery": 0, "late": 0}



# R matrices

R_physical_sum = np.zeros((N, N), dtype=float)

R_driver_sum = np.zeros((N, N), dtype=float)



# Projection accumulator (matching production)

projection = ProjectionTotals()
```

```
# Trajectory accumulators
sample_times = []
density_trajectory = []
similarity_trajectory = []

# For retention/emergence calculations
r_sum = 0.0
r_count = 0
r_late_ema_sum = 0.0
r_late_ema_count = 0
r_late_fixed_sum = 0.0
r_late_fixed_count = 0

B_reference = prefix["B_at_split"]
shuffle_driver = (branch == "future_shuffle")
```

```
for step in range(future_steps):

R_physical, R_driver, info = advance_state(

state, config, rng, shuffle_driver=shuffle_driver

)


# Update projection accumulator

projection.update(info)


# Accumulate R matrices

R_physical_sum += R_physical

R_driver_sum += R_driver


# Determine which horizons this step belongs to for BOTH systems

selected = (state.B &amp;gt; config.B_c).astype(float)


# === EMA-Normalized Horizons ===

# Full: all steps

P_sums_ema["full"] += selected

B_sums_ema["full"] += state.B
```

counts_ema["full"] += 1

immediate: 0 to window_steps (exclusive)

if step < window_steps:

P_sums_ema["immediate"] += selected

B_sums_ema["immediate"] += state.B

counts_ema["immediate"] += 1

recovery: window_steps to recovery_end_ema (exclusive)

if window_steps <= step < recovery_end_ema:

P_sums_ema["recovery"] += selected

B_sums_ema["recovery"] += state.B

counts_ema["recovery"] += 1

late: recovery_end_ema to end

if recovery_end_ema <= step:

P_sums_ema["late"] += selected

B_sums_ema["late"] += state.B

counts_ema["late"] += 1

=== Fixed Physical Horizons ===

Full: all steps (same as EMA)

P_sums_fixed["full"] += selected

B_sums_fixed["full"] += state.B

counts_fixed["full"] += 1

immediate: 0 to fixed_immediate_end (exclusive)

if step < fixed_immediate_end:

P_sums_fixed["immediate"] += selected

B_sums_fixed["immediate"] += state.B

counts_fixed["immediate"] += 1

recovery: fixed_immediate_end to fixed_recovery_end (exclusive)

if fixed_immediate_end <= step < fixed_recovery_end:

P_sums_fixed["recovery"] += selected

B_sums_fixed["recovery"] += state.B

counts_fixed["recovery"] += 1

```
# late: fixed_recovery_end to end

if fixed_recovery_end &lt;= step:

P_sums_fixed["late"] += selected

B_sums_fixed["late"] += state.B

counts_fixed["late"] += 1


# Trajectory sampling - CRITICAL: r_mean_late is ONLY accumulated here

if step % protocol.diagnostic_stride == 0:

t = (step + 1) * config.dt

sample_times.append(t)


# Density: fraction of edges above B_c

density = float(np.count_nonzero(state.B[edges] &gt; config.B_c) / len(edges[0]))

density_trajectory.append(density)


# Similarity: cosine between current and reference B

B_vec = state.B[edges]

B_ref_vec = B_reference[edges]
```

```
norm_B = np.linalg.norm(B_vec)

norm_ref = np.linalg.norm(B_ref_vec)

if norm_B &gt; 0 and norm_ref &gt; 0:

similarity = float([np.dot](http://np.dot)(B_vec, B_ref_vec) / (norm_B * norm_ref))

else:

similarity = float('nan')

similarity_trajectory.append(similarity)


# Track overall r

r_value = float(np.abs(np.mean(np.exp(1j * state.theta))))

r_sum += r_value

r_count += 1


# Track late-phase r for BOTH horizon systems

if recovery_end_ema &lt;= step:

r_late_ema_sum += r_value

r_late_ema_count += 1


if fixed_recovery_end &lt;= step:
```

r_late_fixed_sum += r_value

r_late_fixed_count += 1

Compute mean matrices for BOTH horizon systems

P_ema = {name: P_sums_ema[name] / counts_ema[name] if counts_ema[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

P_fixed = {name: P_sums_fixed[name] / counts_fixed[name] if counts_fixed[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

B_mean_ema = {name: B_sums_ema[name] / counts_ema[name] if counts_ema[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

B_mean_fixed = {name: B_sums_fixed[name] / counts_fixed[name] if counts_fixed[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

Compute R means

```
R_physical_mean = R_physical_sum / future_steps if future_steps &gt; 0 else np.zeros((N, N))

R_driver_mean = R_driver_sum / future_steps if future_steps &gt; 0 else np.zeros((N, N))



# Final state

B_final = state.B.copy()

final_weights = state.B[edges]

final_rows = np.sum(state.B, axis=1)



# Retention and emergence for BOTH horizon systems

split_selected = prefix["selected_at_split"][edges].astype(bool)



# EMA-normalized

p_late_edges_ema = P_ema["late"][edges]

retention_ema = (

float(np.mean(p_late_edges_ema[split_selected]))

if np.any(split_selected)

else float('nan')
```

```
)
emergence_ema = (
float(np.mean(p_late_edges_ema[~split_selected]))
if np.any(~split_selected)
else float('nan')
)

# Fixed physical
p_late_edges_fixed = P_fixed["late"][edges]
retention_fixed = (
float(np.mean(p_late_edges_fixed[split_selected]))
if np.any(split_selected)
else float('nan')
)
emergence_fixed = (
float(np.mean(p_late_edges_fixed[~split_selected]))
if np.any(~split_selected)
```

```
else float('nan')

)



# Minimum similarity from trajectory

similarity_array = np.array(similarity_trajectory, dtype=float)

finite_similarity = similarity_array[np.isfinite(similarity_array)]

minimum_similarity = (

float(np.min(finite_similarity)) if len(finite_similarity) &gt; 0

else float('nan')

)



# Build result with BOTH horizon systems

result = {

# === Matrix outputs for EMA-normalized horizons (backward compatible) ===

"B_final": B_final,

"B_mean_late": B_mean_ema["late"],

"P_full": P_ema["full"],
```

"P_immediate": P_ema["immediate"],

"P_recovery": P_ema["recovery"],

"P_late": P_ema["late"],

"R_physical_future": R_physical_mean,

"R_driver_future": R_driver_mean,

=== Matrix outputs for FIXED-PHYSICAL horizons (NEW) ===

"B_mean_late_fixed_physical": B_mean_fixed["late"],

"P_full_fixed_physical": P_fixed["full"],

"P_immediate_fixed_physical": P_fixed["immediate"],

"P_recovery_fixed_physical": P_fixed["recovery"],

"P_late_fixed_physical": P_fixed["late"],

=== Matrix outputs for NORMALIZED-EMA horizons (NEW - explicit) ===

"B_mean_late_normalized_EMA": B_mean_ema["late"],

"P_full_normalized_EMA": P_ema["full"],

"P_immediate_normalized_EMA": P_ema["immediate"],

"P_recovery_normalized_EMA": P_ema["recovery"],

"P_late_normalized_EMA": P_ema["late"],

Trajectory outputs

"sample_times": np.array(sample_times, dtype=float),

"density_trajectory": np.array(density_trajectory, dtype=float),

"similarity_trajectory": np.array(similarity_trajectory, dtype=float),

Scalar outcomes - EMA-normalized (backward compatible names)

"D_B_final": float(np.count_nonzero(final_weights > config.B_c) / len(edges[0])),

"P_mean_full": float(np.mean(P_ema["full"][edges])),

"P_mean_immediate": float(np.mean(P_ema["immediate"][edges])),

"P_mean_recovery": float(np.mean(P_ema["recovery"][edges])),

"P_mean_late": float(np.mean(P_ema["late"][edges])),

Scalar outcomes - FIXED-PHYSICAL (NEW - primary estimands)

"P_mean_full_fixed_physical": float(np.mean(P_fixed["full"][edges])),

"P_mean_immediate_fixed_physical": float(np.mean(P_fixed["immediate"][edges])),

"P_mean_recovery_fixed_physical": float(np.mean(P_fixed["recovery"][edges])),

"P_mean_late_fixed_physical": float(np.mean(P_fixed["late"][edges])),

Scalar outcomes - NORMALIZED-EMA (NEW - explicit)

"P_mean_full_normalized_EMA": float(np.mean(P_ema["full"][edges])),

"P_mean_immediate_normalized_EMA": float(np.mean(P_ema["immediate"][edges])),

"P_mean_recovery_normalized_EMA": float(np.mean(P_ema["recovery"][edges])),

"P_mean_late_normalized_EMA": float(np.mean(P_ema["late"][edges])),

Similarity metrics - BOTH systems

"Weighted_similarity_final": cosine_edges(B_reference, B_final, edges),

"Weighted_similarity_late": cosine_edges(B_reference, B_mean_ema["late"], edges),

"Weighted_similarity_late_fixed_physical": cosine_edges(B_reference, B_mean_fixed["late"], edges),

"Weighted_similarity_late_normalized_EMA": cosine_edges(B_reference, B_mean_ema["late"], edges),

"Jaccard_final": jaccard_edges(B_reference, B_final, edges, config.B_c),

"Jaccard_final_fixed_physical": jaccard_edges(B_reference, B_final, edges, config.B_c),

Retention/emergence - BOTH systems

"Retention_selected_late": retention_ema,

"Emergence_unselected_late": emergence_ema,

"Retention_contrast_late": retention_ema - emergence_ema,

"Retention_selected_late_fixed_physical": retention_fixed,

"Emergence_unselected_late_fixed_physical": emergence_fixed,

"Retention_contrast_late_fixed_physical": retention_fixed - emergence_fixed,

"Retention_selected_late_normalized_EMA": retention_ema,

"Emergence_unselected_late_normalized_EMA": emergence_ema,

"Retention_contrast_late_normalized_EMA": retention_ema - emergence_ema,

Correlations with past R - BOTH systems

"Corr_Rpast_Pfull": safe_corr(prefix["R_physical_past"][edges], P_ema["full"][edges]),

"Corr_Rpast_Plate": safe_corr(prefix["R_physical_past"][edges], P_ema["late"][edges]),

"Corr_Rpast_Pfull_fixed_physical": safe_corr(prefix["R_physical_past"][edges], P_fixed["full"][edges]),

"Corr_Rpast_Plate_fixed_physical": safe_corr(prefix["R_physical_past"][edges], P_fixed["late"][edges]),

"Corr_Rpast_Pfull_normalized_EMA": safe_corr(prefix["R_physical_past"][edges], P_ema["full"][edges]),

"Corr_Rpast_Plate_normalized_EMA": safe_corr(prefix["R_physical_past"][edges], P_ema["late"][edges]),

Weight statistics (same for both - based on final state)

"Total_edge_weight": float(np.sum(final_weights)),

"Mean_row_sum": float(np.mean(final_rows)),

"Max_row_sum": float(np.max(final_rows)),

"Nonzero_edge_fraction": float(np.mean(final_weights > config.scale_epsilon)),

"Effective_edges": effective_number(final_weights),

"Weight_gini": gini_coefficient(final_weights),

"Weight_entropy": normalized_entropy(final_weights),

Order parameter - BOTH systems

"r_mean_future": float(r_sum / r_count) if r_count > 0 else float('nan'),

"r_mean_late": float(r_late_ema_sum / r_late_ema_count) if r_late_ema_count > 0 else float('nan'),

"r_mean_late_fixed_physical": float(r_late_fixed_sum / r_late_fixed_count) if r_late_fixed_count > 0 else float('nan'),

"r_mean_late_normalized_EMA": float(r_late_ema_sum / r_late_ema_count) if r_late_ema_count > 0 else float('nan'),

Minimum similarity

"Minimum_similarity": minimum_similarity,

Projection diagnostics (from ProjectionTotals)

**projection.metrics(),

Horizon metadata

"window_steps": window_steps,

"recovery_end_ema": recovery_end_ema,

"fixed_immediate_end": fixed_immediate_end,

"fixed_recovery_end": fixed_recovery_end,

}

return result

==========
Metric Helpers
==========

def cosine_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None) -> float:

text

"""Compute cosine similarity between reference and current B matrices over edges.

If edges is None, computes over all upper triangle edges.

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

x = np.asarray(reference[edges], dtype=float)

y = np.asarray(current[edges], dtype=float)

denominator = float(np.linalg.norm(x) * np.linalg.norm(y))

if denominator <= 1e-12:

return float('nan')

return float(np.dot(x, y) / denominator)

def jaccard_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None, threshold: float = FROZEN_ZERO_TOL) -> float:

text

"""Compute Jaccard similarity between reference and current B matrices over edges.

Uses B > threshold as the threshold for edge presence.

If edges is None, computes over all upper triangle edges.

Matches production: jaccard_edges(B_reference, state.B) uses B > B_C

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

x = reference[edges] > threshold

y = current[edges] > threshold

union = np.count_nonzero(x | y)

if union == 0:

return float('nan')

return float(np.count_nonzero(x & y) / union)

def safe_corr(x: np.ndarray, y: np.ndarray) -> float:

text

"""Compute correlation coefficient safely."""

x = np.asarray(x, dtype=float)

y = np.asarray(y, dtype=float)

valid = np.isfinite(x) & np.isfinite(y)

x = x[valid]

y = y[valid]

if len(x) < 3 or np.std(x) < 1e-12 or np.std(y) < 1e-12:

return float('nan')

return float(np.corrcoef(x, y)[0, 1])

def effective_number(weights: np.ndarray) -> float:

text

"""Compute effective number of edges."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

sum_sq = float(np.sum(weights**2))

if total <= 1e-12 or sum_sq <= 1e-12:

return 0.0

```
return total**2 / sum_sq

def gini_coefficient(weights: np.ndarray) -> float:

text

"""Compute Gini coefficient."""

x = np.sort(np.asarray(weights, dtype=float))

total = float(np.sum(x))

if total &lt;= 1e-12:

return 0.0

n = len(x)

ranks = np.arange(1, n + 1, dtype=float)

return float(2.0 *[np.dot](http://np.dot)(ranks, x) / (n* total) - (n + 1.0) / n)

def normalized_entropy(weights: np.ndarray) -> float:

text

"""Compute normalized entropy."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

if total &lt;= 1e-12:

return 0.0
```

```
probabilities = weights / total

positive = probabilities &gt; 0.0

entropy = -float(np.sum(probabilities[positive] * np.log(probabilities[positive])))

return entropy / float(np.log(len(weights)))

def time_to_at_least(

text

sample_times: np.ndarray,

values: np.ndarray,

target: float,

) -> float:

text

"""Find the first time at which values reach at least the target.



Matches frozen production exactly.

Returns NaN if target is never reached.

"""

sample_times = np.asarray(sample_times, dtype=float)
```

values = np.asarray(values, dtype=float)

valid = np.isfinite(values)

indices = np.flatnonzero(valid & (values >= target))

return float(sample_times[indices[0]]) if len(indices) > 0 else float('nan')

def time_to_below(

text

sample_times: np.ndarray,

values: np.ndarray,

threshold: float,

) -> float:

text

"""Find the first time at which values drop below the threshold.

Matches frozen production exactly.

Returns NaN if threshold is never dropped below.

"""

sample_times = np.asarray(sample_times, dtype=float)

values = np.asarray(values, dtype=float)

valid = np.isfinite(values)

indices = np.flatnonzero(valid & (values < threshold))

return float(sample_times[indices[0]]) if len(indices) > 0 else float('nan')

def finalize_historical_branch_metrics(

text

branch_results: dict[str, dict[str, Any]],

reference_density: float,

reference_late_similarity: float,

reference_late_persistence: float,

) -> None:

text

"""Compute and add the 8 historical scalar fields that compare branches to continue.

These fields are computed in frozen production by run_seed() after all branches

are simulated. They must be computed here to match the frozen schema.

The 8 fields are:

- Time_to_50pct_continue_final_density

- Time_to_90pct_continue_final_density

- Time_to_50pct_continue_late_similarity

- Time_to_90pct_continue_late_similarity

- Time_to_similarity_below_0_9

- Time_to_similarity_below_0_5

- Late_persistence_ratio_to_continue

- Late_similarity_ratio_to_continue

These are computed for each branch relative to the continue branch reference values.

"""

Get continue branch results for reference

if "continue" not in branch_results:

raise RuntimeError("continue branch is required for historical metric computation")

continue_result = branch_results["continue"]

```
# Get reference values from continue branch

# These are the target values for the "continue" branch itself

# For other branches, we compare their trajectory to these reference values

reference_density = float(continue_result.get("D_B_final", reference_density))

reference_late_similarity = float(continue_result.get("Weighted_similarity_late",
reference_late_similarity))

reference_late_persistence = float(continue_result.get("P_mean_late", reference_late_persistence))



for branch, result in branch_results.items():

times = result.get("sample_times")

density = result.get("density_trajectory")

similarity = result.get("similarity_trajectory")


if times is None or density is None or similarity is None:

# If trajectory data is missing, set all time-based metrics to NaN

result["Time_to_50pct_continue_final_density"] = float('nan')

result["Time_to_90pct_continue_final_density"] = float('nan')
```

```
result["Time_to_50pct_continue_late_similarity"] = float('nan')

result["Time_to_90pct_continue_late_similarity"] = float('nan')

result["Time_to_similarity_below_0_9"] = float('nan')

result["Time_to_similarity_below_0_5"] = float('nan')

result["Late_persistence_ratio_to_continue"] = float('nan')

result["Late_similarity_ratio_to_continue"] = float('nan')

continue


# Compute time-based metrics relative to continue branch reference

result["Time_to_50pct_continue_final_density"] = time_to_at_least(

times, density, 0.5 * reference_density

)

result["Time_to_90pct_continue_final_density"] = time_to_at_least(

times, density, 0.9 * reference_density

)

result["Time_to_50pct_continue_late_similarity"] = time_to_at_least(

times, similarity, 0.5 * reference_late_similarity
```

```
)

result["Time_to_90pct_continue_late_similarity"] = time_to_at_least(

times, similarity, 0.9 * reference_late_similarity

)

result["Time_to_similarity_below_0_9"] = time_to_below(

times, similarity, 0.9

)

result["Time_to_similarity_below_0_5"] = time_to_below(

times, similarity, 0.5

)


# Compute ratio metrics

branch_late_persistence = float(result.get("P_mean_late", float('nan')))

branch_late_similarity = float(result.get("Weighted_similarity_late", float('nan')))


if abs(reference_late_persistence) &gt; FROZEN_ZERO_TOL:

result["Late_persistence_ratio_to_continue"] = (

branch_late_persistence / reference_late_persistence
```

)

else:

result["Late_persistence_ratio_to_continue"] = float('nan')

if abs(reference_late_similarity) > FROZEN_ZERO_TOL:

result["Late_similarity_ratio_to_continue"] = (

branch_late_similarity / reference_late_similarity

)

else:

result["Late_similarity_ratio_to_continue"] = float('nan')

==========
Required Output Schema for Baseline Equivalence
==========
IMPORTANT: We separate FROZEN keys (must exist in historical NPZ) from
EXTENDED keys (new in v1, only validated internally at rho=1).
The frozen NPZ does NOT contain dual-horizon keys, so baseline equivalence
must only check FROZEN keys against historical data.
Required prefix keys for baseline equivalence

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)
These are the keys that MUST exist in the frozen NPZ for baseline equivalence

FROZEN_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
These are the NEW keys introduced in v1 for dual-horizon support
They will NOT exist in frozen NPZ, so baseline equivalence skips them

EXTENDED_BRANCH_MATRIX_KEYS = (

text

"B_mean_late_fixed_physical",

"P_full_fixed_physical",

"P_immediate_fixed_physical",

"P_recovery_fixed_physical",

"P_late_fixed_physical",

"B_mean_late_normalized_EMA",

"P_full_normalized_EMA",

"P_immediate_normalized_EMA",

"P_recovery_normalized_EMA",

"P_late_normalized_EMA",

)
All matrix keys (frozen + extended)

REQUIRED_BRANCH_MATRIX_KEYS = FROZEN_BRANCH_MATRIX_KEYS + EXTENDED_BRANCH_MATRIX_KEYS

REQUIRED_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

Fixed physical horizon scalars (NEW - PRIMARY ESTIMANDS)

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

Normalized EMA horizon scalars (NEW - explicit)

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

Projection diagnostics (from ProjectionTotals)

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

)
==========
====
Baseline Equivalence Test

```
================================================================================

def exact_array_equal_with_nan(a: np.ndarray, b: np.ndarray) -> bool:

text

"""Compare arrays with NaN-aware exact equality."""

a = np.asarray(a)

b = np.asarray(b)

if a.shape != b.shape or a.dtype != b.dtype:

return False

if np.issubdtype(a.dtype, np.floating) or np.issubdtype(a.dtype, np.complexfloating):

a_nan = np.isnan(a)

b_nan = np.isnan(b)

if not np.array_equal(a_nan, b_nan):

return False

finite = ~a_nan

return np.array_equal(a[finite], b[finite])

return np.array_equal(a, b)

def run_baseline_equivalence(
```

```
text

seeds: list[int],

config: ModelConfig,

protocol: ProtocolConfig,

frozen_manifest_path: str,

frozen_npz_path: str,

branch_order: tuple[str, ...],

frozen_csv_path: str | None = None,

) -> dict[str, Any]:

text

"""Run baseline equivalence test at rho=1 (T_window = T_*).



MANDATORY: Must reproduce frozen run exactly for all seeds.



This function validates that the engine produces EXACTLY the same outputs

as the frozen production run for all required keys.
```

Requires NaN-aware exact equality for ALL required schema keys:

- All prefix keys

- All branch matrix keys

- All branch scalar keys

Returns:

status: "passed" | "failed" | "not_run"

seed_results: per-seed validation details

engine_source_sha256: hash of this engine source

frozen_manifest_hash: hash of frozen manifest

frozen_npz_hash: hash of frozen NPZ

"""

import numpy as np

from pathlib import Path

import hashlib

Helper function to compute source hash

```
def *source*sha256() -&gt; str | None:

try:

return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

return None



# Early failure: frozen scalar CSV is required for scalar equivalence

if frozen_csv_path is None:

return {

"status": "failed",

"error": "frozen scalar CSV is required for scalar equivalence",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": None,

}



# Record engine source hash
```

engine_source_hash = *source*sha256()

results = {

"status": "not_run",

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"mean_omega": config.mean_omega,

"T_window": config.T_window,

"T_star": config.T_star,

"alpha": config.alpha,

"a": config.a,

"q": config.q,

"delta": config.delta,

"eta": config.eta,

"mu": config.mu,

```
"gamma": config.gamma,
"B_c": config.B_c,
"B_init_scale": config.B_init_scale,
"scale_epsilon": config.scale_epsilon,
"capacity": config.capacity,
"delay_steps": config.delay_steps,
},
"protocol": {
"T_sim": protocol.T_sim,
"burn_end_time": protocol.burn_end_time,
"split_time": protocol.split_time,
"future_duration": protocol.future_duration,
"diagnostic_stride": protocol.diagnostic_stride,
"branches": list(protocol.branches),
},
"n_seeds": len(seeds),
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
```

```
"engine_source_sha256": engine_source_hash,

"seed_results": {},

}



# Load frozen manifest

with open(frozen_manifest_path, "r") as f:

frozen_manifest = json.load(f)



# Load frozen data

with np.load(frozen_npz_path, allow_pickle=False) as frozen_data:

frozen_files = dict(frozen_data)



# Load frozen scalar CSV if provided

frozen_scalar_csv: dict[str, dict[str, Any]] = {}

if frozen_csv_path is not None and os.path.exists(frozen_csv_path):

try:

import csv
```

```
with open(frozen_csv_path, "r", newline="", encoding="utf-8") as f:

reader = csv.DictReader(f)

for row in reader:

seed_id = row.get("seed", "")

branch = row.get("branch", "")

if seed_id and branch:

key = f"{branch}_seed_{seed_id}"

if key not in frozen_scalar_csv:

frozen_scalar_csv[key] = {}

for k, v in row.items():

if k not in ("seed", "seed_index", "branch", "rho", "T_window"):

try:

frozen_scalar_csv[key][k] = float(v)

except (ValueError, TypeError):

frozen_scalar_csv[key][k] = v

except Exception as e:

print(f"Warning: Could not load frozen CSV {frozen_csv_path}: {e}")
```

```
# Record frozen hashes - use actual file hashes, not canonical JSON hashes

# Record frozen hashes - use actual file hashes, not canonical JSON hashes

results["frozen_manifest_file_sha256"] = file_sha256(frozen_manifest_path)

results["frozen_npz_file_sha256"] = file_sha256(frozen_npz_path)

if frozen_csv_path is not None and os.path.exists(frozen_csv_path):

results["frozen_scalar_csv_file_sha256"] = file_sha256(frozen_csv_path)



# Also record engine source hash

results["engine_source_sha256"] = engine_source_hash



# Also keep canonical JSON hash for reference

results["frozen_manifest_hash"] = hashlib.sha256(

json.dumps(frozen_manifest, sort_keys=True).encode()

).hexdigest()



# Validate config matches frozen manifest model - use EXACT equality
```

```
frozen_model = frozen_manifest["model"]



# Check N

if frozen_model["N"] != config.N:

results["status"] = "failed"

results["error"] = f"N mismatch: config={config.N}, frozen={frozen_model['N']}"

return results



# Check omega - use exact equality

frozen_omega_arr = np.array(frozen_model["omega"], dtype=float)

config_omega_arr = np.array(list([config.omega](http://config.omega)), dtype=float)

if not np.array_equal(frozen_omega_arr, config_omega_arr):

results["status"] = "failed"

results["error"] = f"omega mismatch: frozen={frozen_model['omega']}, config={list([config.omega]
(http://config.omega))}"

return results



# Check all other model params with EXACT equality (rtol=0, atol=0)
```

```
model_checks = [
("dt", frozen_model["dt"], config.dt),
("tau", frozen_model["tau"], config.tau),
("a", frozen_model["a"], config.a),
("q", frozen_model["q"], config.q),
("delta", frozen_model["delta"], [config.delta](http://config.delta)),
("eta", frozen_model["eta"], config.eta),
("mu", frozen_model["mu"], [config.mu](http://config.mu)),
("gamma", frozen_model["gamma"], config.gamma),
("B_c", frozen_model["B_c"], config.B_c),
("B_init_scale", frozen_model.get("B_init_scale", 0.0), config.B_init_scale),
("T_window", frozen_model["T_window"], config.T_window),
]

for name, frozen_val, config_val in model_checks:
if not np.isclose(frozen_val, config_val, rtol=0.0, atol=0.0):
results["status"] = "failed"
```

```
results["error"] = f"{name} mismatch: frozen={frozen_val}, config={config_val}"

return results



# Check derived values

if not np.isclose(frozen_model.get("alpha", 0.0), config.alpha, rtol=0.0, atol=1e-15):

results["status"] = "failed"

results["error"] = f"alpha mismatch: frozen={frozen_model.get('alpha')}, config={config.alpha}"

return results



# Now run actual equivalence test for each seed

print("Running baseline equivalence test...")

all_passed = True



for seed in seeds:

seed_str = str(seed)

seed_results = {"status": "passed", "checks": {}, "max_diffs": {}, "errors": {}}
```

```
try:

# Simulate with our engine

split_state, prefix = simulate_prefix(seed, config, protocol)

branch_rngs = make_branch_rngs(seed, branch_order)


# Simulate each branch

engine_results = {}

for branch in protocol.branches:

rng = branch_rngs[branch]

engine_results[branch] = simulate_future(

split_state.clone(),

prefix,

branch,

config,

protocol,

rng,

)
```

```
# Check prefix state - iterate over REQUIRED_PREFIX_KEYS

for key in REQUIRED_PREFIX_KEYS:

engine_val = prefix.get(key)

frozen_key = f"prefix_seed_{seed_str}_{key}"

frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"prefix_{key}_missing"] = True

all_passed = False

continue


if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"prefix_{key}_exact"] = True

else:

seed_results["checks"][f"prefix_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False
```

```
if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(frozen_val), 0.0, frozen_val)

seed_results["max_diffs"][f"prefix_{key}"] = float(np.max(np.abs(e - f)))


# Check branch outputs - separate FROZEN vs EXTENDED keys

for branch in protocol.branches:

engine_branch = engine_results[branch]

frozen_prefix = f"{branch}_seed_{seed_str}"


# === FROZEN MATRIX KEYS: Must exist in historical NPZ ===

for key in FROZEN_BRANCH_MATRIX_KEYS:

engine_val = engine_branch.get(key)

frozen_key = f"{frozen_prefix}_{key}"

frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_frozen"] = True
```

```
all_passed = False
continue

if engine_val is None:
seed_results["status"] = "failed"
seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True
all_passed = False
continue

if exact_array_equal_with_nan(engine_val, frozen_val):
seed_results["checks"][f"{branch}_{key}_exact"] = True
else:
seed_results["checks"][f"{branch}_{key}_exact"] = False
seed_results["status"] = "failed"
all_passed = False
if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):
e = np.where(np.isnan(engine_val), 0.0, engine_val)
f = np.where(np.isnan(frozen_val), 0.0, frozen_val)
```

```
seed_results["max_diffs"][f"{branch}_{key}"] = float(np.max(np.abs(e - f)))


# === EXTENDED MATRIX KEYS: Validate internally at rho=1 ===
# These do NOT exist in frozen NPZ. At rho=1, both horizon systems
# should produce identical results (1750 and 8750 steps match).
if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):
for key in EXTENDED_BRANCH_MATRIX_KEYS:
engine_val = engine_branch.get(key)
if engine_val is None:
seed_results["status"] = "failed"
seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True
all_passed = False
continue


# Map extended key to its frozen equivalent
frozen_key_map = {
"B_mean_late_fixed_physical": "B_mean_late",
"P_full_fixed_physical": "P_full",
```

```
"P_immediate_fixed_physical": "P_immediate",

"P_recovery_fixed_physical": "P_recovery",

"P_late_fixed_physical": "P_late",

"B_mean_late_normalized_EMA": "B_mean_late",

"P_full_normalized_EMA": "P_full",

"P_immediate_normalized_EMA": "P_immediate",

"P_recovery_normalized_EMA": "P_recovery",

"P_late_normalized_EMA": "P_late",

}


frozen_equiv_key = frozen_key_map.get(key)

if frozen_equiv_key:

engine_frozen_equiv = engine_branch.get(frozen_equiv_key)

if engine_frozen_equiv is not None:

if exact_array_equal_with_nan(engine_val, engine_frozen_equiv):

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = True

else:
```

```
seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(engine_frozen_equiv, np.ndarray):

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(engine_frozen_equiv), 0.0, engine_frozen_equiv)

seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = float(np.max(np.abs(e - f)))


# === SCALAR KEYS ===

# Separate FROZEN vs EXTENDED scalars

# First, define which scalars are frozen (exist in historical data)

FROZEN_BRANCH_SCALAR_KEYS = (

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",
```

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

Historical time-based metrics (computed cross-branch)

"Time_to_50pct_continue_final_density",

"Time_to_90pct_continue_final_density",

"Time_to_50pct_continue_late_similarity",

"Time_to_90pct_continue_late_similarity",

"Time_to_similarity_below_0_9",

"Time_to_similarity_below_0_5",

```
"Late_persistence_ratio_to_continue",

"Late_similarity_ratio_to_continue",

)


EXTENDED_BRANCH_SCALAR_KEYS = (

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

)


for key in FROZEN_BRANCH_SCALAR_KEYS:
```

```
engine_val = engine_branch.get(key)


# Get from CSV first (production stores scalars in CSV)

frozen_val = None

csv_key = f"{branch}_seed_{seed_str}"

if csv_key in frozen_scalar_csv and key in frozen_scalar_csv[csv_key]:

frozen_val = frozen_scalar_csv[csv_key][key]

else:

# Fallback to NPZ

frozen_key = f"{frozen_prefix}_{key}"

frozen_val = frozen_files.get(frozen_key)


# FROZEN scalars MUST exist in historical data

if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = False

all_passed = False

continue
```

```
else:
seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = True

if engine_val is None:
seed_results["status"] = "failed"
seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False
all_passed = False
continue
else:
seed_results["checks"][f"{branch}_{key}_present_in_engine"] = True

# For scalars, compare directly - DO NOT silently skip
# Normalize to scalar value
try:
engine_scalar = float(engine_val)
except (TypeError, ValueError) as e:
seed_results["status"] = "failed"
seed_results["checks"][f"{branch}_{key}_is_scalar"] = False
```

seed_results["errors"][f"{branch}_{key}_type_error"] = f"Cannot convert to float: {type(engine_val).__name__}: {e}"

all_passed = False

continue

try:

frozen_scalar = float(frozen_val)

except (TypeError, ValueError) as e:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_frozen_is_scalar"] = False

seed_results["errors"][f"{branch}_{key}_frozen_type_error"] = f"Cannot convert frozen to float: {type(frozen_val).__name__}: {e}"

all_passed = False

continue

Compare with exact equality (rtol=0, atol=0)

engine_is_nan = bool(np.isnan(engine_scalar))

frozen_is_nan = bool(np.isnan(frozen_scalar))

if engine_is_nan and frozen_is_nan:

```
seed_results["checks"][f"{branch}_{key}_exact"] = True

elif np.isclose(engine_scalar, frozen_scalar, rtol=0.0, atol=0.0):

seed_results["checks"][f"{branch}_{key}_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

seed_results["max_diffs"][f"{branch}_{key}"] = float(abs(engine_scalar - frozen_scalar)) if not (engine_is_nan or frozen_is_nan) else float('nan')



# === EXTENDED SCALAR KEYS: Validate internally at rho=1 ===

if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):

for key in EXTENDED_BRANCH_SCALAR_KEYS:

engine_val = engine_branch.get(key)

if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False

all_passed = False

continue
```

```
else:
seed_results["checks"][f"{branch}_{key}_present_in_engine"] = True

# Map to frozen equivalent for INTERNAL consistency check
# At rho=1, fixed_physical and normalized_EMA should match
frozen_scalar_map = {
"P_mean_late_fixed_physical": "P_mean_late",
"Weighted_similarity_late_fixed_physical": "Weighted_similarity_late",
"Retention_contrast_late_fixed_physical": "Retention_contrast_late",
"Corr_Rpast_Plate_fixed_physical": "Corr_Rpast_Plate",
"r_mean_late_fixed_physical": "r_mean_late",
"P_mean_late_normalized_EMA": "P_mean_late",
"Weighted_similarity_late_normalized_EMA": "Weighted_similarity_late",
"Retention_contrast_late_normalized_EMA": "Retention_contrast_late",
"Corr_Rpast_Plate_normalized_EMA": "Corr_Rpast_Plate",
"r_mean_late_normalized_EMA": "r_mean_late",
}
```

```
frozen_equiv = frozen_scalar_map.get(key)

if frozen_equiv:

# Get the engine's own frozen-equivalent value

engine_frozen_equiv = engine_branch.get(frozen_equiv)


if engine_frozen_equiv is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{frozen_equiv}_present_in_engine"] = False

all_passed = False

continue


# Validate internal consistency: extended should equal frozen equivalent

try:

engine_scalar = float(engine_val)

equiv_scalar = float(engine_frozen_equiv)

except (TypeError, ValueError) as e:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_internal_consistency"] = False
```

```
seed_results["errors"][f"{branch}_{key}_internal_type_error"] = str(e)

all_passed = False

continue


engine_is_nan = bool(np.isnan(engine_scalar))

equiv_is_nan = bool(np.isnan(equiv_scalar))


if engine_is_nan and equiv_is_nan:

seed_results["checks"][f"{branch}_{key}_internal_exact"] = True

elif np.isclose(engine_scalar, equiv_scalar, rtol=0.0, atol=0.0):

seed_results["checks"][f"{branch}_{key}_internal_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_internal_exact"] = False

seed_results["status"] = "failed"

all_passed = False

seed_results["max_diffs"][f"{branch}_{key}_internal"] = float(abs(engine_scalar - equiv_scalar)) if not (engine_is_nan or equiv_is_nan) else float('nan')


# Also validate against frozen CSV for completeness
```

```
csv_key = f"{branch}_seed_{seed_str}"

if csv_key in frozen_scalar_csv and frozen_equiv in frozen_scalar_csv[csv_key]:

frozen_equiv_val = frozen_scalar_csv[csv_key][frozen_equiv]

try:

frozen_scalar = float(frozen_equiv_val)

if engine_is_nan and bool(np.isnan(frozen_scalar)):

seed_results["checks"][f"{branch}_{key}_equals_frozen_csv"] = True

elif np.isclose(engine_scalar, frozen_scalar, rtol=0.0, atol=0.0):

seed_results["checks"][f"{branch}_{key}_equals_frozen_csv"] = True

else:

seed_results["checks"][f"{branch}_{key}_equals_frozen_csv"] = False

# Don't fail on this - it's informational

except (TypeError, ValueError):

pass


except Exception as e:

seed_results["status"] = "failed"
```

```
seed_results["error"] = str(e)

import traceback

seed_results["traceback"] = traceback.format_exc()

all_passed = False


results["seed_results"][seed_str] = seed_results

print(f" Seed {seed_str}: {seed_results['status']}")



if all_passed:

results["status"] = "passed"

print("Baseline equivalence: PASSED")

else:

results["status"] = "failed"

print("Baseline equivalence: FAILED")

# Print details of failures

for seed_str, seed_results in results["seed_results"].items():

if seed_results.get("status") != "passed":

print(f" Seed {seed_str} failures:")
```

```
for check, val in seed_results.get("checks", {}).items():

if not val:

print(f" CHECK FAILED: {check}")

for key, val in seed_results.get("max_diffs", {}).items():

print(f" MAX DIFF: {key} = {val}")



return results

================================================================================
Main API
================================================================================
Known branches - allowlist for validation

KNOWN_BRANCHES = frozenset({

text

"continue",

"reset_ema",

"reset_B",

"reset_B_ema",

"B_carrier_only",
```

"full_reset",

"future_shuffle",

})

def run_seed(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

branch_order: tuple[str, ...],

) -> dict[str, Any]:

text

"""Run a single seed through prefix + all branches.

CRITICAL: branch_order defines the FULL frozen RNG namespace (7 branches).

protocol.branches defines which branches are actually executed.

This ensures future_shuffle gets the 7th RNG sequence (not the 4th)

as in production, even when only 4 study branches are executed.

Returns complete seed results with all branches.

"""

Validate all protocol branches are known

for branch in protocol.branches:

if branch not in KNOWN_BRANCHES:

raise ValueError(

f"Unknown branch '{branch}'. Known branches: {sorted(KNOWN_BRANCHES)}"

)

Validate all protocol branches are in branch_order (for RNG namespace)

for branch in protocol.branches:

if branch not in branch_order:

raise ValueError(

f"Branch '{branch}' in protocol.branches but not in branch_order. "

f"branch_order must include all protocol branches for correct RNG namespace."

```
)



# Simulate prefix

split_state, prefix = simulate_prefix(seed, config, protocol)



# Create branch RNGs using frozen algorithm with FULL branch_order

branch_rngs = make_branch_rngs(seed, branch_order)



# Execute only the branches in protocol.branches, but use RNGs from branch_order

branch_results = {}

for branch in protocol.branches:

if branch not in branch_rngs:

raise ValueError(

f"Branch '{branch}' not found in branch_rngs. "

f"This indicates branch_order={branch_order} does not match the RNG namespace. "

f"Registered branches: {list(branch_rngs.keys())}"

)
```

```
rng = branch_rngs[branch]


branch_results[branch] = simulate_future(

split_state.clone(),

prefix,

branch,

config,

protocol,

rng,

)



# Compute historical cross-branch metrics (the 8 time-based fields)

# These must be computed AFTER all branches are simulated

if "continue" in branch_results:

# Get reference values from continue branch

continue_result = branch_results["continue"]

reference_density = float(continue_result.get("D_B_final", float('nan')))
```

```
reference_late_similarity = float(continue_result.get("Weighted_similarity_late", float('nan')))

reference_late_persistence = float(continue_result.get("P_mean_late", float('nan')))


finalize_historical_branch_metrics(

branch_results,

reference_density,

reference_late_similarity,

reference_late_persistence,

)



return {

"seed": int(seed),

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"T_window": config.T_window,

"alpha": config.alpha,
```

"mean_omega": config.mean_omega,

"T_star": config.T_star,

},

"protocol": {

"T_sim": protocol.T_sim,

"burn_end_time": protocol.burn_end_time,

"split_time": protocol.split_time,

"future_duration": protocol.future_duration,

},

"prefix": prefix,

"branches": branch_results,

}

if name == "main":

text

Quick validation

config = DEFAULT_MODEL_CONFIG

config.validate()

protocol = DEFAULT_PROTOCOL_CONFIG

protocol.validate(config)

print(f"Engine v1 loaded successfully")

print(f" N={config.N}, dt={config.dt}, tau={config.tau}")

print(f" T_window={config.T_window}, alpha={config.alpha:.6f}")

print(f" mean_omega={config.mean_omega:.6f}, T_*={config.T_star:.6f}")

print(f" delay_steps={config.delay_steps}")

print(f" Protocol: T_sim={protocol.T_sim}, burn={protocol.burn_end_time}, "

f"split={protocol.split_time}, future={protocol.future_duration}")

Ignoring the paste corruption, this revision fixes several earlier issues, but it is still NO-GO for the mandatory 40-seed baseline run. One defect will deterministically make the baseline fail after doing all the expensive simulations.
Critical blocker: baseline never calls the historical-metric finalizer

You added:

Python

finalize_historical_branch_metrics(...)

to run_seed(), but run_baseline_equivalence() does not use run_seed(). It manually runs:

Python

for branch in protocol.branches:
engine_results[branch] = simulate_future(...)

and then immediately begins checking outputs.

Because simulate_future() no longer creates the eight historical fields, every branch will be missing:

text

Time_to_50pct_continue_final_density
Time_to_90pct_continue_final_density
Time_to_50pct_continue_late_similarity
Time_to_90pct_continue_late_similarity
Time_to_similarity_below_0_9
Time_to_similarity_below_0_5
Late_persistence_ratio_to_continue
Late_similarity_ratio_to_continue

The baseline will therefore simulate all 10.88 million steps and then fail every seed.
Required insertion

Immediately after all branches have been simulated in run_baseline_equivalence():

Python

engine_results = {}
for branch in protocol.branches:
rng = branch_rngs[branch]
engine_results[branch] = simulate_future(
split_state.clone(),
prefix,
branch,
config,
protocol,
rng,
)

Required before scalar comparisons

```
finalize_historical_branch_metrics(
engine_results,
reference_density=float(engine_results["continue"]["D_B_final"]),
reference_late_similarity=float(
engine_results["continue"]["Weighted_similarity_late"]
),
reference_late_persistence=float(
engine_results["continue"]["P_mean_late"]
),
)
```

Better still, simplify the helper so it accepts only branch_results; the three arguments are immediately overwritten from the continue result anyway.

Additional must-fix certificate issues

1. Missing prefix/matrix values are still recorded as successful checks

These remain:

Python

```
seed_results["checks"][f"prefix_{key}_missing"] = True
seed_results["checks"][f"{branch}_{key}_missing_in_frozen"] = True
seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True
```

That creates a failed seed containing checks marked True, and the failure printer will not display them.

Use positive predicates:

Python

```
seed_results["checks"][f"prefix_{key}_present_in_frozen"] = False
seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = False
seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False
```

Set the corresponding predicate to True when present.

This needs correction in:

prefix checks;
frozen matrix checks;
extended matrix checks.

You already corrected it for scalar presence.
2. A false “informational” check can coexist with status="passed"

The extended scalar section stores:

Python

```
seed_results["checks"][f"{branch}_{key}_equals_frozen_csv"] = False
# Don't fail on this - it's informational
```

This permits:

text

```
overall status = passed
a check in checks = false
```

That is not acceptable for a proof-oriented certificate.

Either:

remove this redundant comparison;
make it a required check and fail on it; or
store it in a separate informational_checks dictionary.

The cleanest choice is removal. The logic already proves:
extended=engine legacy
extended=engine legacy

and separately:
engine legacy=frozen CSV.
engine legacy=frozen CSV.
3. The module-level scalar schema is still inconsistent

The local baseline tuple now contains all 42 historical scalars, but the module-level:

Python

```
REQUIRED_BRANCH_SCALAR_KEYS
```

still omits the eight historical cross-branch fields.

This leaves two competing definitions of the frozen schema and makes it possible for the driver or future validation code to accept incomplete results.

Use module-level definitions only:

Python

```
FROZEN_BRANCH_SCALAR_KEYS = (
# all 42 historical fields, in frozen production order
)

EXTENDED_BRANCH_SCALAR_KEYS = (
# all declared new fields
)

REQUIRED_BRANCH_SCALAR_KEYS = (
FROZEN_BRANCH_SCALAR_KEYS
+ EXTENDED_BRANCH_SCALAR_KEYS
)
```

Remove the local tuple definitions from run_baseline_equivalence().

Ideally, also require agreement with:

Python

```
frozen_manifest["schema"]["branch_scalar_keys"]
```

4. Extended scalar coverage remains incomplete

The baseline checks only ten late-horizon extensions. It does not validate several other fields that simulate_future() exposes, including:

text

P_mean_full_fixed_physical
P_mean_immediate_fixed_physical
P_mean_recovery_fixed_physical

P_mean_full_normalized_EMA
P_mean_immediate_normalized_EMA
P_mean_recovery_normalized_EMA

Retention_selected_late_fixed_physical
Emergence_unselected_late_fixed_physical
Retention_selected_late_normalized_EMA
Emergence_unselected_late_normalized_EMA

Corr_Rpast_Pfull_fixed_physical
Corr_Rpast_Pfull_normalized_EMA

Jaccard_final_fixed_physical

At ρ=1ρ=1, all applicable fields should be checked through a single module-level mapping:

Python

EXTENDED_SCALAR_EQUIVALENCE_MAP = {
"P_mean_full_fixed_physical": "P_mean_full",
"P_mean_immediate_fixed_physical": "P_mean_immediate",
"P_mean_recovery_fixed_physical": "P_mean_recovery",
"P_mean_late_fixed_physical": "P_mean_late",

"P_mean_full_normalized_EMA": "P_mean_full",

"P_mean_immediate_normalized_EMA": "P_mean_immediate",
"P_mean_recovery_normalized_EMA": "P_mean_recovery",
"P_mean_late_normalized_EMA": "P_mean_late",

"Weighted_similarity_late_fixed_physical":
"Weighted_similarity_late",
"Weighted_similarity_late_normalized_EMA":
"Weighted_similarity_late",

"Retention_selected_late_fixed_physical":
"Retention_selected_late",
"Emergence_unselected_late_fixed_physical":
"Emergence_unselected_late",
"Retention_contrast_late_fixed_physical":
"Retention_contrast_late",

"Retention_selected_late_normalized_EMA":
"Retention_selected_late",
"Emergence_unselected_late_normalized_EMA":
"Emergence_unselected_late",
"Retention_contrast_late_normalized_EMA":
"Retention_contrast_late",

"Corr_Rpast_Pfull_fixed_physical": "Corr_Rpast_Pfull",
"Corr_Rpast_Plate_fixed_physical": "Corr_Rpast_Plate",
"Corr_Rpast_Pfull_normalized_EMA": "Corr_Rpast_Pfull",
"Corr_Rpast_Plate_normalized_EMA": "Corr_Rpast_Plate",

"r_mean_late_fixed_physical": "r_mean_late",
"r_mean_late_normalized_EMA": "r_mean_late",

"Jaccard_final_fixed_physical": "Jaccard_final",
}

Also validate baseline metadata:

Python

window_steps == FROZEN_WINDOW_STEPS
recovery_end_ema == FROZEN_RECOVERY_END
fixed_immediate_end == FIXED_PHYSICAL_IMMEDIATE_STEPS
fixed_recovery_end == FIXED_PHYSICAL_RECOVERY_STEPS

Without this expansion, the certificate should not claim that all extensions were checked.
5. Failure certificates can still be non-JSON-serializable

The writer uses:

Python

allow_nan=False

but mismatch paths can record:

Python

float("nan")

in max_diffs:

Python

... else float('nan')

If a finite/NaN mismatch occurs, the baseline may fail correctly but then fail to save its certificate.

Use None for a nonfinite or undefined difference:

Python

seed_results["max_diffs"][name] = (
float(abs(engine_scalar - frozen_scalar))
if np.isfinite(engine_scalar) and np.isfinite(frozen_scalar)
else None

)

Also guard matrix difference reporting against shape mismatch. If shapes differ, report shapes and dtypes rather than attempting subtraction.
6. CSV errors should fail before simulation

Currently, a missing or unreadable CSV produces either an empty lookup or only a warning. That can waste the complete baseline runtime.

Before any simulation, require:

Python

```
if not Path(frozen_csv_path).is_file():
return failed_result(...)
```

When parsing, fail immediately on:

duplicate (seed, branch) rows;
missing scalar columns;
missing expected seed/branch pairs;
extra or malformed rows;
parse exceptions.

Do not fall back to the matrix NPZ for historical scalars. The frozen production NPZ is the matrix/trajectory authority; the CSV is the scalar authority.
Historical helper: two smaller fidelity corrections
Use direct indexing

For mandatory equivalence, these should not silently fall back:

Python

```
continue_result.get("D_B_final", reference_density)
result.get("sample_times")
result.get("P_mean_late", nan)
```

Use direct indexing so incomplete engine output fails:

Python

```
reference_density = float(continue_result["D_B_final"])
times = result["sample_times"]
```

Match the frozen finite check

Frozen production computes the similarity ratio only when:

Python

```
np.isfinite(reference_late_similarity)
and abs(reference_late_similarity) > FROZEN_ZERO_TOL
```

The new helper only tests the absolute value. It gives the same result for NaN but not necessarily for infinity. Reproduce the frozen condition exactly.
Branch-safety status

The run_seed() distinction is now correct:

branch_order defines the full RNG namespace;
protocol.branches defines execution.

That is a good correction.

However, branch validation is only in run_seed(). Direct calls to simulate_future() and run_baseline_equivalence() can still pass an unknown branch that silently behaves like continue.

Put the allowlist check in apply_intervention() or simulate_future() itself:

Python

```
if branch not in KNOWN_BRANCHES:
raise ValueError(...)
```

Also validate:

Python

```
len(branch_order) == len(set(branch_order))
set(branch_order) <= KNOWN_BRANCHES
len(protocol.branches) == len(set(protocol.branches))
```

For the mandatory frozen gate, require the exact seven-branch order from the frozen manifest.
Preflight validation still needs to be invoked

You improved ModelConfig.validate(), ProtocolConfig.validate(), and DynamicState.validate(), but the main simulation APIs do not call them.

At minimum:

Python

```
def run_seed(...):
config.validate()
protocol.validate(config)
...
```

and:

Python

```
def run_baseline_equivalence(...):
config.validate()
protocol.validate(config)
...
```

Call state.validate(config):

after initialization;
after intervention;

at the prefix split if desired.

Do not run full state validation every timestep.

Also add finite checks for all scalar model parameters, not only omega; comparisons such as nan <= 0 do not reliably reject NaN.
Performance assessment

Two important improvements are present:

cached edge indices;
cached omega arrays.

Two remaining improvements are advisable before the five-level grid:

config.alpha is recomputed through np.exp twice per timestep. Cache the derived alpha once per immutable configuration.
History is still a list with pop(0). A bounded deque, as used by frozen production, avoids repeated shifting and more directly matches the authority implementation.

Make cached omega and edge arrays read-only to prevent accidental global mutation:

Python

arr.setflags(write=False)

These are not correctness blockers for the baseline, but should preferably be settled before the engine is frozen.
What is now correct

The revision successfully addresses several earlier concerns:

np.exp now matches the frozen alpha construction;
omega and edge computations are cached;
state dimension is checked against config.N;
history shapes and finite state values are checked;
protocol split/future consistency is checked;

run_seed() preserves the full seven-branch RNG namespace while executing a subset;
all eight historical field formulas have been reintroduced;
scalar comparisons no longer silently skip ordinary nonnumeric values;
extended late scalars are now compared internally against engine legacy fields;
Jaccard explicitly uses config.B_c;
dual horizon accumulation remains logically correct.

Final decision
Compile/import test

GO, after cleaning the copy-paste corruption in the actual file.
One-seed smoke test

GO only after adding the missing finalizer call to run_baseline_equivalence().
Mandatory 40-seed baseline-only run

NO-GO yet. Before launching it, fix at least:

baseline call to finalize_historical_branch_metrics;
missing prefix/matrix checks currently recorded as True;
contradictory false “informational” checks;
module-level 42-field frozen scalar schema;
JSON-safe failure reporting;
fail-fast frozen CSV validation.

After those corrections, the engine is close enough for compile → import → one-seed smoke → full baseline-only testing. No grid should start until that complete certificate passes exactly.

Now we will do a final pass through before we do our first testing on my system to see any bugs here is the first of three files:

#!/usr/bin/env python3

"""AIEF Experiment Engine v1 - Parameterized simulation kernel.

This is a reusable, validated simulation engine that reproduces the frozen

AIEF-1.4 production run exactly and supports parameterized studies.

FROZEN PRODUCTION VALUES (from AIEF_1_4_memory_ablation_updated_REAL.py):

N = 12

DT = 0.01

TAU = 0.5

OMEGA = [1.00, 1.02, 1.50, 2.00, 2.01, 1.01, 1.03, 1.51, 2.02, 2.00, 1.00, 1.04]

A = 1.0

Q = 1.5

DELTA = 0.05

ETA = 0.04

GAMMA = 0.015

MU = 0.03

T_WINDOW = 25.0 / mean(OMEGA) = 17.5

B_C = 0.12

B_INIT_SCALE = 0.0005

PROTOCOL (from frozen run):

T_SIM = 800.0

N_STEPS = int(round(T_SIM / DT)) = 80000

BURN_END = int(0.20 * N_STEPS) = 16000 steps = 160.0 time units

HIST_END = int(0.60 * N_STEPS) = 48000 steps = 480.0 time units

FUTURE_STEPS = N_STEPS - HIST_END = 32000 steps = 320.0 time units

DELAY_STEPS = int(round(TAU / DT)) = 50

WINDOW_STEPS = int(round(T_WINDOW / DT)) = 1750

RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS) = 8750

Architecture:

ModelConfig: frozen dataclass for all model parameters

ProtocolConfig: frozen dataclass for protocol parameters

DynamicState: state container (immutable where possible)

Core functions: initialize, advance, simulate, interventions

Validation: baseline equivalence test

NOTE: This is a candidate reference implementation. After passing 40/40 baseline

text

equivalence tests, it will be frozen as the reference implementation.

"""

from future import annotations

import copy

```
import hashlib

import json

import math

import os

import tempfile

import time

from dataclasses import dataclass, field

from pathlib import Path

from typing import Any

import numpy as np
================================================================================
Frozen Production Constants (DO NOT CHANGE)
================================================================================

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([
```

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005

FROZEN_ZERO_TOL = 1e-12

FROZEN_PROJECTION_NORM_TOL = 1e-14

FROZEN_GAP_TOL = 1e-9
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA)) # 1.428333...

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.502917152858814

FROZEN_T_WINDOW = FROZEN_T_STAR # 17.502917152858814 (exact, NOT rounded)

FROZEN_ALPHA = float(np.exp(-FROZEN_DT / FROZEN_T_WINDOW))

FROZEN_CAPACITY = (FROZEN_A - FROZEN_DELTA) / FROZEN_Q
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_WINDOW / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
Fixed physical horizon boundaries (in time units, NOT T_window dependent)
These are the PRIMARY estimand horizons

FIXED_PHYSICAL_IMMEDIATE_END = 17.5

FIXED_PHYSICAL_RECOVERY_END = 87.5

FIXED_PHYSICAL_FUTURE_END = 320.0
Derived fixed physical horizon steps

FIXED_PHYSICAL_IMMEDIATE_STEPS = int(round(FIXED_PHYSICAL_IMMEDIATE_END /
FROZEN_DT)) # 1750

FIXED_PHYSICAL_RECOVERY_STEPS = int(round(FIXED_PHYSICAL_RECOVERY_END /
FROZEN_DT)) # 8750

```
================================================================================
Model Configuration
================================================================================

@dataclass(frozen=True)

class ModelConfig:

text

"""Immutable model configuration.



All model-changing quantities must come from here. No hidden module globals.

"""

# Core parameters

N: int

dt: float

tau: float

omega: tuple[float, ...] # Immutable tuple



# Coupling parameters

a: float
```

q: float

delta: float

Dynamics parameters

eta: float

mu: float

gamma: float

EMA/threshold parameters

T_window: float

B_c: float

B_init_scale: float

scale_epsilon: float = 1e-12

Derived properties

@property

def mean_omega(self) -> float:

"""Mean of omega frequencies."""

return float(np.mean(self.omega))

@property

def T_star(self) -> float:

"""T_* = 25 / mean(omega)"""

return 25.0 / self.mean_omega

@property

def alpha(self) -> float:

"""EMA smoothing factor: exp(-dt / T_window)"""

return float(np.exp(-self.dt / self.T_window))

@property

def capacity(self) -> float:

"""Projection capacity: (a - delta) / q"""

return (self.a - self.delta) / self.q

@property

def delay_steps(self) -> int:

"""Delay steps: round(tau / dt)"""

return int(round(self.tau / self.dt))

def validate(self) -> None:

"""Validate model configuration with strict consistency checks.

Uses rtol=0, atol=0 for exact compatibility with frozen production.

All scalar parameters must be finite.

"""

Validate all scalar parameters are finite

if not np.isfinite(self.N):

raise ValueError("N must be finite")

if not np.isfinite(self.dt) or self.dt <= 0:

raise ValueError("dt must be finite and positive")

if not np.isfinite(self.tau) or self.tau <= 0:

raise ValueError("tau must be finite and positive")

if not np.isfinite(self.a) or self.a <= 0:

raise ValueError("a must be finite and positive")

if not np.isfinite(self.q) or self.q <= 0:

raise ValueError("q must be finite and positive")

if not np.isfinite(self.delta) or self.delta < 0:

raise ValueError("delta must be finite and nonnegative")

if not np.isfinite(self.eta) or self.eta < 0:

raise ValueError("eta must be finite and nonnegative")

if not np.isfinite(self.mu) or self.mu < 0:

raise ValueError("mu must be finite and nonnegative")

if not np.isfinite(self.gamma) or self.gamma < 0:

raise ValueError("gamma must be finite and nonnegative")

if not np.isfinite(self.T_window) or self.T_window <= 0:

raise ValueError("T_window must be finite and positive")

if not np.isfinite(self.B_c) or self.B_c < 0:

raise ValueError("B_c must be finite and nonnegative")

if not np.isfinite(self.B_init_scale) or self.B_init_scale < 0:

raise ValueError("B_init_scale must be finite and nonnegative")

if not np.isfinite(self.scale_epsilon) or self.scale_epsilon <= 0:

raise ValueError("scale_epsilon must be finite and positive")

Validate N is positive integer

if self.N <= 0:

raise ValueError("N must be positive")

Validate omega

if len(self.omega) != self.N:

raise ValueError(f"omega must have {self.N} elements")

if not np.all(np.isfinite(self.omega)):

raise ValueError("omega must contain only finite values")

Validate derived values

if not (0 < self.alpha <= 1):

raise ValueError(f"alpha={self.alpha} must be in (0, 1]")

if self.capacity < 0:

raise ValueError(f"capacity={self.capacity} must be nonnegative")

Strict consistency: delay_steps * dt must equal tau (exact)

if not np.isclose(self.delay_steps * self.dt, self.tau, rtol=0.0, atol=0.0):

raise ValueError(

f"delay_steps *dt ({self.delay_steps}* {self.dt} = {self.delay_steps * self.dt}) "

f"must equal tau ({self.tau}) exactly"

)

@dataclass(frozen=True)

class ProtocolConfig:

text

"""Protocol configuration for experiment execution."""

Simulation time parameters (in time units, not steps)

T_sim: float

burn_end_time: float

split_time: float

future_duration: float

Diagnostics

diagnostic_stride: int

Branches to simulate

branches: tuple[str, ...]

def n_steps(self, model: ModelConfig) -> int:

"""Total steps."""

return int(round(self.T_sim / model.dt))

def burn_end(self, model: ModelConfig) -> int:

"""Burn-in end step."""

return int(round(self.burn_end_time / model.dt))

def hist_end(self, model: ModelConfig) -> int:

"""History end step (split point)."""

return int(round(self.split_time / model.dt))

def future_steps(self, model: ModelConfig) -> int:

"""Future simulation steps."""

return int(round(self.future_duration / model.dt))

def validate(self, model: ModelConfig) -> None:

"""Validate protocol against model with strict consistency checks.

Uses rtol=0, atol=0 for exact compatibility with frozen production.

"""

if self.T_sim <= 0:

raise ValueError("T_sim must be positive")

if not (0 < self.burn_end_time < self.split_time < self.T_sim):

raise ValueError("Invalid time parameters: must have 0 < burn_end < split < T_sim")

if self.future_duration <= 0:

raise ValueError("future_duration must be positive")

if self.diagnostic_stride <= 0:

raise ValueError("diagnostic_stride must be positive")

Strict consistency: split_time + future_duration must equal T_sim

if not np.isclose(self.split_time + self.future_duration, self.T_sim, rtol=0.0, atol=0.0):

raise ValueError(

f"split_time ({self.split_time}) + future_duration ({self.future_duration}) "

f"= {self.split_time + self.future_duration} must equal T_sim ({self.T_sim}) exactly"

)

Validate that all time parameters are compatible with dt (exact divisibility)

if not np.isclose(self.T_sim / model.dt, round(self.T_sim / model.dt), rtol=0.0, atol=0.0):

raise ValueError(f"T_sim ({self.T_sim}) must be divisible by dt ({model.dt}) exactly")

if not np.isclose(self.burn_end_time / model.dt, round(self.burn_end_time / model.dt), rtol=0.0, atol=0.0):

raise ValueError(f"burn_end_time ({self.burn_end_time}) must be divisible by dt ({model.dt}) exactly")

if not np.isclose(self.split_time / model.dt, round(self.split_time / model.dt), rtol=0.0, atol=0.0):

raise ValueError(f"split_time ({self.split_time}) must be divisible by dt ({model.dt}) exactly")

if not np.isclose(self.future_duration / model.dt, round(self.future_duration / model.dt), rtol=0.0, atol=0.0):

raise ValueError(f"future_duration ({self.future_duration}) must be divisible by dt ({model.dt}) exactly")

Default frozen production configuration
NOTE: T_window must be the EXACT value 25.0/mean(OMEGA), not rounded

DEFAULT_MODEL_CONFIG = ModelConfig(

text

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=FROZEN_T_WINDOW, # 17.502917152858814 (exact)

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=FROZEN_ZERO_TOL,

)
Default protocol matching frozen run

DEFAULT_PROTOCOL_CONFIG = ProtocolConfig(

text

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=("continue", "reset_ema", "reset_B", "reset_B_ema", "B_carrier_only", "full_reset", "future_shuffle"),

)
==========
Cached Utility Values
==========
Cache for compute_edges to avoid recomputing on every step

EDGESCACHE: dict[int, tuple[np.ndarray, np.ndarray]] = {}
Cache for omega array conversion

OMEGAARRAY_CACHE: dict[tuple[float, ...], np.ndarray] = {}
==========
Utility Functions
==========

def compute_edges(N: int) -> tuple[np.ndarray, np.ndarray]:

text

"""Get upper triangle indices for edges.

Cached to avoid recomputing on every step.

"""

if N not in *EDGES*CACHE:

*EDGES*CACHE[N] = np.triu_indices(N, k=1)

return *EDGES*CACHE[N]

def get_omega_array(omega_tuple: tuple[float, ...]) -> np.ndarray:

text

"""Get omega as numpy array with caching.

Cached to avoid recomputing on every step.

"""

if omega_tuple not in *OMEGA*ARRAY_CACHE:

*OMEGA*ARRAY_CACHE[omega_tuple] = np.asarray(omega_tuple, dtype=float)

return *OMEGA*ARRAY_CACHE[omega_tuple]

def validate_B_matrix(B: np.ndarray, N: int, eps: float = 1e-12) -> None:

text

"""Validate B matrix: symmetric, nonnegative, zero diagonal."""

if B.shape != (N, N):

raise ValueError(f"B shape {B.shape} != ({N}, {N})")

if not np.allclose(B, B.T, atol=eps):

raise ValueError("B is not symmetric")

if not np.allclose(np.diag(B), 0, atol=eps):

raise ValueError("B diagonal is not zero")

if np.any(B < -eps):

raise ValueError("B has negative entries")

def file_sha256(path: str | os.PathLike[str]) -> str:

text

"""Compute SHA-256 hash of a file."""

```
digest = hashlib.sha256()
with open(path, "rb") as handle:
for block in iter(lambda: [handle.read](http://handle.read)(1024 * 1024), b""):
digest.update(block)
return digest.hexdigest()
def array_sha256(arr: np.ndarray) -> str:
text
"""Compute SHA-256 hash of a numpy array."""
digest = hashlib.sha256()
shape_bytes = np.asarray(arr.shape, dtype=np.int64).tobytes()
digest.update(shape_bytes)
digest.update(arr.dtype.str.encode())
contiguous = np.ascontiguousarray(arr)
if contiguous.size &gt; 0:
digest.update(contiguous.tobytes())
return digest.hexdigest()
def atomic_write_text(path: Path, text: str) -> None:
```

```
text

"""Write text atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{[path.name](http://path.name)}.", suffix=".tmp")

try:

with os.fdopen(fd, "w", encoding="utf-8") as handle:

handle.write(text)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))

finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

def atomic_write_json(path: Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically with fsync."""

text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"
```

atomic_write_text(path, text)

def atomic_write_npz(path: Path, **kwargs: Any) -> None:

text

"""Write NPZ atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{path.name}.", suffix=".tmp.npz")

try:

with os.fdopen(fd, "wb") as handle:

np.savez(handle, **kwargs)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))

finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

==========
RNG Utilities (matching frozen production exactly)
==========

def make_seed_sequence(seed: int, child_index: int) -> int:

text

"""Generate a child seed from SeedSequence(seed).spawn(...)[child_index].

This matches the frozen production RNG hierarchy exactly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

children = root.spawn(child_index + 1)

return int(children[child_index].generate_state(1, dtype=np.uint64)[0])

def make_prefix_rng(seed: int) -> np.random.Generator:

text

"""Create prefix RNG matching frozen production exactly.

Frozen production does:

root = np.random.SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = np.random.default_rng(init_sequence)

NOTE: default_rng(SeedSequence_object) is NOT the same as

default_rng(SeedSequence_object.generate_state(...))

The production uses the SeedSequence object directly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

return np.random.default_rng(init_sequence)

def make_branch_rngs(seed: int, branch_order: tuple[str, ...]) -> dict[str, np.random.Generator]:

text

"""Create branch RNGs matching frozen production algorithm.

Frozen: *, branch*root = SeedSequence(seed).spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))

branch_rng_seeds = {branch: int(sequence.generate_state(1, dtype=np.uint64)[0])

for branch, sequence in zip(branch_order, branch_sequences)}

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

*, branch*root = root.spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))

return {

branch: np.random.default_rng(int(sequence.generate_state(1, dtype=np.uint64)[0]))

for branch, sequence in zip(branch_order, branch_sequences)

}

==
Projection Accumulator (matching production)
==

@dataclass

class ProjectionTotals:

text

"""Accumulator for projection diagnostics - exact match to production."""

steps: int = 0

violations: int = 0

active: int = 0

norm_sum: float = 0.0

norm_max: float = 0.0

relative_state_sum: float = 0.0

relative_state_max: float = 0.0

relative_update_sum: float = 0.0

relative_update_max: float = 0.0

g_min: float = np.inf

g_min_pre: float = np.inf

def update(self, info: dict[str, Any]) -> None:

"""Update accumulator with step info."""

self.steps += 1

self.violations += int(info["pre_violation"])

self.g_min = min(self.g_min, float(info["g_min"]))

self.g_min_pre = min(self.g_min_pre, float(info["g_min_pre"]))

```
if info["projection_active"]:

self.active += 1

norm = float(info["projection_norm"])

rel_state = float(info["projection_relative_state"])

rel_update = float(info["projection_relative_update"])

self.norm_sum += norm

self.norm_max = max(self.norm_max, norm)

self.relative_state_sum += rel_state

self.relative_state_max = max(self.relative_state_max, rel_state)

self.relative_update_sum += rel_update

self.relative_update_max = max(self.relative_update_max, rel_update)

def metrics(self) -&gt; dict[str, float]:

"""Compute final metrics from accumulated values."""

active_denominator = max(self.active, 1)

return {

"G_min_future": float(self.g_min),

"G_min_pre_future": float(self.g_min_pre),
```

"Pre_gap_violation_fraction": float(self.violations / self.steps),

"Projection_active_fraction": float(self.active / self.steps),

"Projection_active_given_violation": float(self.active / self.violations)

if self.violations > 0

else float('nan'),

"Projection_norm_mean_active": float(

self.norm_sum / active_denominator

),

"Projection_norm_max": float(self.norm_max),

"Projection_relative_state_mean_active": float(

self.relative_state_sum / active_denominator

),

"Projection_relative_state_max": float(self.relative_state_max),

"Projection_relative_update_mean_active": float(

self.relative_update_sum / active_denominator

),

"Projection_relative_update_max": float(self.relative_update_max),

}

```
================================================================================
Dynamic State
================================================================================

class DynamicState:

text

"""State container for AIEF-1.4 dynamics.



Matches the frozen production implementation exactly.

"""



def **init**(

self,

theta: np.ndarray,

B: np.ndarray,

coherence_acc: np.ndarray,

history: list[np.ndarray] | np.ndarray,

):
```

```
self.theta = np.asarray(theta, dtype=float).copy()

self.B = np.asarray(B, dtype=float).copy()

self.coherence_acc = np.asarray(coherence_acc, dtype=complex).copy()


# Handle history input - ensure it's a list of arrays

if isinstance(history, np.ndarray):

# If it's a 2D array, convert to list of 1D arrays

self.history = [np.asarray(h, dtype=float).copy() for h in history]

else:

self.history = [np.asarray(h, dtype=float).copy() for h in history]



def clone(self) -&gt; "DynamicState":

"""Create a deep copy of the state."""

return DynamicState(

theta=self.theta.copy(),

B=self.B.copy(),

coherence_acc=self.coherence_acc.copy(),

history=[h.copy() for h in self.history],
```

)

@property

def N(self) -> int:

return len(self.theta)

def validate(self, config: ModelConfig, step: int | None = None) -> None:

"""Validate state consistency.

Checks:

- State dimension matches config.N

- History length is exactly delay_steps + 1

- All arrays have correct shapes

- B matrix is symmetric, zero-diagonal, non-negative

- All history vectors have shape (N,)

- All arrays are finite

"""

```
# Check state dimension matches config

if self.N != config.N:

raise RuntimeError(

f"State N ({self.N}) does not match config N ({config.N})"

)


# Check history length

expected_history_len = config.delay_steps + 1

if len(self.history) != expected_history_len:

raise RuntimeError(

f"History length {len(self.history)} != expected {expected_history_len} "

f"(delay_steps={config.delay_steps}) at step {step}"

)


# Check all history vectors have correct shape

for i, h in enumerate(self.history):

if h.shape != (self.N,):

raise RuntimeError(
```

```
f"History[{i}] shape {h.shape} != ({self.N},)"

)

# Check theta shape

if self.theta.shape != (self.N,):

raise RuntimeError(f"theta shape {self.theta.shape} != ({self.N},)")

# Check B shape

if self.B.shape != (self.N, self.N):

raise RuntimeError(f"B shape {self.B.shape} != ({self.N}, {self.N})")

# Check coherence_acc shape

if self.coherence_acc.shape != (self.N, self.N):

raise RuntimeError(f"coherence_acc shape {self.coherence_acc.shape} != ({self.N}, {self.N})")

# Check all arrays are finite

if not np.all(np.isfinite(self.theta)):

raise RuntimeError("theta contains non-finite values")

if not np.all(np.isfinite(self.B)):
```

raise RuntimeError("B contains non-finite values")

if not np.all(np.isfinite(self.coherence_acc)):

raise RuntimeError("coherence_acc contains non-finite values")

for i, h in enumerate(self.history):

if not np.all(np.isfinite(h)):

raise RuntimeError(f"History[{i}] contains non-finite values")

Check B symmetry

if not np.allclose(self.B, self.B.T, atol=1e-12):

raise RuntimeError("B matrix is not symmetric")

Check B diagonal

if not np.allclose(np.diag(self.B), 0, atol=1e-12):

raise RuntimeError("B matrix diagonal is not zero")

Check B non-negativity

if np.any(self.B < -1e-12):

raise RuntimeError("B matrix has negative entries")

```
================================================================================
Simulation Core (exact reproduction of frozen production)
================================================================================

def initialize_state(

text

seed: int,

config: ModelConfig,

) -> DynamicState:

text

"""Initialize state from seed using frozen algorithm exactly.



Matches production: simulate_prefix() initialization.

Uses: root = SeedSequence(seed); init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence)

"""

N = config.N

rng = make_prefix_rng(seed)
```

```
# Initialize theta: uniform in [0, 2*pi) - matching production

theta = rng.random(N) *2.0* np.pi



# Initialize B: small random symmetric matrix

B = config.B_init_scale * rng.random((N, N))

B = 0.5 * (B + B.T)

np.fill_diagonal(B, 0.0)



# Initialize coherence accumulator

coherence_acc = np.zeros((N, N), dtype=complex)



# Initialize history: DELAY_STEPS + 1 copies of theta

history = [theta.copy() for *in range(config.delay*steps + 1)]



return DynamicState(

theta=theta,
```

```
B=B,

coherence_acc=coherence_acc,

history=history,

)

def compute_R(

text

state: DynamicState,

config: ModelConfig,

) -> np.ndarray:

text

"""Compute coherence matrix R from current state.



Exact reproduction of production compute_R().

"""

N = state.N

delayed = state.history[0]
```

```
# Convert omega to numpy array for indexing (cached)

omega_arr = get_omega_array([config.omega](http://config.omega))



dphi = (

state.theta[:, None]

- delayed[None, :]

- ((omega_arr[:, None] + omega_arr[None, :]) / 2.0) * config.tau

)

z_inst = np.exp(1j * dphi)



# Update EMA

state.coherence_acc[:] = (

config.alpha *state.coherence_acc + (1.0 - config.alpha)* z_inst

)



# Compute magnitude and symmetrize

result = np.abs(state.coherence_acc)
```

result = 0.5 * (result + result.T)

np.fill_diagonal(result, 0.0)

return result

def advance_state(

text

state: DynamicState,

config: ModelConfig,

rng: np.random.Generator,

shuffle_driver: bool = False,

) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:

text

"""Advance state by one time step.

Exact reproduction of production advance_state().

Returns (R_physical, R_driver, info).

Production also returns projection diagnostics:

- projection_active: whether correction was applied

- projection_norm: norm of correction

- projection_relative_state: correction_norm / ||B_unprojected||

- projection_relative_update: correction_norm / ||B_unprojected - B_previous||

"""

N = state.N

edges = compute_edges(N)

Store previous B for projection

B_previous = state.B.copy()

Compute R matrices

R_physical = compute_R(state, config)

R_driver = R_physical

if shuffle_driver:

```
values = R_physical[edges].copy()

rng.shuffle(values)

R_driver = np.zeros_like(R_physical)

R_driver[edges] = values

R_driver[(edges[1], edges[0])] = values



# Update B

B_unprojected = state.B + config.dt * (

(config.eta + [config.mu](http://config.mu)) * R_driver

- [config.mu](http://config.mu)

- config.gamma * state.B

)

B_unprojected = np.maximum(B_unprojected, 0.0)

B_unprojected = 0.5 * (B_unprojected + B_unprojected.T)

np.fill_diagonal(B_unprojected, 0.0)



# Projection diagnostics (matching production exactly)
```

proposed_update_norm = float(np.linalg.norm(B_unprojected - B_previous))

Check gap constraint

row_sum_pre = np.sum(B_unprojected, axis=1)

g_min_pre = float(np.min(config.a - config.q * row_sum_pre))

pre_violation = bool(g_min_pre < config.delta)

Project if needed

B_projected = B_unprojected

correction_norm = 0.0

if pre_violation:

scale = np.minimum(1.0, config.capacity / (row_sum_pre + config.scale_epsilon))

B_projected = B_unprojected * np.minimum.outer(scale, scale)

np.fill_diagonal(B_projected, 0.0)

correction_norm = float(np.linalg.norm(B_projected - B_unprojected))

projection_active = bool(correction_norm > FROZEN_PROJECTION_NORM_TOL)

```
relative_state = correction_norm / max(

float(np.linalg.norm(B_unprojected)), FROZEN_ZERO_TOL

)

relative_update = correction_norm / max(proposed_update_norm, FROZEN_ZERO_TOL)



# Validate projection

state.B = B_projected

row_sum = np.sum(state.B, axis=1)

g_min = float(np.min(config.a - config.q * row_sum))

if g_min &lt; [config.delta](http://config.delta) - 1e-9:

raise RuntimeError(f"Gap projection failed: g_min={g_min}")



# Update theta

delayed = state.history[0]

omega_arr = get_omega_array([config.omega](http://config.omega))

coupling = np.sum(

state.B * np.sin(delayed[None, :] - state.theta[:, None]), axis=1
```

```
)

state.theta = np.mod(

state.theta + config.dt * (omega_arr + coupling),

2.0 * np.pi

)



# Update history

state.history.append(state.theta.copy())

if len(state.history) &gt; config.delay_steps + 1:

state.history.pop(0)



info = {

"pre_violation": pre_violation,

"projection_active": projection_active,

"projection_norm": correction_norm,

"projection_relative_state": relative_state,

"projection_relative_update": relative_update,
```

"g_min_pre": g_min_pre,

"g_min": g_min,

}

return R_physical, R_driver, info

def simulate_prefix(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

) -> tuple[DynamicState, dict[str, Any]]:

text

"""Simulate prefix up to split_time.

Exact reproduction of production simulate_prefix().

Returns (state, prefix_dict).

Production uses:

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence) # for initialization

Then in the loop, uses:

unused_rng = default_rng(0) # for advance_state when shuffle_driver=False

"""

N = config.N

edges = compute_edges(N)

Initialize state (uses init_sequence via make_prefix_rng)

state = initialize_state(seed, config)

Get RNG for prefix simulation loop

Production uses unused_rng = default_rng(0) for the advance_state calls

when shuffle_driver=False

unused_rng = np.random.default_rng(0)

Simulate to split point

R_past_sum = np.zeros((N, N), dtype=float)

R_past_count = 0

burn_end = protocol.burn_end(config)

hist_end = protocol.hist_end(config)

for step in range(hist_end):

R_physical, *,* = advance_state(state, config, unused_rng, shuffle_driver=False)

if burn_end <= step < hist_end:

R_past_sum += R_physical

R_past_count += 1

prefix = {

"B_at_split": state.B.copy(),

"R_physical_past": R_past_sum / R_past_count if R_past_count > 0 else np.zeros((N, N)),

"theta_at_split": state.theta.copy(),

"coherence_acc_at_split": state.coherence_acc.copy(),

"history_at_split": np.stack(state.history),

"selected_at_split": (state.B > config.B_c).astype(np.uint8),

}

return state, prefix

def apply_intervention(

text

state: DynamicState,

branch: str,

config: ModelConfig,

rng: np.random.Generator,

) -> None:

text

"""Apply intervention to state. Mutates in place.

Exact reproduction of production apply_intervention().

"""

Validate branch is known

if branch not in KNOWN_BRANCHES:

raise ValueError(f"Unknown branch '{branch}'. Known branches: {sorted(KNOWN_BRANCHES)}")

N = state.N

if branch in {"reset_ema", "reset_B_ema", "B_carrier_only", "full_reset"}:

Zero coherence accumulator

state.coherence_acc.fill(0.0)

if branch in {"reset_B", "reset_B_ema", "full_reset"}:

Zero B

state.B.fill(0.0)

```
if branch in {"B_carrier_only", "full_reset"}:

# Randomize theta and history

new_theta = rng.random(N) *2.0* np.pi

state.theta = new_theta

state.history = [new_theta.copy() for *in range(config.delay*steps + 1)]



# Ensure symmetry

state.B = 0.5 * (state.B + state.B.T)

np.fill_diagonal(state.B, 0.0)

def simulate_future(

text

split_state: DynamicState,

prefix: dict[str, Any],

branch: str,

config: ModelConfig,

protocol: ProtocolConfig,
```

rng: np.random.Generator,

) -> dict[str, Any]:

text

"""Simulate future trajectory from split state.

Reproduces production simulate_future() with ALL required outputs.

CRITICAL: Now accumulates BOTH horizon systems:

Fixed Physical Horizons (time-based, NOT T_window dependent):

- immediate: 0 to 17.5 time units

- recovery: 17.5 to 87.5 time units

- late: 87.5 to 320 time units

EMA-Normalized Horizons (T_window dependent):

- immediate: 0 to 1*T_window

- recovery: 1*T_window to 5*T_window

- late: 5*T_window to end

At rho=1 (T_window=17.502917...), the fixed physical and normalized horizons

are approximately aligned but use exact step boundaries.

Production accumulates:

- P_full, P_immediate, P_recovery, P_late (incidence matrices)

- B_mean_full, B_mean_immediate, B_mean_recovery, B_mean_late

- R_physical_future, R_driver_future

- Projection diagnostics via ProjectionTotals

- Various scalar metrics

Uses horizon system from production for normalized:

- WINDOW_STEPS = round(T_window / dt)

- RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS)

For fixed physical:

- FIXED_IMMEDIATE_END = round(17.5 / dt)

- FIXED_RECOVERY_END = round(87.5 / dt)

"""

state = split_state.clone()

apply_intervention(state, branch, config, rng)

N = state.N

edges = compute_edges(N)

future_steps = protocol.future_steps(config)

Compute horizon step boundaries

Normalized EMA horizons (T_window dependent)

window_steps = int(round(config.T_window / config.dt))

recovery_end_ema = min(5 * window_steps, future_steps)

Fixed physical horizons (NOT T_window dependent)

These use the production's physical time boundaries: 17.5, 87.5, 320

```
fixed_immediate_end = int(round(17.5 / config.dt)) # 17.5 time units

fixed_recovery_end = int(round(87.5 / config.dt)) # 87.5 time units

# Late goes from 87.5 to end (320 time units)



# Clamp all boundaries to future_steps

recovery_end_ema = min(recovery_end_ema, future_steps)

fixed_immediate_end = min(fixed_immediate_end, future_steps)

fixed_recovery_end = min(fixed_recovery_end, future_steps)



# Initialize accumulators for BOTH horizon systems

# P matrices (incidence: B &gt; B_c as 0/1)

P_sums_ema = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}
```

```
P_sums_fixed = {
"full": np.zeros((N, N), dtype=float),
"immediate": np.zeros((N, N), dtype=float),
"recovery": np.zeros((N, N), dtype=float),
"late": np.zeros((N, N), dtype=float),
}

# B matrices (mean B values)
B_sums_ema = {
"full": np.zeros((N, N), dtype=float),
"immediate": np.zeros((N, N), dtype=float),
"recovery": np.zeros((N, N), dtype=float),
"late": np.zeros((N, N), dtype=float),
}
B_sums_fixed = {
"full": np.zeros((N, N), dtype=float),
"immediate": np.zeros((N, N), dtype=float),
```

```
"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}



# Counts for each horizon in both systems

counts_ema = {"full": 0, "immediate": 0, "recovery": 0, "late": 0}

counts_fixed = {"full": 0, "immediate": 0, "recovery": 0, "late": 0}



# R matrices

R_physical_sum = np.zeros((N, N), dtype=float)

R_driver_sum = np.zeros((N, N), dtype=float)



# Projection accumulator (matching production)

projection = ProjectionTotals()



# Trajectory accumulators

sample_times = []
```

```
density_trajectory = []

similarity_trajectory = []



# For retention/emergence calculations

r_sum = 0.0

r_count = 0

r_late_ema_sum = 0.0

r_late_ema_count = 0

r_late_fixed_sum = 0.0

r_late_fixed_count = 0



B_reference = prefix["B_at_split"]

shuffle_driver = (branch == "future_shuffle")



for step in range(future_steps):

R_physical, R_driver, info = advance_state(

state, config, rng, shuffle_driver=shuffle_driver
```

)

Update projection accumulator

projection.update(info)

Accumulate R matrices

R_physical_sum += R_physical

R_driver_sum += R_driver

Determine which horizons this step belongs to for BOTH systems

selected = (state.B > config.B_c).astype(float)

=== EMA-Normalized Horizons ===

Full: all steps

P_sums_ema["full"] += selected

B_sums_ema["full"] += state.B

counts_ema["full"] += 1

immediate: 0 to window_steps (exclusive)

if step < window_steps:

P_sums_ema["immediate"] += selected

B_sums_ema["immediate"] += state.B

counts_ema["immediate"] += 1

recovery: window_steps to recovery_end_ema (exclusive)

if window_steps <= step < recovery_end_ema:

P_sums_ema["recovery"] += selected

B_sums_ema["recovery"] += state.B

counts_ema["recovery"] += 1

late: recovery_end_ema to end

if recovery_end_ema <= step:

P_sums_ema["late"] += selected

B_sums_ema["late"] += state.B

counts_ema["late"] += 1

=== Fixed Physical Horizons ===

Full: all steps (same as EMA)

P_sums_fixed["full"] += selected

B_sums_fixed["full"] += state.B

counts_fixed["full"] += 1

immediate: 0 to fixed_immediate_end (exclusive)

if step < fixed_immediate_end:

P_sums_fixed["immediate"] += selected

B_sums_fixed["immediate"] += state.B

counts_fixed["immediate"] += 1

recovery: fixed_immediate_end to fixed_recovery_end (exclusive)

if fixed_immediate_end <= step < fixed_recovery_end:

P_sums_fixed["recovery"] += selected

B_sums_fixed["recovery"] += state.B

counts_fixed["recovery"] += 1

late: fixed_recovery_end to end

if fixed_recovery_end <= step:

P_sums_fixed["late"] += selected

```
B_sums_fixed["late"] += state.B

counts_fixed["late"] += 1


# Trajectory sampling - CRITICAL: r_mean_late is ONLY accumulated here

if step % protocol.diagnostic_stride == 0:

t = (step + 1) * config.dt

sample_times.append(t)


# Density: fraction of edges above B_c

density = float(np.count_nonzero(state.B[edges] &gt; config.B_c) / len(edges[0]))

density_trajectory.append(density)


# Similarity: cosine between current and reference B

B_vec = state.B[edges]

B_ref_vec = B_reference[edges]

norm_B = np.linalg.norm(B_vec)

norm_ref = np.linalg.norm(B_ref_vec)

if norm_B &gt; 0 and norm_ref &gt; 0:
```

```
similarity = float([np.dot](http://np.dot)(B_vec, B_ref_vec) / (norm_B * norm_ref))

else:

similarity = float('nan')

similarity_trajectory.append(similarity)


# Track overall r

r_value = float(np.abs(np.mean(np.exp(1j * state.theta))))

r_sum += r_value

r_count += 1


# Track late-phase r for BOTH horizon systems

if recovery_end_ema &lt;= step:

r_late_ema_sum += r_value

r_late_ema_count += 1


if fixed_recovery_end &lt;= step:

r_late_fixed_sum += r_value

r_late_fixed_count += 1
```

Compute mean matrices for BOTH horizon systems

P_ema = {name: P_sums_ema[name] / counts_ema[name] if counts_ema[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

P_fixed = {name: P_sums_fixed[name] / counts_fixed[name] if counts_fixed[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

B_mean_ema = {name: B_sums_ema[name] / counts_ema[name] if counts_ema[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

B_mean_fixed = {name: B_sums_fixed[name] / counts_fixed[name] if counts_fixed[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

Compute R means

R_physical_mean = R_physical_sum / future_steps if future_steps > 0 else np.zeros((N, N))

R_driver_mean = R_driver_sum / future_steps if future_steps > 0 else np.zeros((N, N))

```
# Final state

B_final = state.B.copy()

final_weights = state.B[edges]

final_rows = np.sum(state.B, axis=1)



# Retention and emergence for BOTH horizon systems

split_selected = prefix["selected_at_split"][edges].astype(bool)



# EMA-normalized

p_late_edges_ema = P_ema["late"][edges]

retention_ema = (

float(np.mean(p_late_edges_ema[split_selected]))

if np.any(split_selected)

else float('nan')

)

emergence_ema = (

float(np.mean(p_late_edges_ema[~split_selected]))
```

```
if np.any(~split_selected)

else float('nan')

)



# Fixed physical

p_late_edges_fixed = P_fixed["late"][edges]

retention_fixed = (

float(np.mean(p_late_edges_fixed[split_selected]))

if np.any(split_selected)

else float('nan')

)

emergence_fixed = (

float(np.mean(p_late_edges_fixed[~split_selected]))

if np.any(~split_selected)

else float('nan')

)
```

```
# Minimum similarity from trajectory

similarity_array = np.array(similarity_trajectory, dtype=float)

finite_similarity = similarity_array[np.isfinite(similarity_array)]

minimum_similarity = (

float(np.min(finite_similarity)) if len(finite_similarity) &gt; 0

else float('nan')

)



# Build result with BOTH horizon systems

result = {

# === Matrix outputs for EMA-normalized horizons (backward compatible) ===

"B_final": B_final,

"B_mean_late": B_mean_ema["late"],

"P_full": P_ema["full"],

"P_immediate": P_ema["immediate"],

"P_recovery": P_ema["recovery"],

"P_late": P_ema["late"],
```

"R_physical_future": R_physical_mean,

"R_driver_future": R_driver_mean,

=== Matrix outputs for FIXED-PHYSICAL horizons (NEW) ===

"B_mean_late_fixed_physical": B_mean_fixed["late"],

"P_full_fixed_physical": P_fixed["full"],

"P_immediate_fixed_physical": P_fixed["immediate"],

"P_recovery_fixed_physical": P_fixed["recovery"],

"P_late_fixed_physical": P_fixed["late"],

=== Matrix outputs for NORMALIZED-EMA horizons (NEW - explicit) ===

"B_mean_late_normalized_EMA": B_mean_ema["late"],

"P_full_normalized_EMA": P_ema["full"],

"P_immediate_normalized_EMA": P_ema["immediate"],

"P_recovery_normalized_EMA": P_ema["recovery"],

"P_late_normalized_EMA": P_ema["late"],

Trajectory outputs

"sample_times": np.array(sample_times, dtype=float),

"density_trajectory": np.array(density_trajectory, dtype=float),

"similarity_trajectory": np.array(similarity_trajectory, dtype=float),

Scalar outcomes - EMA-normalized (backward compatible names)

"D_B_final": float(np.count_nonzero(final_weights > config.B_c) / len(edges[0])),

"P_mean_full": float(np.mean(P_ema["full"][edges])),

"P_mean_immediate": float(np.mean(P_ema["immediate"][edges])),

"P_mean_recovery": float(np.mean(P_ema["recovery"][edges])),

"P_mean_late": float(np.mean(P_ema["late"][edges])),

Scalar outcomes - FIXED-PHYSICAL (NEW - primary estimands)

"P_mean_full_fixed_physical": float(np.mean(P_fixed["full"][edges])),

"P_mean_immediate_fixed_physical": float(np.mean(P_fixed["immediate"][edges])),

"P_mean_recovery_fixed_physical": float(np.mean(P_fixed["recovery"][edges])),

"P_mean_late_fixed_physical": float(np.mean(P_fixed["late"][edges])),

Scalar outcomes - NORMALIZED-EMA (NEW - explicit)

"P_mean_full_normalized_EMA": float(np.mean(P_ema["full"][edges])),

"P_mean_immediate_normalized_EMA": float(np.mean(P_ema["immediate"][edges])),

"P_mean_recovery_normalized_EMA": float(np.mean(P_ema["recovery"][edges])),

"P_mean_late_normalized_EMA": float(np.mean(P_ema["late"][edges])),

Similarity metrics - BOTH systems

"Weighted_similarity_final": cosine_edges(B_reference, B_final, edges),

"Weighted_similarity_late": cosine_edges(B_reference, B_mean_ema["late"], edges),

"Weighted_similarity_late_fixed_physical": cosine_edges(B_reference, B_mean_fixed["late"], edges),

"Weighted_similarity_late_normalized_EMA": cosine_edges(B_reference, B_mean_ema["late"], edges),

"Jaccard_final": jaccard_edges(B_reference, B_final, edges, config.B_c),

"Jaccard_final_fixed_physical": jaccard_edges(B_reference, B_final, edges, config.B_c),

Retention/emergence - BOTH systems

"Retention_selected_late": retention_ema,

"Emergence_unselected_late": emergence_ema,

"Retention_contrast_late": retention_ema - emergence_ema,

"Retention_selected_late_fixed_physical": retention_fixed,

"Emergence_unselected_late_fixed_physical": emergence_fixed,

"Retention_contrast_late_fixed_physical": retention_fixed - emergence_fixed,

"Retention_selected_late_normalized_EMA": retention_ema,

"Emergence_unselected_late_normalized_EMA": emergence_ema,

"Retention_contrast_late_normalized_EMA": retention_ema - emergence_ema,

Correlations with past R - BOTH systems

"Corr_Rpast_Pfull": safe_corr(prefix["R_physical_past"][edges], P_ema["full"][edges]),

"Corr_Rpast_Plate": safe_corr(prefix["R_physical_past"][edges], P_ema["late"][edges]),

"Corr_Rpast_Pfull_fixed_physical": safe_corr(prefix["R_physical_past"][edges], P_fixed["full"][edges]),

"Corr_Rpast_Plate_fixed_physical": safe_corr(prefix["R_physical_past"][edges], P_fixed["late"][edges]),

"Corr_Rpast_Pfull_normalized_EMA": safe_corr(prefix["R_physical_past"][edges], P_ema["full"][edges]),

"Corr_Rpast_Plate_normalized_EMA": safe_corr(prefix["R_physical_past"][edges], P_ema["late"][edges]),

Weight statistics (same for both - based on final state)

"Total_edge_weight": float(np.sum(final_weights)),

"Mean_row_sum": float(np.mean(final_rows)),

"Max_row_sum": float(np.max(final_rows)),

"Nonzero_edge_fraction": float(np.mean(final_weights > config.scale_epsilon)),

"Effective_edges": effective_number(final_weights),

"Weight_gini": gini_coefficient(final_weights),

"Weight_entropy": normalized_entropy(final_weights),

Order parameter - BOTH systems

"r_mean_future": float(r_sum / r_count) if r_count > 0 else float('nan'),

"r_mean_late": float(r_late_ema_sum / r_late_ema_count) if r_late_ema_count > 0 else float('nan'),

"r_mean_late_fixed_physical": float(r_late_fixed_sum / r_late_fixed_count) if r_late_fixed_count > 0 else float('nan'),

"r_mean_late_normalized_EMA": float(r_late_ema_sum / r_late_ema_count) if r_late_ema_count > 0 else float('nan'),

Minimum similarity

"Minimum_similarity": minimum_similarity,

Projection diagnostics (from ProjectionTotals)

**projection.metrics(),

Horizon metadata

"window_steps": window_steps,

"recovery_end_ema": recovery_end_ema,

"fixed_immediate_end": fixed_immediate_end,

"fixed_recovery_end": fixed_recovery_end,

}

return result

==========
Metric Helpers
==========

def cosine_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None) -> float:

text

"""Compute cosine similarity between reference and current B matrices over edges.

If edges is None, computes over all upper triangle edges.

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

x = np.asarray(reference[edges], dtype=float)

y = np.asarray(current[edges], dtype=float)

denominator = float(np.linalg.norm(x) * np.linalg.norm(y))

if denominator <= 1e-12:

return float('nan')

return float(np.dot(x, y) / denominator)

def jaccard_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None, threshold: float = FROZEN_ZERO_TOL) -> float:

text

"""Compute Jaccard similarity between reference and current B matrices over edges.

Uses B > threshold as the threshold for edge presence.

If edges is None, computes over all upper triangle edges.

Matches production: jaccard_edges(B_reference, state.B) uses B > B_C

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

x = reference[edges] > threshold

y = current[edges] > threshold

union = np.count_nonzero(x | y)

if union == 0:

return float('nan')

return float(np.count_nonzero(x & y) / union)

def safe_corr(x: np.ndarray, y: np.ndarray) -> float:

text

"""Compute correlation coefficient safely."""

x = np.asarray(x, dtype=float)

```
y = np.asarray(y, dtype=float)

valid = np.isfinite(x) &amp;amp; np.isfinite(y)

x = x[valid]

y = y[valid]

if len(x) &amp;lt; 3 or np.std(x) &amp;lt; 1e-12 or np.std(y) &amp;lt; 1e-12:

return float('nan')

return float(np.corrcoef(x, y)[0, 1])

def effective_number(weights: np.ndarray) -> float:

text

"""Compute effective number of edges."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

sum_sq = float(np.sum(weights**2))

if total &amp;lt;= 1e-12 or sum_sq &amp;lt;= 1e-12:

return 0.0

return total**2 / sum_sq

def gini_coefficient(weights: np.ndarray) -> float:

text
```

```
"""Compute Gini coefficient."""

x = np.sort(np.asarray(weights, dtype=float))

total = float(np.sum(x))

if total &lt;= 1e-12:

return 0.0

n = len(x)

ranks = np.arange(1, n + 1, dtype=float)

return float(2.0 *[np.dot](http://np.dot)(ranks, x) / (n* total) - (n + 1.0) / n)

def normalized_entropy(weights: np.ndarray) -> float:

text

"""Compute normalized entropy."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

if total &lt;= 1e-12:

return 0.0

probabilities = weights / total

positive = probabilities &gt; 0.0
```

```
entropy = -float(np.sum(probabilities[positive] * np.log(probabilities[positive])))

return entropy / float(np.log(len(weights)))

def time_to_at_least(

text

sample_times: np.ndarray,

values: np.ndarray,

target: float,

) -> float:

text

"""Find the first time at which values reach at least the target.



Matches frozen production exactly.

Returns NaN if target is never reached.

"""

sample_times = np.asarray(sample_times, dtype=float)

values = np.asarray(values, dtype=float)

valid = np.isfinite(values)
```

indices = np.flatnonzero(valid & (values >= target))

return float(sample_times[indices[0]]) if len(indices) > 0 else float('nan')

def time_to_below(

text

sample_times: np.ndarray,

values: np.ndarray,

threshold: float,

) -> float:

text

"""Find the first time at which values drop below the threshold.

Matches frozen production exactly.

Returns NaN if threshold is never dropped below.

"""

sample_times = np.asarray(sample_times, dtype=float)

values = np.asarray(values, dtype=float)

valid = np.isfinite(values)

indices = np.flatnonzero(valid & (values < threshold))

return float(sample_times[indices[0]]) if len(indices) > 0 else float('nan')

def finalize_historical_branch_metrics(

text

branch_results: dict[str, dict[str, Any]],

) -> None:

text

"""Compute and add the 8 historical scalar fields that compare branches to continue.

These fields are computed in frozen production by run_seed() after all branches

are simulated. They must be computed here to match the frozen schema.

The 8 fields are:

- Time_to_50pct_continue_final_density

- Time_to_90pct_continue_final_density

- Time_to_50pct_continue_late_similarity

- Time_to_90pct_continue_late_similarity

- Time_to_similarity_below_0_9

- Time_to_similarity_below_0_5

- Late_persistence_ratio_to_continue

- Late_similarity_ratio_to_continue

These are computed for each branch relative to the continue branch reference values.

"""

Get continue branch results for reference

if "continue" not in branch_results:

raise RuntimeError("continue branch is required for historical metric computation")

continue_result = branch_results["continue"]

Get reference values from continue branch

reference_density = float(continue_result["D_B_final"])

reference_late_similarity = float(continue_result["Weighted_similarity_late"])

reference_late_persistence = float(continue_result["P_mean_late"])

```
for branch, result in branch_results.items():

# Use direct indexing to fail if keys are missing

times = result["sample_times"]

density = result["density_trajectory"]

similarity = result["similarity_trajectory"]


# Compute time-based metrics relative to continue branch reference


# Compute time-based metrics relative to continue branch reference

result["Time_to_50pct_continue_final_density"] = time_to_at_least(

times, density, 0.5 * reference_density

)

result["Time_to_90pct_continue_final_density"] = time_to_at_least(

times, density, 0.9 * reference_density

)

result["Time_to_50pct_continue_late_similarity"] = time_to_at_least(

times, similarity, 0.5 * reference_late_similarity
```

```
)

result["Time_to_90pct_continue_late_similarity"] = time_to_at_least(

times, similarity, 0.9 * reference_late_similarity

)

result["Time_to_similarity_below_0_9"] = time_to_below(

times, similarity, 0.9

)

result["Time_to_similarity_below_0_5"] = time_to_below(

times, similarity, 0.5

)


# Compute ratio metrics - use direct indexing to fail if missing

branch_late_persistence = float(result["P_mean_late"])

branch_late_similarity = float(result["Weighted_similarity_late"])


# Match frozen production exactly: check both finite and &gt; ZERO_TOL

if np.isfinite(reference_late_persistence) and abs(reference_late_persistence) &gt;
FROZEN_ZERO_TOL:

result["Late_persistence_ratio_to_continue"] = (
```

branch_late_persistence / reference_late_persistence

)

else:

result["Late_persistence_ratio_to_continue"] = float('nan')

if np.isfinite(reference_late_similarity) and abs(reference_late_similarity) > FROZEN_ZERO_TOL:

result["Late_similarity_ratio_to_continue"] = (

branch_late_similarity / reference_late_similarity

)

else:

result["Late_similarity_ratio_to_continue"] = float('nan')

==========
Required Output Schema for Baseline Equivalence
==========
IMPORTANT: We separate FROZEN keys (must exist in historical NPZ) from
EXTENDED keys (new in v1, only validated internally at rho=1).
The frozen NPZ does NOT contain dual-horizon keys, so baseline equivalence
must only check FROZEN keys against historical data.
Required prefix keys for baseline equivalence

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)
These are the keys that MUST exist in the frozen NPZ for baseline equivalence

FROZEN_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
These are the NEW keys introduced in v1 for dual-horizon support
They will NOT exist in frozen NPZ, so baseline equivalence skips them

EXTENDED_BRANCH_MATRIX_KEYS = (

text

"B_mean_late_fixed_physical",

"P_full_fixed_physical",

"P_immediate_fixed_physical",

"P_recovery_fixed_physical",

"P_late_fixed_physical",

"B_mean_late_normalized_EMA",

"P_full_normalized_EMA",

"P_immediate_normalized_EMA",

"P_recovery_normalized_EMA",

"P_late_normalized_EMA",

)

All matrix keys (frozen + extended)

REQUIRED_BRANCH_MATRIX_KEYS = FROZEN_BRANCH_MATRIX_KEYS + EXTENDED_BRANCH_MATRIX_KEYS
FROZEN BRANCH SCALAR KEYS: All 42 historical fields from frozen production
These MUST exist in the frozen CSV and are validated against it

FROZEN_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

Historical cross-branch time-based metrics (from frozen production)

"Time_to_50pct_continue_final_density",

"Time_to_90pct_continue_final_density",

"Time_to_50pct_continue_late_similarity",

"Time_to_90pct_continue_late_similarity",

"Time_to_similarity_below_0_9",

"Time_to_similarity_below_0_5",

"Late_persistence_ratio_to_continue",

"Late_similarity_ratio_to_continue",

)
EXTENDED BRANCH SCALAR KEYS: New fields in v1 for dual-horizon support
These do NOT exist in frozen CSV/NPZ, so are validated internally at rho=1

EXTENDED_BRANCH_SCALAR_KEYS = (

text

Fixed physical horizon scalars (PRIMARY ESTIMANDS)

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

Normalized EMA horizon scalars (NEW - explicit)

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

Additional extended scalars for complete coverage

"P_mean_full_fixed_physical",

"P_mean_immediate_fixed_physical",

"P_mean_recovery_fixed_physical",

"P_mean_full_normalized_EMA",

"P_mean_immediate_normalized_EMA",

"P_mean_recovery_normalized_EMA",

"Retention_selected_late_fixed_physical",

"Emergence_unselected_late_fixed_physical",

"Retention_selected_late_normalized_EMA",

"Emergence_unselected_late_normalized_EMA",

"Corr_Rpast_Pfull_fixed_physical",

"Corr_Rpast_Pfull_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"Jaccard_final_fixed_physical",

)
All required scalar keys (frozen + extended)

REQUIRED_BRANCH_SCALAR_KEYS = FROZEN_BRANCH_SCALAR_KEYS +
EXTENDED_BRANCH_SCALAR_KEYS
Extended scalar equivalence map for internal consistency validation at rho=1
Maps extended keys to their frozen equivalents

EXTENDED_SCALAR_EQUIVALENCE_MAP = {

text

"P_mean_full_fixed_physical": "P_mean_full",

"P_mean_immediate_fixed_physical": "P_mean_immediate",

"P_mean_recovery_fixed_physical": "P_mean_recovery",

"P_mean_late_fixed_physical": "P_mean_late",

"P_mean_full_normalized_EMA": "P_mean_full",

"P_mean_immediate_normalized_EMA": "P_mean_immediate",

"P_mean_recovery_normalized_EMA": "P_mean_recovery",

"P_mean_late_normalized_EMA": "P_mean_late",

"Weighted_similarity_late_fixed_physical": "Weighted_similarity_late",

"Weighted_similarity_late_normalized_EMA": "Weighted_similarity_late",

"Retention_contrast_late_fixed_physical": "Retention_contrast_late",

"Retention_contrast_late_normalized_EMA": "Retention_contrast_late",

"Corr_Rpast_Plate_fixed_physical": "Corr_Rpast_Plate",

"Corr_Rpast_Plate_normalized_EMA": "Corr_Rpast_Plate",

"Corr_Rpast_Pfull_fixed_physical": "Corr_Rpast_Pfull",

"Corr_Rpast_Pfull_normalized_EMA": "Corr_Rpast_Pfull",

"r_mean_late_fixed_physical": "r_mean_late",

"r_mean_late_normalized_EMA": "r_mean_late",

"Retention_selected_late_fixed_physical": "Retention_selected_late",

"Retention_selected_late_normalized_EMA": "Retention_selected_late",

"Emergence_unselected_late_fixed_physical": "Emergence_unselected_late",

"Emergence_unselected_late_normalized_EMA": "Emergence_unselected_late",

"Jaccard_final_fixed_physical": "Jaccard_final",

}
All required scalar keys (frozen + extended)

REQUIRED_BRANCH_SCALAR_KEYS = FROZEN_BRANCH_SCALAR_KEYS + EXTENDED_BRANCH_SCALAR_KEYS
==========
Baseline Equivalence Test
==========

def exact_array_equal_with_nan(a: np.ndarray, b: np.ndarray) -> bool:

text

"""Compare arrays with NaN-aware exact equality."""

a = np.asarray(a)

b = np.asarray(b)

if a.shape != b.shape or a.dtype != b.dtype:

return False

if np.issubdtype(a.dtype, np.floating) or np.issubdtype(a.dtype, np.complexfloating):

a_nan = np.isnan(a)

```
b_nan = np.isnan(b)

if not np.array_equal(a_nan, b_nan):

return False

finite = ~a_nan

return np.array_equal(a[finite], b[finite])

return np.array_equal(a, b)

def run_baseline_equivalence(

text

seeds: list[int],

config: ModelConfig,

protocol: ProtocolConfig,

frozen_manifest_path: str,

frozen_npz_path: str,

branch_order: tuple[str, ...],

frozen_csv_path: str | None = None,

) -> dict[str, Any]:

text

"""Run baseline equivalence test at rho=1 (T_window = T_*).
```

MANDATORY: Must reproduce frozen run exactly for all seeds.

This function validates that the engine produces EXACTLY the same outputs

as the frozen production run for all required keys.

Requires NaN-aware exact equality for ALL required schema keys:

- All prefix keys

- All branch matrix keys

- All branch scalar keys

Returns:

status: "passed" | "failed" | "not_run"

seed_results: per-seed validation details

engine_source_sha256: hash of this engine source

frozen_manifest_hash: hash of frozen manifest

```
frozen_npz_hash: hash of frozen NPZ

"""

import numpy as np

from pathlib import Path

import hashlib



# Preflight validation: validate config and protocol before any simulation

try:

config.validate()

except Exception as e:

return {

"status": "failed",

"error": f"Config validation failed: {e}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()) if frozen_manifest_path else None,

"frozen_npz": str(Path(frozen_npz_path).resolve()) if frozen_npz_path else None,

"frozen_csv_path": frozen_csv_path,

}
```

try:

protocol.validate(config)

except Exception as e:

return {

"status": "failed",

"error": f"Protocol validation failed: {e}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()) if frozen_manifest_path else None,

"frozen_npz": str(Path(frozen_npz_path).resolve()) if frozen_npz_path else None,

"frozen_csv_path": frozen_csv_path,

}

Helper function to compute source hash

def *source*sha256() -> str | None:

try:

return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

```
return None



# Early failure: frozen scalar CSV is required for scalar equivalence

if frozen_csv_path is None:

return {

"status": "failed",

"error": "frozen scalar CSV is required for scalar equivalence",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": None,

}



# Record engine source hash

engine_source_hash = *source*sha256()



results = {

"status": "not_run",
```

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"mean_omega": config.mean_omega,

"T_window": config.T_window,

"T_star": config.T_star,

"alpha": config.alpha,

"a": config.a,

"q": config.q,

"delta": config.delta,

"eta": config.eta,

"mu": config.mu,

"gamma": config.gamma,

"B_c": config.B_c,

"B_init_scale": config.B_init_scale,

"scale_epsilon": config.scale_epsilon,

"capacity": config.capacity,

```
"delay_steps": config.delay_steps,

},

"protocol": {

"T_sim": protocol.T_sim,

"burn_end_time": protocol.burn_end_time,

"split_time": protocol.split_time,

"future_duration": protocol.future_duration,

"diagnostic_stride": protocol.diagnostic_stride,

"branches": list(protocol.branches),

},

"n_seeds": len(seeds),

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"engine_source_sha256": engine_source_hash,

"seed_results": {},

}
```

```
# Load frozen manifest

with open(frozen_manifest_path, "r") as f:

frozen_manifest = json.load(f)



# Load frozen data

with np.load(frozen_npz_path, allow_pickle=False) as frozen_data:

frozen_files = dict(frozen_data)



# Load frozen scalar CSV - REQUIRED for scalar equivalence

# Fail immediately if CSV is missing or invalid

if frozen_csv_path is None or not os.path.isfile(frozen_csv_path):

return {

"status": "failed",

"error": f"Frozen scalar CSV not found: {frozen_csv_path}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,
```

```
}



frozen_scalar_csv: dict[str, dict[str, Any]] = {}

try:

import csv

with open(frozen_csv_path, "r", newline="", encoding="utf-8") as f:

reader = csv.DictReader(f)

for row in reader:

seed_id = row.get("seed", "")

branch = row.get("branch", "")

if seed_id and branch:

key = f"{branch}_seed_{seed_id}"

if key not in frozen_scalar_csv:

frozen_scalar_csv[key] = {}

for k, v in row.items():

if k not in ("seed", "seed_index", "branch", "rho", "T_window"):

try:
```

```
frozen_scalar_csv[key][k] = float(v)

except (ValueError, TypeError):

frozen_scalar_csv[key][k] = v

except Exception as e:

return {

"status": "failed",

"error": f"Could not load frozen CSV {frozen_csv_path}: {e}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# Validate frozen CSV has all expected seeds and branches

# Expected: 40 seeds × 7 branches = 280 rows

expected_csv_rows = len(seeds) * len(branch_order)

actual_csv_rows = len(frozen_scalar_csv)

if actual_csv_rows != expected_csv_rows:
```

```
return {

"status": "failed",

"error": f"Frozen CSV has {actual_csv_rows} rows, expected {expected_csv_rows} (40 seeds × 7 branches)",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# Validate all expected scalar columns exist in frozen CSV

# Check a sample row to see what columns are available

sample_key = None

for key in frozen_scalar_csv:

sample_key = key

break



if sample_key and frozen_scalar_csv[sample_key]:
```

```
available_columns = set(frozen_scalar_csv[sample_key].keys())

missing_columns = set(FROZEN_BRANCH_SCALAR_KEYS) - available_columns

if missing_columns:

return {

"status": "failed",

"error": f"Frozen CSV missing {len(missing_columns)} required scalar columns:
{sorted(missing_columns)[:5]}",

"missing_columns": sorted(missing_columns),

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# Record frozen hashes - use actual file hashes, not canonical JSON hashes

# Record frozen hashes - use actual file hashes, not canonical JSON hashes

results["frozen_manifest_file_sha256"] = file_sha256(frozen_manifest_path)

results["frozen_npz_file_sha256"] = file_sha256(frozen_npz_path)

if frozen_csv_path is not None and os.path.exists(frozen_csv_path):
```

```
results["frozen_scalar_csv_file_sha256"] = file_sha256(frozen_csv_path)



# Also record engine source hash

results["engine_source_sha256"] = engine_source_hash



# Also keep canonical JSON hash for reference

results["frozen_manifest_hash"] = hashlib.sha256(

json.dumps(frozen_manifest, sort_keys=True).encode()

).hexdigest()



# Validate config matches frozen manifest model - use EXACT equality

frozen_model = frozen_manifest["model"]



# Check N

if frozen_model["N"] != config.N:

results["status"] = "failed"

results["error"] = f"N mismatch: config={config.N}, frozen={frozen_model['N']}"
```

return results

Check omega - use exact equality

frozen_omega_arr = np.array(frozen_model["omega"], dtype=float)

config_omega_arr = np.array(list(config.omega), dtype=float)

if not np.array_equal(frozen_omega_arr, config_omega_arr):

results["status"] = "failed"

results["error"] = f"omega mismatch: frozen={frozen_model['omega']}, config={list(config.omega)}"

return results

Check all other model params with EXACT equality (rtol=0, atol=0)

model_checks = [

("dt", frozen_model["dt"], config.dt),

("tau", frozen_model["tau"], config.tau),

("a", frozen_model["a"], config.a),

("q", frozen_model["q"], config.q),

("delta", frozen_model["delta"], config.delta),

("eta", frozen_model["eta"], config.eta),

("mu", frozen_model["mu"], config.mu),

("gamma", frozen_model["gamma"], config.gamma),

("B_c", frozen_model["B_c"], config.B_c),

("B_init_scale", frozen_model.get("B_init_scale", 0.0), config.B_init_scale),

("T_window", frozen_model["T_window"], config.T_window),

]

for name, frozen_val, config_val in model_checks:

if not np.isclose(frozen_val, config_val, rtol=0.0, atol=0.0):

results["status"] = "failed"

results["error"] = f"{name} mismatch: frozen={frozen_val}, config={config_val}"

return results

Check derived values

if not np.isclose(frozen_model.get("alpha", 0.0), config.alpha, rtol=0.0, atol=1e-15):

results["status"] = "failed"

```
results["error"] = f"alpha mismatch: frozen={frozen_model.get('alpha')}, config={config.alpha}"
return results


# Now run actual equivalence test for each seed
print("Running baseline equivalence test...")
all_passed = True


for seed in seeds:
seed_str = str(seed)
seed_results = {"status": "passed", "checks": {}, "max_diffs": {}, "errors": {}}


try:
# Simulate with our engine
split_state, prefix = simulate_prefix(seed, config, protocol)
branch_rngs = make_branch_rngs(seed, branch_order)


# Simulate each branch
engine_results = {}
```

```
for branch in protocol.branches:

rng = branch_rngs[branch]

engine_results[branch] = simulate_future(

split_state.clone(),

prefix,

branch,

config,

protocol,

rng,

)


# CRITICAL: Compute historical cross-branch metrics (the 8 time-based fields)

# These must be computed AFTER all branches are simulated

if "continue" in engine_results:

finalize_historical_branch_metrics(engine_results)


# Check prefix state - iterate over REQUIRED_PREFIX_KEYS

for key in REQUIRED_PREFIX_KEYS:
```

```
engine_val = prefix.get(key)

frozen_key = f"prefix_seed_{seed_str}_{key}"

frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"prefix_{key}_present_in_frozen"] = False

all_passed = False

continue

else:

seed_results["checks"][f"prefix_{key}_present_in_frozen"] = True


if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"prefix_{key}_exact"] = True

else:

seed_results["checks"][f"prefix_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False
```

```
if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):

if engine_val.shape != frozen_val.shape:

seed_results["errors"][f"prefix_{key}_shape_mismatch"] = f"engine shape {engine_val.shape} != frozen shape {frozen_val.shape}"

seed_results["max_diffs"][f"prefix_{key}"] = None

else:

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(frozen_val), 0.0, frozen_val)

seed_results["max_diffs"][f"prefix_{key}"] = float(np.max(np.abs(e - f)))

else:

seed_results["max_diffs"][f"prefix_{key}"] = None


# Check branch outputs - separate FROZEN vs EXTENDED keys

for branch in protocol.branches:

engine_branch = engine_results[branch]

frozen_prefix = f"{branch}_seed_{seed_str}"


# === FROZEN MATRIX KEYS: Must exist in historical NPZ ===
```

```
for key in FROZEN_BRANCH_MATRIX_KEYS:

engine_val = engine_branch.get(key)

frozen_key = f"{frozen_prefix}_{key}"

frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = False

all_passed = False

continue

else:

seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = True


if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False

all_passed = False

continue
```

```
else:

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = True


if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"{branch}_{key}_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):

if engine_val.shape != frozen_val.shape:

seed_results["errors"][f"{branch}_{key}_shape_mismatch"] = f"engine shape {engine_val.shape} != frozen shape {frozen_val.shape}"

seed_results["max_diffs"][f"{branch}_{key}"] = None

else:

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(frozen_val), 0.0, frozen_val)

seed_results["max_diffs"][f"{branch}_{key}"] = float(np.max(np.abs(e - f)))

else:
```

```
seed_results["max_diffs"][f"{branch}_{key}"] = None


# === EXTENDED MATRIX KEYS: Validate internally at rho=1 ===
# These do NOT exist in frozen NPZ. At rho=1, both horizon systems
# should produce identical results (1750 and 8750 steps match).
if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):
for key in EXTENDED_BRANCH_MATRIX_KEYS:
engine_val = engine_branch.get(key)
if engine_val is None:
seed_results["status"] = "failed"
seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False
all_passed = False
continue
else:
seed_results["checks"][f"{branch}_{key}_present_in_engine"] = True


# Map extended key to its frozen equivalent
frozen_key_map = {
```

"B_mean_late_fixed_physical": "B_mean_late",

"P_full_fixed_physical": "P_full",

"P_immediate_fixed_physical": "P_immediate",

"P_recovery_fixed_physical": "P_recovery",

"P_late_fixed_physical": "P_late",

"B_mean_late_normalized_EMA": "B_mean_late",

"P_full_normalized_EMA": "P_full",

"P_immediate_normalized_EMA": "P_immediate",

"P_recovery_normalized_EMA": "P_recovery",

"P_late_normalized_EMA": "P_late",

}

frozen_equiv_key = frozen_key_map.get(key)

if frozen_equiv_key:

engine_frozen_equiv = engine_branch.get(frozen_equiv_key)

if engine_frozen_equiv is not None:

if exact_array_equal_with_nan(engine_val, engine_frozen_equiv):

```
seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = True

else:

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(engine_frozen_equiv, np.ndarray):

if engine_val.shape != engine_frozen_equiv.shape:

seed_results["errors"][f"{branch}_{key}_shape_mismatch"] = f"engine shape {engine_val.shape} != equiv shape {engine_frozen_equiv.shape}"

seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = None

else:

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(engine_frozen_equiv), 0.0, engine_frozen_equiv)

seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = float(np.max(np.abs(e - f)))

else:

seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = None



# === SCALAR KEYS ===

# Use module-level constants for consistency
```

```
# FROZEN_BRANCH_SCALAR_KEYS: All 42 historical fields from frozen production

# EXTENDED_BRANCH_SCALAR_KEYS: New fields in v1 for dual-horizon support


for key in FROZEN_BRANCH_SCALAR_KEYS:

engine_val = engine_branch.get(key)


# Get from CSV first (production stores scalars in CSV)

frozen_val = None

csv_key = f"{branch}_seed_{seed_str}"

if csv_key in frozen_scalar_csv and key in frozen_scalar_csv[csv_key]:

frozen_val = frozen_scalar_csv[csv_key][key]

else:

# Fallback to NPZ

frozen_key = f"{frozen_prefix}_{key}"

frozen_val = frozen_files.get(frozen_key)


# FROZEN scalars MUST exist in historical data

if frozen_val is None:
```

```
seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = False

all_passed = False

continue

else:

seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = True


if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False

all_passed = False

continue

else:

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = True


# For scalars, compare directly - DO NOT silently skip

# Normalize to scalar value

try:
```

```
engine_scalar = float(engine_val)

except (TypeError, ValueError) as e:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_is_scalar"] = False

seed_results["errors"][f"{branch}_{key}_type_error"] = f"Cannot convert to float:
{type(engine_val).__name__}: {e}"

all_passed = False

continue



try:

frozen_scalar = float(frozen_val)

except (TypeError, ValueError) as e:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_frozen_is_scalar"] = False

seed_results["errors"][f"{branch}_{key}_frozen_type_error"] = f"Cannot convert frozen to float:
{type(frozen_val).__name__}: {e}"

all_passed = False

continue



# Compare with exact equality (rtol=0, atol=0)
```

```
engine_is_nan = bool(np.isnan(engine_scalar))

frozen_is_nan = bool(np.isnan(frozen_scalar))


if engine_is_nan and frozen_is_nan:

seed_results["checks"][f"{branch}_{key}_exact"] = True

elif np.isclose(engine_scalar, frozen_scalar, rtol=0.0, atol=0.0):

seed_results["checks"][f"{branch}_{key}_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

seed_results["max_diffs"][f"{branch}_{key}"] = float(abs(engine_scalar - frozen_scalar)) if not
(engine_is_nan or frozen_is_nan) else None


# === EXTENDED SCALAR KEYS: Validate internally at rho=1 ===

if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):

for key in EXTENDED_BRANCH_SCALAR_KEYS:

engine_val = engine_branch.get(key)
```

```
if engine_val is None:
seed_results["status"] = "failed"
seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False
all_passed = False
continue
else:
seed_results["checks"][f"{branch}_{key}_present_in_engine"] = True

# Map to frozen equivalent for INTERNAL consistency check
# At rho=1, fixed_physical and normalized_EMA should match
frozen_scalar_map = {
"P_mean_late_fixed_physical": "P_mean_late",
"Weighted_similarity_late_fixed_physical": "Weighted_similarity_late",
"Retention_contrast_late_fixed_physical": "Retention_contrast_late",
"Corr_Rpast_Plate_fixed_physical": "Corr_Rpast_Plate",
"r_mean_late_fixed_physical": "r_mean_late",
"P_mean_late_normalized_EMA": "P_mean_late",
"Weighted_similarity_late_normalized_EMA": "Weighted_similarity_late",
```

```
"Retention_contrast_late_normalized_EMA": "Retention_contrast_late",

"Corr_Rpast_Plate_normalized_EMA": "Corr_Rpast_Plate",

"r_mean_late_normalized_EMA": "r_mean_late",

}


frozen_equiv = frozen_scalar_map.get(key)

if frozen_equiv:

# Get the engine's own frozen-equivalent value

engine_frozen_equiv = engine_branch.get(frozen_equiv)


if engine_frozen_equiv is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{frozen_equiv}_present_in_engine"] = False

all_passed = False

continue


# Validate internal consistency: extended should equal frozen equivalent

try:

engine_scalar = float(engine_val)
```

```
equiv_scalar = float(engine_frozen_equiv)

except (TypeError, ValueError) as e:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_internal_consistency"] = False

seed_results["errors"][f"{branch}_{key}_internal_type_error"] = str(e)

all_passed = False

continue


engine_is_nan = bool(np.isnan(engine_scalar))

equiv_is_nan = bool(np.isnan(equiv_scalar))


if engine_is_nan and equiv_is_nan:

seed_results["checks"][f"{branch}_{key}_internal_exact"] = True

elif np.isclose(engine_scalar, equiv_scalar, rtol=0.0, atol=0.0):

seed_results["checks"][f"{branch}_{key}_internal_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_internal_exact"] = False

seed_results["status"] = "failed"
```

```
all_passed = False

seed_results["max_diffs"][f"{branch}_{key}_internal"] = float(abs(engine_scalar - equiv_scalar)) if not (engine_is_nan or equiv_is_nan) else None



except Exception as e:

seed_results["status"] = "failed"

seed_results["error"] = str(e)

import traceback

seed_results["traceback"] = traceback.format_exc()

all_passed = False



results["seed_results"][seed_str] = seed_results

print(f" Seed {seed_str}: {seed_results['status']}")



if all_passed:

results["status"] = "passed"

print("Baseline equivalence: PASSED")

else:
```

```
results["status"] = "failed"

print("Baseline equivalence: FAILED")

# Print details of failures

for seed_str, seed_results in results["seed_results"].items():

if seed_results.get("status") != "passed":

print(f" Seed {seed_str} failures:")

for check, val in seed_results.get("checks", {}).items():

if not val:

print(f" CHECK FAILED: {check}")

for key, val in seed_results.get("max_diffs", {}).items():

print(f" MAX DIFF: {key} = {val}")



return results

================================================================================
Main API
================================================================================
Known branches - allowlist for validation

KNOWN_BRANCHES = frozenset({
```

```
text

"continue",

"reset_ema",

"reset_B",

"reset_B_ema",

"B_carrier_only",

"full_reset",

"future_shuffle",

})

def run_seed(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

branch_order: tuple[str, ...],

) -> dict[str, Any]:

text

"""Run a single seed through prefix + all branches.
```

CRITICAL: branch_order defines the FULL frozen RNG namespace (7 branches).

protocol.branches defines which branches are actually executed.

This ensures future_shuffle gets the 7th RNG sequence (not the 4th)

as in production, even when only 4 study branches are executed.

Returns complete seed results with all branches.

"""

Preflight validation: validate config and protocol

config.validate()

protocol.validate(config)

Validate all protocol branches are known

for branch in protocol.branches:

if branch not in KNOWN_BRANCHES:

```
raise ValueError(
f"Unknown branch '{branch}'. Known branches: {sorted(KNOWN_BRANCHES)}"
)

# Validate all protocol branches are in branch_order (for RNG namespace)
for branch in protocol.branches:
if branch not in branch_order:
raise ValueError(
f"Branch '{branch}' in protocol.branches but not in branch_order. "
f"branch_order must include all protocol branches for correct RNG namespace."
)

# Validate branch_order has no duplicates and all are known
if len(branch_order) != len(set(branch_order)):
raise ValueError(f"branch_order has duplicate branches: {branch_order}")
for branch in branch_order:
if branch not in KNOWN_BRANCHES:
```

```
raise ValueError(f"Unknown branch '{branch}' in branch_order. Known branches:
{sorted(KNOWN_BRANCHES)}")



# Simulate prefix

split_state, prefix = simulate_prefix(seed, config, protocol)



# Create branch RNGs using frozen algorithm with FULL branch_order

branch_rngs = make_branch_rngs(seed, branch_order)



# Execute only the branches in protocol.branches, but use RNGs from branch_order

branch_results = {}

for branch in protocol.branches:

if branch not in branch_rngs:

raise ValueError(

f"Branch '{branch}' not found in branch_rngs. "

f"This indicates branch_order={branch_order} does not match the RNG namespace. "

f"Registered branches: {list(branch_rngs.keys())}"

)
```

```
rng = branch_rngs[branch]


branch_results[branch] = simulate_future(

split_state.clone(),

prefix,

branch,

config,

protocol,

rng,

)



# Compute historical cross-branch metrics (the 8 time-based fields)

# These must be computed AFTER all branches are simulated

if "continue" in branch_results:

finalize_historical_branch_metrics(branch_results)



return {
```

```
"seed": int(seed),
"config": {
"N": config.N,
"dt": config.dt,
"tau": config.tau,
"T_window": config.T_window,
"alpha": config.alpha,
"mean_omega": config.mean_omega,
"T_star": config.T_star,
},
"protocol": {
"T_sim": protocol.T_sim,
"burn_end_time": protocol.burn_end_time,
"split_time": protocol.split_time,
"future_duration": protocol.future_duration,
},
"prefix": prefix,
"branches": branch_results,
```

```
}

if name == "main":

text

# Quick validation

config = DEFAULT_MODEL_CONFIG

config.validate()

protocol = DEFAULT_PROTOCOL_CONFIG

protocol.validate(config)



print(f"Engine v1 loaded successfully")

print(f" N={config.N}, dt={config.dt}, tau={config.tau}")

print(f" T_window={config.T_window}, alpha={config.alpha:.6f}")

print(f" mean_omega={config.mean_omega:.6f}, T_*={config.T_star:.6f}")

print(f" delay_steps={config.delay_steps}")

print(f" Protocol: T_sim={protocol.T_sim}, burn={protocol.burn_end_time}, "

f"split={protocol.split_time}, future={protocol.future_duration}")
```

Ignoring the paste/HTML corruption, this is substantially improved. The frozen numerical kernel still looks faithful, and the previously fatal omission—failure to finalize the eight historical cross-branch metrics inside run_baseline_equivalence()—is now fixed.

However, I would make a few final corrections before spending time on the 40-seed baseline.
Verdict

Core numerical dynamics: approximately 9.5/10
Frozen-output gate: approximately 9/10
Dual-horizon gate: still incomplete because of one concrete mapping bug
Compile/import: GO
Expensive mandatory baseline: hold until the P0 items below are fixed

No finite replay has occurred yet; this remains a static source assessment.
P0: fix before the baseline
1. The extended scalar equivalence map is defined but not used

You correctly created the complete module-level map:

Python

EXTENDED_SCALAR_EQUIVALENCE_MAP

but the actual baseline loop still constructs and uses an older local ten-field map:

Python

frozen_scalar_map = {
"P_mean_late_fixed_physical": "P_mean_late",
...
}

Consequently, the newly added fields are checked only for presence, not equality. These include, among others:

text

P_mean_full_fixed_physical
P_mean_immediate_fixed_physical
P_mean_recovery_fixed_physical
P_mean_full_normalized_EMA
P_mean_immediate_normalized_EMA
P_mean_recovery_normalized_EMA
Retention_selected_late_fixed_physical
Emergence_unselected_late_fixed_physical
Corr_Rpast_Pfull_fixed_physical
Jaccard_final_fixed_physical

A baseline could pass even if these fields were numerically wrong.
Correction

Remove the local frozen_scalar_map completely and use the module-level authority:

Python

for key in EXTENDED_BRANCH_SCALAR_KEYS:
engine_val = engine_branch.get(key)

if engine_val is None:
seed_results["status"] = "failed"
seed_results["checks"][
f"{branch}_{key}_present_in_engine"
] = False
all_passed = False
continue

seed_results["checks"][
f"{branch}_{key}_present_in_engine"
] = True

frozen_equiv = EXTENDED_SCALAR_EQUIVALENCE_MAP[key]
engine_frozen_equiv = engine_branch.get(frozen_equiv)

if engine_frozen_equiv is None:
seed_results["status"] = "failed"
seed_results["checks"][

f"{branch}_{frozen_equiv}_present_in_engine"
] = False
all_passed = False
continue

Continue with NaN-aware exact scalar comparison.

Use direct indexing:

Python

EXTENDED_SCALAR_EQUIVALENCE_MAP[key]

rather than .get(key), so an unmapped declared field fails immediately.
2. EXTENDED_BRANCH_SCALAR_KEYS contains a duplicate

This field appears twice:

Python

"Corr_Rpast_Plate_normalized_EMA"

It appears once in the primary normalized block and again under additional fields.

Remove the second occurrence. Also remove the duplicate assignment:

Python

REQUIRED_BRANCH_SCALAR_KEYS = (
FROZEN_BRANCH_SCALAR_KEYS + EXTENDED_BRANCH_SCALAR_KEYS
)

which currently appears twice.

Add module-level integrity checks:

Python

```
if len(EXTENDED_BRANCH_SCALAR_KEYS) != len(
set(EXTENDED_BRANCH_SCALAR_KEYS)
):
raise RuntimeError("Duplicate extended scalar schema keys")


if set(EXTENDED_BRANCH_SCALAR_KEYS) != set(
EXTENDED_SCALAR_EQUIVALENCE_MAP
):
missing_map = (
set(EXTENDED_BRANCH_SCALAR_KEYS)
- set(EXTENDED_SCALAR_EQUIVALENCE_MAP)
)
extra_map = (
set(EXTENDED_SCALAR_EQUIVALENCE_MAP)
- set(EXTENDED_BRANCH_SCALAR_KEYS)
)
raise RuntimeError(
f"Extended scalar map mismatch: "
f"missing mappings={sorted(missing_map)}, "
f"extra mappings={sorted(extra_map)}"
)
```

An even cleaner design is to define the map first and derive:

Python

```
EXTENDED_BRANCH_SCALAR_KEYS = tuple(
EXTENDED_SCALAR_EQUIVALENCE_MAP
)
```

That makes schema-map drift impossible.
3. Validate the baseline horizon metadata

The engine emits:

text

window_steps
recovery_end_ema
fixed_immediate_end
fixed_recovery_end

but the gate does not verify them.

At the frozen baseline require:

Python

```
BASELINE_HORIZON_METADATA = {
"window_steps": FROZEN_WINDOW_STEPS,
"recovery_end_ema": FROZEN_RECOVERY_END,
"fixed_immediate_end": FIXED_PHYSICAL_IMMEDIATE_STEPS,
"fixed_recovery_end": FIXED_PHYSICAL_RECOVERY_STEPS,
}
```

For every branch:

Python

```
for key, expected in BASELINE_HORIZON_METADATA.items():
observed = engine_branch.get(key)
passed = (
isinstance(observed, (int, np.integer))
and int(observed) == expected
)
seed_results["checks"][
f"{branch}_{key}_baseline_exact"
] = passed

if not passed:
seed_results["status"] = "failed"
all_passed = False
```

This certifies the boundary definitions that justify the internal fixed-versus-normalized equivalences.
4. The mandatory gate still does not verify the exact seed/branch authority

The CSV row-count test ensures only the correct number of unique dictionary keys. It does not establish that they are the correct seed/branch pairs.

Before simulation, require:

Python

```
manifest_seeds = [
int(x)
for x in frozen_manifest["randomization"]["paired_seeds"]
]
manifest_branches = tuple(
frozen_manifest["protocol"]["branches"]
)

if list(seeds) != manifest_seeds:
return failed_result("Baseline seed list/order differs from manifest")

if tuple(branch_order) != manifest_branches:
return failed_result("RNG branch order differs from manifest")

if tuple(protocol.branches) != manifest_branches:
return failed_result("Executed baseline branches differ from manifest")
```

This also prevents someone from supplying 40 incorrect seeds or seven incorrect branch names and discovering the problem only after simulation.

For subset smoke testing, use a separately labelled smoke path rather than allowing this mandatory function to report "passed".
P1: strongly recommended before the baseline
5. CSV duplicate detection is not implemented yet

The parser currently does:

Python

if key not in frozen_scalar_csv:
frozen_scalar_csv[key] = {}

A duplicate (seed, branch) row silently overwrites the earlier row. Because len(frozen_scalar_csv) counts unique keys rather than physical rows, a 281-row file containing all 280 unique keys still passes the row-count test.

Track duplicates explicitly:

Python

seen_csv_keys: set[str] = set()
csv_row_count = 0

for line_number, row in enumerate(reader, start=2):
csv_row_count += 1
...
key = f"{branch}_seed_{seed_id}"

if key in seen_csv_keys:
return failed_result(
f"Duplicate frozen CSV row for {key} "
f"at line {line_number}"
)

seen_csv_keys.add(key)

Then compare exact keys:

Python

expected_csv_keys = {
f"{branch}_seed_{seed}"
for seed in seeds
for branch in branch_order
}

actual_csv_keys = set(frozen_scalar_csv)

```
if actual_csv_keys != expected_csv_keys:
missing = sorted(expected_csv_keys - actual_csv_keys)
extra = sorted(actual_csv_keys - expected_csv_keys)
return failed_result(
f"Frozen CSV key mismatch; "
f"missing={missing[:10]}, extra={extra[:10]}"
)
```

Validate required columns in every row, not only one sample row.
6. Preflight all three authority files before opening any

Currently, the manifest and NPZ are opened before the CSV existence check, and their errors are not caught.

Before loading anything:

Python

```
authority_paths = {
"manifest": Path(frozen_manifest_path),
"npz": Path(frozen_npz_path),
"csv": Path(frozen_csv_path),
}

for label, path in authority_paths.items():
if not path.is_file():
return {
"status": "failed",
"error": f"Frozen {label} file not found: {path}",
...
}
```

Wrap JSON and NPZ loading in fail-fast exception handling as well. A typo in a path should produce a saved failure certificate, not an uncaught exception.
7. Preflight the frozen NPZ schema before simulation

The expected NPZ keys can be determined without running any trajectories:

Python

```
expected_npz_keys = {
f"prefix_seed_{seed}_{key}"
for seed in seeds
for key in REQUIRED_PREFIX_KEYS
}

expected_npz_keys |= {
f"{branch}_seed_{seed}_{key}"
for seed in seeds
for branch in branch_order
for key in FROZEN_BRANCH_MATRIX_KEYS
}
```

Then:

Python

```
missing_npz_keys = expected_npz_keys - set(frozen_files)

if missing_npz_keys:
return failed_result(
f"Frozen NPZ missing {len(missing_npz_keys)} keys"
)
```

This avoids discovering an incomplete frozen artifact after 10.88 million simulation steps.

8. Remove the scalar fallback to the NPZ

The frozen CSV is the scalar authority; the frozen NPZ is the matrix/trajectory authority. The current code still says:

Python

```
else:
frozen_key = f"{frozen_prefix}_{key}"
frozen_val = frozen_files.get(frozen_key)
```

Remove that fallback:

Python

```
csv_key = f"{branch}_seed_{seed_str}"
frozen_val = frozen_scalar_csv[csv_key].get(key)
```

Because exact CSV coverage is preflight-validated, a missing field at this point is an internal validation error.

This also prevents a scalar-like one-element NPZ array from being accepted by float(...).
9. Match the frozen persistence-ratio condition literally

The frozen source uses:

Python

```
if abs(reference_late_persistence) > ZERO_TOL
```

It does not explicitly call np.isfinite() for this persistence denominator. The frozen similarity ratio does use the finite check.

Therefore, literal source matching is:

Python

```
result["Late_persistence_ratio_to_continue"] = (
float(branch_late_persistence / reference_late_persistence)
if abs(reference_late_persistence) > FROZEN_ZERO_TOL
else float("nan")
)

result["Late_similarity_ratio_to_continue"] = (
float(branch_late_similarity / reference_late_similarity)
if np.isfinite(reference_late_similarity)
and abs(reference_late_similarity) > FROZEN_ZERO_TOL
```

else float("nan")
)

The saved frozen continuation values should be finite, so the current code likely reproduces them. This is an implementation-identity correction rather than an expected finite-replay difference.
Validation hardening
10. Validate integer-valued fields as integers

This passes the current N validation:

Python

N = 12.0

but will later fail when passed as an array dimension.

Use:

Python

if (
isinstance(self.N, (bool, np.bool_))
or not isinstance(self.N, (int, np.integer))
or self.N <= 0
):
raise ValueError("N must be a positive integer")

Do the same for:

Python

protocol.diagnostic_stride

Also validate:

protocol.branches is nonempty;
no duplicate protocol branches;

all protocol times are finite.

11. Invoke state validation at transition boundaries

DynamicState.validate() is now good, but it is never called.

Reasonable low-cost locations are:

Python

```
state = initialize_state(...)
state.validate(config)
```

and after intervention:

Python

```
apply_intervention(...)
state.validate(config)
```

and optionally at the prefix split. Do not invoke it every timestep.

For symmetry checks, specify rtol=0.0:

Python

```
np.allclose(self.B, self.B.T, rtol=0.0, atol=1e-12)
```

Failure-certificate robustness

Your scalar mismatches now use None when a difference is undefined, which fixes the main allow_nan=False issue.

Two residual cases remain:

Array values containing infinity could produce inf in max_diffs.
A dtype mismatch is not reported explicitly.

A small helper would make all mismatch reporting safe:

Python

```
def json_safe_max_abs_diff(
a: np.ndarray,
b: np.ndarray,
) -> float | None:
a = np.asarray(a)
b = np.asarray(b)

if a.shape != b.shape:
return None

valid = np.isfinite(a) & np.isfinite(b)
if not np.any(valid):
return None

value = float(np.max(np.abs(a[valid] - b[valid])))
return value if np.isfinite(value) else None
```

Also record:

Python

```
engine_dtype
frozen_dtype
engine_shape
frozen_shape
```

when exact equality fails.
Performance before the full grid

The edge and omega caches are worthwhile improvements. Remaining nonblocking optimizations:

config.alpha evaluates np.exp twice per timestep. Cache alpha once per immutable configuration.
config.delay_steps is recomputed during every history update. Cache it as a derived value.
A length-51 list uses pop(0) every step. Frozen production's bounded deque more directly matches the authority and avoids shifting.
Mark cached omega and edge arrays read-only.
simulate_prefix() computes edges but never uses it.

These should ideally be settled before the successful baseline, because the exact source hash that passes should be the source used for the grid.
Cosmetic cleanup

These do not affect results:

duplicate "Compute time-based metrics" comment;
duplicate "Record frozen hashes" comment;
unused imports: copy, time, and field;
unused make_seed_sequence();
unused N in compute_R();
top documentation should say
T_WINDOW = 17.502917152858814... ≈ 17.5, not equality to 17.5.

What now looks correct

The important corrections are present:

frozen np.exp alpha construction;
exact prefix RNG object use;
full frozen branch RNG namespace;
subset execution separated from RNG namespace;
branch allowlist at intervention;
all 42 historical scalar names at module level;
historical finalizer called in both run_seed() and baseline equivalence;
missing outputs represented by false presence checks;
no contradictory informational checks;
NaN-aware exact arrays;
dual fixed-physical and normalized-EMA accumulation;
explicit B_c threshold for Jaccard;
file hashes separated from canonical JSON hash;
JSON-safe None for ordinary NaN scalar mismatches.

Bottom line

I do not see a remaining obvious defect in the numerical step map. The main remaining bug is in the certificate logic: the complete EXTENDED_SCALAR_EQUIVALENCE_MAP is not used, and the schema has a duplicate key.

Fix that, the exact seed/branch preflight, and CSV duplicate/exact-key validation before launching the expensive baseline. After those changes, this engine is ready for:

py_compile;
import/validation;
a separately labelled one-seed smoke test;
the full 40-seed, seven-branch baseline-only run.

Do not launch the five-level grid until the resulting complete certificate passes exactly.

The second file:

#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Driver.

Parameterized study varying T_window = rho T_ where T_* = 25 / mean(OMEGA) = 17.5

and rho ∈ {0.5, 0.75, 1, 1.5, 2}.

Study Design:

For each seed s ∈ seeds (40 seeds)

For each rho ∈ {0.5, 0.75, 1, 1.5, 2}

Set T_window = rho * 17.5

Set alpha = exp(-dt / T_window)

Full regeneration for each (s, rho) - NO split state reuse

Branches: continue, reset_ema, reset_B, future_shuffle

Fixed physical horizons: immediate=0-17.5, recovery=17.5-87.5, late=87.5-320

EMA-normalized horizons: immediate=0-1 T_window, recovery=1-5 T_window

Output:

Scalars in CSV: per (seed, rho, branch) scalar outcomes

Arrays in NPZ: per (seed, rho, branch) trajectory and matrix outcomes

MANDATORY: Baseline equivalence test at rho=1 must pass against frozen production.

"""

```
from future import annotations

import argparse

import csv

import hashlib

import json

import os

import platform

import shutil

import sys
```

```
import time

import tempfile

from pathlib import Path

from typing import Any

try:

text

import pandas as pd

except ImportError:

text

print("ERROR: pandas is required for production T_window study")

print("Install with: pip install pandas")

sys.exit(1)

import numpy as np
Import the experiment engine

import aief_experiment_engine_v1 as engine
============================================================================
====
Study Configuration
============================================================================
====

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"
```

WORKFLOW_VERSION = "v1.0"
Frozen production values (from AIEF_1_4_memory_ablation_updated_REAL.py)

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005

Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA))

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.5
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_STAR / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
Rho values for T_window sensitivity

RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)
Frozen branch order from AIEF_1_4_memory_ablation_updated_REAL.py

FROZEN_BRANCH_ORDER = (

text

"continue",

"reset_ema",

"reset_B",

"reset_B_ema",

"B_carrier_only",

"full_reset",

"future_shuffle",

)
Study subset of branches

STUDY_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Fixed physical horizon times (in time units, not steps)

PHYSICAL_IMMEDIATE_END = 17.5

PHYSICAL_RECOVERY_END = 87.5 # Also PHYSICAL_LATE_START

PHYSICAL_FUTURE_END = 320.0
Defaults

DEFAULT_MASTER_SEED = 20260714

DEFAULT_N_SEEDS = 40

DEFAULT_RUN_ROOT = "runs"
==========
REQUIRED OUTPUT SCHEMA
==========
These are the MINIMUM required outputs that must be present in engine results
If any are missing, the study should fail with a clear error

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)

REQUIRED_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
Primary outcomes (must be present)
NOW USING FIXED-PHYSICAL HORIZONS as the primary estimands

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"P_mean_late_fixed_physical",

)
Secondary outcomes for comparison (normalized EMA)

SECONDARY_OUTCOMES = (

text

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"P_mean_late_normalized_EMA",

)
Backward compatible names (map to normalized EMA for legacy compatibility)

LEGACY_OUTCOMES = (

text

"Weighted_similarity_late",

"Retention_contrast_late",

"P_mean_late",

)
Separate FROZEN vs EXTENDED schemas
FROZEN keys must exist in historical frozen data

FROZEN_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

Projection diagnostics (in frozen data)

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

Metadata

"T_window",

"rho",

)
EXTENDED keys are new in v1 (dual-horizon)

EXTENDED_BRANCH_SCALAR_KEYS = (

text

Fixed physical horizon scalars (PRIMARY ESTIMANDS)

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

Normalized EMA horizon scalars (explicit)

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

)
All required scalar keys (frozen + extended)

REQUIRED_BRANCH_SCALAR_KEYS = FROZEN_BRANCH_SCALAR_KEYS + EXTENDED_BRANCH_SCALAR_KEYS
Desired scalar keys (not yet produced by engine, but included in CSV for future)

DESIRED_BRANCH_SCALAR_KEYS = (

text

Time-based metrics (not yet in engine)

"Time_to_similarity_below_0_9",

"Time_to_similarity_below_0_5",

Ratio metrics (not yet in engine)

"Late_persistence_ratio_to_continue",

"Late_similarity_ratio_to_continue",

)
All scalar keys (required + desired) for CSV output

BRANCH_SCALAR_KEYS = REQUIRED_BRANCH_SCALAR_KEYS +
DESIRED_BRANCH_SCALAR_KEYS
All matrix keys

BRANCH_MATRIX_KEYS = REQUIRED_BRANCH_MATRIX_KEYS

def is_finite_number(value: Any) -> bool:

text

"""Check if a value is a finite number.

Handles int, float, numpy scalar types, and excludes NaN, inf, -inf, None, strings.

"""

return (

isinstance(

value,

(

int,

float,

np.integer,

np.floating,

),

)

and np.isfinite(value)

)

def validate_branch_outputs(branch: str, branch_result: dict[str, Any]) -> None:

text

"""Validate that a branch result contains all required outputs.

Raises RuntimeError if required keys are missing.

"""

missing_scalars = set(REQUIRED_BRANCH_SCALAR_KEYS) - set(branch_result.keys())

missing_matrices = set(REQUIRED_BRANCH_MATRIX_KEYS) - set(branch_result.keys())

if missing_scalars or missing_matrices:

missing = list(missing_scalars) + list(missing_matrices)

raise RuntimeError(

f"Branch '{branch}' missing required outputs: {missing[:10]}... "

f"({len(missing)} total missing)"

)

==========
Utility Functions
==========
Note: master-seed is now type=int, no longer supports 'random' for production

def generate_seed_set(master_seed: int, count: int) -> list[int]:

text

"""Generate deterministic seed set from master seed."""

if count <= 0:

raise ValueError("seed count must be positive")

children = np.random.SeedSequence(master_seed).spawn(count)

```
seeds = [
int(child.generate_state(1, dtype=np.uint64)[0])
for child in children
]
if len(set(seeds)) != len(seeds):
raise RuntimeError("duplicate generated seeds")
return seeds
def source_sha256() -> str | None:
text
"""Compute SHA-256 of this source file."""
try:
return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
except (NameError, OSError):
return None
def atomic_write_text(path: str | Path, text: str) -> None:
text
"""Write text atomically with fsync."""
```

```
path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

temporary = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temporary, "w", encoding="utf-8", newline="") as handle:

handle.write(text)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(temporary), str(path))

finally:

if temporary.exists():

temporary.unlink()

def atomic_write_json(path: str | Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically with fsync."""

text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"

atomic_write_text(path, text)

def save_npz_atomic(path: str | Path, payload: dict[str, Any]) -> None:
```

```
text

"""Save NPZ atomically with fsync.



Uses a temporary file with .npz suffix so np.savez_compressed doesn't add extra suffix.

"""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

# Use tempfile.mkstemp to create a temporary file in the same directory

fd, temporary = tempfile.mkstemp(

dir=str(path.parent),

prefix=f".{[path.name](http://path.name)}.",

suffix=".tmp.npz",

)

try:

with os.fdopen(fd, "wb") as handle:

np.savez_compressed(handle, **payload)

handle.flush()
```

```
os.fsync(handle.fileno())

os.replace(temporary, str(path))

finally:

if os.path.exists(temporary):

os.unlink(temporary)

def load_npz(path: str | Path) -> dict[str, Any]:

text

"""Load NPZ file."""

payload: dict[str, Any] = {}

with np.load(str(path), allow_pickle=False) as data:

for key in data.files:

value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

return payload

def save_csv_atomic(path: str | Path, dataframe) -> None:

text

"""Save CSV atomically with fsync."""
```

```
if pd is None:

raise RuntimeError("pandas is required for CSV output but is not installed")



path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

fd, temporary = tempfile.mkstemp(

dir=str(path.parent),

prefix=f".{[path.name](http://path.name)}.",

suffix=".tmp",

)

try:

with os.fdopen(fd, "w", encoding="utf-8", newline="") as handle:

[dataframe.to](http://dataframe.to)_csv(handle, index=False)

handle.flush()

os.fsync(handle.fileno())

os.replace(temporary, str(path))

finally:
```

```
if os.path.exists(temporary):

os.unlink(temporary)

================================================================================
Configuration and Validation
================================================================================

def create_model_config(rho: float) -> engine.ModelConfig:

text

"""Create ModelConfig for a given rho value."""

T_window = rho * FROZEN_T_STAR

return engine.ModelConfig(

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,
```

```
mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=T_window,

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=1e-12,

)

def create_protocol_config() -> engine.ProtocolConfig:

text

"""Create ProtocolConfig matching frozen production."""

return engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=STUDY_BRANCHES,

)
```

```
================================================================================
Simulation Functions
================================================================================

def simulate_seed_rho(

text

seed: int,

rho: float,

config: engine.ModelConfig,

protocol: engine.ProtocolConfig,

study_branches: tuple[str, ...],

) -> dict[str, Any]:

text

"""Simulate a single (seed, rho) combination.



CRITICAL: Uses FROZEN_BRANCH_ORDER for RNG namespace, but only simulates

study_branches subset. This ensures future_shuffle gets the 7th RNG sequence

(not the 4th) as in production.
```

```
Returns complete results for all branches at this (seed, rho).

"""

# Create branch RNGs using FULL frozen branch order for correct namespace

branch_rngs = engine.make_branch_rngs(seed, FROZEN_BRANCH_ORDER)



# Create a protocol with just the study branches

study_protocol = engine.ProtocolConfig(

T_sim=protocol.T_sim,

burn_end_time=protocol.burn_end_time,

split_time=protocol.split_time,

future_duration=protocol.future_duration,

diagnostic_stride=protocol.diagnostic_stride,

branches=study_branches,

)



# Simulate prefix
```

```
split_state, prefix = engine.simulate_prefix(seed, config, study_protocol)



# Simulate only study branches, using RNGs from frozen namespace

branch_results = {}

for branch in study_branches:

if branch not in branch_rngs:

raise RuntimeError(

f"Branch '{branch}' not in FROZEN_BRANCH_ORDER. "

f"Cannot map to production RNG namespace."

)

rng = branch_rngs[branch]

branch_results[branch] = engine.simulate_future(

split_state.clone(),

prefix,

branch,

config,

study_protocol,
```

```
rng,

)

# Add metadata

branch_results[branch]["T_window"] = config.T_window

branch_results[branch]["rho"] = rho



return {

"seed": int(seed),

"rho": rho,

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"T_window": config.T_window,

"alpha": config.alpha,

"mean_omega": config.mean_omega,

"T_star": config.T_star,
```

```
},

"protocol": {

"T_sim": study_protocol.T_sim,

"burn_end_time": study_protocol.burn_end_time,

"split_time": study_protocol.split_time,

"future_duration": study_protocol.future_duration,

},

"prefix": prefix,

"branches": branch_results,

}

def run_baseline_equivalence(

text

seeds: list[int],

frozen_manifest_path: str,

frozen_npz_path: str,

frozen_csv_path: str,

) -> dict[str, Any]:

text
```

"""Run baseline equivalence test at rho=1 (T_window = T_*).

MANDATORY: Must reproduce frozen run exactly for ALL seeds.

Verifies:

1. Seed list matches frozen manifest exactly (same seeds, same order)

2. All 40 seeds pass equivalence (if 40 seeds provided)

3. Engine produces all required outputs

This helper ONLY performs equivalence and returns a result.

Main handles manifests and output paths.

"""

import json

Load frozen manifest to compare seeds

try:

```
with open(frozen_manifest_path, 'r', encoding='utf-8') as f:

frozen_manifest = json.load(f)

except FileNotFoundError:

return {

"status": "failed",

"error": f"Frozen manifest not found: {frozen_manifest_path}",

"seeds_match": False,

}

except json.JSONDecodeError as e:

return {

"status": "failed",

"error": f"Frozen manifest is not valid JSON: {frozen_manifest_path} ({e})",

"seeds_match": False,

}



# Validate manifest has required structure

if "randomization" not in frozen_manifest or "paired_seeds" not in frozen_manifest["randomization"]:
```

```
return {
"status": "failed",
"error": f"Frozen manifest missing required structure: {frozen_manifest_path}",
"seeds_match": False,
}

frozen_seeds = frozen_manifest["randomization"]["paired_seeds"]

# Verify seed list matches frozen manifest EXACTLY
# This is critical: baseline equivalence tests a specific cohort
if len(seeds) != len(frozen_seeds):
return {
"status": "failed",
"error": f"Seed count mismatch: generated={len(seeds)}, frozen={len(frozen_seeds)}",
"frozen_seeds": [int(s) for s in frozen_seeds],
"generated_seeds": [int(s) for s in seeds],
"seeds_match": False,
```

```
}



for i, (gen_seed, frozen_seed) in enumerate(zip(seeds, frozen_seeds)):

if int(gen_seed) != int(frozen_seed):

return {

"status": "failed",

"error": f"Seed mismatch at index {i}: generated={gen_seed}, frozen={frozen_seed}",

"frozen_seeds": [int(s) for s in frozen_seeds],

"generated_seeds": [int(s) for s in seeds],

"seeds_match": False,

}



# Verify master seed matches

frozen_master_seed = frozen_manifest["randomization"]["master_seed"]

# Note: We can't check the generated master seed here as it's not passed,

# but the seed generation method should match
```

```
# Validate frozen CSV path exists

if not os.path.exists(frozen_csv_path):

return {

"status": "failed",

"error": f"Frozen scalar CSV not found: {frozen_csv_path}",

"seeds_match": True,

}



# Create config for rho=1

config = create_model_config(rho=1.0)



# For baseline equivalence, we need to validate ALL 7 branches

# to certify the engine as reusable infrastructure

baseline_protocol = engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,

burn_end_time=160.0,

split_time=480.0,
```

```
future_duration=320.0,

diagnostic_stride=50,

branches=FROZEN_BRANCH_ORDER, # All 7 branches for engine certification

)



# Run the engine's baseline equivalence test with ALL seeds and ALL 7 branches

results = [engine.run](http://engine.run)_baseline_equivalence(

seeds=seeds,

config=config,

protocol=baseline_protocol,

frozen_manifest_path=frozen_manifest_path,

frozen_npz_path=frozen_npz_path,

frozen_csv_path=frozen_csv_path,

branch_order=FROZEN_BRANCH_ORDER, # Use FULL branch order for RNG namespace

)



# Add seed comparison info
```

```
results["seed_comparison"] = {

"generated_seeds": [int(s) for s in seeds],

"frozen_seeds": [int(s) for s in frozen_seeds],

"seeds_match": True,

}



return results

# ============================================================================
# Manifest and Output
# ============================================================================

def build_manifest(

text

master_seed: int,

seeds: list[int],

rho_values: tuple[float, ...],

) -> dict[str, Any]:

text

"""Build study manifest."""
```

```
# Compute config hash from model parameters

model_params = {

"N": FROZEN_N,

"dt": FROZEN_DT,

"tau": FROZEN_TAU,

"omega": FROZEN_OMEGA.tolist(),

"a": FROZEN_A,

"q": FROZEN_Q,

"delta": FROZEN_DELTA,

"eta": FROZEN_ETA,

"gamma": FROZEN_GAMMA,

"mu": FROZEN_MU,

"B_c": FROZEN_B_C,

"B_init_scale": FROZEN_B_INIT_SCALE,

"T_star": FROZEN_T_STAR,

}
```

```
protocol_params = {

"T_sim": FROZEN_T_SIM,

"burn_end_time": 160.0,

"split_time": 480.0,

"future_duration": 320.0,

"diagnostic_stride": 50,

"branches": list(STUDY_BRANCHES),

}



study_params = {

"rho_values": list(rho_values),

"T_window_values": [rho * FROZEN_T_STAR for rho in rho_values],

"n_seeds": len(seeds),

"statistical_unit": "40 seed/graphs",

}



# Add dependencies - engine and driver source hashes
```

```
dependencies = {}



# Engine source hash

try:

engine_path = Path(engine.__file__).resolve()

engine_hash = file_sha256(engine_path)

dependencies["aief_experiment_engine_v1"] = {

"path": str(engine_path),

"sha256": engine_hash,

}

except Exception as e:

dependencies["aief_experiment_engine_v1"] = {

"path": "NOT_FOUND",

"sha256": "NOT_AVAILABLE",

"error": str(e),

}
```

```
# Driver source hash (this file)
try:
driver_path = Path(__file__).resolve()
driver_hash = file_sha256(driver_path)
dependencies["AIEF_1_4_Twindow_sensitivity"] = {
"path": str(driver_path),
"sha256": driver_hash,
}
except Exception as e:
dependencies["AIEF_1_4_Twindow_sensitivity"] = {
"path": "NOT_FOUND",
"sha256": "NOT_AVAILABLE",
"error": str(e),
}


manifest = {
"study_name": STUDY_NAME,
```

"workflow_version": WORKFLOW_VERSION,

"source_sha256": source_sha256(),

"dependencies": dependencies,

"model": model_params,

"protocol": protocol_params,

"study": study_params,

"randomization": {

"master_seed": int(master_seed),

"paired_seeds": [int(seed) for seed in seeds],

"seed_generation": "SeedSequence(master_seed).spawn(n_seeds)",

"branch_generation": "second per-seed child sequence spawns named uint64 branch seeds",

"bit_generator": np.random.default_rng(0).bit_generator.__class__.__name__,

},

"environment": {

"python": platform.python_version(),

"numpy": np.__version__,

"platform": platform.platform(),

},

"schema": {

"prefix_keys": list(REQUIRED_PREFIX_KEYS),

"branch_scalar_keys": list(BRANCH_SCALAR_KEYS),

"branch_matrix_keys": list(BRANCH_MATRIX_KEYS),

"required_branch_scalar_keys": list(REQUIRED_BRANCH_SCALAR_KEYS),

"required_branch_matrix_keys": list(REQUIRED_BRANCH_MATRIX_KEYS),

},

"notes": {

"purpose": "T_window sensitivity study - varying T_window = rho *T_*",

"T_*": FROZEN_T_STAR,

"fixed_physical_horizons": {

"immediate": f"0-{PHYSICAL_IMMEDIATE_END} time units",

"recovery": f"{PHYSICAL_IMMEDIATE_END}-{PHYSICAL_RECOVERY_END} time units",

"late": f"{PHYSICAL_RECOVERY_END}-{PHYSICAL_FUTURE_END} time units",

},

"ema_normalized_horizons": {

"immediate": "0-1 T_window",

"recovery": "1-5 T_window",

"late": "5 T_window to end",

},

"full_regeneration": "Each (seed, rho) pair is independently initialized",

},

}

return manifest

def flatten_result(

text

seed: int,

rho: float,

result: dict[str, Any],

) -> dict[str, Any]:

text

"""Flatten a (seed, rho) result into a flat dict for NPZ storage."""

payload: dict[str, Any] = {

```
f"seed_{seed}_rho_{rho}_config_T_window": result["config"]["T_window"],

f"seed_{seed}_rho_{rho}_config_alpha": result["config"]["alpha"],

}



# Add prefix keys

for key, value in result["prefix"].items():

payload[f"prefix_seed_{seed}_rho_{rho}_{key}"] = value



# Add branch results

for branch, branch_result in result["branches"].items():

for key, value in branch_result.items():

if isinstance(value, np.ndarray):

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value

else:

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value



return payload
```

```
def extract_scalar_row(
text
seed: int,
seed_index: int,
rho: float,
branch: str,
branch_result: dict[str, Any],
) -> dict[str, Any]:
text
"""Extract a scalar row for CSV output.


Validates that all required outputs are present before extraction.
"""
# Validate required outputs
validate_branch_outputs(branch, branch_result)


row = {
```

"seed_index": seed_index,

"seed": seed,

"rho": rho,

"T_window": branch_result.get("T_window", float('nan')),

"branch": branch,

}

for key in BRANCH_SCALAR_KEYS:

if key in branch_result:

row[key] = branch_result[key]

else:

row[key] = float('nan')

return row

==
Main Execution
==

```
def parse_arguments() -> argparse.Namespace:

text

"""Parse command line arguments."""

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)

parser.add_argument(

"--master-seed",

type=int,

default=DEFAULT_MASTER_SEED,

help="fixed integer master seed (default: 20260714). For production, must match frozen data.",

)

parser.add_argument(

"--n-seeds",

type=int,

default=DEFAULT_N_SEEDS,

help=f"number of seeds (default: {DEFAULT_N_SEEDS})",
```

```
)

parser.add_argument(

"--run-root",

default=DEFAULT_RUN_ROOT,

help=f"output root directory (default: {DEFAULT_RUN_ROOT})",

)

parser.add_argument(

"--frozen-manifest",

default=None,

help="path to frozen manifest for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-npz",

default=None,

help="path to frozen NPZ for baseline equivalence test. "
```

"Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-scalar",

default=None,

help="path to frozen scalar CSV for baseline equivalence test. "

"MANDATORY for scalar validation. Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--overwrite",

action="store_true",

help="overwrite existing run directory. Use with caution - will delete existing data.",

)

NOTE: --skip-baseline removed for production safety

Baseline equivalence is MANDATORY and cannot be skipped

For debugging, use a separate nonproduction namespace

```
parser.add_argument(

"--dry-run",

action="store_true",

help="validate configuration and exit without running",

)

return parser.parse_args()

def main() -> None:

text

"""Main entry point."""

args = parse_arguments()



# Generate seeds

seeds = generate_seed_set(int(args.master_seed), args.n_seeds)



# Build manifest

manifest = build_manifest(

master_seed=int(args.master_seed),
```

```
seeds=seeds,

rho_values=RHO_VALUES,

)



# Compute config hash

manifest_text = json.dumps(

manifest, sort_keys=True, separators=(",", ":"), allow_nan=False

)

config_hash = hashlib.sha256(manifest_text.encode()).hexdigest()[:12]



# Check dry-run mode FIRST to avoid contaminating production directory

if args.dry_run:

print("Dry run: configuration validated successfully.")

print(f"Configuration hash: {config_hash}")

print("To execute, run without --dry-run")

print(f"Expected run directory: {Path([args.run](http://args.run)_root) / config_hash}")

return
```

Create run directory

run_dir = Path(args.run_root) / config_hash

Check if run directory already exists

if run_dir.exists():

if args.overwrite:

Overwrite mode: remove existing directory

print(f"Overwriting existing run directory: {run_dir}")

try:

shutil.rmtree(run_dir)

except Exception as e:

print(f"Cannot overwrite: error removing directory: {e}")

sys.exit(1)

run_dir.mkdir(parents=True, exist_ok=True)

else:

Default: fail if directory exists

```
print(f"Run directory {run_dir} already exists")

print("Use --overwrite to replace")

sys.exit(1)

else:

# New run directory

run_dir.mkdir(parents=True, exist_ok=True)



# Save manifest

manifest["config_hash"] = config_hash

manifest_path = run_dir / "manifest.json"

atomic_write_json(manifest_path, manifest)



print(f"{STUDY_NAME}")

print(f" master seed: {args.master_seed}")

print(f" n_seeds: {args.n_seeds}")

print(f" rho values: {RHO_VALUES}")

print(f" configuration: {config_hash}")
```

```
print(f" output: {run_dir}")

print()



# Create protocol config (same for all rho)

protocol = create_protocol_config()



# MANDATORY: Run baseline equivalence test at rho=1

# This cannot be skipped - baseline equivalence is required for freezeability

print("=" * 80)

print("Running MANDATORY baseline equivalence test at rho=1...")

print("=" * 80)



# Validate all frozen paths are provided (MANDATORY)

for path_name, path_value in [

("frozen-manifest", args.frozen_manifest),

("frozen-npz", args.frozen_npz),

("frozen-scalar", args.frozen_scalar),
```

]:

if path_value is None:

print(f"\n{path_name} path not provided!")

print("All frozen baseline artifacts are required for production runs.")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40")

print("Then provide all paths with --frozen-manifest, --frozen-npz, --frozen-scalar")

sys.exit(1)

if not os.path.isfile(path_value):

print(f"\n{path_name} file not found: {path_value}")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path.")

sys.exit(1)

Run baseline equivalence - MANDATORY for all seeds

```
try:
baseline_results = run_baseline_equivalence(
seeds=seeds,
frozen_manifest_path=args.frozen_manifest,
frozen_npz_path=args.frozen_npz,
frozen_csv_path=args.frozen_scalar,
)
except Exception as exc:
# Handle baseline equivalence exception - save failure manifest
failure_manifest = {
**manifest,
"config_hash": config_hash,
"status": "FAILED",
"failure_reason": "baseline_equivalence_exception",
"exception_type": type(exc).__name__,
"exception_message": str(exc),
"freezeable": False,
}
```

```
failure_manifest_path = run_dir / "FAILED_manifest.json"

atomic_write_json(failure_manifest_path, failure_manifest)

print(f"\nBaseline equivalence exception occurred!")

print(f"Failure manifest saved to {failure_manifest_path}")

print(f"Exception: {type(exc).__name__}: {exc}")

sys.exit(1)



if baseline_results["status"] != "passed":

error_msg = baseline_results.get('error', 'unknown')

print(f"\nBaseline equivalence test FAILED: {error_msg}")


# Check if it's a configuration mismatch

if "mismatch" in error_msg.lower():

print("\nConfiguration mismatch detected!")

print("The frozen data does not match the expected production configuration.")

print("Expected configuration (from
AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py)):")

print(f" N={FROZEN_N}, DT={FROZEN_DT}, TAU={FROZEN_TAU}")
```

```
print(f" OMEGA={FROZEN_OMEGA.tolist()}")

print(f" T_WINDOW={FROZEN_T_STAR:.6f}")

print("\nTo generate correct frozen data:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path with --frozen-manifest and --frozen-npz")



# Save failure manifest with detailed results

failure_manifest = {

**manifest,

"config_hash": config_hash,

"status": "FAILED",

"failure_reason": "baseline_equivalence_failed",

"failure_details": baseline_results,

"freezeable": False,

}

failure_manifest_path = run_dir / "FAILED_manifest.json"

atomic_write_json(failure_manifest_path, failure_manifest)
```

```
# Also save detailed per-seed results

if "seed_results" in baseline_results:

baseline_eq_path = run_dir / "baseline_equivalence_by_seed.json"

atomic_write_json(baseline_eq_path, baseline_results["seed_results"])

print(f" Per-seed results saved to {baseline_eq_path}")


print(f"\nFailure manifest saved to {failure_manifest_path}")


sys.exit(1)

else:

print("\nBaseline equivalence test PASSED")

print("=" * 80)


# Save detailed baseline equivalence certificate

baseline_cert_path = run_dir / "baseline_equivalence_certificate.json"

atomic_write_json(baseline_cert_path, baseline_results)

print(f" Detailed certificate saved to {baseline_cert_path}")


# Save per-seed baseline results separately for easier audit
```

```
if "seed_results" in baseline_results:

baseline_seed_path = run_dir / "baseline_equivalence_by_seed.json"

atomic_write_json(baseline_seed_path, baseline_results["seed_results"])

print(f" Per-seed results saved to {baseline_seed_path}")


# Update manifest with baseline equivalence certificate

# Record all frozen input hashes for complete provenance

frozen_input_hashes = {}


# Get frozen manifest hash

if "frozen_manifest_hash" in baseline_results:

frozen_input_hashes["frozen_manifest"] = baseline_results["frozen_manifest_hash"]

if "frozen_manifest_path" in baseline_results:

frozen_input_hashes["frozen_manifest_path"] = baseline_results["frozen_manifest_path"]


# Get frozen NPZ hash

if "frozen_npz_hash" in baseline_results:

frozen_input_hashes["frozen_npz"] = baseline_results["frozen_npz_hash"]
```

```
if "frozen_npz_path" in baseline_results:

frozen_input_hashes["frozen_npz_path"] = baseline_results["frozen_npz_path"]


# Get frozen CSV hash

if "frozen_csv_hash" in baseline_results:

frozen_input_hashes["frozen_csv"] = baseline_results["frozen_csv_hash"]

if "frozen_csv_path" in baseline_results:

frozen_input_hashes["frozen_csv_path"] = baseline_results["frozen_csv_path"]


# Get frozen production source hash

if "frozen_source_hash" in baseline_results:

frozen_input_hashes["frozen_production_source"] = baseline_results["frozen_source_hash"]


# Compute frozen production source hash directly if not in results

# Check both possible filenames: with and without _REAL suffix

frozen_production_paths = [

Path(args.frozen_manifest).parent /
"AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py)",

Path(args.frozen_manifest).parent / "AIEF_1_4_memory_ablation_[updated.py](http://updated.py)",
```

```
]
for candidate_path in frozen_production_paths:
if candidate_path.exists():
try:
frozen_prod_hash = file_sha256(candidate_path)
frozen_input_hashes["frozen_production_source"] = frozen_prod_hash
frozen_input_hashes["frozen_production_path"] = str(candidate_path)
break
except Exception:
pass

manifest["baseline_equivalence"] = {
"status": "passed",
"n_seeds_tested": baseline_results["n_seeds"],
"n_seeds_passed": baseline_results["n_seeds"],
"engine_source_sha256": baseline_results.get("engine_source_sha256"),
"frozen_manifest_hash": baseline_results.get("frozen_manifest_hash"),
"frozen_npz_hash": baseline_results.get("frozen_npz_hash"),
```

```
"frozen_csv_hash": baseline_results.get("frozen_csv_hash", "N/A"),

"frozen_production_source_hash": frozen_input_hashes.get("frozen_production_source", "N/A"),

"frozen_input_hashes": frozen_input_hashes,

"certificate_path": str(baseline_cert_path),

"seed_results_path": str(baseline_seed_path) if "seed_results" in baseline_results else "N/A",

"branches_validated": list(FROZEN_BRANCH_ORDER), # All 7 branches

}


# Re-save manifest with baseline certificate

atomic_write_json(manifest_path, manifest)



# Run the study

print("\nRunning T_window sensitivity study...")

print(f" rho values: {RHO_VALUES}")

print(f" branches: {STUDY_BRANCHES}")

print(f" seeds: {len(seeds)}")



all_scalar_rows: list[dict[str, Any]] = []
```

all_matrix_payload: dict[str, Any] = {} # Full payload in memory; checkpoints also written per-condition

Track condition status

condition_status = []

successful_conditions = 0

failed_conditions = 0

started = time.time()

total_combinations = len(seeds) * len(RHO_VALUES)

Fatal error flag for breaking outer loop on schema/config mismatch

fatal_error = False

for seed_index, seed in enumerate(seeds):

Check for fatal error from previous seed

if fatal_error:

```
print(f" Skipping seed {seed} due to fatal schema/config error")

break


for rho in RHO_VALUES:

condition_start = time.time()


# Create config for this rho

config = create_model_config(rho)


# Checkpoint path for this condition (saved for diagnostic recovery, not resume)

checkpoint_path = run_dir / f"condition_seed_{seed}_rho_{rho:.2f}.npz"


try:

# Validate config before simulation

config.validate()

protocol.validate(config)


# Run simulation

result = simulate_seed_rho(

seed=seed,
```

```
rho=rho,

config=config,

protocol=protocol,

study_branches=STUDY_BRANCHES,

)


# Extract scalar rows

for branch in STUDY_BRANCHES:

branch_result = result["branches"][branch]

row = extract_scalar_row(

seed=seed,

seed_index=seed_index,

rho=rho,

branch=branch,

branch_result=branch_result,

)

all_scalar_rows.append(row)
```

```
# Extract matrix payload

matrix_payload = flatten_result(seed, rho, result)

all_matrix_payload.update(matrix_payload)


# Save checkpoint atomically

save_npz_atomic(checkpoint_path, matrix_payload)


condition_status.append({

"seed": seed,

"rho": rho,

"status": "success",

"elapsed_seconds": time.time() - condition_start,

})

successful_conditions += 1


except Exception as e:

error_str = str(e)

condition_status.append({
```

```
"seed": seed,

"rho": rho,

"status": "failed",

"error": error_str,

"elapsed_seconds": time.time() - condition_start,

})

failed_conditions += 1

print(f" ERROR: seed={seed}, rho={rho:.2f}: {e}")


# Stop immediately on schema/config mismatch - use fatal flag to break outer loop

if "schema" in error_str.lower() or "config" in error_str.lower() or "missing" in error_str.lower():

print(f"\n FATAL: Schema/config mismatch detected - stopping immediately")

print(f" This error will affect all remaining conditions")

# Set fatal error flag to break outer loop after inner loop

fatal_error = True

break


# Status
```

```
elapsed = time.time() - started

current_combination = seed_index * len(RHO_VALUES) + list(RHO_VALUES).index(rho) + 1

status_str = f"[{current_combination}/{total_combinations}] seed={seed}, rho={rho:.2f}
({elapsed:.1f}s)"

if successful_conditions + failed_conditions == current_combination:

print(f" {status_str}")



# Save condition status table

if condition_status:

if pd is not None:

status_df = pd.DataFrame(condition_status)

status_path = run_dir / "twindow_condition_status.csv"

save_csv_atomic(status_path, status_df)

print(f"\nCondition status saved to {status_path}")

else:

# Fallback: save as JSON if pandas not available

status_path = run_dir / "twindow_condition_status.json"

atomic_write_json(status_path, condition_status)
```

```
print(f"\nCondition status saved to {status_path} (pandas not available)")


# Record condition status in manifest

manifest["condition_status"] = {

"path": str(status_path),

"sha256": file_sha256(status_path),

"expected_rows": total_combinations,

"successful_rows": successful_conditions,

"failed_rows": failed_conditions,

}


print(f" Successful: {successful_conditions}/{total_combinations}")

print(f" Failed: {failed_conditions}/{total_combinations}")



# If any conditions failed, mark as non-freezeable

if failed_conditions &gt; 0:

manifest["freezeable"] = False

manifest["failed_conditions"] = failed_conditions
```

```
atomic_write_json(manifest_path, manifest)

print(f"\nWARNING: {failed_conditions} conditions failed - run is NOT freezeable")



# Validate we have the expected number of scalar rows

expected_rows = total_combinations * len(STUDY_BRANCHES)

actual_rows = len(all_scalar_rows)

if actual_rows != expected_rows:

print(f"\nERROR: Expected {expected_rows} scalar rows, got {actual_rows}")

print("Run is INCOMPLETE and NOT freezeable")



# Validate primary outcomes are present and finite

missing_primary = []

for row in all_scalar_rows:

for key in PRIMARY_OUTCOMES:

if key not in row or not is_finite_number(row[key]):

missing_primary.append(f"seed={row['seed']}, rho={row['rho']}, branch={row['branch']},
key={key}")
```

```
if missing_primary:
print(f"\nERROR: {len(missing_primary)} primary outcomes are missing or non-finite!")
print(f" First 5 missing: {missing_primary[:5]}")
print(" This indicates engine is not producing required outputs.")
print("Run is INCOMPLETE and NOT freezeable")



# Save outputs
csv_path = run_dir / f"{STUDY_NAME}_{config_hash}_scalar_summary.csv"
npz_path = run_dir / f"{STUDY_NAME}_{config_hash}_branch_diagnostics.npz"



# Write CSV
if all_scalar_rows:
# Sort by seed_index, rho, branch for consistent ordering
all_scalar_rows.sort(key=lambda r: (r["seed_index"], r["rho"], r["branch"]))


# Get all fieldnames
fieldnames = list(all_scalar_rows[0].keys())
```

```
temporary_csv = csv_path.with_suffix(csv_path.suffix + ".tmp")

try:

with open(temporary_csv, "w", newline="", encoding="utf-8") as handle:

writer = csv.DictWriter(handle, fieldnames=fieldnames)

writer.writeheader()

writer.writerows(all_scalar_rows)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(temporary_csv), str(csv_path))

finally:

if temporary_csv.exists():

temporary_csv.unlink()



# Write NPZ

if all_matrix_payload:

save_npz_atomic(npz_path, all_matrix_payload)
```

```
# Determine completion status
complete = (
failed_conditions == 0
and successful_conditions == total_combinations
and len(all_scalar_rows) == expected_rows
and len(missing_primary) == 0
)

# Top-level freezeable flag
baseline_passed = baseline_results.get("status") == "passed"
manifest["freezeable"] = complete and baseline_passed

# Final manifest update with completion status
manifest["completion"] = {
"status": "completed" if complete else "incomplete",
"n_conditions_requested": total_combinations,
"n_conditions_successful": successful_conditions,
```

```
"n_conditions_failed": failed_conditions,

"n_scalar_rows": len(all_scalar_rows),

"n_matrix_entries": len(all_matrix_payload),

"n_missing_primary_outcomes": len(missing_primary),

"elapsed_seconds": time.time() - started,

"freezeable": manifest["freezeable"],

}



# Add output hashes - hash ALL final outputs for complete provenance

output_hashes = {}



# Main outputs

if csv_path.exists():

output_hashes["scalar_csv"] = file_sha256(csv_path)

if npz_path.exists():

output_hashes["branch_npz"] = file_sha256(npz_path)
```

```
# Condition status

condition_status_path = run_dir / "twindow_condition_status.csv"

if condition_status_path.exists():

output_hashes["condition_status_csv"] = file_sha256(condition_status_path)



# Baseline equivalence certificate

baseline_cert_path = run_dir / "baseline_equivalence_certificate.json"

if baseline_cert_path.exists():

output_hashes["baseline_certificate"] = file_sha256(baseline_cert_path)



# Baseline per-seed results (if saved separately)

baseline_seed_path = run_dir / "baseline_equivalence_by_seed.json"

if baseline_seed_path.exists():

output_hashes["baseline_seed_results"] = file_sha256(baseline_seed_path)



if output_hashes:

manifest["output_hashes"] = output_hashes
```

```
# Re-save final manifest
atomic_write_json(manifest_path, manifest)


# Print completion status
if complete and baseline_passed:
print(f"\nRun COMPLETED successfully and is FREEZEABLE")
elif complete and not baseline_passed:
print(f"\nRun COMPLETED but baseline equivalence FAILED - NOT freezeable")
else:
print(f"\nRun INCOMPLETE - NOT freezeable")
if failed_conditions &gt; 0:
print(f" Failed conditions: {failed_conditions}/{total_combinations}")
if len(all_scalar_rows) != expected_rows:
print(f" Missing scalar rows: {expected_rows - len(all_scalar_rows)}")
if len(missing_primary) &gt; 0:
print(f" Missing primary outcomes: {len(missing_primary)}")
```

```
# Print summary

elapsed = time.time() - started

print(f"\nCompleted in {elapsed:.1f}s")

print(f"Saved {csv_path}")

print(f"Saved {npz_path}")

print(f"Manifest: {manifest_path}")

# Print top-level freezeable flag

freezeable_status = manifest.get("freezeable", False)

print(f"Freezeable: {freezeable_status}")



# Quick summary statistics

print("\n" + "=" * 80)

print("Summary Statistics by (rho, branch)")

print("=" * 80)



# Use the missing_primary count already computed earlier
```

```
if missing_primary:
print(f"WARNING: {len(missing_primary)} primary outcomes are non-finite or missing!")
print(f" First 5 missing: {missing_primary[:5]}")
print(" This indicates engine is not producing required outputs.")


# Group by rho and branch
for rho in RHO_VALUES:
for branch in STUDY_BRANCHES:
rows = [r for r in all_scalar_rows if r["rho"] == rho and r["branch"] == branch]
if rows:
# Only show keys that are actually present and not all NaN
available_keys = [k for k in PRIMARY_OUTCOMES if any(
k in r and not (isinstance(r[k], float) and np.isnan(r[k]))
for r in rows
)]

if not available_keys:
print(f" rho={rho:.2f}, branch={branch:&lt;15} | ALL PRIMARY OUTCOMES MISSING/NaN!")
```

continue

stats = {}

for key in available_keys:

values = [r[key] for r in rows if key in r and isinstance(r[key], (int, float)) and not np.isnan(r[key])]

if values:

Use sample SD (ddof=1) for descriptive statistics

stats[key] = f"{np.mean(values):.4f}±{np.std(values, ddof=1):.4f}"

else:

stats[key] = "N/A"

print(f" rho={rho:.2f}, branch={branch:<15} | " +

" | ".join(f"{k}={stats.get(k, 'N/A')}" for k in available_keys))

if name == "main":

text

main()

Ignoring paste corruption, the experimental design is mostly correct: every (s,ρ)(s,ρ) condition regenerates its prefix, the full seven-branch RNG namespace is preserved, and only the four study branches are executed. However, this driver has several concrete bugs that should be fixed before testing.
Verdict

Scientific design: strong
Full-regeneration logic: correct
Current execution readiness: NO-GO
Main immediate blockers: undefined hash function and missing --baseline-only

P0 blockers
1. file_sha256() is not defined

The driver calls:

Python

file_sha256(engine_path)
file_sha256(driver_path)
file_sha256(status_path)
file_sha256(csv_path)
file_sha256(npz_path)

but defines only:

Python

source_sha256()

The first two calls are inside try blocks, so the NameError is silently converted into "NOT_AVAILABLE". That means dry-run may misleadingly succeed with missing dependency hashes.

Later calls are not protected. A full study can finish all simulations and then crash here:

Python

"sha256": file_sha256(status_path)

Fix

Add:

Python

```
def file_sha256(path: str | os.PathLike[str]) -> str:
digest = hashlib.sha256()
with open(path, "rb") as handle:
for block in iter(
lambda: handle.read(1024 * 1024),
b"",
):
digest.update(block)
return digest.hexdigest()
```

Alternatively, consistently call:

Python

```
engine.file_sha256(...)
```

A local helper is safer because the driver should not depend on an undocumented engine utility.
2. --baseline-only is absent

The planned first run must certify the frozen baseline without automatically launching the 200-condition grid. This driver currently runs the full grid immediately after a successful baseline.

Add:

Python

```
parser.add_argument(
"--baseline-only",
action="store_true",
help=(
"run and save the mandatory frozen-baseline "
"certificate, then exit without launching the grid"
),
)
```

After the baseline certificate and updated manifest are saved:

Python

```
if args.baseline_only:
manifest["execution_mode"] = "baseline_only"
manifest["baseline_certified"] = True
manifest["freezeable"] = False
manifest["completion"] = {
"status": "baseline_only_completed",
"study_grid_started": False,
"baseline_passed": True,
}

manifest["output_hashes"] = {
"baseline_certificate": file_sha256(
baseline_cert_path
),
"baseline_seed_results": file_sha256(
baseline_seed_path
),
}

atomic_write_json(manifest_path, manifest)

print("\nBaseline-only run completed successfully.")
print("The T_window grid was not started.")
return
```

A baseline-only directory is an engine-certification artifact, not a completed sensitivity study, so freezeable should remain false while baseline_certified is true.
3. Baseline hash keys do not match the engine

The engine returns names such as:

text

frozen_manifest_file_sha256
frozen_npz_file_sha256
frozen_scalar_csv_file_sha256
frozen_manifest
frozen_npz
frozen_manifest_hash

The driver looks for nonexistent names:

text

frozen_npz_hash
frozen_csv_hash
frozen_manifest_path
frozen_npz_path
frozen_csv_path
frozen_source_hash

Therefore, even a successful baseline manifest will record None or "N/A" for the NPZ and CSV hashes.
Correct extraction

Python

```
frozen_input_hashes = {
"manifest_file_sha256": baseline_results.get(
"frozen_manifest_file_sha256"
),
"npz_file_sha256": baseline_results.get(
"frozen_npz_file_sha256"
),
"scalar_csv_file_sha256": baseline_results.get(
"frozen_scalar_csv_file_sha256"
),
"manifest_canonical_json_sha256": baseline_results.get(
"frozen_manifest_hash"
),
"manifest_path": baseline_results.get(
"frozen_manifest"
),
```

"npz_path": baseline_results.get(
"frozen_npz"
),
"scalar_csv_path": str(
Path(args.frozen_scalar).resolve()
),
}

Then use those same names in:

Python

manifest["baseline_equivalence"]

Do not search for production source files beside the run manifest. In the frozen workflow, the source need not reside inside the run directory. Read:

Python

frozen_manifest["source_sha256"]

from the frozen manifest and record it as the frozen production source hash.
4. Non-freezeable study runs still exit successfully

If conditions fail or primary outcomes are missing, the driver prints:

text

Run INCOMPLETE - NOT freezeable

but reaches the end of main() without a nonzero exit status.

That allows batch systems to interpret an incomplete run as successful.

After saving all diagnostics and the final manifest:

Python

if not manifest["freezeable"]:
raise SystemExit(1)

The failure outputs should still be preserved before exiting.
P1 correctness and schema issues
5. Secondary outcomes are not checked for finiteness

The study explicitly requires both:

fixed-physical primary outcomes;
normalized-EMA secondary outcomes.

Only PRIMARY_OUTCOMES are checked after execution. A run can therefore be marked complete and freezeable with invalid normalized outcomes.

Add:

Python

missing_secondary = []

for row in all_scalar_rows:
for key in SECONDARY_OUTCOMES:
if key not in row or not is_finite_number(row[key]):
missing_secondary.append(
f"seed={row['seed']}, "
f"rho={row['rho']}, "
f"branch={row['branch']}, "
f"key={key}"
)

Require:

Python

and len(missing_secondary) == 0

in complete, and record the count in the manifest.

Preferably validate primary and secondary fields immediately after each condition, before saving its checkpoint:

Python

```
for branch in STUDY_BRANCHES:
branch_result = result["branches"][branch]

for key in PRIMARY_OUTCOMES + SECONDARY_OUTCOMES:
if key not in branch_result:
raise RuntimeError(
f"schema: {branch} missing {key}"
)
if not is_finite_number(branch_result[key]):
raise RuntimeError(
f"schema: {branch} has non-finite {key}"
)
```

6. The “desired” historical fields are always NaN

The driver declares:

Python

```
DESIRED_BRANCH_SCALAR_KEYS = (
"Time_to_similarity_below_0_9",
"Time_to_similarity_below_0_5",
"Late_persistence_ratio_to_continue",
"Late_similarity_ratio_to_continue",
)
```

But simulate_seed_rho() never calls:

Python

engine.finalize_historical_branch_metrics(branch_results)

Thus extract_scalar_row() fills these fields with NaN for every condition.

There are two defensible choices.
Preferred: remove them

The new analysis script computes richer condition-specific recovery estimands from trajectories.
Remove DESIRED_BRANCH_SCALAR_KEYS to avoid publishing columns that are universally missing.
Alternative: compute the complete historical set

After all four branches are simulated:

Python

engine.finalize_historical_branch_metrics(branch_results)

Then include all eight historical fields, not only four. Clearly label them as legacy auxiliary metrics; they are not substitutes for sustained/held recovery analysis.

Do not retain four permanently NaN columns described as “desired.”
7. Study metadata is misclassified as frozen scalar output

This tuple labels:

Python

"T_window",
"rho",

as frozen scalar fields, but they were not part of the historical frozen branch scalar schema. They are study metadata.

Separate them:

Python

```
STUDY_METADATA_KEYS = (
"T_window",
"rho",
)
```

Then:

Python

```
BRANCH_SCALAR_KEYS = (
CONDITION_SCALAR_KEYS
+ EXTENDED_BRANCH_SCALAR_KEYS
+ STUDY_METADATA_KEYS
)
```

This avoids confusing historical equivalence fields with newly introduced condition identifiers.
8. Matrix schema validation is incomplete

The engine produces dual-horizon matrices, but the driver requires only the historical matrix fields. It does not validate:

text

```
B_mean_late_fixed_physical
P_full_fixed_physical
P_immediate_fixed_physical
P_recovery_fixed_physical
P_late_fixed_physical
B_mean_late_normalized_EMA
P_full_normalized_EMA
P_immediate_normalized_EMA
P_recovery_normalized_EMA
P_late_normalized_EMA
```

They are written because flatten_result() serializes every branch field, but their presence is not part of completion.

Use the corrected engine schema:

Python

```
REQUIRED_BRANCH_MATRIX_KEYS = (
engine.REQUIRED_BRANCH_MATRIX_KEYS
)
```

or reproduce the complete explicit tuple in the driver.

At minimum, validate the arrays required downstream:

text

```
B_final
B_mean_late_fixed_physical
B_mean_late_normalized_EMA
P_late_fixed_physical
P_late_normalized_EMA
sample_times
density_trajectory
similarity_trajectory
```

Also validate basic shapes:

Python

```
if branch_result["B_final"].shape != (FROZEN_N, FROZEN_N):
raise RuntimeError(...)

expected_samples = (
FROZEN_FUTURE_STEPS + protocol.diagnostic_stride - 1
) // protocol.diagnostic_stride
```

for key in (
"sample_times",
"density_trajectory",
"similarity_trajectory",
):
if branch_result[key].shape != (expected_samples,):
raise RuntimeError(...)

Similarity trajectories may contain legitimate early NaNs after resets, so do not require every similarity sample to be finite.
9. validate_branch_outputs() validates only presence

Despite its name, it does not validate:

scalar type;
primary/secondary finiteness;
array shape;
trajectory length;
horizon metadata;
condition metadata consistency.

At minimum, require:

Python

branch_result["rho"] == rho
branch_result["T_window"] == config.T_window
branch_result["window_steps"] == round(
config.T_window / config.dt
)
branch_result["fixed_immediate_end"] == 1750
branch_result["fixed_recovery_end"] == 8750

This should happen per condition, before that condition is counted as successful.
Baseline and preflight improvements
10. Validate mandatory input paths before creating/deleting a run directory

The driver currently creates the run directory and writes its initial manifest before checking whether the three frozen paths were supplied and valid.

Move frozen-argument validation before:

Python

```
run_dir.mkdir(...)
shutil.rmtree(...)
```

This prevents:

contaminating a production root after a simple path error;
deleting an existing run under --overwrite before discovering that a baseline input path is invalid.

Dry-run can remain path-independent if desired, but its output should say configuration validation only.
11. Dry-run does not actually validate configuration

The message says:

text

```
Dry run: configuration validated successfully.
```

but dry-run has not called:

Python

```
config.validate()
protocol.validate(config)
```

Validate all levels:

Python

```
protocol = create_protocol_config()
protocol.validate(create_model_config(1.0))
```

for rho in RHO_VALUES:
config = create_model_config(rho)
config.validate()
protocol.validate(config)

Also validate:

Python

set(STUDY_BRANCHES) <= set(FROZEN_BRANCH_ORDER)
len(STUDY_BRANCHES) == len(set(STUDY_BRANCHES))
len(RHO_VALUES) == len(set(RHO_VALUES))
1.0 in RHO_VALUES

Then the dry-run claim is accurate.
12. Assert driver constants agree with the engine

This driver duplicates all frozen constants. Add a fail-fast consistency check:

Python

def validate_engine_contract() -> None:
checks = {
"N": (FROZEN_N, engine.FROZEN_N),
"dt": (FROZEN_DT, engine.FROZEN_DT),
"tau": (FROZEN_TAU, engine.FROZEN_TAU),
"T_star": (FROZEN_T_STAR, engine.FROZEN_T_STAR),
"B_c": (FROZEN_B_C, engine.FROZEN_B_C),
}

for name, (driver_value, engine_value) in checks.items():
if driver_value != engine_value:
raise RuntimeError(
f"Driver/engine {name} mismatch: "
f"{driver_value} != {engine_value}"
)

if not np.array_equal(FROZEN_OMEGA, engine.FROZEN_OMEGA):

raise RuntimeError("Driver/engine omega mismatch")

Call it in dry-run and normal execution.
Manifest and provenance
13. The configuration hash currently includes absolute file paths

build_manifest() includes:

Python

"path": str(engine_path)
"path": str(driver_path)

and the complete manifest is hashed. Therefore, identical code and scientific configuration in different directories can receive different configuration hashes.

The environment and platform are also included, so identical experiments on different systems receive different IDs.

Prefer a separate canonical hash payload:

Python

config_identity = {
"workflow_version": WORKFLOW_VERSION,
"model": model_params,
"protocol": protocol_params,
"study": study_params,
"engine_source_sha256": engine_hash,
"driver_source_sha256": driver_hash,
}

Hash only that. Keep absolute paths and environment in the full manifest for provenance, but outside configuration identity.

Also include:

text

scale_epsilon
full seven-branch RNG namespace
executed four-branch subset
fixed physical boundaries

in the identity payload.
14. Record the full RNG namespace explicitly

The manifest currently records only the executed branches. Add:

Python

"randomization": {
...
"branch_rng_namespace": list(FROZEN_BRANCH_ORDER),
"executed_branches": list(STUDY_BRANCHES),
}

This is important because future_shuffle receives child index 6 rather than child index 3.
15. Condition checkpoints are not covered by output hashes

The comment says all final outputs are hashed, but the 200 condition checkpoints are not.

Either:

declare checkpoints non-authoritative diagnostic artifacts and say the consolidated NPZ is the authority; or
record their hashes in a separate checkpoint manifest.

The former is simpler:

Python

"checkpoints": {
"role": "diagnostic, non-authoritative",

"count": successful_conditions,
"authoritative_consolidation": str(npz_path),
}

Operational cleanup
16. Pandas fallback is unreachable

At import time, missing pandas causes sys.exit(1), but later code contains:

Python

if pd is not None:
...
else:
JSON fallback

The fallback can never execute.

Either make pandas mandatory and remove the dead fallback, or avoid pandas entirely for the condition-status CSV. The standard csv module is already imported.

If pandas remains mandatory, record:

Python

"pandas": pd.__version__

in the environment manifest.
17. Partial outputs are reported as saved even when absent

At the end, the driver always prints:

Python

print(f"Saved {csv_path}")
print(f"Saved {npz_path}")

even if no scalar or matrix payload was generated.

Print conditionally:

Python

```
if csv_path.exists():
print(f"Saved {csv_path}")
if npz_path.exists():
print(f"Saved {npz_path}")
```

What is correct

The key scientific architecture is sound:

ρ∈{0.5,0.75,1,1.5,2}ρ∈{0.5,0.75,1,1.5,2};
exact T∗=25/ω¯T∗=25/ω¯;
alpha regenerated for every condition;
prefix regenerated separately for each (s,ρ)(s,ρ);
no incompatible split-state reuse;
paired seed cohort retained across levels;
full frozen RNG namespace retained;
four study branches selected correctly;
baseline certifies all seven historical branches;
fixed-physical primary and normalized-EMA secondary names are correct;
successful-condition checkpoints are atomic;
consolidated CSV/NPZ outputs are atomic;
run directory reuse fails unless explicitly overwritten;
condition failures make the manifest non-freezeable;
primary fields are checked for finiteness;
final output hashes are intended.

Required changes before testing

At minimum, fix these before the first run:

```
define file_sha256;
restore --baseline-only;
```

align frozen input hash key names with the engine;
validate normalized secondary outcomes;
exit nonzero for incomplete/non-freezeable runs;
remove or actually compute the always-NaN desired fields;
validate frozen paths before creating the run directory.

After that:

compile all three files;
run driver --dry-run;
run the mandatory baseline with --baseline-only in a fresh baseline-test root;
inspect the complete certificate;
only after an exact pass, launch the five-level grid in a separate fresh production root.

And the third:

#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Analysis Driver v3.

Correct implementation of the prespecified estimands:

Regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}

Branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]

Cross-T topology: S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Recovery scaling: t_rec / T_window for both horizon systems

Output:

twindow_paired_regime_contrasts.csv

twindow_branch_interactions.csv

twindow_cross_condition_topology.csv

twindow_recovery_summary.csv

twindow_sensitivity_report.txt

analysis_manifest.json

This script implements the 5-stage analysis structure as specified.

"""

```
from future import annotations

import argparse

import csv

import hashlib

import json

import os

import sys

import time

from pathlib import Path

from typing import Any

import numpy as np

import pandas as pd
```

```
================================================================================
Configuration
================================================================================

ANALYSIS_VERSION = "v3.0"

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"
Expected values (for validation)

EXPECTED_N_SEEDS = 40

EXPECTED_RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)

EXPECTED_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Primary outcomes - FIXED-PHYSICAL HORIZONS (corrected from legacy)

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"P_mean_late_fixed_physical",

)
Secondary outcomes - NORMALIZED EMA HORIZONS

SECONDARY_NORMALIZED_OUTCOMES = (

text

"Weighted_similarity_late_normalized_EMA",
```

"Retention_contrast_late_normalized_EMA",

"P_mean_late_normalized_EMA",

)
Expected counts

EXPECTED_N_CONDITIONS = EXPECTED_N_SEEDS * len(EXPECTED_RHO_VALUES)

EXPECTED_N_ROWS = EXPECTED_N_CONDITIONS * len(EXPECTED_BRANCHES)

EXPECTED_N_REGIME_CONTRASTS = EXPECTED_N_SEEDS
(len(EXPECTED_RHO_VALUES) - 1) len(PRIMARY_OUTCOMES)

EXPECTED_N_DID = EXPECTED_N_SEEDS (len(EXPECTED_RHO_VALUES) - 1)
(len(EXPECTED_BRANCHES) - 1) * len(PRIMARY_OUTCOMES)

EXPECTED_N_TOPOLOGY = EXPECTED_N_SEEDS * (len(EXPECTED_RHO_VALUES) - 1)
Non-continue branches for DID

NON_CONTINUE_BRANCHES = ("reset_ema", "reset_B", "future_shuffle")
Matrix keys needed for cross-T topology

MATRIX_KEYS = ("B_final",)
==========
====
Utility Functions
==========
====

def load_csv_atomic(path: str | Path) -> pd.DataFrame:

text

"""Load CSV file atomically with validation."""

```
path = Path(path)



# Use pandas with explicit dtypes

df = [pd.read](http://pd.read)_csv(

path,

dtype={"seed": str, "branch": str}, # Keep seeds as strings

)



# Convert numeric columns, coercing errors to NaN

for col in df.columns:

if col not in ("seed", "seed_index", "branch", "rho"):

try:

df[col] = [pd.to](http://pd.to)_numeric(df[col], errors="coerce")

except Exception:

pass



return df
```

```
def load_npz_selective(path: str | Path, keys: list[str] | None = None) -> dict[str, Any]:

text

"""Load only specified keys from NPZ file to save memory."""

path = Path(path)

payload: dict[str, Any] = {}



if not path.exists():

print(f"Warning: NPZ file not found: {path}")

return payload



try:

with np.load(str(path), allow_pickle=False) as data:

if keys is None:

keys = list(data.files)

for key in keys:

if key in data:

value = data[key]
```

payload[key] = value.item() if value.ndim == 0 else value.copy()

except Exception as e:

print(f"Warning: Could not load NPZ {path}: {e}")

return payload

def save_csv_atomic(path: str | Path, df: pd.DataFrame) -> None:

text

"""Save CSV atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

df.to_csv(temp_path, index=False)

temp_path.replace(path)

fsync the directory to ensure the replace is visible

dir_fd = os.open(str(path.parent), os.O_RDONLY)

```
try:
os.fsync(dir_fd)
finally:
os.close(dir_fd)
finally:
if temp_path.exists():
temp_path.unlink()
def save_text_atomic(path: str | Path, text: str) -> None:
text
"""Save text file atomically with fsync."""
path = Path(path)
path.parent.mkdir(parents=True, exist_ok=True)



temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")
try:
with open(temp_path, "w", encoding="utf-8") as f:
f.write(text)
```

```
f.flush()

os.fsync(f.fileno())

temp_path.replace(path)

# fsync the directory

dir_fd = [os.open](http://os.open)(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

def save_json_atomic(path: str | Path, data: dict[str, Any]) -> None:

text

"""Save JSON atomically with fsync, handling NaN values."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)
```

```
# Convert NaN and infinity to None for JSON serialization

# Recursively handles nested structures including numpy arrays

def sanitize(obj: Any) -&gt; Any:

if isinstance(obj, dict):

return {k: sanitize(v) for k, v in obj.items()}

elif isinstance(obj, list):

return [sanitize(v) for v in obj]

elif isinstance(obj, tuple):

return tuple(sanitize(v) for v in obj)

elif isinstance(obj, float):

if np.isnan(obj) or np.isinf(obj):

return None

return obj

elif isinstance(obj, (np.integer, np.floating)):

val = obj.item()

if isinstance(val, float) and (np.isnan(val) or np.isinf(val)):

return None
```

return val

elif isinstance(obj, np.ndarray):

Convert to list and recursively sanitize to preserve structure

return sanitize(obj.tolist())

else:

return obj

temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temp_path, "w", encoding="utf-8") as f:

json.dump(sanitize(data), f, indent=2, sort_keys=True, allow_nan=False)

f.write("\n")

f.flush()

os.fsync(f.fileno())

temp_path.replace(path)

fsync the directory

dir_fd = os.open(str(path.parent), os.O_RDONLY)

```
try:
os.fsync(dir_fd)
finally:
os.close(dir_fd)
finally:
if temp_path.exists():
temp_path.unlink()
def file_sha256(path: str | Path) -> str:
text
"""Compute SHA-256 hash of a file."""
import hashlib as hl
digest = hl.sha256()
path = Path(path)
if not path.exists():
return "FILE_NOT_FOUND"
with open(path, "rb") as f:
for block in iter(lambda: [f.read](http://f.read)(1024 * 1024), b""):
digest.update(block)
```

```
return digest.hexdigest()

============================================================================
====
Metric Functions
============================================================================
====

def first_crossing_time(times: np.ndarray, values: np.ndarray, threshold: float) -> tuple[float, bool]:

text

"""Find first time when values cross above threshold.



Returns (first_crossing_time, observed).

"""

times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)



# Handle NaN values by treating them as below threshold

above_threshold = (np.isfinite(values) &amp;amp; (values &amp;gt;= threshold))



crossing_indices = np.flatnonzero(above_threshold)
```

if len(crossing_indices) > 0:

return float(times[crossing_indices[0]]), True

else:

return float('nan'), False

def first_time_sustained_for_rest(times: np.ndarray, values: np.ndarray, threshold: float) -> tuple[float, bool]:

text

"""Find earliest time after which values stay above threshold for rest of trajectory.

Returns (sustained_time, observed).

"""

times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)

Handle NaN values by treating them as below threshold

condition = (np.isfinite(values) & (values >= threshold))

Check from the end backwards: find first index where all subsequent values are above threshold

suffix_all = np.logical_and.accumulate(condition[::-1])[::-1]

sustained_indices = np.flatnonzero(suffix_all)

if len(sustained_indices) > 0:

return float(times[sustained_indices[0]]), True

else:

return float('nan'), False

def first_time_held_for_duration(times: np.ndarray, values: np.ndarray, threshold: float, duration: float) -> tuple[float, bool]:

text

"""Find earliest time after which values stay above threshold for at least the specified duration.

Returns (held_time, observed).

"""

times = np.asarray(times, dtype=float)

```
values = np.asarray(values, dtype=float)



if len(times) &lt; 2:

return float('nan'), False



# Calculate sample step

sample_step = float(times[1] - times[0])

if sample_step &lt;= 0:

return float('nan'), False



# Required number of consecutive samples

required_samples = max(1, int(np.ceil(duration / sample_step)))



# Handle NaN values by treating them as below threshold

condition = (np.isfinite(values) &amp; (values &gt;= threshold))
```

Use sliding window to find runs of sufficient length

held_time = float('nan')

observed = False

for i in range(len(condition) - required_samples + 1):

window = condition[i:i + required_samples]

if np.all(window):

held_time = float(times[i])

observed = True

break

return held_time, observed

def trapezoidal_auc_deficit(times: np.ndarray, reference: np.ndarray, target: np.ndarray) -> float:

text

"""Calculate one-sided AUC deficit: integral of max(0, reference - target) using trapezoidal rule.

Handles NaN values by treating them as zero for the deficit calculation.

"""

times = np.asarray(times, dtype=float)

reference = np.asarray(reference, dtype=float)

target = np.asarray(target, dtype=float)

if len(times) < 2:

return float('nan')

Calculate one-sided deficit: max(0, reference - target)

Handle NaN by replacing with 0

ref_safe = np.where(np.isfinite(reference), reference, 0.0)

tgt_safe = np.where(np.isfinite(target), target, 0.0)

deficit = np.maximum(0.0, ref_safe - tgt_safe)

Trapezoidal integration

return float(np.trapz(deficit, x=times))

def cosine_similarity(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute cosine similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]

norm_x = np.linalg.norm(x)

norm_y = np.linalg.norm(y)

if norm_x <= 1e-12 or norm_y <= 1e-12:

return float('nan')

return float(np.dot(x, y) / (norm_x * norm_y))

def jaccard_similarity(B1: np.ndarray, B2: np.ndarray, threshold: float) -> float:

text

"""Compute Jaccard similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges] > threshold

y = B2[edges] > threshold

union = np.count_nonzero(x | y)

if union == 0:

return float('nan')

return float(np.count_nonzero(x & y) / union)

def normalized_distance(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute normalized L2 distance between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]

diff = x - y

norm_diff = np.linalg.norm(diff)

norm_y = np.linalg.norm(y)

if norm_y <= 1e-12:

return float('nan')

return float(norm_diff / norm_y)

def compute_recovery_scaling(

text

df: pd.DataFrame,

npz_path: Path,

b_c: float,

) -> pd.DataFrame:

text

"""Compute recovery scaling metrics for both horizon systems.

For each seed, rho, branch, and horizon system:

1. Load: sample_times, density_trajectory, similarity_trajectory

2. Use the same seed/rho continue branch as reference

3. Calculate condition-specific targets:

- Density target: 0.9 * D_B_final_continue (fraction of edges above B_c)

- Similarity target: 0.9 * S_late_continue (for appropriate horizon system)

4. Calculate:

- first_crossing_time: first time metric > target

- sustained_time: earliest time after which metric stays above target for rest of trajectory

- held_time: earliest time after which metric stays above target for at least T_window duration

- censoring indicators for each event type

- restricted_time: horizon for censored events, event_time for observed events

- auc_deficit: one-sided integral of (continue_trajectory - branch_trajectory) using trapezoidal rule

- t_rec / T_window: normalized recovery times

5. Output separate rows for fixed_physical and normalized_EMA horizon systems

Returns DataFrame with complete recovery metrics.

"""

Get unique seeds, rho values, and branches from the data

```
seeds = sorted(df["seed"].unique())

rho_values = sorted([float(r) for r in df["rho"].unique()])

branches = sorted(df["branch"].unique())



# Build complete list of required keys for all conditions

# We need: sample_times, density_trajectory, similarity_trajectory for all seed/rho/branch

all_keys = set()

for seed in seeds:

for rho in rho_values:

for branch in branches:

prefix = f"{branch}_seed_{seed}_rho_{rho}"

all_keys.update([

f"{prefix}_sample_times",

f"{prefix}_density_trajectory",

f"{prefix}_similarity_trajectory",

])
```

```
# Load all required data in one operation

print(f" Loading {len(all_keys)} trajectory arrays from NPZ...")

npz_data = load_npz_selective(npz_path, keys=list(all_keys))



# Check for missing keys - this should be fatal for production

missing_keys = [k for k in all_keys if k not in npz_data]

if missing_keys:

raise RuntimeError(

f"Missing trajectory keys in NPZ: {missing_keys[:10]}... "

f"({len(missing_keys)} total missing). "

f"This indicates schema mismatch or incomplete simulation."

)



recovery_rows = []



for seed in seeds:

for rho in rho_values:
```

```
# Get T_window for this rho from the data

t_window = float(df[(df["seed"] == seed) &amp;amp; (df["rho"] == rho)]["T_window"].iloc[0])


# Get continue branch reference values for this seed/rho

cont_row = df[(df["seed"] == seed) &amp;amp; (df["rho"] == rho) &amp;amp; (df["branch"] ==
"continue")].iloc[0]


# Get reference targets for both horizon systems

# Density target: 0.9 * D_B_final_continue (fraction of edges above B_c)

density_target_continue = 0.9 * float(cont_row["D_B_final"])


# Similarity targets for both horizon systems

similarity_target_fixed = 0.9 * float(cont_row["Weighted_similarity_late_fixed_physical"])

similarity_target_normalized = 0.9 * float(cont_row["Weighted_similarity_late_normalized_EMA"])


for branch in branches:

# INCLUDE continue branch for DID calculation

# For continue: compare trajectory with itself, AUC deficit = 0

# This makes recovery DID well-defined: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]
```

```
prefix = f"{branch}_seed_{seed}_rho_{rho}"

cont_prefix = f"continue_seed_{seed}_rho_{rho}"


# Load trajectory data

try:

sample_times = np.asarray(npz_data[f"{prefix}_sample_times"], dtype=float)

density_trajectory = np.asarray(npz_data[f"{prefix}_density_trajectory"], dtype=float)

similarity_trajectory = np.asarray(npz_data[f"{prefix}_similarity_trajectory"], dtype=float)


# Load continue (reference) trajectory data

cont_sample_times = np.asarray(npz_data[f"{cont_prefix}_sample_times"], dtype=float)

cont_density_trajectory = np.asarray(npz_data[f"{cont_prefix}_density_trajectory"], dtype=float)

cont_similarity_trajectory = np.asarray(npz_data[f"{cont_prefix}_similarity_trajectory"], dtype=float)


except Exception as e:

raise RuntimeError(f"Error loading trajectory data for seed={seed}, rho={rho}, branch={branch}:
{e}")


# Validate that sample times match between branch and continue

# Use exact equality since all trajectories use the same diagnostic schedule
```

```
if not np.array_equal(sample_times, cont_sample_times):
raise RuntimeError(
f"Sample times mismatch for seed={seed}, rho={rho}, branch={branch}: "
f"branch has {len(sample_times)} times, continue has {len(cont_sample_times)}"
)

# Also validate that trajectory lengths match
if len(density_trajectory) != len(cont_density_trajectory):
raise RuntimeError(
f"Density trajectory length mismatch for seed={seed}, rho={rho}, branch={branch}: "
f"branch has {len(density_trajectory)} points, continue has {len(cont_density_trajectory)}"
)
if len(similarity_trajectory) != len(cont_similarity_trajectory):
raise RuntimeError(
f"Similarity trajectory length mismatch for seed={seed}, rho={rho}, branch={branch}: "
f"branch has {len(similarity_trajectory)} points, continue has {len(cont_similarity_trajectory)}"
)
```

```
# Process each horizon system

for horizon_system, similarity_target in [

("fixed_physical", similarity_target_fixed),

("normalized_EMA", similarity_target_normalized),

]:

# Process density recovery

density_values = density_trajectory

density_target = density_target_continue


# Calculate first crossing for density

density_first_crossing, density_first_observed = first_crossing_time(

sample_times, density_values, density_target

)


# Calculate sustained recovery for density (stays above target for rest of trajectory)

density_sustained, density_sustained_observed = first_time_sustained_for_rest(

sample_times, density_values, density_target

)
```

```
# Calculate held recovery for density (stays above target for T_window duration)
density_held, density_held_observed = first_time_held_for_duration(
sample_times, density_values, density_target, t_window
)

# Calculate AUC deficit for density (one-sided: continue - branch)
density_auc_deficit = trapezoidal_auc_deficit(
sample_times, cont_density_trajectory, density_values
)

# Process similarity recovery
similarity_values = similarity_trajectory
similarity_target = similarity_target

# Calculate first crossing for similarity
sim_first_crossing, sim_first_observed = first_crossing_time(
sample_times, similarity_values, similarity_target
)
```

```
# Calculate sustained recovery for similarity
sim_sustained, sim_sustained_observed = first_time_sustained_for_rest(
sample_times, similarity_values, similarity_target
)

# Calculate held recovery for similarity
sim_held, sim_held_observed = first_time_held_for_duration(
sample_times, similarity_values, similarity_target, t_window
)

# Calculate AUC deficit for similarity
sim_auc_deficit = trapezoidal_auc_deficit(
sample_times, cont_similarity_trajectory, similarity_values
)

# Calculate restricted times (horizon for censored, event_time for observed)
# Use the last sample time as the horizon (not the elapsed difference)
horizon_time = float(sample_times[-1]) if len(sample_times) &gt; 0 else float('nan')
```

```
# For density metrics

density_restricted_first = density_first_crossing if density_first_observed else horizon_time

density_restricted_sustained = density_sustained if density_sustained_observed else horizon_time

density_restricted_held = density_held if density_held_observed else horizon_time


# For similarity metrics

sim_restricted_first = sim_first_crossing if sim_first_observed else horizon_time

sim_restricted_sustained = sim_sustained if sim_sustained_observed else horizon_time

sim_restricted_held = sim_held if sim_held_observed else horizon_time


# Calculate normalized times (t_rec / T_window)

# Raw observed times (conditional on recovery)

density_first_over_Twindow = density_first_crossing / t_window if np.isfinite(density_first_crossing) and t_window &gt; 0 else float('nan')

density_sustained_over_Twindow = density_sustained / t_window if np.isfinite(density_sustained) and t_window &gt; 0 else float('nan')

density_held_over_Twindow = density_held / t_window if np.isfinite(density_held) and t_window &gt; 0 else float('nan')
```

sim_first_over_Twindow = sim_first_crossing / t_window if np.isfinite(sim_first_crossing) and t_window > 0 else float('nan')

sim_sustained_over_Twindow = sim_sustained / t_window if np.isfinite(sim_sustained) and t_window > 0 else float('nan')

sim_held_over_Twindow = sim_held / t_window if np.isfinite(sim_held) and t_window > 0 else float('nan')

Censoring-aware restricted normalized times (cohort-level estimates)

These use restricted times (horizon for censored, event_time for observed) divided by T_window

density_restricted_first_over_Twindow = density_restricted_first / t_window if np.isfinite(density_restricted_first) and t_window > 0 else float('nan')

density_restricted_sustained_over_Twindow = density_restricted_sustained / t_window if np.isfinite(density_restricted_sustained) and t_window > 0 else float('nan')

density_restricted_held_over_Twindow = density_restricted_held / t_window if np.isfinite(density_restricted_held) and t_window > 0 else float('nan')

sim_restricted_first_over_Twindow = sim_restricted_first / t_window if np.isfinite(sim_restricted_first) and t_window > 0 else float('nan')

sim_restricted_sustained_over_Twindow = sim_restricted_sustained / t_window if np.isfinite(sim_restricted_sustained) and t_window > 0 else float('nan')

sim_restricted_held_over_Twindow = sim_restricted_held / t_window if np.isfinite(sim_restricted_held) and t_window > 0 else float('nan')

Add rows for density metrics

```
# Note: For continue branch, AUC deficit = 0 (comparing trajectory with itself)

# Held duration uses T_window for both horizon systems (Option B: documented below)

recovery_rows.append({

"seed": seed,

"rho": rho,

"T_window": t_window,

"branch": branch,

"horizon_system": horizon_system,

"metric": "density",

"target_value": density_target,

"target_source": "0.9 * D_B_final_continue",

"target_horizon_dependence": "horizon_independent",

"held_duration_definition": "one_current_T_window",

"auc_horizon": "full_future",

"first_crossing_time": density_first_crossing,

"first_crossing_observed": density_first_observed,

"sustained_time": density_sustained,

"sustained_observed": density_sustained_observed,
```

```
"held_time": density_held,
"held_observed": density_held_observed,
"restricted_first_crossing_time": density_restricted_first,
"restricted_sustained_time": density_restricted_sustained,
"restricted_held_time": density_restricted_held,
"auc_deficit_vs_continue": density_auc_deficit,
"first_crossing_over_Twindow": density_first_over_Twindow,
"sustained_time_over_Twindow": density_sustained_over_Twindow,
"held_time_over_Twindow": density_held_over_Twindow,
"restricted_first_over_Twindow": density_restricted_first_over_Twindow,
"restricted_sustained_over_Twindow": density_restricted_sustained_over_Twindow,
"restricted_held_over_Twindow": density_restricted_held_over_Twindow,
})

# Add rows for similarity metrics
recovery_rows.append({
"seed": seed,
```

"rho": rho,

"T_window": t_window,

"branch": branch,

"horizon_system": horizon_system,

"metric": "similarity",

"target_value": similarity_target,

"target_source": f"0.9 * Weighted_similarity_late_{horizon_system}",

"target_horizon_dependence": "horizon_independent",

"held_duration_definition": "one_current_T_window",

"auc_horizon": "full_future",

"first_crossing_time": sim_first_crossing,

"first_crossing_observed": sim_first_observed,

"sustained_time": sim_sustained,

"sustained_observed": sim_sustained_observed,

"held_time": sim_held,

"held_observed": sim_held_observed,

"restricted_first_crossing_time": sim_restricted_first,

"restricted_sustained_time": sim_restricted_sustained,

"restricted_held_time": sim_restricted_held,

"auc_deficit_vs_continue": sim_auc_deficit,

"first_crossing_over_Twindow": sim_first_over_Twindow,

"sustained_time_over_Twindow": sim_sustained_over_Twindow,

"held_time_over_Twindow": sim_held_over_Twindow,

"restricted_first_over_Twindow": sim_restricted_first_over_Twindow,

"restricted_sustained_over_Twindow": sim_restricted_sustained_over_Twindow,

"restricted_held_over_Twindow": sim_restricted_held_over_Twindow,

})

if not recovery_rows:

raise RuntimeError("No recovery scaling data computed - check NPZ contents and data structure")

Validate expected count: 40 seeds × 5 rho × 4 branches × 2 metrics × 2 horizon systems = 3200

Now includes continue branch for DID calculation

expected_count = len(seeds) *len(rho_values)* len(branches) *2* 2

if len(recovery_rows) != expected_count:

```
raise RuntimeError(
f"Expected {expected_count} recovery rows, got {len(recovery_rows)}. "
f"This may indicate a logic error in recovery calculation."
)


recovery_df = pd.DataFrame(recovery_rows)


print(f"✅ Stage 4b: Recovery Scaling - {len(recovery_rows)} recovery metrics computed")
print(f" Seeds: {recovery_df['seed'].nunique()}, Rho: {recovery_df['rho'].nunique()}, "
f"Branches: {recovery_df['branch'].nunique()}, Metrics: {recovery_df['metric'].nunique()}, "
f"Systems: {recovery_df['horizon_system'].nunique()}")


return recovery_df

================================================================================
Stage 1: Validation
================================================================================

def validate_run(
```

text

csv_path: Path,

npz_path: Path,

manifest_path: Path,

) -> dict[str, Any]:

text

"""Validate the run before analysis.

Checks:

- 800 scalar rows (40 seeds × 5 rho × 4 branches)

- 40 unique seeds

- 5 rho values per seed

- 4 branches per seed/rho

- baseline equivalence passed

- run is freezeable

- no duplicate condition keys

- all primary outcomes are present and numeric

"""

results = {

"status": "passed",

"errors": [],

"warnings": [],

}

Load manifest

try:

with open(manifest_path, "r") as f:

manifest = json.load(f)

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Cannot load manifest: {e}")

return results

Validate condition status - MANDATORY for production

```
condition_status_path = Path(csv_path.parent) / "twindow_condition_status.csv"

if not condition_status_path.exists():

results["status"] = "failed"

results["errors"].append("Condition status file not found - REQUIRED for production analysis")

return results



try:

condition_df = [pd.read](http://pd.read)_csv(condition_status_path)

successful_conditions = (condition_df["status"] == "success").sum()

failed_conditions = (condition_df["status"] == "failed").sum()

total_conditions = len(condition_df)


expected_conditions = EXPECTED_N_CONDITIONS

if total_conditions != expected_conditions:

results["status"] = "failed"

results["errors"].append(

f"Condition status: expected {expected_conditions} conditions, "
```

```
f"got {total_conditions}"

)

return results


if failed_conditions &gt; 0:

results["status"] = "failed"

results["errors"].append(

f"Condition status: {failed_conditions} conditions failed, "

f"{successful_conditions} succeeded"

)

return results


# Hash the condition status file

condition_status_hash = file_sha256(condition_status_path)

results["condition_status_hash"] = condition_status_hash


except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Could not validate condition status: {e}")
```

```
return results



# Check baseline equivalence

baseline_eq = manifest.get("baseline_equivalence", {})

if baseline_eq.get("status") != "passed":

results["status"] = "failed"

results["errors"].append(

f"Baseline equivalence not passed: {baseline_eq.get('status', 'unknown')}"

)

return results



# Check freezeable status - FAIL if not freezeable

# Check both top-level and completion.freezeable, default to False

completion = manifest.get("completion", {})

freezeable = manifest.get("freezeable", completion.get("freezeable", False))



# Also check completion status
```

```
completion_status = completion.get("status", "")

n_successful = completion.get("n_conditions_successful", 0)

n_failed = completion.get("n_conditions_failed", 0)

n_expected = EXPECTED_N_CONDITIONS



# Must have all conditions successful and explicit freezeable=true

if not freezeable:

results["status"] = "failed"

results["errors"].append("Run is marked as non-freezeable")

return results



if completion_status != "completed":

results["status"] = "failed"

results["errors"].append(f"Run not completed: status={completion_status}")

return results



if n_successful != n_expected or n_failed != 0:
```

```
results["status"] = "failed"

results["errors"].append(

f"Incomplete run: {n_successful}/{n_expected} successful, {n_failed} failed"

)

return results



# Load CSV

try:

df = load_csv_atomic(csv_path)

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Cannot load CSV: {e}")

return results



# Validate row count

expected_rows = EXPECTED_N_ROWS

actual_rows = len(df)
```

```
if actual_rows != expected_rows:
results["status"] = "failed"
results["errors"].append(
f"Row count mismatch: expected {expected_rows}, got {actual_rows}"
)


# Validate unique seeds
unique_seeds = df["seed"].unique()
if len(unique_seeds) != EXPECTED_N_SEEDS:
results["status"] = "failed"
results["errors"].append(
f"Seed count mismatch: expected {EXPECTED_N_SEEDS}, got {len(unique_seeds)}"
)


# Validate rho values
unique_rho = sorted([float(r) for r in df["rho"].unique()])
expected_rho = sorted([float(r) for r in EXPECTED_RHO_VALUES])
```

```
if unique_rho != expected_rho:
results["status"] = "failed"
results["errors"].append(
f"Rho values mismatch: expected {expected_rho}, got {unique_rho}"
)


# Validate branches
unique_branches = sorted(df["branch"].unique())
expected_branches = sorted(EXPECTED_BRANCHES)
if unique_branches != expected_branches:
results["status"] = "failed"
results["errors"].append(
f"Branches mismatch: expected {expected_branches}, got {unique_branches}"
)


# Validate completeness: each seed/rho/branch combination should exist
missing_combinations = []
```

```
for seed in unique_seeds:

seed_data = df[df["seed"] == seed]

for rho in [float(r) for r in df["rho"].unique()]:

rho_data = seed_data[seed_data["rho"] == rho]

for branch in unique_branches:

if not ((rho_data["branch"] == branch).any()):

missing_combinations.append(f"seed={seed}, rho={rho}, branch={branch}")



if missing_combinations:

results["status"] = "failed"

results["errors"].append(

f"Missing {len(missing_combinations)} combinations: {missing_combinations[:5]}"

)



# Check for duplicates

duplicate_rows = df.duplicated(subset=["seed", "rho", "branch"])

if duplicate_rows.any():
```

```
results["status"] = "failed"

results["errors"].append(

f"Duplicate rows found: {duplicate_rows.sum()} duplicates"

)



# Validate primary outcomes are present and finite

for outcome in PRIMARY_OUTCOMES:

if outcome not in df.columns:

results["status"] = "failed"

results["errors"].append(f"Primary outcome '{outcome}' missing from CSV")

else:

# Check that values are numeric

non_numeric = df[outcome].apply(lambda x: not isinstance(x, (int, float, np.number)))

if non_numeric.any():

results["status"] = "failed"

results["errors"].append(

f"Primary outcome '{outcome}' has {non_numeric.sum()} non-numeric values"
```

```
)

# Check that values are finite (not NaN, inf, or -inf)
non_finite = ~np.isfinite(df[outcome])
if non_finite.any():
results["status"] = "failed"
results["errors"].append(
f"Primary outcome '{outcome}' has {non_finite.sum()} non-finite values"
)

# Get baseline equivalence certificate hash - MANDATORY for production
baseline_cert_path = Path(csv_path.parent) / "baseline_equivalence_certificate.json"
if not baseline_cert_path.exists():
results["status"] = "failed"
results["errors"].append("Baseline equivalence certificate not found - REQUIRED for production analysis")
return results
baseline_cert_hash = file_sha256(baseline_cert_path)
```

```
# Record validation info
results["validation"] = {
"n_rows": actual_rows,
"n_seeds": len(unique_seeds),
"n_rho": len(unique_rho),
"n_branches": len(unique_branches),
"n_duplicates": int(duplicate_rows.sum()),
"n_missing_combinations": len(missing_combinations),
"baseline_equivalence": baseline_eq.get("status"),
"freezeable": manifest.get("freezeable", False),
"baseline_certificate_hash": baseline_cert_hash,
}

if results["status"] == "passed":
print("✅ Stage 1: Validation PASSED")
```

```
print(f" {actual_rows} rows, {len(unique_seeds)} seeds, {len(unique_rho)} rho,
{len(unique_branches)} branches")

else:

print("❌ Stage 1: Validation FAILED")

for error in results["errors"]:

print(f" {error}")

for warning in results["warnings"]:

print(f" WARNING: {warning}")




return results

================================================================================
Stage 2: Regime Contrasts
================================================================================

def compute_regime_contrasts(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:
```

```
text

"""Compute regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}



Uses direct merge instead of pivot to avoid MultiIndex column issues.



For each seed and non-baseline rho, compute the difference between

the outcome at that rho and the outcome at rho=1 (baseline).

"""

# Filter to continue branch only

cont = df.loc[

df["branch"] == "continue",

["seed", "rho", *outcomes],

].copy()



# Get baseline (rho=1.0) values

baseline = (

cont.loc[cont["rho"] == 1.0]
```

```
.drop(columns="rho")

.rename(

columns={

outcome: f"{outcome}_baseline"

for outcome in outcomes

}

)

)



# Merge with all rho values

merged = cont.merge(

baseline,

on="seed",

how="inner",

validate="many_to_one",

)
```

```
# Filter to non-baseline rho

merged = merged.loc[merged["rho"] != 1.0]



# Compute contrasts

rows = []

for _, row in merged.iterrows():

for outcome in outcomes:

value = row[outcome]

reference = row[f"{outcome}_baseline"]



# Validate that both values are numeric

if not isinstance(value, (int, float, np.number)) or not isinstance(reference, (int, float, np.number)):

raise RuntimeError(f"Non-numeric value for {outcome} at seed={row['seed']}, rho={row['rho']}")



rows.append({

"seed": row["seed"],

"rho": row["rho"],

"outcome": outcome,
```

```
"Y_rho_continue": float(value),

"Y_baseline_continue": float(reference),

"paired_regime_contrast": float(value - reference),

})



if not rows:

raise RuntimeError("No regime contrasts computed - check input data")



# Validate expected count

expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)* len(outcomes)

if len(rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} regime contrasts, got {len(rows)}"

)



contrasts_df = pd.DataFrame(rows)
```

```
print(f"✅ Stage 2: Regime Contrasts - {len(rows)} contrasts computed")

print(f" Seeds: {contrasts_df['seed'].nunique()}, Rho values: {contrasts_df['rho'].nunique()}, Outcomes: {contrasts_df['outcome'].nunique()}")




return contrasts_df

# ============================================================================
# Stage 3: Branch Difference-in-Differences
# ============================================================================

def compute_branch_did(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:

text

"""Compute branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]




For each seed, non-baseline rho, and non-continue branch, compute the

difference-in-differences relative to continue branch.
```

```
Skips rho=1.0 (baseline) as DID should be zero by construction.

"""

did_rows = []



for seed in df["seed"].unique():

seed_data = df[df["seed"] == seed]


for rho in sorted([float(r) for r in df["rho"].unique()]):

# Skip baseline rho=1.0

if rho == 1.0:

continue


rho_data = seed_data[seed_data["rho"] == rho]


# Get baseline (rho=1.0) data for this seed

baseline_data = seed_data[seed_data["rho"] == 1.0]
```

```
for branch in NON_CONTINUE_BRANCHES:

for outcome in outcomes:

# Get Y_{s,ρ,b}

y_rho_b_match = rho_data[rho_data["branch"] == branch][outcome]

if len(y_rho_b_match) == 0:

raise RuntimeError(f"Missing data: seed={seed}, rho={rho}, branch={branch}, outcome={outcome}")

y_rho_b = float(y_rho_b_match.iloc[0])


# Get Y_{s,ρ,continue}

y_rho_c_match = rho_data[rho_data["branch"] == "continue"][outcome]

if len(y_rho_c_match) == 0:

raise RuntimeError(f"Missing continue data: seed={seed}, rho={rho}, outcome={outcome}")

y_rho_c = float(y_rho_c_match.iloc[0])


# Get Y_{s,1,b}

y_base_b_match = baseline_data[baseline_data["branch"] == branch][outcome]

if len(y_base_b_match) == 0:

raise RuntimeError(f"Missing baseline data: seed={seed}, branch={branch}, outcome={outcome}")
```

```
y_base_b = float(y_base_b_match.iloc[0])


# Get Y_{s,1,continue}

y_base_c_match = baseline_data[baseline_data["branch"] == "continue"][outcome]

if len(y_base_c_match) == 0:

raise RuntimeError(f"Missing baseline continue data: seed={seed}, outcome={outcome}")

y_base_c = float(y_base_c_match.iloc[0])


# Compute DID

branch_effect_rho = y_rho_b - y_rho_c

branch_effect_baseline = y_base_b - y_base_c

did = branch_effect_rho - branch_effect_baseline


did_rows.append({

"seed": seed,

"rho": rho,

"branch": branch,

"outcome": outcome,

"Y_rho_branch": y_rho_b,
```

```
"Y_rho_continue": y_rho_c,

"branch_effect_rho": branch_effect_rho,

"Y_baseline_branch": y_base_b,

"Y_baseline_continue": y_base_c,

"branch_effect_baseline": branch_effect_baseline,

"difference_in_differences": did,

})



# Validate expected count

expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)*
len(NON_CONTINUE_BRANCHES) * len(outcomes)

if len(did_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} DID values, got {len(did_rows)}"

)



did_df = pd.DataFrame(did_rows)
```

print(f"✅ Stage 3: Branch DID - {len(did_rows)} DID values computed")

print(f" Seeds: {did_df['seed'].nunique()}, Rho: {did_df['rho'].nunique()}, Branches: {did_df['branch'].nunique()}")

return did_df

==
Stage 4: Cross-T Topology and Recovery
==

def build_continue_b_final_key(seed: str, rho: float) -> str:

text

"""Build canonical key for B_final matrix."""

return f"continue_seed_{seed}_rho_{rho}_B_final"

def compute_cross_t_topology(

text

npz_path: Path,

df: pd.DataFrame,

seeds: list[str],

rho_values: list[float],

b_c: float,

) -> pd.DataFrame:

text

"""Compute cross-T topology metrics.

For each seed and non-baseline rho, compute:

- S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- J_{s,ρ} = Jaccard(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- D_{s,ρ} = normalized_L2_distance(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Requires exactly EXPECTED_N_TOPOLOGY rows (40 seeds × 4 non-baseline rho = 160).

Any missing matrix is a schema failure.

"""

Build list of exact keys to load

keys_to_load = []

for seed in seeds:

```
for rho in rho_values:

keys_to_load.append(build_continue_b_final_key(seed, rho))



# Load only the required B_final matrices

npz_data = load_npz_selective(npz_path, keys=keys_to_load)



topology_rows = []

missing_keys = []



for seed in seeds:

for rho in rho_values:

if rho == 1.0:

continue # Skip baseline



# Get B_final for this (seed, rho, continue)

rho_key = build_continue_b_final_key(seed, rho)

baseline_key = build_continue_b_final_key(seed, 1.0)
```

```
if rho_key not in npz_data:
missing_keys.append(rho_key)
continue
if baseline_key not in npz_data:
missing_keys.append(baseline_key)
continue

B_rho = npz_data[rho_key]
B_baseline = npz_data[baseline_key]

# Compute metrics
cos_sim = cosine_similarity(B_rho, B_baseline)
jaccard = jaccard_similarity(B_rho, B_baseline, threshold=b_c)
norm_dist = normalized_distance(B_rho, B_baseline)

topology_rows.append({
"seed": seed,
"rho": rho,
"cross_T_cosine": cos_sim,
```

```
"cross_T_Jaccard": jaccard,

"cross_T_normalized_distance": norm_dist,

})



# Validate we have all expected comparisons

expected_count = EXPECTED_N_TOPOLOGY

if len(topology_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} topology comparisons, got {len(topology_rows)}. "

f"Missing keys: {missing_keys[:10]}"

)



topology_df = pd.DataFrame(topology_rows)



print(f"✅ Stage 4: Cross-T Topology - {len(topology_rows)} topology comparisons computed")

print(f" Seeds: {topology_df['seed'].nunique()}, Rho: {topology_df['rho'].nunique()}")
```

return topology_df

def compute_recovery_did(

text

recovery: pd.DataFrame,

n_bootstrap: int = 1000,

bootstrap_seed: int | None = None,

) -> pd.DataFrame:

text

"""Compute recovery interaction DID: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]

For each seed, non-baseline rho, and non-continue branch, compute the

difference-in-differences for recovery outcomes relative to continue branch.

Uses restricted times (censoring-aware) and normalized t/T_window as primary outcomes.

Skips rho=1.0 (baseline) as DID should be zero by construction.

"""

```
if recovery.empty:

return pd.DataFrame()



# Create seeded RNG for reproducibility

if bootstrap_seed is None:

raise ValueError("bootstrap_seed is required")

rng = np.random.default_rng(bootstrap_seed)



did_rows = []



seeds = sorted(recovery["seed"].unique())

rho_values = sorted([float(r) for r in recovery["rho"].unique()])

branches = sorted(recovery["branch"].unique())

horizon_systems = sorted(recovery["horizon_system"].unique())

metrics = sorted(recovery["metric"].unique())
```

```
# Recovery outcomes to compute DID for

# Prioritize censoring-aware restricted outcomes as primary

recovery_outcomes = [

"restricted_first_crossing_time",

"restricted_sustained_time",

"restricted_held_time",

"restricted_first_over_Twindow",

"restricted_sustained_over_Twindow",

"restricted_held_over_Twindow",

"auc_deficit_vs_continue",

"first_crossing_over_Twindow",

"sustained_time_over_Twindow",

"held_time_over_Twindow",

]



# Also compute DID for recovery indicators

indicator_outcomes = [
```

```
"first_crossing_observed",

"sustained_observed",

"held_observed",

]



for seed in seeds:

seed_data = recovery[recovery["seed"] == seed]


for rho in rho_values:

# Skip baseline rho=1.0

if rho == 1.0:

continue


rho_data = seed_data[seed_data["rho"] == rho]


# Get baseline (rho=1.0) data for this seed

baseline_data = seed_data[seed_data["rho"] == 1.0]


for branch in branches:
```

```
# INCLUDE continue branch for DID calculation

# For continue: [R_{s,ρ,c} - R_{s,ρ,c}] - [R_{s,1,c} - R_{s,1,c}] = 0

# This ensures consistency with recovery scaling which includes all branches

for horizon_system in horizon_systems:

for metric in metrics:

# Compute DID for each recovery outcome

for outcome in recovery_outcomes:

# Get values: R_{s,ρ,b}, R_{s,ρ,c}, R_{s,1,b}, R_{s,1,c}


# R_{s,ρ,b}

r_rho_b_match = rho_data[

(rho_data["branch"] == branch) &amp;amp;

(rho_data["horizon_system"] == horizon_system) &amp;amp;

(rho_data["metric"] == metric)

][outcome]


# R_{s,ρ,c} (continue branch at same rho)

r_rho_c_match = rho_data[
```

```
(rho_data["branch"] == "continue") &amp;amp;

(rho_data["horizon_system"] == horizon_system) &amp;amp;

(rho_data["metric"] == metric)

][outcome]


# R_{s,1,b} (baseline rho=1.0, same branch)

r_base_b_match = baseline_data[

(baseline_data["branch"] == branch) &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]


# R_{s,1,c} (baseline rho=1.0, continue)

r_base_c_match = baseline_data[

(baseline_data["branch"] == "continue") &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]
```

```
if len(r_rho_b_match) == 0 or len(r_rho_c_match) == 0 or len(r_base_b_match) == 0 or
len(r_base_c_match) == 0:

# Skip if any data is missing

continue


r_rho_b = float(r_rho_b_match.iloc[0])

r_rho_c = float(r_rho_c_match.iloc[0])

r_base_b = float(r_base_b_match.iloc[0])

r_base_c = float(r_base_c_match.iloc[0])


# Compute DID: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]

branch_effect_rho = r_rho_b - r_rho_c

branch_effect_baseline = r_base_b - r_base_c

did = branch_effect_rho - branch_effect_baseline


did_rows.append({

"seed": seed,

"rho": rho,

"branch": branch,
```

"horizon_system": horizon_system,

"metric": metric,

"outcome": outcome,

"R_rho_branch": r_rho_b,

"R_rho_continue": r_rho_c,

"branch_effect_rho": branch_effect_rho,

"R_baseline_branch": r_base_b,

"R_baseline_continue": r_base_c,

"branch_effect_baseline": branch_effect_baseline,

"recovery_difference_in_differences": did,

})

Compute DID for recovery indicators (binary outcomes)

for outcome in indicator_outcomes:

Get values

r_rho_b_match = rho_data[

(rho_data["branch"] == branch) &

(rho_data["horizon_system"] == horizon_system) &

(rho_data["metric"] == metric)

][outcome]

r_rho_c_match = rho_data[

(rho_data["branch"] == "continue") &

(rho_data["horizon_system"] == horizon_system) &

(rho_data["metric"] == metric)

][outcome]

r_base_b_match = baseline_data[

(baseline_data["branch"] == branch) &

(baseline_data["horizon_system"] == horizon_system) &

(baseline_data["metric"] == metric)

][outcome]

r_base_c_match = baseline_data[

(baseline_data["branch"] == "continue") &

(baseline_data["horizon_system"] == horizon_system) &

```
(baseline_data["metric"] == metric)

][outcome]


if len(r_rho_b_match) == 0 or len(r_rho_c_match) == 0 or len(r_base_b_match) == 0 or
len(r_base_c_match) == 0:

continue


r_rho_b = float(r_rho_b_match.iloc[0])

r_rho_c = float(r_rho_c_match.iloc[0])

r_base_b = float(r_base_b_match.iloc[0])

r_base_c = float(r_base_c_match.iloc[0])


# For binary outcomes, use difference in proportions

branch_effect_rho = r_rho_b - r_rho_c

branch_effect_baseline = r_base_b - r_base_c

did = branch_effect_rho - branch_effect_baseline


did_rows.append({

"seed": seed,
```

"rho": rho,

"branch": branch,

"horizon_system": horizon_system,

"metric": metric,

"outcome": outcome,

"R_rho_branch": r_rho_b,

"R_rho_continue": r_rho_c,

"branch_effect_rho": branch_effect_rho,

"R_baseline_branch": r_base_b,

"R_baseline_continue": r_base_c,

"branch_effect_baseline": branch_effect_baseline,

"recovery_difference_in_differences": did,

})

if not did_rows:

return pd.DataFrame()

```
did_df = pd.DataFrame(did_rows)



print(f"✅ Stage 4c: Recovery DID - {len(did_rows)} recovery DID values computed")

print(f" Seeds: {did_df['seed'].nunique()}, Rho: {did_df['rho'].nunique()}, "

f"Branches: {did_df['branch'].nunique()}, Systems: {did_df['horizon_system'].nunique()}")



return did_df

def compute_seed_level_summaries(

text

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

recovery: pd.DataFrame | None = None,

n_bootstrap: int = 1000,

bootstrap_seed: int | None = None,

) -> dict[str, pd.DataFrame]:

text
```

"""Compute seed-level summaries with bootstrap confidence intervals.

For each outcome and rho, compute:

- Mean contrast

- Std error

- 95% bootstrap CI

Uses seeded bootstrap for reproducibility.

"""

summaries = {}

Create seeded RNG for bootstrap

if bootstrap_seed is None:

raise ValueError("--bootstrap-seed is required for reproducible analysis")

rng = np.random.default_rng(bootstrap_seed)

Regime contrast summaries

```
if not regime_contrasts.empty:

# Use named aggregation to avoid MultiIndex columns

regime_summary = regime_contrasts.groupby(

["rho", "outcome"],

as_index=False,

).agg(

mean=("paired_regime_contrast", "mean"),

std=("paired_regime_contrast", "std"),

n=("paired_regime_contrast", "count"),

)


# Add bootstrap CI

for (rho, outcome), group in regime_contrasts.groupby(["rho", "outcome"]):

values = group["paired_regime_contrast"].dropna().values

if len(values) &gt; 0:

# Seeded bootstrap

indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)
```

```
ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = regime_summary[(regime_summary["rho"] == rho) &amp;amp; (regime_summary["outcome"] == outcome)].index

regime_summary.loc[idx, "ci_lower"] = ci_lower

regime_summary.loc[idx, "ci_upper"] = ci_upper


summaries["regime_contrasts"] = regime_summary

print(f"✅ Stage 5: Regime Contrast Summaries - {len(regime_summary)} groups")



# Branch DID summaries

if not branch_did.empty:

did_summary = branch_did.groupby(

["rho", "branch", "outcome"],

as_index=False,

).agg(

mean=("difference_in_differences", "mean"),
```

```
std=("difference_in_differences", "std"),

n=("difference_in_differences", "count"),

)


# Add bootstrap CI

for (rho, branch, outcome), group in branch_did.groupby(["rho", "branch", "outcome"]):

values = group["difference_in_differences"].dropna().values

if len(values) &gt; 0:

indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)


ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = did_summary[(did_summary["rho"] == rho) &amp;amp; (did_summary["branch"] == branch) &amp;amp;
(did_summary["outcome"] == outcome)].index

did_summary.loc[idx, "ci_lower"] = ci_lower

did_summary.loc[idx, "ci_upper"] = ci_upper


summaries["branch_did"] = did_summary
```

```
print(f"✅ Stage 5: Branch DID Summaries - {len(did_summary)} groups")



# Topology summaries

if not topology.empty:

# Flatten the aggregation to avoid MultiIndex columns

topology_summary = topology.groupby("rho", as_index=False).agg(

cosine_mean=("cross_T_cosine", "mean"),

cosine_std=("cross_T_cosine", "std"),

cosine_n=("cross_T_cosine", "count"),

jaccard_mean=("cross_T_Jaccard", "mean"),

jaccard_std=("cross_T_Jaccard", "std"),

jaccard_n=("cross_T_Jaccard", "count"),

distance_mean=("cross_T_normalized_distance", "mean"),

distance_std=("cross_T_normalized_distance", "std"),

distance_n=("cross_T_normalized_distance", "count"),

)
```

```
# Add bootstrap CI for topology metrics (seed-level bootstrap by rho)

for rho, group in topology.groupby("rho"):

# Bootstrap cosine similarity

cosine_values = group["cross_T_cosine"].dropna().values

if len(cosine_values) &gt; 0:

indices = rng.integers(0, len(cosine_values), size=(n_bootstrap, len(cosine_values)))

bootstrap_means = np.mean(cosine_values[indices], axis=1)

cosine_ci_lower = float(np.percentile(bootstrap_means, 2.5))

cosine_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "cosine_ci_lower"] = cosine_ci_lower

topology_summary.loc[idx, "cosine_ci_upper"] = cosine_ci_upper


# Bootstrap Jaccard similarity

jaccard_values = group["cross_T_Jaccard"].dropna().values

if len(jaccard_values) &gt; 0:

indices = rng.integers(0, len(jaccard_values), size=(n_bootstrap, len(jaccard_values)))
```

```
bootstrap_means = np.mean(jaccard_values[indices], axis=1)

jaccard_ci_lower = float(np.percentile(bootstrap_means, 2.5))

jaccard_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "jaccard_ci_lower"] = jaccard_ci_lower

topology_summary.loc[idx, "jaccard_ci_upper"] = jaccard_ci_upper


# Bootstrap normalized distance

distance_values = group["cross_T_normalized_distance"].dropna().values

if len(distance_values) &gt; 0:

indices = rng.integers(0, len(distance_values), size=(n_bootstrap, len(distance_values)))

bootstrap_means = np.mean(distance_values[indices], axis=1)

distance_ci_lower = float(np.percentile(bootstrap_means, 2.5))

distance_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "distance_ci_lower"] = distance_ci_lower

topology_summary.loc[idx, "distance_ci_upper"] = distance_ci_upper
```

```
summaries["topology"] = topology_summary

print(f"✅ Stage 5: Topology Summaries - {len(topology_summary)} rho values")



# Recovery scaling summaries with bootstrap CI

if recovery is not None and not recovery.empty:

# Group by metric, horizon_system, branch, rho and compute summaries

# Include both raw event times AND censoring-aware restricted times

recovery_summary = recovery.groupby(

["rho", "branch", "horizon_system", "metric"],

as_index=False,

).agg(

# Raw event times (conditional on observed recovery)

mean_first_crossing=("first_crossing_time", "mean"),

std_first_crossing=("first_crossing_time", "std"),

n_first_crossing=("first_crossing_time", "count"),

mean_sustained=("sustained_time", "mean"),
```

```
std_sustained=("sustained_time", "std"),

n_sustained=("sustained_time", "count"),

mean_held=("held_time", "mean"),

std_held=("held_time", "std"),

n_held=("held_time", "count"),

mean_auc_deficit=("auc_deficit_vs_continue", "mean"),

std_auc_deficit=("auc_deficit_vs_continue", "std"),

n_auc_deficit=("auc_deficit_vs_continue", "count"),


# Censoring-aware restricted times (cohort-level estimates)

mean_restricted_first=("restricted_first_crossing_time", "mean"),

std_restricted_first=("restricted_first_crossing_time", "std"),

n_restricted_first=("restricted_first_crossing_time", "count"),

mean_restricted_sustained=("restricted_sustained_time", "mean"),

std_restricted_sustained=("restricted_sustained_time", "std"),

n_restricted_sustained=("restricted_sustained_time", "count"),

mean_restricted_held=("restricted_held_time", "mean"),

std_restricted_held=("restricted_held_time", "std"),
```

```
n_restricted_held=("restricted_held_time", "count"),


# Normalized times (t/T_window) - raw observed (conditional on recovery)
mean_first_over_Twindow=("first_crossing_over_Twindow", "mean"),
std_first_over_Twindow=("first_crossing_over_Twindow", "std"),
n_first_over_Twindow=("first_crossing_over_Twindow", "count"),
mean_sustained_over_Twindow=("sustained_time_over_Twindow", "mean"),
std_sustained_over_Twindow=("sustained_time_over_Twindow", "std"),
n_sustained_over_Twindow=("sustained_time_over_Twindow", "count"),
mean_held_over_Twindow=("held_time_over_Twindow", "mean"),
std_held_over_Twindow=("held_time_over_Twindow", "std"),
n_held_over_Twindow=("held_time_over_Twindow", "count"),


# Censoring-aware restricted normalized times (cohort-level estimates)
mean_restricted_first_over_Twindow=("restricted_first_over_Twindow", "mean"),
std_restricted_first_over_Twindow=("restricted_first_over_Twindow", "std"),
n_restricted_first_over_Twindow=("restricted_first_over_Twindow", "count"),
mean_restricted_sustained_over_Twindow=("restricted_sustained_over_Twindow", "mean"),
```

```
std_restricted_sustained_over_Twindow=("restricted_sustained_over_Twindow", "std"),

n_restricted_sustained_over_Twindow=("restricted_sustained_over_Twindow", "count"),

mean_restricted_held_over_Twindow=("restricted_held_over_Twindow", "mean"),

std_restricted_held_over_Twindow=("restricted_held_over_Twindow", "std"),

n_restricted_held_over_Twindow=("restricted_held_over_Twindow", "count"),


# Recovery probabilities

observed_rate_first=("first_crossing_observed", "mean"),

observed_rate_sustained=("sustained_observed", "mean"),

observed_rate_held=("held_observed", "mean"),

)


# Add bootstrap CI for recovery metrics

for (rho, branch, horizon_system, metric), group in recovery.groupby(["rho", "branch",
"horizon_system", "metric"]):

# Bootstrap first crossing time (conditional on observed)

first_crossing_values = group["first_crossing_time"].dropna().values

if len(first_crossing_values) &gt; 0:

indices = rng.integers(0, len(first_crossing_values), size=(n_bootstrap, len(first_crossing_values)))
```

```
bootstrap_means = np.nanmean(first_crossing_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_first_crossing"] = ci_lower

recovery_summary.loc[idx, "ci_upper_first_crossing"] = ci_upper


# Bootstrap restricted first crossing time (cohort-level, includes censored)

restricted_first_values = group["restricted_first_crossing_time"].dropna().values

if len(restricted_first_values) &gt; 0:

indices = rng.integers(0, len(restricted_first_values), size=(n_bootstrap, len(restricted_first_values)))

bootstrap_means = np.nanmean(restricted_first_values[indices], axis=1)
```

```
ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_first"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_first"] = ci_upper


# Bootstrap restricted sustained time

restricted_sustained_values = group["restricted_sustained_time"].dropna().values

if len(restricted_sustained_values) &gt; 0:

indices = rng.integers(0, len(restricted_sustained_values), size=(n_bootstrap,
len(restricted_sustained_values)))

bootstrap_means = np.nanmean(restricted_sustained_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))
```

```
ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_sustained"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_sustained"] = ci_upper


# Bootstrap restricted held time

restricted_held_values = group["restricted_held_time"].dropna().values

if len(restricted_held_values) &amp;gt; 0:

indices = rng.integers(0, len(restricted_held_values), size=(n_bootstrap, len(restricted_held_values)))

bootstrap_means = np.nanmean(restricted_held_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))
```

```
idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_held"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_held"] = ci_upper



# Bootstrap normalized first crossing time over T_window

first_over_Twindow_values = group["first_crossing_over_Twindow"].dropna().values

if len(first_over_Twindow_values) &amp;gt; 0:

indices = rng.integers(0, len(first_over_Twindow_values), size=(n_bootstrap,
len(first_over_Twindow_values)))

bootstrap_means = np.nanmean(first_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))
```

```
idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_first_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_first_over_Twindow"] = ci_upper


# Bootstrap normalized sustained time over T_window

sustained_over_Twindow_values = group["sustained_time_over_Twindow"].dropna().values

if len(sustained_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(sustained_over_Twindow_values), size=(n_bootstrap,
len(sustained_over_Twindow_values)))

bootstrap_means = np.nanmean(sustained_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[
```

```
(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_sustained_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_sustained_over_Twindow"] = ci_upper


# Bootstrap normalized held time over T_window

held_over_Twindow_values = group["held_time_over_Twindow"].dropna().values

if len(held_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(held_over_Twindow_values), size=(n_bootstrap,
len(held_over_Twindow_values)))

bootstrap_means = np.nanmean(held_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;
```

```
(recovery_summary["branch"] == branch) &amp;amp;
(recovery_summary["horizon_system"] == horizon_system) &amp;amp;
(recovery_summary["metric"] == metric)
].index
recovery_summary.loc[idx, "ci_lower_held_over_Twindow"] = ci_lower
recovery_summary.loc[idx, "ci_upper_held_over_Twindow"] = ci_upper

# Bootstrap AUC deficit
auc_values = group["auc_deficit_vs_continue"].dropna().values
if len(auc_values) &gt; 0:
indices = rng.integers(0, len(auc_values), size=(n_bootstrap, len(auc_values)))
bootstrap_means = np.nanmean(auc_values[indices], axis=1)
ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))
ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))

idx = recovery_summary[
(recovery_summary["rho"] == rho) &amp;amp;
(recovery_summary["branch"] == branch) &amp;amp;
```

```
(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_auc_deficit"] = ci_lower

recovery_summary.loc[idx, "ci_upper_auc_deficit"] = ci_upper


# Bootstrap restricted normalized first crossing time over T_window

restricted_first_over_Twindow_values = group["restricted_first_over_Twindow"].dropna().values

if len(restricted_first_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(restricted_first_over_Twindow_values), size=(n_bootstrap,
len(restricted_first_over_Twindow_values)))

bootstrap_means = np.nanmean(restricted_first_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;
```

```
(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_first_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_first_over_Twindow"] = ci_upper



# Bootstrap restricted normalized sustained time over T_window

restricted_sustained_over_Twindow_values =
group["restricted_sustained_over_Twindow"].dropna().values

if len(restricted_sustained_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(restricted_sustained_over_Twindow_values), size=(n_bootstrap,
len(restricted_sustained_over_Twindow_values)))

bootstrap_means = np.nanmean(restricted_sustained_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))



idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)
```

```
].index

recovery_summary.loc[idx, "ci_lower_restricted_sustained_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_sustained_over_Twindow"] = ci_upper


# Bootstrap restricted normalized held time over T_window

restricted_held_over_Twindow_values = group["restricted_held_over_Twindow"].dropna().values

if len(restricted_held_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(restricted_held_over_Twindow_values), size=(n_bootstrap,
len(restricted_held_over_Twindow_values)))

bootstrap_means = np.nanmean(restricted_held_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)
```

```
].index

recovery_summary.loc[idx, "ci_lower_restricted_held_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_held_over_Twindow"] = ci_upper


# Bootstrap recovery probabilities (observed rates)

# First crossing observed rate

first_observed_values = group["first_crossing_observed"].dropna().values

if len(first_observed_values) &gt; 0:

indices = rng.integers(0, len(first_observed_values), size=(n_bootstrap, len(first_observed_values)))

bootstrap_means = np.nanmean(first_observed_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)
```

```
].index

recovery_summary.loc[idx, "ci_lower_observed_rate_first"] = ci_lower

recovery_summary.loc[idx, "ci_upper_observed_rate_first"] = ci_upper


# Sustained observed rate

sustained_observed_values = group["sustained_observed"].dropna().values

if len(sustained_observed_values) &gt; 0:

indices = rng.integers(0, len(sustained_observed_values), size=(n_bootstrap,
len(sustained_observed_values)))

bootstrap_means = np.nanmean(sustained_observed_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index
```

```
recovery_summary.loc[idx, "ci_lower_observed_rate_sustained"] = ci_lower

recovery_summary.loc[idx, "ci_upper_observed_rate_sustained"] = ci_upper


# Held observed rate

held_observed_values = group["held_observed"].dropna().values

if len(held_observed_values) &gt; 0:

indices = rng.integers(0, len(held_observed_values), size=(n_bootstrap, len(held_observed_values)))

bootstrap_means = np.nanmean(held_observed_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_observed_rate_held"] = ci_lower
```

```
recovery_summary.loc[idx, "ci_upper_observed_rate_held"] = ci_upper


summaries["recovery"] = recovery_summary

print(f"✅ Stage 5: Recovery Summaries - {len(recovery_summary)} groups")



return summaries

# ============================================================================
# Report Generation
# ============================================================================

def generate_text_report(

text

validation: dict[str, Any],

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

regime_summaries: pd.DataFrame,

did_summaries: pd.DataFrame,

topology_summaries: pd.DataFrame,
```

```
recovery_summaries: pd.DataFrame | None = None,

regime_summaries_secondary: pd.DataFrame | None = None,

did_summaries_secondary: pd.DataFrame | None = None,

recovery_did_summary: pd.DataFrame | None = None,

) -> str:

text

"""Generate human-readable text report."""

lines = []

lines.append("=" * 80)

lines.append("AIEF-1.4 T_window Sensitivity Study - Analysis Report")

lines.append("=" * 80)

lines.append("")



# Validation summary

lines.append("VALIDATION SUMMARY")

lines.append("-" * 40)

val = validation.get("validation", {})
```

```
lines.append(f" Status: {validation.get('status', 'unknown')}")

lines.append(f" Rows: {val.get('n_rows', 0)} (expected: {EXPECTED_N_ROWS})")

lines.append(f" Seeds: {val.get('n_seeds', 0)} (expected: {EXPECTED_N_SEEDS})")

lines.append(f" Rho values: {val.get('n_rho', 0)} (expected: {len(EXPECTED_RHO_VALUES)})")

lines.append(f" Branches: {val.get('n_branches', 0)} (expected: {len(EXPECTED_BRANCHES)})")

lines.append(f" Duplicates: {val.get('n_duplicates', 0)}")

lines.append(f" Missing combinations: {val.get('n_missing_combinations', 0)}")

lines.append(f" Baseline equivalence: {val.get('baseline_equivalence', 'unknown')}")

lines.append(f" Freezeable: {val.get('freezeable', False)}")

lines.append("")



# Regime contrasts (Primary - fixed_physical horizons)

lines.append("REGIME CONTRASTS (Δ = Y_{s,ρ,continue} - Y_{s,1,continue})")

lines.append("-" * 40)

lines.append("Primary outcomes: fixed_physical horizons")

lines.append("")

if not regime_summaries.empty:
```

```
for rho in sorted(regime_summaries["rho"].unique()):

rho_data = regime_summaries[regime_summaries["rho"] == rho]

for outcome in PRIMARY_OUTCOMES:

out_data = rho_data[rho_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No regime contrast data")

lines.append("")



# Secondary regime contrasts (normalized_EMA horizons)
```

```
lines.append("SECONDARY REGIME CONTRASTS (Normalized EMA horizons)")

lines.append("-" * 40)

if regime_summaries_secondary is not None and not regime_summaries_secondary.empty:

for rho in sorted(regime_summaries_secondary["rho"].unique()):

rho_data = regime_summaries_secondary[regime_summaries_secondary["rho"] == rho]

for outcome in SECONDARY_NORMALIZED_OUTCOMES:

out_data = rho_data[rho_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:
```

```
lines.append(" No secondary regime contrast data")

lines.append("")



# Branch DID (Primary - fixed_physical horizons)

lines.append("BRANCH DIFFERENCE-IN-DIFFERENCES")

lines.append("-" * 40)

lines.append("I_{s,ρ,b} = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]")

lines.append("Primary outcomes: fixed_physical horizons")

lines.append("")

if not did_summaries.empty:

for rho in sorted(did_summaries["rho"].unique()):

rho_data = did_summaries[did_summaries["rho"] == rho]

for branch in sorted(rho_data["branch"].unique()):

branch_data = rho_data[rho_data["branch"] == branch]

for outcome in PRIMARY_OUTCOMES:

out_data = branch_data[branch_data["outcome"] == outcome]

if not out_data.empty:
```

```
mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])



ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, branch={branch}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No branch DID data")

lines.append("")



# Secondary Branch DID (normalized_EMA horizons)

lines.append("SECONDARY BRANCH DIFFERENCE-IN-DIFFERENCES (Normalized EMA)")

lines.append("-" * 40)

if did_summaries_secondary is not None and not did_summaries_secondary.empty:

for rho in sorted(did_summaries_secondary["rho"].unique()):
```

```
rho_data = did_summaries_secondary[did_summaries_secondary["rho"] == rho]

for branch in sorted(rho_data["branch"].unique()):

branch_data = rho_data[rho_data["branch"] == branch]

for outcome in SECONDARY_NORMALIZED_OUTCOMES:

out_data = branch_data[branch_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])



ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, branch={branch}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str}
(n={n_val})")

else:

lines.append(" No secondary branch DID data")

lines.append("")
```

```
# Cross-T topology

lines.append("CROSS-T TOPOLOGY METRICS")

lines.append("-" * 40)

if not topology_summaries.empty:

for *, row in topology*summaries.iterrows():

rho = float(row["rho"])

lines.append(f" rho={rho:.2f}:")

lines.append(f" Cosine similarity: {float(row['cosine_mean']):.4f} ± {float(row['cosine_std']):.4f}
(n={int(row['cosine_n'])})")

if "cosine_ci_lower" in row and not np.isnan(float(row.get("cosine_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['cosine_ci_lower']):.4f}, {float(row['cosine_ci_upper']):.4f}]")

lines.append(f" Jaccard similarity: {float(row['jaccard_mean']):.4f} ± {float(row['jaccard_std']):.4f}
(n={int(row['jaccard_n'])})")

if "jaccard_ci_lower" in row and not np.isnan(float(row.get("jaccard_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['jaccard_ci_lower']):.4f}, {float(row['jaccard_ci_upper']):.4f}]")

lines.append(f" Normalized distance: {float(row['distance_mean']):.4f} ±
{float(row['distance_std']):.4f} (n={int(row['distance_n'])})")

if "distance_ci_lower" in row and not np.isnan(float(row.get("distance_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['distance_ci_lower']):.4f}, {float(row['distance_ci_upper']):.4f}]")
```

```
else:

lines.append(" No cross-T topology data")

lines.append("")



# Recovery scaling metrics

lines.append("RECOVERY SCALING METRICS")

lines.append("-" * 40)

lines.append("Cohort-level estimates (censoring-aware) and conditional-on-recovery estimates")

lines.append("")

if recovery_summaries is not None and not recovery_summaries.empty:

for *, row in recovery*summaries.iterrows():

rho = float(row["rho"])

branch = str(row["branch"])

horizon_system = str(row["horizon_system"])

metric = str(row["metric"])

lines.append(f" rho={rho:.2f}, branch={branch}, {horizon_system}, {metric}:")
```

```
# Cohort-level estimates (includes censored) - restricted times

lines.append(f" Cohort-level estimates (all seeds, censoring-aware):")

lines.append(f" Mean restricted first crossing: {float(row['mean_restricted_first']):.4f} ±
{float(row['std_restricted_first']):.4f} (n={int(row['n_restricted_first'])})")

if "ci_lower_restricted_first" in row and not np.isnan(float(row.get("ci_lower_restricted_first",
float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_first']):.4f},
{float(row['ci_upper_restricted_first']):.4f}]")

lines.append(f" Mean restricted sustained: {float(row['mean_restricted_sustained']):.4f} ±
{float(row['std_restricted_sustained']):.4f} (n={int(row['n_restricted_sustained'])})")

if "ci_lower_restricted_sustained" in row and not
np.isnan(float(row.get("ci_lower_restricted_sustained", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_sustained']):.4f},
{float(row['ci_upper_restricted_sustained']):.4f}]")

lines.append(f" Mean restricted held: {float(row['mean_restricted_held']):.4f} ±
{float(row['std_restricted_held']):.4f} (n={int(row['n_restricted_held'])})")

if "ci_lower_restricted_held" in row and not np.isnan(float(row.get("ci_lower_restricted_held",
float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_held']):.4f},
{float(row['ci_upper_restricted_held']):.4f}]")

lines.append(f" AUC deficit: {float(row['mean_auc_deficit']):.4f} ± {float(row['std_auc_deficit']):.4f}
(n={int(row['n_auc_deficit'])})")

if "ci_lower_auc_deficit" in row and not np.isnan(float(row.get("ci_lower_auc_deficit", float('nan')))):
```

```
lines.append(f" 95% CI: [{float(row['ci_lower_auc_deficit']):.4f},
{float(row['ci_upper_auc_deficit']):.4f}]")



# Cohort-level normalized times (restricted t/T_window)

lines.append(f" Cohort-level normalized times (restricted, all seeds):")

lines.append(f" Mean restricted first/t_window: {float(row.get('mean_restricted_first_over_Twindow',
float('nan'))):.4f} ± {float(row.get('std_restricted_first_over_Twindow', float('nan'))):.4f}
(n={int(row.get('n_restricted_first_over_Twindow', 0))})")

if "ci_lower_restricted_first_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_restricted_first_over_Twindow", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_first_over_Twindow']):.4f},
{float(row['ci_upper_restricted_first_over_Twindow']):.4f}]")

lines.append(f" Mean restricted sustained/t_window:
{float(row.get('mean_restricted_sustained_over_Twindow', float('nan'))):.4f} ±
{float(row.get('std_restricted_sustained_over_Twindow', float('nan'))):.4f}
(n={int(row.get('n_restricted_sustained_over_Twindow', 0))})")

if "ci_lower_restricted_sustained_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_restricted_sustained_over_Twindow", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_sustained_over_Twindow']):.4f},
{float(row['ci_upper_restricted_sustained_over_Twindow']):.4f}]")

lines.append(f" Mean restricted held/t_window: {float(row.get('mean_restricted_held_over_Twindow',
float('nan'))):.4f} ± {float(row.get('std_restricted_held_over_Twindow', float('nan'))):.4f}
(n={int(row.get('n_restricted_held_over_Twindow', 0))})")

if "ci_lower_restricted_held_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_restricted_held_over_Twindow", float('nan')))):
```

```
lines.append(f" 95% CI: [{float(row['ci_lower_restricted_held_over_Twindow']):.4f},
{float(row['ci_upper_restricted_held_over_Twindow']):.4f}]")



# Recovery probabilities with CI

lines.append(f" Recovery probabilities (with 95% bootstrap CI):")

lines.append(f" First crossing observed: {float(row['observed_rate_first']):.4f}")

if "ci_lower_observed_rate_first" in row and not
np.isnan(float(row.get("ci_lower_observed_rate_first", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_observed_rate_first']):.4f},
{float(row['ci_upper_observed_rate_first']):.4f}]")

lines.append(f" Sustained recovery observed: {float(row['observed_rate_sustained']):.4f}")

if "ci_lower_observed_rate_sustained" in row and not
np.isnan(float(row.get("ci_lower_observed_rate_sustained", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_observed_rate_sustained']):.4f},
{float(row['ci_upper_observed_rate_sustained']):.4f}]")

lines.append(f" Held recovery observed: {float(row['observed_rate_held']):.4f}")

if "ci_lower_observed_rate_held" in row and not
np.isnan(float(row.get("ci_lower_observed_rate_held", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_observed_rate_held']):.4f},
{float(row['ci_upper_observed_rate_held']):.4f}]")



# Conditional-on-recovery estimates (observed seeds only)

lines.append(f" Conditional-on-recovery estimates (observed seeds only):")
```

```
lines.append(f" First crossing: {float(row['mean_first_crossing']):.4f} ±
{float(row['std_first_crossing']):.4f} (n={int(row['n_first_crossing'])})")

if "ci_lower_first_crossing" in row and not np.isnan(float(row.get("ci_lower_first_crossing",
float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_first_crossing']):.4f},
{float(row['ci_upper_first_crossing']):.4f}]")

lines.append(f" Sustained: {float(row['mean_sustained']):.4f} ± {float(row['std_sustained']):.4f}
(n={int(row['n_sustained'])})")

if "ci_lower_sustained" in row and not np.isnan(float(row.get("ci_lower_sustained", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_sustained']):.4f},
{float(row['ci_upper_sustained']):.4f}]")

lines.append(f" Held: {float(row['mean_held']):.4f} ± {float(row['std_held']):.4f}
(n={int(row['n_held'])})")

if "ci_lower_held" in row and not np.isnan(float(row.get("ci_lower_held", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_held']):.4f},
{float(row['ci_upper_held']):.4f}]")



# Conditional-on-recovery normalized times (observed seeds only)

lines.append(f" Conditional-on-recovery normalized times (observed seeds only):")

lines.append(f" First crossing/t_window: {float(row['mean_first_over_Twindow']):.4f} ±
{float(row['std_first_over_Twindow']):.4f} (n={int(row['n_first_over_Twindow'])})")

if "ci_lower_first_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_first_over_Twindow", float('nan')))):
```

```
lines.append(f" 95% CI (conditional): [{float(row['ci_lower_first_over_Twindow']):.4f},
{float(row['ci_upper_first_over_Twindow']):.4f}]")

lines.append(f" Sustained/t_window: {float(row['mean_sustained_over_Twindow']):.4f} ±
{float(row['std_sustained_over_Twindow']):.4f} (n={int(row['n_sustained_over_Twindow'])})")

if "ci_lower_sustained_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_sustained_over_Twindow", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_sustained_over_Twindow']):.4f},
{float(row['ci_upper_sustained_over_Twindow']):.4f}]")

lines.append(f" Held/t_window: {float(row['mean_held_over_Twindow']):.4f} ±
{float(row['std_held_over_Twindow']):.4f} (n={int(row['n_held_over_Twindow'])})")

if "ci_lower_held_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_held_over_Twindow", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_held_over_Twindow']):.4f},
{float(row['ci_upper_held_over_Twindow']):.4f}]")

else:

lines.append(" No recovery scaling data")

lines.append("")



# Recovery interaction DID

lines.append("RECOVERY INTERACTION DID")

lines.append("-" * 40)
```

```
lines.append("I_{s,ρ,b}^recovery = [R_{s,ρ,b} - R_{s,ρ,continue}] - [R_{s,1,b} - R_{s,1,continue}]")

lines.append("")

if recovery_did_summary is not None and not recovery_did_summary.empty:

for *, row in recovery*did_summary.iterrows():

rho = float(row["rho"])

branch = str(row["branch"])

horizon_system = str(row["horizon_system"])

metric = str(row["metric"])

outcome = str(row["outcome"])

lines.append(f" rho={rho:.2f}, branch={branch}, {horizon_system}, {metric}, {outcome}:")

mean_val = float(row["mean"])

std_val = float(row["std"])

n_val = int(row["n"])

ci_lower = float(row.get("ci_lower", float('nan')))

ci_upper = float(row.get("ci_upper", float('nan')))


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" DID: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")
```

```
else:

lines.append(" No recovery DID data")

lines.append("")



lines.append("=" * 80)

lines.append("END OF REPORT")

lines.append("=" * 80)



return "\n".join(lines)

def generate_analysis_manifest(

text

csv_path: Path,

npz_path: Path,

manifest_path: Path,

validation: dict[str, Any],

output_files: dict[str, Path],

source_path: Path,
```

```
bootstrap_seed: int,

n_bootstrap: int,

command_line: str,

) -> dict[str, Any]:

text

"""Generate analysis manifest with complete provenance."""

# Compute hashes

csv_hash = file_sha256(csv_path)

npz_hash = file_sha256(npz_path)

manifest_hash = file_sha256(manifest_path)

source_hash = file_sha256(source_path)



output_hashes = {}

for name, path in output_files.items():

output_hashes[name] = file_sha256(path)



manifest = {
```

"analysis_version": ANALYSIS_VERSION,

"timestamp": int(time.time()),

"study_name": STUDY_NAME,

"source": {

"path": str(source_path),

"sha256": source_hash,

},

"command_line": command_line,

"bootstrap": {

"seed": bootstrap_seed,

"n_iterations": n_bootstrap,

},

"input_hashes": {

"scalar_csv": csv_hash,

"branch_npz": npz_hash,

"simulation_manifest": manifest_hash,

"condition_status": validation.get("condition_status_hash", "NOT_FOUND"),

"baseline_certificate": validation.get("baseline_certificate_hash", "NOT_FOUND"),

},

"validation": validation,

"output_hashes": output_hashes,

"environment": {

"python": sys.version,

"numpy": np.__version__,

"pandas": pd.__version__,

},

"notes": {

"primary_outcomes": list(PRIMARY_OUTCOMES),

"secondary_outcomes": list(SECONDARY_NORMALIZED_OUTCOMES),

"expected_n_seeds": EXPECTED_N_SEEDS,

"expected_rho_values": [float(r) for r in EXPECTED_RHO_VALUES],

"expected_branches": list(EXPECTED_BRANCHES),

"expected_n_regime_contrasts": EXPECTED_N_REGIME_CONTRASTS,

"expected_n_did": EXPECTED_N_DID,

"expected_n_topology": EXPECTED_N_TOPOLOGY,

"horizon_definitions": {

"fixed_physical": {

"immediate": "0-17.5 time units",

"recovery": "17.5-87.5 time units",

"late": "87.5-320 time units"

},

"normalized_EMA": {

"immediate": "0-1×T_window",

"recovery": "1-5×T_window",

"late": "5×T_window to end"

}

},

"interpretation_constraints": {

"primary_estimand": "fixed_physical horizons",

"secondary_estimand": "normalized_EMA horizons",

"recovery_metrics": "t_rec/T_window for both horizon systems"

}

},

}

return manifest

==
Main Execution
==

def parse_arguments() -> argparse.Namespace:

text

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)

```
parser.add_argument(
"run_dir",
type=str,
help="run directory containing study outputs",
)
parser.add_argument(
"--output",
type=str,
default=None,
help="output directory for analysis (default: &lt;run_dir&gt;/analysis)",
)
parser.add_argument(
"--n-bootstrap",
type=int,
default=1000,
help="number of bootstrap iterations (default: 1000)",
)
parser.add_argument(
```

```
"--bootstrap-seed",

type=int,

required=True,

help="random seed for bootstrap reproducibility (REQUIRED)",

)

parser.add_argument(

"--overwrite",

action="store_true",

default=False,

help="overwrite existing output files (default: fail if outputs exist)",

)

return parser.parse_args()

def validate_bootstrap_args(args):

text

"""Validate bootstrap arguments."""

if args.n_bootstrap &lt;= 0:

raise ValueError("--n-bootstrap must be positive")
```

```
if args.bootstrap_seed is None:

raise ValueError("--bootstrap-seed is required")

return True

def main() -> None:

text

args = parse_arguments()



# Validate bootstrap arguments

validate_bootstrap_args(args)



# Parse run directory

run_dir = Path([args.run](http://args.run)_dir)

csv_files = list(run_dir.glob(f"{STUDY_NAME}_*_scalar_summary.csv"))

npz_files = list(run_dir.glob(f"{STUDY_NAME}_*_branch_diagnostics.npz"))

manifest_files = list(run_dir.glob("manifest.json"))



if not csv_files:
```

```
print(f"No CSV file found in {run_dir}")

sys.exit(1)

if not npz_files:

print(f"No NPZ file found in {run_dir}")

sys.exit(1)

if not manifest_files:

print(f"No manifest found in {run_dir}")

sys.exit(1)



csv_path = csv_files[0]

npz_path = npz_files[0]

manifest_path = manifest_files[0]



# Set up output directory with stale-file protection

if args.output is None:

output_dir = run_dir / "analysis"

else:
```

```
output_dir = Path(args.output)



# Check for stale files in output directory

expected_outputs = [

"twindow_paired_regime_contrasts.csv",

"twindow_paired_regime_contrasts_fixed_physical.csv",

"twindow_paired_regime_contrasts_normalized_EMA.csv",

"twindow_branch_interactions.csv",

"twindow_branch_interactions_fixed_physical.csv",

"twindow_branch_interactions_normalized_EMA.csv",

"twindow_cross_condition_topology.csv",

"twindow_recovery_by_seed.csv",

"twindow_recovery_summary.csv",

"twindow_regime_summary.csv",

"twindow_branch_DID_summary.csv",

"twindow_topology_summary.csv",

"twindow_regime_secondary_summary.csv",
```

```
"twindow_branch_DID_secondary_summary.csv",

"twindow_sensitivity_report.txt",

"analysis_manifest.json",

]



stale_files = []

if output_dir.exists():

for expected_file in expected_outputs:

file_path = output_dir / expected_file

if file_path.exists():

stale_files.append(expected_file)



if stale_files and not args.overwrite:

print(f"❌ Output directory contains stale files: {stale_files[:5]}...")

print(f" Use --overwrite to force overwrite, or specify a different --output directory")

print(f" This prevents mixing outputs from different analysis versions.")

sys.exit(1)
```

```
# If --overwrite is passed, remove ALL expected files before starting

if stale_files and args.overwrite:

print(f" Removing {len(stale_files)} stale files from output directory...")

for expected_file in expected_outputs:

file_path = output_dir / expected_file

if file_path.exists():

try:

file_path.unlink()

except Exception as e:

print(f" WARNING: Could not remove stale file {expected_file}: {e}")



output_dir.mkdir(parents=True, exist_ok=True)



# Save command line for manifest

command_line = " ".join(sys.argv)



print(f"Analyzing {run_dir}")
```

```
print(f" CSV: {csv_[path.name](http://path.name)}")

print(f" NPZ: {npz_[path.name](http://path.name)}")

print(f" Manifest: {manifest_[path.name](http://path.name)}")

print(f" Output: {output_dir}")

print(f" Bootstrap: {args.n_bootstrap} iterations, seed={args.bootstrap_seed}")

print()



# Stage 1: Validation

print("Stage 1: Validating run...")

validation = validate_run(csv_path, npz_path, manifest_path)



if validation["status"] == "failed":

print("\n❌ VALIDATION FAILED - Cannot proceed with analysis")

for error in validation["errors"]:

print(f" {error}")

for warning in validation["warnings"]:

print(f" WARNING: {warning}")
```

```
sys.exit(1)



# Load CSV data

print("\nLoading scalar data...")

df = load_csv_atomic(csv_path)



# Get study parameters from manifest

with open(manifest_path, "r") as f:

manifest = json.load(f)



seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]

rho_values = [float(r) for r in manifest["study"]["rho_values"]]

branches = manifest["protocol"]["branches"]

b_c = float(manifest["model"]["B_c"])



print(f" Loaded {len(df)} rows")

print(f" Seeds: {len(seeds)}, Rho: {len(rho_values)}, Branches: {len(branches)}")
```

```
print(f" B_c: {b_c}")



# Stage 2: Regime Contrasts (Primary outcomes - fixed_physical)

print("\nStage 2: Computing regime contrasts...")

regime_contrasts = compute_regime_contrasts(df, PRIMARY_OUTCOMES)



# Add horizon_system column to primary regime contrasts

regime_contrasts["horizon_system"] = "fixed_physical"



# Stage 2b: Regime Contrasts (Secondary normalized-EMA outcomes)

print("\nStage 2b: Computing secondary regime contrasts...")

regime_contrasts_secondary = compute_regime_contrasts(df,
SECONDARY_NORMALIZED_OUTCOMES)



# Add horizon_system column to secondary regime contrasts

regime_contrasts_secondary["horizon_system"] = "normalized_EMA"
```

```
# Combine primary and secondary regime contrasts into long format

regime_contrasts_combined = pd.concat([regime_contrasts, regime_contrasts_secondary],
ignore_index=True)



# Stage 3: Branch DID (Primary outcomes - fixed_physical)

print("\nStage 3: Computing branch difference-in-differences...")

branch_did = compute_branch_did(df, PRIMARY_OUTCOMES)



# Add horizon_system column to primary branch DID

branch_did["horizon_system"] = "fixed_physical"



# Stage 3b: Branch DID (Secondary normalized-EMA outcomes)

print("\nStage 3b: Computing secondary branch difference-in-differences...")

branch_did_secondary = compute_branch_did(df, SECONDARY_NORMALIZED_OUTCOMES)



# Add horizon_system column to secondary branch DID

branch_did_secondary["horizon_system"] = "normalized_EMA"
```

```
# Combine primary and secondary branch DID into long format
branch_did_combined = pd.concat([branch_did, branch_did_secondary], ignore_index=True)


# Stage 4: Cross-T Topology
print("\nStage 4: Computing cross-T topology metrics...")
topology = compute_cross_t_topology(npz_path, df, seeds, rho_values, b_c)


# Stage 4b: Recovery Scaling (REQUIRED for production)
print("\nStage 4b: Computing recovery scaling metrics...")
recovery = compute_recovery_scaling(df, npz_path, b_c)


# Stage 4c: Recovery Interaction DID
print("\nStage 4c: Computing recovery interaction DID...")
recovery_did = compute_recovery_did(recovery,
n_bootstrap=args.n_bootstrap,
bootstrap_seed=args.bootstrap_seed)
```

```
# Stage 5: Seed-Level Summaries

print("\nStage 5: Computing seed-level summaries...")

summaries = compute_seed_level_summaries(

regime_contrasts, branch_did, topology, recovery,

n_bootstrap=args.n_bootstrap,

bootstrap_seed=args.bootstrap_seed,

)



# Stage 5b: Secondary normalized-EMA summaries

print("\nStage 5b: Computing secondary seed-level summaries...")

secondary_summaries = compute_seed_level_summaries(

regime_contrasts_secondary, branch_did_secondary, pd.DataFrame(), None,

n_bootstrap=args.n_bootstrap,

bootstrap_seed=args.bootstrap_seed,

)



# Stage 5c: Recovery DID summaries
```

```
print("\nStage 5c: Computing recovery DID summaries...")

recovery_did_summaries = {}

if not recovery_did.empty:

recovery_did_summary = recovery_did.groupby(

["rho", "branch", "horizon_system", "metric", "outcome"],

as_index=False,

).agg(

mean=("recovery_difference_in_differences", "mean"),

std=("recovery_difference_in_differences", "std"),

n=("recovery_difference_in_differences", "count"),

)


# Add bootstrap CI

rng_did = np.random.default_rng(args.bootstrap_seed)

for (rho, branch, horizon_system, metric, outcome), group in recovery_did.groupby(

["rho", "branch", "horizon_system", "metric", "outcome"]

):
```

```
values = group["recovery_difference_in_differences"].dropna().values

if len(values) &gt; 0:

indices = rng_did.integers(0, len(values), size=(args.n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)

ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = recovery_did_summary[

(recovery_did_summary["rho"] == rho) &amp;amp;

(recovery_did_summary["branch"] == branch) &amp;amp;

(recovery_did_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_did_summary["metric"] == metric) &amp;amp;

(recovery_did_summary["outcome"] == outcome)

].index

recovery_did_summary.loc[idx, "ci_lower"] = ci_lower

recovery_did_summary.loc[idx, "ci_upper"] = ci_upper


recovery_did_summaries["recovery_did"] = recovery_did_summary
```

```
print(f"✅ Stage 5c: Recovery DID Summaries - {len(recovery_did_summary)} groups")



regime_summaries = summaries.get("regime_contrasts", pd.DataFrame())

did_summaries = summaries.get("branch_did", pd.DataFrame())

topology_summaries = summaries.get("topology", pd.DataFrame())

recovery_summaries = summaries.get("recovery", pd.DataFrame())

regime_summaries_secondary = secondary_summaries.get("regime_contrasts", pd.DataFrame())

did_summaries_secondary = secondary_summaries.get("branch_did", pd.DataFrame())

recovery_did_summary = recovery_did_summaries.get("recovery_did", pd.DataFrame())



# Save outputs - construct output_files incrementally

print("\nSaving outputs...")

output_files: dict[str, Path] = {}



# Paired regime contrasts (combined primary + secondary)

regime_contrast_combined_path = output_dir / "twindow_paired_regime_contrasts.csv"

save_csv_atomic(regime_contrast_combined_path, regime_contrasts_combined)
```

```
output_files["regime_contrasts_combined"] = regime_contrast_combined_path

print(f" Saved: {regime_contrast_combined_[path.name](http://path.name)}")



# Paired regime contrasts (primary only - fixed_physical)

regime_contrast_path = output_dir / "twindow_paired_regime_contrasts_fixed_physical.csv"

save_csv_atomic(regime_contrast_path, regime_contrasts)

output_files["regime_contrasts"] = regime_contrast_path

print(f" Saved: {regime_contrast_[path.name](http://path.name)}")



# Paired regime contrasts (secondary only - normalized_EMA)

regime_contrast_secondary_path = output_dir /
"twindow_paired_regime_contrasts_normalized_EMA.csv"

save_csv_atomic(regime_contrast_secondary_path, regime_contrasts_secondary)

output_files["regime_contrasts_secondary"] = regime_contrast_secondary_path

print(f" Saved: {regime_contrast_secondary_[path.name](http://path.name)}")



# Branch DID (combined primary + secondary)

branch_did_combined_path = output_dir / "twindow_branch_interactions.csv"
```

```
save_csv_atomic(branch_did_combined_path, branch_did_combined)

output_files["branch_did_combined"] = branch_did_combined_path

print(f" Saved: {branch_did_combined_[path.name](http://path.name)}")



# Branch DID (primary only - fixed_physical)

branch_did_path = output_dir / "twindow_branch_interactions_fixed_physical.csv"

save_csv_atomic(branch_did_path, branch_did)

output_files["branch_did"] = branch_did_path

print(f" Saved: {branch_did_[path.name](http://path.name)}")



# Branch DID (secondary only - normalized_EMA)

branch_did_secondary_path = output_dir / "twindow_branch_interactions_normalized_EMA.csv"

save_csv_atomic(branch_did_secondary_path, branch_did_secondary)

output_files["branch_did_secondary"] = branch_did_secondary_path

print(f" Saved: {branch_did_secondary_[path.name](http://path.name)}")



# Cross-T topology
```

```
if not topology.empty:

topology_path = output_dir / "twindow_cross_condition_topology.csv"

save_csv_atomic(topology_path, topology)

output_files["topology"] = topology_path

print(f" Saved: {topology_[path.name](http://path.name)}")



# Recovery scaling raw output

if not recovery.empty:

recovery_raw_path = output_dir / "twindow_recovery_by_seed.csv"

save_csv_atomic(recovery_raw_path, recovery)

output_files["recovery_by_seed"] = recovery_raw_path

print(f" Saved: {recovery_raw_[path.name](http://path.name)}")



# Summaries

if not regime_summaries.empty:

regime_summary_path = output_dir / "twindow_regime_summary.csv"

save_csv_atomic(regime_summary_path, regime_summaries)
```

```
output_files["regime_summary"] = regime_summary_path

print(f" Saved: {regime_summary_[path.name](http://path.name)}")



if not did_summaries.empty:

did_summary_path = output_dir / "twindow_branch_DID_summary.csv"

save_csv_atomic(did_summary_path, did_summaries)

output_files["did_summary"] = did_summary_path

print(f" Saved: {did_summary_[path.name](http://path.name)}")



if not topology_summaries.empty:

topology_summary_path = output_dir / "twindow_topology_summary.csv"

save_csv_atomic(topology_summary_path, topology_summaries)

output_files["topology_summary"] = topology_summary_path

print(f" Saved: {topology_summary_[path.name](http://path.name)}")



# Recovery summaries (censoring-aware)

if not recovery_summaries.empty:
```

```
recovery_summary_path = output_dir / "twindow_recovery_summary.csv"

save_csv_atomic(recovery_summary_path, recovery_summaries)

output_files["recovery_summary"] = recovery_summary_path

print(f" Saved: {recovery_summary_[path.name](http://path.name)}")



# Recovery DID (raw)

if not recovery_did.empty:

recovery_did_path = output_dir / "twindow_recovery_interactions.csv"

save_csv_atomic(recovery_did_path, recovery_did)

output_files["recovery_did"] = recovery_did_path

print(f" Saved: {recovery_did_[path.name](http://path.name)}")



# Recovery DID summaries

if not recovery_did_summary.empty:

recovery_did_summary_path = output_dir / "twindow_recovery_interactions_summary.csv"

save_csv_atomic(recovery_did_summary_path, recovery_did_summary)

output_files["recovery_did_summary"] = recovery_did_summary_path
```

```
print(f" Saved: {recovery_did_summary_[path.name](http://path.name)}")


# Secondary summaries
if not regime_summaries_secondary.empty:
regime_summary_secondary_path = output_dir / "twindow_regime_secondary_summary.csv"
save_csv_atomic(regime_summary_secondary_path, regime_summaries_secondary)
output_files["regime_summary_secondary"] = regime_summary_secondary_path
print(f" Saved: {regime_summary_secondary_[path.name](http://path.name)}")


if not did_summaries_secondary.empty:
did_summary_secondary_path = output_dir / "twindow_branch_DID_secondary_summary.csv"
save_csv_atomic(did_summary_secondary_path, did_summaries_secondary)
output_files["did_summary_secondary"] = did_summary_secondary_path
print(f" Saved: {did_summary_secondary_[path.name](http://path.name)}")


# Text report
report = generate_text_report(
```

```
validation, regime_contrasts, branch_did, topology,
regime_summaries, did_summaries, topology_summaries,
recovery_summaries, regime_summaries_secondary, did_summaries_secondary,
recovery_did_summary
)
report_path = output_dir / "twindow_sensitivity_report.txt"
save_text_atomic(report_path, report)
output_files["report"] = report_path
print(f" Saved: {report_[path.name](http://path.name)}")


# Analysis manifest
analysis_manifest = generate_analysis_manifest(
csv_path, npz_path, manifest_path, validation, output_files,
source_path=Path(__file__),
bootstrap_seed=args.bootstrap_seed,
n_bootstrap=args.n_bootstrap,
command_line=command_line,
```

```
)

manifest_path_out = output_dir / "analysis_manifest.json"

save_json_atomic(manifest_path_out, analysis_manifest)

output_files["analysis_manifest"] = manifest_path_out

print(f" Saved: {manifest_path_[out.name](http://out.name)}")



# Print report to console

print("\n" + "=" * 80)

print("ANALYSIS REPORT")

print("=" * 80)

print(report)



print("\n" + "=" * 80)

print("ANALYSIS COMPLETE")

print("=" * 80)

if name == "main":

text
```

main()

Ignoring paste corruption, this analysis script has the correct high-level estimands and preserves the seed as the inferential unit. The primary fixed-physical and secondary normalized-EMA analyses are properly separated. Several concrete issues remain, including one that changes the recovery-DID estimand and expected row count.
Verdict

Regime contrasts: correct
Primary branch DID: correct
Cross-TT topology: correct in structure
Recovery raw table: mostly correct
Recovery DID: currently incorrect because it includes continue as an output branch
Execution readiness: NO-GO until the P0 items are fixed

This file will not be used during the first baseline-only run, but it should be corrected before the grid analysis.
P0 corrections
1. Recovery DID incorrectly includes the continue branch

The intended recovery interaction is:
Is,ρ,b=
Is,ρ,b=
[R_{s,\rho,b}-R_{s,\rho,c}]

[R_{s,1,b}-R_{s,1,c}],
]

where bb is a non-continue branch.

Current code loops over:

Python

branches = sorted(recovery["branch"].unique())

for branch in branches:

That includes continue, generating trivial zero-DID rows. It changes the expected count from:
40×4×3×2×2×13=24,960
40×4×3×2×2×13=24,960

to:
40×4×4×2×2×13=33,280.
40×4×4×2×2×13=33,280.
Fix

Use:

Python

for branch in NON_CONTINUE_BRANCHES:

Remove the comments claiming that continue should be included in the DID output. continue must remain in the raw recovery table because it supplies the reference values, but it should not be an output branch in recovery_did.

Add an exact count check:

Python

```
expected_count = (
EXPECTED_N_SEEDS
* (len(EXPECTED_RHO_VALUES) - 1)
* len(NON_CONTINUE_BRANCHES)
* 2 # horizon systems
* 2 # metrics: density, similarity
* 13 # recovery outcomes
)

if len(did_rows) != expected_count:
raise RuntimeError(
f"Expected {expected_count} recovery DID rows, "
f"got {len(did_rows)}"
)
```

The expected result is 24,960 rows.
2. Recovery DID silently skips missing cells

Current code does:

Python

```
if len(r_rho_b_match) == 0 or ...:
continue
```

Missing cells indicate an internal schema error because the raw recovery table has already been declared complete. Silently skipping them can yield an incomplete but apparently valid DID table.

Require exactly one match for each cell:

Python

```
matches = {
"rho_branch": r_rho_b_match,
"rho_continue": r_rho_c_match,
"baseline_branch": r_base_b_match,
"baseline_continue": r_base_c_match,
}

for label, match in matches.items():
if len(match) != 1:
raise RuntimeError(
f"Recovery DID requires exactly one {label} row; "
f"seed={seed}, rho={rho}, branch={branch}, "
f"horizon={horizon_system}, metric={metric}, "
f"outcome={outcome}, found={len(match)}"
)
```

This should be applied to both continuous outcomes and binary indicators.
3. Secondary normalized outcomes are not validated in Stage 1

validate_run() validates only:

Python

PRIMARY_OUTCOMES

A run can pass validation and proceed with NaN or missing normalized-EMA outcomes.

Use:

Python

```
for outcome in (
PRIMARY_OUTCOMES
+ SECONDARY_NORMALIZED_OUTCOMES
):
...
```

Require every value to be finite.

This is necessary because the script immediately computes secondary regime contrasts and secondary branch DID after validation.
4. Similarity targets are mislabeled as horizon-independent

Density targets are correctly horizon-independent because both duplicated density rows use:

Python

```
0.9 * D_B_final_continue
```

Similarity targets differ between:

text

```
Weighted_similarity_late_fixed_physical
Weighted_similarity_late_normalized_EMA
```

Yet similarity rows currently say:

Python

"target_horizon_dependence": "horizon_independent",

Change similarity rows to:

Python

"target_horizon_dependence": "horizon_dependent",

Keep density rows as:

Python

"horizon_independent"

Density rows are intentionally duplicated across horizon systems. They must remain separated by horizon_system and must never be pooled.
5. Recovery interaction outputs are missing from stale-file protection

The script writes:

text

twindow_recovery_interactions.csv
twindow_recovery_interactions_summary.csv

but they are absent from expected_outputs.

Add:

Python

"twindow_recovery_interactions.csv",
"twindow_recovery_interactions_summary.csv",

Otherwise stale recovery-interaction files may survive or avoid triggering the no-overwrite guard.
6. Require every recovery summary group to contain 40 seeds

Before aggregating recovery summaries, validate:

Python

```
recovery_group_sizes = recovery.groupby(
["rho", "branch", "horizon_system", "metric"]
).size()

bad_groups = recovery_group_sizes[
recovery_group_sizes != EXPECTED_N_SEEDS
]

if not bad_groups.empty:
raise RuntimeError(
"Recovery groups do not all contain 40 seed-level rows: "
f"{bad_groups.head().to_dict()}"
)
```

For censoring-aware fields, every group should have 40 finite values:

Python

```
RESTRICTED_RECOVERY_FIELDS = (
"restricted_first_crossing_time",
"restricted_sustained_time",
"restricted_held_time",
"restricted_first_over_Twindow",
"restricted_sustained_over_Twindow",
"restricted_held_over_Twindow",
"auc_deficit_vs_continue",
"first_crossing_observed",
"sustained_observed",
```

"held_observed",
)

for field in RESTRICTED_RECOVERY_FIELDS:
non_finite = ~np.isfinite(
pd.to_numeric(recovery[field], errors="coerce")
)
if non_finite.any():
raise RuntimeError(
f"Recovery field {field} has "
f"{int(non_finite.sum())} non-finite values"
)

Raw event times may legitimately be NaN when recovery is unobserved; restricted times and indicators may not.

Apply analogous group-size validation to recovery_did:

Python

did_group_sizes = recovery_did.groupby(
["rho", "branch", "horizon_system", "metric", "outcome"]
).size()

if not (did_group_sizes == EXPECTED_N_SEEDS).all():
raise RuntimeError(...)

The summary n for conditional raw outcomes may be below 40 because pandas count excludes NaNs. The physical number of seed rows must still be 40.
Input validation and provenance
7. The baseline certificate hash is not exposed where the analysis manifest expects it

validate_run() calculates:

Python

baseline_cert_hash = file_sha256(baseline_cert_path)

but only stores it inside:

Python

results["validation"]["baseline_certificate_hash"]

Later, generate_analysis_manifest() asks for:

Python

validation.get("baseline_certificate_hash", "NOT_FOUND")

Therefore, the analysis manifest records "NOT_FOUND".

Add:

Python

results["baseline_certificate_hash"] = baseline_cert_hash

before returning.

The condition-status hash is already stored at the required top level.
8. Validate the baseline certificate contents, not only its existence

Load:

Python

baseline_equivalence_certificate.json

and require:

Python

certificate["status"] == "passed"
certificate["n_seeds"] == 40
len(certificate["seed_results"]) == 40
all(
result["status"] == "passed"
for result in certificate["seed_results"].values()
)

Also verify its hash against the simulation manifest’s recorded output hash, if present.

The current code trusts only the simulation manifest’s statement that baseline equivalence passed.
9. Verify simulation output hashes before analysis

The simulation manifest should contain hashes for:

text

scalar_csv
branch_npz
condition_status_csv
baseline_certificate
baseline_seed_results

Compare them to the actual files before analysis:

Python

recorded_hashes = manifest.get("output_hashes", {})

hash_checks = {
"scalar_csv": csv_path,
"branch_npz": npz_path,
"condition_status_csv": condition_status_path,
"baseline_certificate": baseline_cert_path,
}

for label, path in hash_checks.items():
expected = recorded_hashes.get(label)

observed = file_sha256(path)

if expected is None:
results["status"] = "failed"
results["errors"].append(
f"Manifest missing hash for {label}"
)
elif expected != observed:
results["status"] = "failed"
results["errors"].append(
f"Hash mismatch for {label}: "
f"expected={expected}, observed={observed}"
)

Adjust names to the corrected second-file manifest schema.
10. Require exactly one input CSV and NPZ

The script currently selects:

Python

csv_path = csv_files[0]
npz_path = npz_files[0]

If multiple matching outputs exist, file ordering may be arbitrary.

Require:

Python

if len(csv_files) != 1:
raise RuntimeError(
f"Expected exactly one scalar CSV, found {len(csv_files)}: "
f"{[p.name for p in csv_files]}"
)

if len(npz_files) != 1:
raise RuntimeError(

f"Expected exactly one branch NPZ, found {len(npz_files)}: "
f"{[p.name for p in npz_files]}"
)

Also ensure their embedded configuration hash matches the simulation manifest’s config_hash.
11. Validate exact condition-status coverage

The current condition-status validation checks 200 rows and zero "failed" rows, but does not reject:

duplicate (seed,\rho) rows;
missing conditions replaced by duplicates;
unknown status strings;
a different 40-seed cohort.

Require the exact set:

Python

```
condition_df["seed"] = condition_df["seed"].astype(str)
condition_df["rho"] = pd.to_numeric(
condition_df["rho"], errors="raise"
)

if condition_df.duplicated(["seed", "rho"]).any():
raise RuntimeError("Duplicate condition-status rows")

if not (condition_df["status"] == "success").all():
raise RuntimeError("Not all condition-status rows are success")
```

Compare against the manifest cohort and rho grid:

Python

```
expected_conditions = {
(str(seed), float(rho))
for seed in manifest["randomization"]["paired_seeds"]
for rho in manifest["study"]["rho_values"]
}
```

observed_conditions = set(
zip(condition_df["seed"], condition_df["rho"])
)

if observed_conditions != expected_conditions:
...

Do the same exact cohort comparison for the scalar CSV.
Recovery implementation details
12. Remove unused bootstrap arguments from compute_recovery_did()

The function accepts:

Python

n_bootstrap
bootstrap_seed

and constructs:

Python

rng = np.random.default_rng(bootstrap_seed)

but never uses the RNG or n_bootstrap.

Change the signature to:

Python

def compute_recovery_did(
recovery: pd.DataFrame,
) -> pd.DataFrame:

and call:

Python

recovery_did = compute_recovery_did(recovery)

Bootstrap intervals belong to Stage 5c, where they are already computed.
13. Use np.trapezoid

Replace:

Python

np.trapz(deficit, x=times)

with:

Python

if hasattr(np, "trapezoid"):
return float(np.trapezoid(deficit, x=times))
return float(np.trapz(deficit, x=times))

The fallback preserves compatibility with older NumPy versions.
14. Record the zero-imputation convention explicitly

For recovery/loss, NaN cosine values correspond to zero-vector weights. Treating them as below threshold and imputing zero for AUC is consistent with the earlier recovery workflow, but it must be recorded.

Add to similarity rows:

Python

"target_nan_samples_imputed_zero": int(
np.sum(~np.isfinite(similarity_trajectory))
),

"reference_nan_samples_imputed_zero": int(
np.sum(~np.isfinite(cont_similarity_trajectory))
),
"similarity_nan_convention": (
"undefined zero-vector cosine imputed as zero "
"for recovery and AUC only"
),

The current trapezoidal_auc_deficit() already effectively performs this imputation. Its docstring should state that this is a prespecified recovery-only convention, not an assertion that the underlying cosine is mathematically zero.
15. The held-duration definition is discrete-sample based

The implementation uses:

Python

required_samples = ceil(duration / sample_step)

This matches the earlier recovery analysis, so it should not be silently changed now. But its precise meaning is:

at least ceil(T_window / diagnostic_stride_time) consecutive saved samples above threshold

It is not a continuous-time proof of persistence over an exact interval. Record this in the row metadata and report.

Using the first difference:

Python

times[1] - times[0]

is safe for current uniform samples, but robust validation should verify all sample differences are equal. The earlier analysis used the median difference.
Summary validation
16. Validate all expected group counts

Before saving summaries, require:
Regime summaries

text

4 rho × 3 outcomes = 12 groups
n = 40 in every group

for both primary and secondary.
Branch DID summaries

text

4 rho × 3 branches × 3 outcomes = 36 groups
n = 40 in every group

for both primary and secondary.
Topology summaries

text

4 rho groups
cosine_n = jaccard_n = distance_n = 40

Recovery summaries

text

5 rho × 4 branches × 2 systems × 2 metrics = 80 groups

with 40 seed rows per group.
Recovery DID summaries

text

4 rho × 3 branches × 2 systems × 2 metrics × 13 outcomes

= 624 groups

Each physical group must contain 40 seed rows, although conditional raw-time n may be below 40 due to censoring.

These checks turn silent grouping errors into explicit failures.
Secondary and optional improvements
17. Conditional sustained/held CIs are absent

The text report checks for:

text

ci_lower_sustained
ci_upper_sustained
ci_lower_held
ci_upper_held

but those columns are never computed.

Either:

add bootstrap CIs for raw conditional sustained and held times; or
remove the unused report checks.

This is optional because restricted-time outcomes and recovery probabilities are the primary censoring-aware summaries.
18. Summary docstring says standard error but computes standard deviation

compute_seed_level_summaries() says:

text

Std error

but computes:

Python

std=("...", "std")

and reports mean ± standard deviation.

Change the docstring to “sample standard deviation,” or explicitly calculate:

Python

se = std / sqrt(n)

Do not label SD as SE.
19. CSV atomic writer does not fsync the temporary file

save_csv_atomic() lets pandas write directly to a temporary path and then replaces it. The directory is fsynced, but the file itself is not.

For full parity with the text/JSON writers, write through a file handle:

Python

with open(temp_path, "w", encoding="utf-8", newline="") as handle:
df.to_csv(handle, index=False)
handle.flush()
os.fsync(handle.fileno())

This is operational hardening, not a scientific blocker.
What is correct

The core analysis architecture is sound:

seed-level paired regime contrasts;
seed-level branch difference-in-differences;
fixed-physical primary outcomes;
normalized-EMA secondary outcomes;
non-baseline ρρ levels compared with ρ=1ρ=1;

cross-TT cosine, threshold Jaccard, and normalized edge distance;
condition-specific continue references;
first crossing, sustained recovery, and held recovery;
physical restricted times;
restricted times divided by current TwindowTwindow;
recovery indicators;
full-future one-sided AUC deficit;
separate horizon-system labels;
seed-level bootstrap intervals;
stale-output protection;
atomic text/JSON outputs;
analysis source/input/output hashes;
explicit freezeability and baseline checks.

The density rows are duplicated across horizon systems by design. Their target is horizon-independent, and the code keeps them separated. Do not pool them.
Required pre-analysis corrections

Before running this on the full grid, fix at least:

restrict recovery DID to the three non-continue branches;
require exactly 24,960 recovery-DID rows;
fail rather than skip missing recovery-DID cells;
validate all secondary normalized outcomes;
mark similarity targets as horizon-dependent;
add both recovery-interaction files to stale protection;
require 40 seed rows in every recovery group;
expose and verify the baseline certificate hash;
require exactly one input CSV/NPZ and verify simulation hashes.

After the three source files are corrected, the safe order remains:

compile all three;
import/self-test the engine;
driver dry-run;
mandatory --baseline-only run;
inspect the complete baseline certificate;
only after exact passage, launch the five-level grid;
run this analysis with a fixed bootstrap seed and the prespecified bootstrap count.

Lets do another evaluation of the three files we have been working on one by one, here is the first:

#!/usr/bin/env python3

"""AIEF Experiment Engine v1 - Parameterized simulation kernel.

This is a reusable, validated simulation engine that reproduces the frozen

AIEF-1.4 production run exactly and supports parameterized studies.

FROZEN PRODUCTION VALUES (from AIEF_1_4_memory_ablation_updated_REAL.py):

N = 12

DT = 0.01

TAU = 0.5

OMEGA = [1.00, 1.02, 1.50, 2.00, 2.01, 1.01, 1.03, 1.51, 2.02, 2.00, 1.00, 1.04]

A = 1.0

Q = 1.5

DELTA = 0.05

ETA = 0.04

GAMMA = 0.015

MU = 0.03

T_WINDOW = 25.0 / mean(OMEGA) = 17.5

B_C = 0.12

B_INIT_SCALE = 0.0005

PROTOCOL (from frozen run):

T_SIM = 800.0

N_STEPS = int(round(T_SIM / DT)) = 80000

BURN_END = int(0.20 * N_STEPS) = 16000 steps = 160.0 time units

HIST_END = int(0.60 * N_STEPS) = 48000 steps = 480.0 time units

FUTURE_STEPS = N_STEPS - HIST_END = 32000 steps = 320.0 time units

DELAY_STEPS = int(round(TAU / DT)) = 50

WINDOW_STEPS = int(round(T_WINDOW / DT)) = 1750

RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS) = 8750

Architecture:

ModelConfig: frozen dataclass for all model parameters

ProtocolConfig: frozen dataclass for protocol parameters

DynamicState: state container (immutable where possible)

Core functions: initialize, advance, simulate, interventions

Validation: baseline equivalence test

NOTE: This is a candidate reference implementation. After passing 40/40 baseline

text

equivalence tests, it will be frozen as the reference implementation.

"""

from future import annotations

import copy

import hashlib

import json

import math

import os

import tempfile

import time

from dataclasses import dataclass, field

from pathlib import Path

from typing import Any

import numpy as np
==
====
Frozen Production Constants (DO NOT CHANGE)
==
====

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005

FROZEN_ZERO_TOL = 1e-12

FROZEN_PROJECTION_NORM_TOL = 1e-14

FROZEN_GAP_TOL = 1e-9
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA)) # 1.428333...

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.502917152858814

FROZEN_T_WINDOW = FROZEN_T_STAR # 17.502917152858814 (exact, NOT rounded)

FROZEN_ALPHA = float(np.exp(-FROZEN_DT / FROZEN_T_WINDOW))

FROZEN_CAPACITY = (FROZEN_A - FROZEN_DELTA) / FROZEN_Q
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_WINDOW / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
Fixed physical horizon boundaries (in time units, NOT T_window dependent)
These are the PRIMARY estimand horizons

FIXED_PHYSICAL_IMMEDIATE_END = 17.5

FIXED_PHYSICAL_RECOVERY_END = 87.5

FIXED_PHYSICAL_FUTURE_END = 320.0
Derived fixed physical horizon steps

FIXED_PHYSICAL_IMMEDIATE_STEPS = int(round(FIXED_PHYSICAL_IMMEDIATE_END / FROZEN_DT)) # 1750

FIXED_PHYSICAL_RECOVERY_STEPS = int(round(FIXED_PHYSICAL_RECOVERY_END / FROZEN_DT)) # 8750
Baseline horizon metadata for validation

BASELINE_HORIZON_METADATA = {

text

"window_steps": FROZEN_WINDOW_STEPS,

"recovery_end_ema": FROZEN_RECOVERY_END,

"fixed_immediate_end": FIXED_PHYSICAL_IMMEDIATE_STEPS,

"fixed_recovery_end": FIXED_PHYSICAL_RECOVERY_STEPS,

}
==========
Model Configuration
==========

@dataclass(frozen=True)

class ModelConfig:

text

"""Immutable model configuration.

All model-changing quantities must come from here. No hidden module globals.

"""

Core parameters

N: int

dt: float

tau: float

omega: tuple[float, ...] # Immutable tuple

Coupling parameters

a: float

q: float

delta: float

Dynamics parameters

eta: float

mu: float

gamma: float

EMA/threshold parameters

T_window: float

B_c: float

B_init_scale: float

scale_epsilon: float = 1e-12

Derived properties

@property

def mean_omega(self) -> float:

"""Mean of omega frequencies."""

return float(np.mean(self.omega))

@property

def T_star(self) -> float:

"""T_* = 25 / mean(omega)"""

return 25.0 / self.mean_omega

@property

def alpha(self) -> float:

"""EMA smoothing factor: exp(-dt / T_window)"""

return float(np.exp(-self.dt / self.T_window))

@property

def capacity(self) -> float:

"""Projection capacity: (a - delta) / q"""

return (self.a - self.delta) / self.q

@property

def delay_steps(self) -> int:

"""Delay steps: round(tau / dt)"""

return int(round(self.tau / self.dt))

def validate(self) -> None:

"""Validate model configuration with strict consistency checks.

Uses rtol=0, atol=0 for exact compatibility with frozen production.

All scalar parameters must be finite.

"""

Validate all scalar parameters are finite

if not np.isfinite(self.N):

raise ValueError("N must be finite")

if not np.isfinite(self.dt) or self.dt <= 0:

raise ValueError("dt must be finite and positive")

if not np.isfinite(self.tau) or self.tau <= 0:

raise ValueError("tau must be finite and positive")

if not np.isfinite(self.a) or self.a <= 0:

raise ValueError("a must be finite and positive")

if not np.isfinite(self.q) or self.q <= 0:

raise ValueError("q must be finite and positive")

if not np.isfinite(self.delta) or self.delta < 0:

raise ValueError("delta must be finite and nonnegative")

if not np.isfinite(self.eta) or self.eta < 0:

raise ValueError("eta must be finite and nonnegative")

if not np.isfinite(self.mu) or self.mu < 0:

raise ValueError("mu must be finite and nonnegative")

if not np.isfinite(self.gamma) or self.gamma < 0:

raise ValueError("gamma must be finite and nonnegative")

if not np.isfinite(self.T_window) or self.T_window <= 0:

raise ValueError("T_window must be finite and positive")

if not np.isfinite(self.B_c) or self.B_c < 0:

raise ValueError("B_c must be finite and nonnegative")

if not np.isfinite(self.B_init_scale) or self.B_init_scale < 0:

raise ValueError("B_init_scale must be finite and nonnegative")

if not np.isfinite(self.scale_epsilon) or self.scale_epsilon <= 0:

raise ValueError("scale_epsilon must be finite and positive")

```
# Validate N is positive integer

if self.N &lt;= 0:

raise ValueError("N must be positive")


# Validate omega

if len([self.omega](http://self.omega)) != self.N:

raise ValueError(f"omega must have {self.N} elements")

if not np.all(np.isfinite([self.omega](http://self.omega))):

raise ValueError("omega must contain only finite values")


# Validate derived values

if not (0 &lt; self.alpha &lt;= 1):

raise ValueError(f"alpha={self.alpha} must be in (0, 1]")

if self.capacity &lt; 0:

raise ValueError(f"capacity={self.capacity} must be nonnegative")


# Strict consistency: delay_steps * dt must equal tau (exact)

if not np.isclose(self.delay_steps * self.dt, self.tau, rtol=0.0, atol=0.0):

raise ValueError(
```

f"delay_steps *dt ({self.delay_steps}* {self.dt} = {self.delay_steps * self.dt}) "

f"must equal tau ({self.tau}) exactly"

)

@dataclass(frozen=True)

class ProtocolConfig:

text

"""Protocol configuration for experiment execution."""

Simulation time parameters (in time units, not steps)

T_sim: float

burn_end_time: float

split_time: float

future_duration: float

Diagnostics

diagnostic_stride: int

Branches to simulate

branches: tuple[str, ...]

def n_steps(self, model: ModelConfig) -> int:

"""Total steps."""

return int(round(self.T_sim / model.dt))

def burn_end(self, model: ModelConfig) -> int:

"""Burn-in end step."""

return int(round(self.burn_end_time / model.dt))

def hist_end(self, model: ModelConfig) -> int:

"""History end step (split point)."""

return int(round(self.split_time / model.dt))

def future_steps(self, model: ModelConfig) -> int:

"""Future simulation steps."""

return int(round(self.future_duration / model.dt))

def validate(self, model: ModelConfig) -> None:

"""Validate protocol against model with strict consistency checks.

Uses rtol=0, atol=0 for exact compatibility with frozen production.

"""

P1 #10-11: Validation hardening - integer validation

if not isinstance(self.diagnostic_stride, int):

raise ValueError(f"diagnostic_stride must be an integer, got {type(self.diagnostic_stride).__name__}")

if self.T_sim <= 0:

raise ValueError("T_sim must be positive")

if not (0 < self.burn_end_time < self.split_time < self.T_sim):

raise ValueError("Invalid time parameters: must have 0 < burn_end < split < T_sim")

if self.future_duration <= 0:

raise ValueError("future_duration must be positive")

if self.diagnostic_stride <= 0:

```
raise ValueError("diagnostic_stride must be positive")


# Strict consistency: split_time + future_duration must equal T_sim

if not np.isclose(self.split_time + self.future_duration, self.T_sim, rtol=0.0, atol=0.0):

raise ValueError(

f"split_time ({self.split_time}) + future_duration ({self.future_duration}) "

f"= {self.split_time + self.future_duration} must equal T_sim ({self.T_sim}) exactly"

)


# Validate that all time parameters are compatible with dt (exact divisibility)

if not np.isclose(self.T_sim / model.dt, round(self.T_sim / model.dt), rtol=0.0, atol=0.0):

raise ValueError(f"T_sim ({self.T_sim}) must be divisible by dt ({model.dt}) exactly")

if not np.isclose(self.burn_end_time / model.dt, round(self.burn_end_time / model.dt), rtol=0.0,
atol=0.0):

raise ValueError(f"burn_end_time ({self.burn_end_time}) must be divisible by dt ({model.dt})
exactly")

if not np.isclose(self.split_time / model.dt, round(self.split_time / model.dt), rtol=0.0, atol=0.0):

raise ValueError(f"split_time ({self.split_time}) must be divisible by dt ({model.dt}) exactly")

if not np.isclose(self.future_duration / model.dt, round(self.future_duration / model.dt), rtol=0.0,
atol=0.0):
```

raise ValueError(f"future_duration ({self.future_duration}) must be divisible by dt ({model.dt}) exactly")

Default frozen production configuration
NOTE: T_window must be the EXACT value 25.0/mean(OMEGA), not rounded

DEFAULT_MODEL_CONFIG = ModelConfig(

text

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=FROZEN_T_WINDOW, # 17.502917152858814 (exact)

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=FROZEN_ZERO_TOL,

)
Default protocol matching frozen run

DEFAULT_PROTOCOL_CONFIG = ProtocolConfig(

text

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=("continue", "reset_ema", "reset_B", "reset_B_ema", "B_carrier_only", "full_reset", "future_shuffle"),

)
==========
Cached Utility Values
==========
Cache for compute_edges to avoid recomputing on every step

EDGESCACHE: dict[int, tuple[np.ndarray, np.ndarray]] = {}
Cache for omega array conversion

OMEGAARRAY_CACHE: dict[tuple[float, ...], np.ndarray] = {}
==========

Utility Functions
==========

def compute_edges(N: int) -> tuple[np.ndarray, np.ndarray]:

text

"""Get upper triangle indices for edges.

Cached to avoid recomputing on every step.

"""

if N not in *EDGES*CACHE:

*EDGES*CACHE[N] = np.triu_indices(N, k=1)

return *EDGES*CACHE[N]

def get_omega_array(omega_tuple: tuple[float, ...]) -> np.ndarray:

text

"""Get omega as numpy array with caching.

Cached to avoid recomputing on every step.

"""

```
if omega_tuple not in *OMEGA*ARRAY_CACHE:

*OMEGA*ARRAY_CACHE[omega_tuple] = np.asarray(omega_tuple, dtype=float)

return *OMEGA*ARRAY_CACHE[omega_tuple]

def validate_B_matrix(B: np.ndarray, N: int, eps: float = 1e-12) -> None:

text

"""Validate B matrix: symmetric, nonnegative, zero diagonal."""

if B.shape != (N, N):

raise ValueError(f"B shape {B.shape} != ({N}, {N})")

if not np.allclose(B, B.T, atol=eps):

raise ValueError("B is not symmetric")

if not np.allclose(np.diag(B), 0, atol=eps):

raise ValueError("B diagonal is not zero")

if np.any(B &lt; -eps):

raise ValueError("B has negative entries")

def file_sha256(path: str | os.PathLike[str]) -> str:

text

"""Compute SHA-256 hash of a file."""

digest = hashlib.sha256()
```

```
with open(path, "rb") as handle:
for block in iter(lambda: [handle.read](http://handle.read)(1024 * 1024), b""):
digest.update(block)
return digest.hexdigest()
def array_sha256(arr: np.ndarray) -> str:
text
"""Compute SHA-256 hash of a numpy array."""
digest = hashlib.sha256()
shape_bytes = np.asarray(arr.shape, dtype=np.int64).tobytes()
digest.update(shape_bytes)
digest.update(arr.dtype.str.encode())
contiguous = np.ascontiguousarray(arr)
if contiguous.size &gt; 0:
digest.update(contiguous.tobytes())
return digest.hexdigest()
def atomic_write_text(path: Path, text: str) -> None:
text
```

"""Write text atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{path.name}.", suffix=".tmp")

try:

with os.fdopen(fd, "w", encoding="utf-8") as handle:

handle.write(text)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))

finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

def atomic_write_json(path: Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically with fsync."""

text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"

atomic_write_text(path, text)

```
def atomic_write_npz(path: Path, **kwargs: Any) -> None:

text

"""Write NPZ atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{[path.name](http://path.name)}.",
suffix=".tmp.npz")

try:

with os.fdopen(fd, "wb") as handle:

np.savez(handle, **kwargs)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))

finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

================================================================================
RNG Utilities (matching frozen production exactly)
================================================================================

def make_seed_sequence(seed: int, child_index: int) -> int:
```

text

"""Generate a child seed from SeedSequence(seed).spawn(...)[child_index].

This matches the frozen production RNG hierarchy exactly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

children = root.spawn(child_index + 1)

return int(children[child_index].generate_state(1, dtype=np.uint64)[0])

def make_prefix_rng(seed: int) -> np.random.Generator:

text

"""Create prefix RNG matching frozen production exactly.

Frozen production does:

root = np.random.SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = np.random.default_rng(init_sequence)

NOTE: default_rng(SeedSequence_object) is NOT the same as

default_rng(SeedSequence_object.generate_state(...))

The production uses the SeedSequence object directly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

return np.random.default_rng(init_sequence)

def make_branch_rngs(seed: int, branch_order: tuple[str, ...]) -> dict[str, np.random.Generator]:

text

"""Create branch RNGs matching frozen production algorithm.

Frozen: *, branch*root = SeedSequence(seed).spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))

branch_rng_seeds = {branch: int(sequence.generate_state(1, dtype=np.uint64)[0])

for branch, sequence in zip(branch_order, branch_sequences)}

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

*, branch*root = root.spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))

return {

branch: np.random.default_rng(int(sequence.generate_state(1, dtype=np.uint64)[0]))

for branch, sequence in zip(branch_order, branch_sequences)

}

==
====
Projection Accumulator (matching production)
==
====

@dataclass

class ProjectionTotals:

text

"""Accumulator for projection diagnostics - exact match to production."""

steps: int = 0

violations: int = 0

active: int = 0

norm_sum: float = 0.0

norm_max: float = 0.0

relative_state_sum: float = 0.0

relative_state_max: float = 0.0

relative_update_sum: float = 0.0

relative_update_max: float = 0.0

g_min: float = np.inf

g_min_pre: float = np.inf

def update(self, info: dict[str, Any]) -> None:

"""Update accumulator with step info."""

self.steps += 1

self.violations += int(info["pre_violation"])

self.g_min = min(self.g_min, float(info["g_min"]))

self.g_min_pre = min(self.g_min_pre, float(info["g_min_pre"]))

if info["projection_active"]:

self.active += 1

norm = float(info["projection_norm"])

rel_state = float(info["projection_relative_state"])

rel_update = float(info["projection_relative_update"])

self.norm_sum += norm

self.norm_max = max(self.norm_max, norm)

self.relative_state_sum += rel_state

self.relative_state_max = max(self.relative_state_max, rel_state)

self.relative_update_sum += rel_update

self.relative_update_max = max(self.relative_update_max, rel_update)

def metrics(self) -> dict[str, float]:

"""Compute final metrics from accumulated values."""

active_denominator = max(self.active, 1)

return {

"G_min_future": float(self.g_min),

"G_min_pre_future": float(self.g_min_pre),

"Pre_gap_violation_fraction": float(self.violations / self.steps),

"Projection_active_fraction": float(self.active / self.steps),

"Projection_active_given_violation": float(self.active / self.violations)

if self.violations > 0

else float('nan'),

"Projection_norm_mean_active": float(

self.norm_sum / active_denominator

),

"Projection_norm_max": float(self.norm_max),

"Projection_relative_state_mean_active": float(

self.relative_state_sum / active_denominator

),

"Projection_relative_state_max": float(self.relative_state_max),

"Projection_relative_update_mean_active": float(

self.relative_update_sum / active_denominator

),

"Projection_relative_update_max": float(self.relative_update_max),

}

```
================================================================================
Dynamic State
================================================================================

class DynamicState:

text

"""State container for AIEF-1.4 dynamics.



Matches the frozen production implementation exactly.

"""



def **init**(

self,

theta: np.ndarray,

B: np.ndarray,

coherence_acc: np.ndarray,

history: list[np.ndarray] | np.ndarray,

):

self.theta = np.asarray(theta, dtype=float).copy()
```

```
self.B = np.asarray(B, dtype=float).copy()

self.coherence_acc = np.asarray(coherence_acc, dtype=complex).copy()


# Handle history input - ensure it's a list of arrays

if isinstance(history, np.ndarray):

# If it's a 2D array, convert to list of 1D arrays

self.history = [np.asarray(h, dtype=float).copy() for h in history]

else:

self.history = [np.asarray(h, dtype=float).copy() for h in history]



def clone(self) -&gt; "DynamicState":

"""Create a deep copy of the state."""

return DynamicState(

theta=self.theta.copy(),

B=self.B.copy(),

coherence_acc=self.coherence_acc.copy(),

history=[h.copy() for h in self.history],
```

)

@property

def N(self) -> int:

return len(self.theta)

def validate(self, config: ModelConfig, step: int | None = None) -> None:

"""Validate state consistency.

Checks:

- State dimension matches config.N

- History length is exactly delay_steps + 1

- All arrays have correct shapes

- B matrix is symmetric, zero-diagonal, non-negative

- All history vectors have shape (N,)

- All arrays are finite

"""

Check state dimension matches config

```
if self.N != config.N:
raise RuntimeError(
f"State N ({self.N}) does not match config N ({config.N})"
)

# Check history length
expected_history_len = config.delay_steps + 1
if len(self.history) != expected_history_len:
raise RuntimeError(
f"History length {len(self.history)} != expected {expected_history_len} "
f"(delay_steps={config.delay_steps}) at step {step}"
)

# Check all history vectors have correct shape
for i, h in enumerate(self.history):
if h.shape != (self.N,):
raise RuntimeError(
f"History[{i}] shape {h.shape} != ({self.N},)"
```

```
)

# Check theta shape
if self.theta.shape != (self.N,):
raise RuntimeError(f"theta shape {self.theta.shape} != ({self.N},)")

# Check B shape
if self.B.shape != (self.N, self.N):
raise RuntimeError(f"B shape {self.B.shape} != ({self.N}, {self.N})")

# Check coherence_acc shape
if self.coherence_acc.shape != (self.N, self.N):
raise RuntimeError(f"coherence_acc shape {self.coherence_acc.shape} != ({self.N}, {self.N})")

# Check all arrays are finite
if not np.all(np.isfinite(self.theta)):
raise RuntimeError("theta contains non-finite values")
if not np.all(np.isfinite(self.B)):
raise RuntimeError("B contains non-finite values")
```

if not np.all(np.isfinite(self.coherence_acc)):

raise RuntimeError("coherence_acc contains non-finite values")

for i, h in enumerate(self.history):

if not np.all(np.isfinite(h)):

raise RuntimeError(f"History[{i}] contains non-finite values")

Check B symmetry

if not np.allclose(self.B, self.B.T, atol=1e-12):

raise RuntimeError("B matrix is not symmetric")

Check B diagonal

if not np.allclose(np.diag(self.B), 0, atol=1e-12):

raise RuntimeError("B matrix diagonal is not zero")

Check B non-negativity

if np.any(self.B < -1e-12):

raise RuntimeError("B matrix has negative entries")

==========

Simulation Core (exact reproduction of frozen production)
==========
====

def initialize_state(

text

seed: int,

config: ModelConfig,

) -> DynamicState:

text

"""Initialize state from seed using frozen algorithm exactly.

Matches production: simulate_prefix() initialization.

Uses: root = SeedSequence(seed); init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence)

"""

N = config.N

rng = make_prefix_rng(seed)

```
# Initialize theta: uniform in [0, 2*pi) - matching production

theta = rng.random(N) *2.0* np.pi



# Initialize B: small random symmetric matrix

B = config.B_init_scale * rng.random((N, N))

B = 0.5 * (B + B.T)

np.fill_diagonal(B, 0.0)



# Initialize coherence accumulator

coherence_acc = np.zeros((N, N), dtype=complex)



# Initialize history: DELAY_STEPS + 1 copies of theta

history = [theta.copy() for *in range(config.delay*steps + 1)]



return DynamicState(

theta=theta,

B=B,
```

```
coherence_acc=coherence_acc,

history=history,

)

def compute_R(

text

state: DynamicState,

config: ModelConfig,

) -> np.ndarray:

text

"""Compute coherence matrix R from current state.



Exact reproduction of production compute_R().

"""

N = state.N

delayed = state.history[0]



# Convert omega to numpy array for indexing (cached)
```

```
omega_arr = get_omega_array([config.omega](http://config.omega))



dphi = (

state.theta[:, None]

- delayed[None, :]

- ((omega_arr[:, None] + omega_arr[None, :]) / 2.0) * config.tau

)

z_inst = np.exp(1j * dphi)



# Update EMA

state.coherence_acc[:] = (

config.alpha *state.coherence_acc + (1.0 - config.alpha)* z_inst

)



# Compute magnitude and symmetrize

result = np.abs(state.coherence_acc)

result = 0.5 * (result + result.T)
```

np.fill_diagonal(result, 0.0)

return result

def advance_state(

text

state: DynamicState,

config: ModelConfig,

rng: np.random.Generator,

shuffle_driver: bool = False,

) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:

text

"""Advance state by one time step.

Exact reproduction of production advance_state().

Returns (R_physical, R_driver, info).

Production also returns projection diagnostics:

- projection_active: whether correction was applied

- projection_norm: norm of correction

- projection_relative_state: correction_norm / ||B_unprojected||

- projection_relative_update: correction_norm / ||B_unprojected - B_previous||

"""

N = state.N

edges = compute_edges(N)

Store previous B for projection

B_previous = state.B.copy()

Compute R matrices

R_physical = compute_R(state, config)

R_driver = R_physical

if shuffle_driver:

values = R_physical[edges].copy()

```
rng.shuffle(values)

R_driver = np.zeros_like(R_physical)

R_driver[edges] = values

R_driver[(edges[1], edges[0])] = values



# Update B

B_unprojected = state.B + config.dt * (

(config.eta + [config.mu](http://config.mu)) * R_driver

- [config.mu](http://config.mu)

- config.gamma * state.B

)

B_unprojected = np.maximum(B_unprojected, 0.0)

B_unprojected = 0.5 * (B_unprojected + B_unprojected.T)

np.fill_diagonal(B_unprojected, 0.0)



# Projection diagnostics (matching production exactly)

proposed_update_norm = float(np.linalg.norm(B_unprojected - B_previous))
```

```
# Check gap constraint

row_sum_pre = np.sum(B_unprojected, axis=1)

g_min_pre = float(np.min(config.a - config.q * row_sum_pre))

pre_violation = bool(g_min_pre &lt; [config.delta](http://config.delta))



# Project if needed

B_projected = B_unprojected

correction_norm = 0.0

if pre_violation:

scale = np.minimum(1.0, config.capacity / (row_sum_pre + config.scale_epsilon))

B_projected = B_unprojected * np.minimum.outer(scale, scale)

np.fill_diagonal(B_projected, 0.0)

correction_norm = float(np.linalg.norm(B_projected - B_unprojected))



projection_active = bool(correction_norm &gt; FROZEN_PROJECTION_NORM_TOL)

relative_state = correction_norm / max(
```

```
float(np.linalg.norm(B_unprojected)), FROZEN_ZERO_TOL

)

relative_update = correction_norm / max(proposed_update_norm, FROZEN_ZERO_TOL)



# Validate projection

state.B = B_projected

row_sum = np.sum(state.B, axis=1)

g_min = float(np.min(config.a - config.q * row_sum))

if g_min &lt; [config.delta](http://config.delta) - 1e-9:

raise RuntimeError(f"Gap projection failed: g_min={g_min}")



# Update theta

delayed = state.history[0]

omega_arr = get_omega_array([config.omega](http://config.omega))

coupling = np.sum(

state.B * np.sin(delayed[None, :] - state.theta[:, None]), axis=1

)
```

```
state.theta = np.mod(

state.theta + config.dt * (omega_arr + coupling),

2.0 * np.pi

)



# Update history

state.history.append(state.theta.copy())

if len(state.history) &gt; config.delay_steps + 1:

state.history.pop(0)



info = {

"pre_violation": pre_violation,

"projection_active": projection_active,

"projection_norm": correction_norm,

"projection_relative_state": relative_state,

"projection_relative_update": relative_update,

"g_min_pre": g_min_pre,
```

"g_min": g_min,

}

return R_physical, R_driver, info

def simulate_prefix(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

) -> tuple[DynamicState, dict[str, Any]]:

text

"""Simulate prefix up to split_time.

Exact reproduction of production simulate_prefix().

Returns (state, prefix_dict).

Production uses:

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence) # for initialization

Then in the loop, uses:

unused_rng = default_rng(0) # for advance_state when shuffle_driver=False

"""

N = config.N

edges = compute_edges(N)

Initialize state (uses init_sequence via make_prefix_rng)

state = initialize_state(seed, config)

Get RNG for prefix simulation loop

Production uses unused_rng = default_rng(0) for the advance_state calls

when shuffle_driver=False

unused_rng = np.random.default_rng(0)

```
# Simulate to split point

R_past_sum = np.zeros((N, N), dtype=float)

R_past_count = 0



burn_end = protocol.burn_end(config)

hist_end = protocol.hist_end(config)



for step in range(hist_end):

R_physical, *,* = advance_state(state, config, unused_rng, shuffle_driver=False)


# P1 #10-11: State validation at transition boundaries

if step == burn_end or step == hist_end - 1:

state.validate(config, step)


if burn_end &lt;= step &lt; hist_end:

R_past_sum += R_physical

R_past_count += 1
```

```
prefix = {

"B_at_split": state.B.copy(),

"R_physical_past": R_past_sum / R_past_count if R_past_count &gt; 0 else np.zeros((N, N)),

"theta_at_split": state.theta.copy(),

"coherence_acc_at_split": state.coherence_acc.copy(),

"history_at_split": np.stack(state.history),

"selected_at_split": (state.B &gt; config.B_c).astype(np.uint8),

}



return state, prefix

def apply_intervention(

text

state: DynamicState,

branch: str,

config: ModelConfig,

rng: np.random.Generator,

) -> None:
```

text

"""Apply intervention to state. Mutates in place.

Exact reproduction of production apply_intervention().

"""

Validate branch is known

if branch not in KNOWN_BRANCHES:

raise ValueError(f"Unknown branch '{branch}'. Known branches: {sorted(KNOWN_BRANCHES)}")

N = state.N

if branch in {"reset_ema", "reset_B_ema", "B_carrier_only", "full_reset"}:

Zero coherence accumulator

state.coherence_acc.fill(0.0)

if branch in {"reset_B", "reset_B_ema", "full_reset"}:

```
# Zero B

state.B.fill(0.0)



if branch in {"B_carrier_only", "full_reset"}:

# Randomize theta and history

new_theta = rng.random(N) *2.0* np.pi

state.theta = new_theta

state.history = [new_theta.copy() for *in range(config.delay*steps + 1)]



# Ensure symmetry

state.B = 0.5 * (state.B + state.B.T)

np.fill_diagonal(state.B, 0.0)

def simulate_future(

text

split_state: DynamicState,

prefix: dict[str, Any],

branch: str,
```

config: ModelConfig,

protocol: ProtocolConfig,

rng: np.random.Generator,

) -> dict[str, Any]:

text

"""Simulate future trajectory from split state.

Reproduces production simulate_future() with ALL required outputs.

CRITICAL: Now accumulates BOTH horizon systems:

Fixed Physical Horizons (time-based, NOT T_window dependent):

- immediate: 0 to 17.5 time units

- recovery: 17.5 to 87.5 time units

- late: 87.5 to 320 time units

EMA-Normalized Horizons (T_window dependent):

- immediate: 0 to 1*T_window

- recovery: 1*T_window to 5*T_window

- late: 5*T_window to end

At rho=1 (T_window=17.502917...), the fixed physical and normalized horizons

are approximately aligned but use exact step boundaries.

Production accumulates:

- P_full, P_immediate, P_recovery, P_late (incidence matrices)

- B_mean_full, B_mean_immediate, B_mean_recovery, B_mean_late

- R_physical_future, R_driver_future

- Projection diagnostics via ProjectionTotals

- Various scalar metrics

Uses horizon system from production for normalized:

- WINDOW_STEPS = round(T_window / dt)

- RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS)

For fixed physical:

- FIXED_IMMEDIATE_END = round(17.5 / dt)

- FIXED_RECOVERY_END = round(87.5 / dt)

"""

state = split_state.clone()

apply_intervention(state, branch, config, rng)

N = state.N

edges = compute_edges(N)

future_steps = protocol.future_steps(config)

Compute horizon step boundaries

Normalized EMA horizons (T_window dependent)

window_steps = int(round(config.T_window / config.dt))

recovery_end_ema = min(5 * window_steps, future_steps)

```
# Fixed physical horizons (NOT T_window dependent)

# These use the production's physical time boundaries: 17.5, 87.5, 320

fixed_immediate_end = int(round(17.5 / config.dt)) # 17.5 time units

fixed_recovery_end = int(round(87.5 / config.dt)) # 87.5 time units

# Late goes from 87.5 to end (320 time units)



# Clamp all boundaries to future_steps

recovery_end_ema = min(recovery_end_ema, future_steps)

fixed_immediate_end = min(fixed_immediate_end, future_steps)

fixed_recovery_end = min(fixed_recovery_end, future_steps)



# Initialize accumulators for BOTH horizon systems

# P matrices (incidence: B &gt; B_c as 0/1)

P_sums_ema = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),
```

```
"late": np.zeros((N, N), dtype=float),

}

P_sums_fixed = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}



# B matrices (mean B values)

B_sums_ema = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}

B_sums_fixed = {
```

```
"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}



# Counts for each horizon in both systems

counts_ema = {"full": 0, "immediate": 0, "recovery": 0, "late": 0}

counts_fixed = {"full": 0, "immediate": 0, "recovery": 0, "late": 0}



# R matrices

R_physical_sum = np.zeros((N, N), dtype=float)

R_driver_sum = np.zeros((N, N), dtype=float)



# Projection accumulator (matching production)

projection = ProjectionTotals()
```

```
# Trajectory accumulators

sample_times = []

density_trajectory = []

similarity_trajectory = []



# For retention/emergence calculations

r_sum = 0.0

r_count = 0

r_late_ema_sum = 0.0

r_late_ema_count = 0

r_late_fixed_sum = 0.0

r_late_fixed_count = 0



B_reference = prefix["B_at_split"]

shuffle_driver = (branch == "future_shuffle")



for step in range(future_steps):
```

```
R_physical, R_driver, info = advance_state(

state, config, rng, shuffle_driver=shuffle_driver

)


# Update projection accumulator

projection.update(info)


# P1 #10-11: State validation at transition boundaries

if step == 0 or step == window_steps or step == recovery_end_ema or step == fixed_immediate_end or step == fixed_recovery_end:

state.validate(config, step)


# Accumulate R matrices

R_physical_sum += R_physical

R_driver_sum += R_driver


# Determine which horizons this step belongs to for BOTH systems

selected = (state.B &gt; config.B_c).astype(float)


# === EMA-Normalized Horizons ===
```

Full: all steps

P_sums_ema["full"] += selected

B_sums_ema["full"] += state.B

counts_ema["full"] += 1

immediate: 0 to window_steps (exclusive)

if step < window_steps:

P_sums_ema["immediate"] += selected

B_sums_ema["immediate"] += state.B

counts_ema["immediate"] += 1

recovery: window_steps to recovery_end_ema (exclusive)

if window_steps <= step < recovery_end_ema:

P_sums_ema["recovery"] += selected

B_sums_ema["recovery"] += state.B

counts_ema["recovery"] += 1

late: recovery_end_ema to end

if recovery_end_ema <= step:

```
P_sums_ema["late"] += selected

B_sums_ema["late"] += state.B

counts_ema["late"] += 1


# === Fixed Physical Horizons ===

# Full: all steps (same as EMA)

P_sums_fixed["full"] += selected

B_sums_fixed["full"] += state.B

counts_fixed["full"] += 1


# immediate: 0 to fixed_immediate_end (exclusive)

if step &lt; fixed_immediate_end:

P_sums_fixed["immediate"] += selected

B_sums_fixed["immediate"] += state.B

counts_fixed["immediate"] += 1


# recovery: fixed_immediate_end to fixed_recovery_end (exclusive)

if fixed_immediate_end &lt;= step &lt; fixed_recovery_end:
```

```
P_sums_fixed["recovery"] += selected

B_sums_fixed["recovery"] += state.B

counts_fixed["recovery"] += 1


# late: fixed_recovery_end to end

if fixed_recovery_end &lt;= step:

P_sums_fixed["late"] += selected

B_sums_fixed["late"] += state.B

counts_fixed["late"] += 1


# Trajectory sampling - CRITICAL: r_mean_late is ONLY accumulated here

if step % protocol.diagnostic_stride == 0:

t = (step + 1) * config.dt

sample_times.append(t)


# Density: fraction of edges above B_c

density = float(np.count_nonzero(state.B[edges] &gt; config.B_c) / len(edges[0]))

density_trajectory.append(density)
```

```
# Similarity: cosine between current and reference B

B_vec = state.B[edges]

B_ref_vec = B_reference[edges]

norm_B = np.linalg.norm(B_vec)

norm_ref = np.linalg.norm(B_ref_vec)

if norm_B &gt; 0 and norm_ref &gt; 0:

similarity = float([np.dot](http://np.dot)(B_vec, B_ref_vec) / (norm_B * norm_ref))

else:

similarity = float('nan')

similarity_trajectory.append(similarity)


# Track overall r

r_value = float(np.abs(np.mean(np.exp(1j * state.theta))))

r_sum += r_value

r_count += 1


# Track late-phase r for BOTH horizon systems

if recovery_end_ema &lt;= step:
```

r_late_ema_sum += r_value

r_late_ema_count += 1

if fixed_recovery_end <= step:

r_late_fixed_sum += r_value

r_late_fixed_count += 1

Compute mean matrices for BOTH horizon systems

P_ema = {name: P_sums_ema[name] / counts_ema[name] if counts_ema[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

P_fixed = {name: P_sums_fixed[name] / counts_fixed[name] if counts_fixed[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

B_mean_ema = {name: B_sums_ema[name] / counts_ema[name] if counts_ema[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

B_mean_fixed = {name: B_sums_fixed[name] / counts_fixed[name] if counts_fixed[name] > 0 else np.zeros((N, N))

```
for name in ["full", "immediate", "recovery", "late"]}



# Compute R means

R_physical_mean = R_physical_sum / future_steps if future_steps &gt; 0 else np.zeros((N, N))

R_driver_mean = R_driver_sum / future_steps if future_steps &gt; 0 else np.zeros((N, N))



# Final state

B_final = state.B.copy()

final_weights = state.B[edges]

final_rows = np.sum(state.B, axis=1)



# Retention and emergence for BOTH horizon systems

split_selected = prefix["selected_at_split"][edges].astype(bool)



# EMA-normalized

p_late_edges_ema = P_ema["late"][edges]

retention_ema = (
```

```
float(np.mean(p_late_edges_ema[split_selected]))

if np.any(split_selected)

else float('nan')

)

emergence_ema = (

float(np.mean(p_late_edges_ema[~split_selected]))

if np.any(~split_selected)

else float('nan')

)



# Fixed physical

p_late_edges_fixed = P_fixed["late"][edges]

retention_fixed = (

float(np.mean(p_late_edges_fixed[split_selected]))

if np.any(split_selected)

else float('nan')

)
```

```
emergence_fixed = (
float(np.mean(p_late_edges_fixed[~split_selected]))
if np.any(~split_selected)
else float('nan')
)

# Minimum similarity from trajectory
similarity_array = np.array(similarity_trajectory, dtype=float)
finite_similarity = similarity_array[np.isfinite(similarity_array)]
minimum_similarity = (
float(np.min(finite_similarity)) if len(finite_similarity) &gt; 0
else float('nan')
)

# Build result with BOTH horizon systems
result = {
# === Matrix outputs for EMA-normalized horizons (backward compatible) ===
```

"B_final": B_final,

"B_mean_late": B_mean_ema["late"],

"P_full": P_ema["full"],

"P_immediate": P_ema["immediate"],

"P_recovery": P_ema["recovery"],

"P_late": P_ema["late"],

"R_physical_future": R_physical_mean,

"R_driver_future": R_driver_mean,

=== Matrix outputs for FIXED-PHYSICAL horizons (NEW) ===

"B_mean_late_fixed_physical": B_mean_fixed["late"],

"P_full_fixed_physical": P_fixed["full"],

"P_immediate_fixed_physical": P_fixed["immediate"],

"P_recovery_fixed_physical": P_fixed["recovery"],

"P_late_fixed_physical": P_fixed["late"],

=== Matrix outputs for NORMALIZED-EMA horizons (NEW - explicit) ===

"B_mean_late_normalized_EMA": B_mean_ema["late"],

"P_full_normalized_EMA": P_ema["full"],

"P_immediate_normalized_EMA": P_ema["immediate"],

"P_recovery_normalized_EMA": P_ema["recovery"],

"P_late_normalized_EMA": P_ema["late"],

Trajectory outputs

"sample_times": np.array(sample_times, dtype=float),

"density_trajectory": np.array(density_trajectory, dtype=float),

"similarity_trajectory": np.array(similarity_trajectory, dtype=float),

Scalar outcomes - EMA-normalized (backward compatible names)

"D_B_final": float(np.count_nonzero(final_weights > config.B_c) / len(edges[0])),

"P_mean_full": float(np.mean(P_ema["full"][edges])),

"P_mean_immediate": float(np.mean(P_ema["immediate"][edges])),

"P_mean_recovery": float(np.mean(P_ema["recovery"][edges])),

"P_mean_late": float(np.mean(P_ema["late"][edges])),

Scalar outcomes - FIXED-PHYSICAL (NEW - primary estimands)

"P_mean_full_fixed_physical": float(np.mean(P_fixed["full"][edges])),

"P_mean_immediate_fixed_physical": float(np.mean(P_fixed["immediate"][edges])),

"P_mean_recovery_fixed_physical": float(np.mean(P_fixed["recovery"][edges])),

"P_mean_late_fixed_physical": float(np.mean(P_fixed["late"][edges])),

Scalar outcomes - NORMALIZED-EMA (NEW - explicit)

"P_mean_full_normalized_EMA": float(np.mean(P_ema["full"][edges])),

"P_mean_immediate_normalized_EMA": float(np.mean(P_ema["immediate"][edges])),

"P_mean_recovery_normalized_EMA": float(np.mean(P_ema["recovery"][edges])),

"P_mean_late_normalized_EMA": float(np.mean(P_ema["late"][edges])),

Similarity metrics - BOTH systems

"Weighted_similarity_final": cosine_edges(B_reference, B_final, edges),

"Weighted_similarity_late": cosine_edges(B_reference, B_mean_ema["late"], edges),

"Weighted_similarity_late_fixed_physical": cosine_edges(B_reference, B_mean_fixed["late"], edges),

"Weighted_similarity_late_normalized_EMA": cosine_edges(B_reference, B_mean_ema["late"], edges),

"Jaccard_final": jaccard_edges(B_reference, B_final, edges, config.B_c),

"Jaccard_final_fixed_physical": jaccard_edges(B_reference, B_final, edges, config.B_c),

```
# Retention/emergence - BOTH systems

"Retention_selected_late": retention_ema,

"Emergence_unselected_late": emergence_ema,

"Retention_contrast_late": retention_ema - emergence_ema,

"Retention_selected_late_fixed_physical": retention_fixed,

"Emergence_unselected_late_fixed_physical": emergence_fixed,

"Retention_contrast_late_fixed_physical": retention_fixed - emergence_fixed,

"Retention_selected_late_normalized_EMA": retention_ema,

"Emergence_unselected_late_normalized_EMA": emergence_ema,

"Retention_contrast_late_normalized_EMA": retention_ema - emergence_ema,


# Correlations with past R - BOTH systems

"Corr_Rpast_Pfull": safe_corr(prefix["R_physical_past"][edges], P_ema["full"][edges]),

"Corr_Rpast_Plate": safe_corr(prefix["R_physical_past"][edges], P_ema["late"][edges]),

"Corr_Rpast_Pfull_fixed_physical": safe_corr(prefix["R_physical_past"][edges], P_fixed["full"]
[edges]),

"Corr_Rpast_Plate_fixed_physical": safe_corr(prefix["R_physical_past"][edges], P_fixed["late"]
[edges]),
```

"Corr_Rpast_Pfull_normalized_EMA": safe_corr(prefix["R_physical_past"][edges], P_ema["full"][edges]),

"Corr_Rpast_Plate_normalized_EMA": safe_corr(prefix["R_physical_past"][edges], P_ema["late"][edges]),

Weight statistics (same for both - based on final state)

"Total_edge_weight": float(np.sum(final_weights)),

"Mean_row_sum": float(np.mean(final_rows)),

"Max_row_sum": float(np.max(final_rows)),

"Nonzero_edge_fraction": float(np.mean(final_weights > config.scale_epsilon)),

"Effective_edges": effective_number(final_weights),

"Weight_gini": gini_coefficient(final_weights),

"Weight_entropy": normalized_entropy(final_weights),

Order parameter - BOTH systems

"r_mean_future": float(r_sum / r_count) if r_count > 0 else float('nan'),

"r_mean_late": float(r_late_ema_sum / r_late_ema_count) if r_late_ema_count > 0 else float('nan'),

"r_mean_late_fixed_physical": float(r_late_fixed_sum / r_late_fixed_count) if r_late_fixed_count > 0 else float('nan'),

"r_mean_late_normalized_EMA": float(r_late_ema_sum / r_late_ema_count) if r_late_ema_count > 0 else float('nan'),

```
# Minimum similarity

"Minimum_similarity": minimum_similarity,


# Projection diagnostics (from ProjectionTotals)

**projection.metrics(),


# Horizon metadata

"window_steps": window_steps,

"recovery_end_ema": recovery_end_ema,

"fixed_immediate_end": fixed_immediate_end,

"fixed_recovery_end": fixed_recovery_end,

}



return result

============================================================================
====
Metric Helpers
============================================================================
====
```

def cosine_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None) -> float:

text

"""Compute cosine similarity between reference and current B matrices over edges.

If edges is None, computes over all upper triangle edges.

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

x = np.asarray(reference[edges], dtype=float)

y = np.asarray(current[edges], dtype=float)

denominator = float(np.linalg.norm(x) * np.linalg.norm(y))

if denominator <= 1e-12:

return float('nan')

return float(np.dot(x, y) / denominator)

def jaccard_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None, threshold: float = FROZEN_ZERO_TOL) -> float:

text

"""Compute Jaccard similarity between reference and current B matrices over edges.

Uses B > threshold as the threshold for edge presence.

If edges is None, computes over all upper triangle edges.

Matches production: jaccard_edges(B_reference, state.B) uses B > B_C

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

x = reference[edges] > threshold

y = current[edges] > threshold

union = np.count_nonzero(x | y)

if union == 0:

```
return float('nan')

return float(np.count_nonzero(x &amp;amp; y) / union)

def safe_corr(x: np.ndarray, y: np.ndarray) -> float:

text

"""Compute correlation coefficient safely."""

x = np.asarray(x, dtype=float)

y = np.asarray(y, dtype=float)

valid = np.isfinite(x) &amp;amp; np.isfinite(y)

x = x[valid]

y = y[valid]

if len(x) &amp;lt; 3 or np.std(x) &amp;lt; 1e-12 or np.std(y) &amp;lt; 1e-12:

return float('nan')

return float(np.corrcoef(x, y)[0, 1])

def effective_number(weights: np.ndarray) -> float:

text

"""Compute effective number of edges."""

weights = np.asarray(weights, dtype=float)
```

```
total = float(np.sum(weights))

sum_sq = float(np.sum(weights**2))

if total &lt;= 1e-12 or sum_sq &lt;= 1e-12:

return 0.0

return total**2 / sum_sq

def gini_coefficient(weights: np.ndarray) -> float:

text

"""Compute Gini coefficient."""

x = np.sort(np.asarray(weights, dtype=float))

total = float(np.sum(x))

if total &lt;= 1e-12:

return 0.0

n = len(x)

ranks = np.arange(1, n + 1, dtype=float)

return float(2.0 *[np.dot](http://np.dot)(ranks, x) / (n* total) - (n + 1.0) / n)

def normalized_entropy(weights: np.ndarray) -> float:

text

"""Compute normalized entropy."""
```

```
weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

if total &lt;= 1e-12:

return 0.0

probabilities = weights / total

positive = probabilities &gt; 0.0

entropy = -float(np.sum(probabilities[positive] * np.log(probabilities[positive])))

return entropy / float(np.log(len(weights)))

def time_to_at_least(

text

sample_times: np.ndarray,

values: np.ndarray,

target: float,

) -> float:

text

"""Find the first time at which values reach at least the target.
```

Matches frozen production exactly.

Returns NaN if target is never reached.

"""

sample_times = np.asarray(sample_times, dtype=float)

values = np.asarray(values, dtype=float)

valid = np.isfinite(values)

indices = np.flatnonzero(valid & (values >= target))

return float(sample_times[indices[0]]) if len(indices) > 0 else float('nan')

def time_to_below(

text

sample_times: np.ndarray,

values: np.ndarray,

threshold: float,

) -> float:

text

"""Find the first time at which values drop below the threshold.

Matches frozen production exactly.

Returns NaN if threshold is never dropped below.

"""

sample_times = np.asarray(sample_times, dtype=float)

values = np.asarray(values, dtype=float)

valid = np.isfinite(values)

indices = np.flatnonzero(valid & (values < threshold))

return float(sample_times[indices[0]]) if len(indices) > 0 else float('nan')

def finalize_historical_branch_metrics(

text

branch_results: dict[str, dict[str, Any]],

) -> None:

text

"""Compute and add the 8 historical scalar fields that compare branches to continue.

These fields are computed in frozen production by run_seed() after all branches

are simulated. They must be computed here to match the frozen schema.

The 8 fields are:

- Time_to_50pct_continue_final_density

- Time_to_90pct_continue_final_density

- Time_to_50pct_continue_late_similarity

- Time_to_90pct_continue_late_similarity

- Time_to_similarity_below_0_9

- Time_to_similarity_below_0_5

- Late_persistence_ratio_to_continue

- Late_similarity_ratio_to_continue

These are computed for each branch relative to the continue branch reference values.

"""

Get continue branch results for reference

if "continue" not in branch_results:

raise RuntimeError("continue branch is required for historical metric computation")

```
continue_result = branch_results["continue"]


# Get reference values from continue branch

reference_density = float(continue_result["D_B_final"])

reference_late_similarity = float(continue_result["Weighted_similarity_late"])

reference_late_persistence = float(continue_result["P_mean_late"])


for branch, result in branch_results.items():

# Use direct indexing to fail if keys are missing

times = result["sample_times"]

density = result["density_trajectory"]

similarity = result["similarity_trajectory"]


# Compute time-based metrics relative to continue branch reference


# Compute time-based metrics relative to continue branch reference

result["Time_to_50pct_continue_final_density"] = time_to_at_least(

times, density, 0.5 * reference_density
```

```
)
result["Time_to_90pct_continue_final_density"] = time_to_at_least(
times, density, 0.9 * reference_density
)
result["Time_to_50pct_continue_late_similarity"] = time_to_at_least(
times, similarity, 0.5 * reference_late_similarity
)
result["Time_to_90pct_continue_late_similarity"] = time_to_at_least(
times, similarity, 0.9 * reference_late_similarity
)
result["Time_to_similarity_below_0_9"] = time_to_below(
times, similarity, 0.9
)
result["Time_to_similarity_below_0_5"] = time_to_below(
times, similarity, 0.5
)
```

```
# Compute ratio metrics - use direct indexing to fail if missing

branch_late_persistence = float(result["P_mean_late"])

branch_late_similarity = float(result["Weighted_similarity_late"])



# Match frozen production exactly: check both finite and &gt; ZERO_TOL

if np.isfinite(reference_late_persistence) and abs(reference_late_persistence) &gt;
FROZEN_ZERO_TOL:

result["Late_persistence_ratio_to_continue"] = (

branch_late_persistence / reference_late_persistence

)

else:

result["Late_persistence_ratio_to_continue"] = float('nan')



if np.isfinite(reference_late_similarity) and abs(reference_late_similarity) &gt; FROZEN_ZERO_TOL:

result["Late_similarity_ratio_to_continue"] = (

branch_late_similarity / reference_late_similarity

)

else:

result["Late_similarity_ratio_to_continue"] = float('nan')
```

==========
Required Output Schema for Baseline Equivalence
==========
IMPORTANT: We separate FROZEN keys (must exist in historical NPZ) from
EXTENDED keys (new in v1, only validated internally at rho=1).
The frozen NPZ does NOT contain dual-horizon keys, so baseline equivalence
must only check FROZEN keys against historical data.
Required prefix keys for baseline equivalence

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)
These are the keys that MUST exist in the frozen NPZ for baseline equivalence

FROZEN_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
These are the NEW keys introduced in v1 for dual-horizon support
They will NOT exist in frozen NPZ, so baseline equivalence skips them

EXTENDED_BRANCH_MATRIX_KEYS = (

text

"B_mean_late_fixed_physical",

"P_full_fixed_physical",

"P_immediate_fixed_physical",

"P_recovery_fixed_physical",

"P_late_fixed_physical",

"B_mean_late_normalized_EMA",

"P_full_normalized_EMA",

"P_immediate_normalized_EMA",

"P_recovery_normalized_EMA",

"P_late_normalized_EMA",

)
All matrix keys (frozen + extended)

REQUIRED_BRANCH_MATRIX_KEYS = FROZEN_BRANCH_MATRIX_KEYS + EXTENDED_BRANCH_MATRIX_KEYS
FROZEN BRANCH SCALAR KEYS: All 42 historical fields from frozen production
These MUST exist in the frozen CSV and are validated against it

FROZEN_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

Historical cross-branch time-based metrics (from frozen production)

"Time_to_50pct_continue_final_density",

"Time_to_90pct_continue_final_density",

"Time_to_50pct_continue_late_similarity",

"Time_to_90pct_continue_late_similarity",

"Time_to_similarity_below_0_9",

"Time_to_similarity_below_0_5",

"Late_persistence_ratio_to_continue",

"Late_similarity_ratio_to_continue",

)
EXTENDED BRANCH SCALAR KEYS: New fields in v1 for dual-horizon support
These do NOT exist in frozen CSV/NPZ, so are validated internally at rho=1

EXTENDED_BRANCH_SCALAR_KEYS = (

text

Fixed physical horizon scalars (PRIMARY ESTIMANDS)

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

Normalized EMA horizon scalars (NEW - explicit)

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

Additional extended scalars for complete coverage

"P_mean_full_fixed_physical",

"P_mean_immediate_fixed_physical",

"P_mean_recovery_fixed_physical",

"P_mean_full_normalized_EMA",

"P_mean_immediate_normalized_EMA",

"P_mean_recovery_normalized_EMA",

"Retention_selected_late_fixed_physical",

"Emergence_unselected_late_fixed_physical",

"Retention_selected_late_normalized_EMA",

"Emergence_unselected_late_normalized_EMA",

"Corr_Rpast_Pfull_fixed_physical",

"Corr_Rpast_Pfull_normalized_EMA",

"Jaccard_final_fixed_physical",

)
Extended scalar equivalence map for internal consistency validation at rho=1
Maps extended keys to their frozen equivalents

EXTENDED_SCALAR_EQUIVALENCE_MAP = {

text

"P_mean_full_fixed_physical": "P_mean_full",

"P_mean_immediate_fixed_physical": "P_mean_immediate",

"P_mean_recovery_fixed_physical": "P_mean_recovery",

"P_mean_late_fixed_physical": "P_mean_late",

"P_mean_full_normalized_EMA": "P_mean_full",

"P_mean_immediate_normalized_EMA": "P_mean_immediate",

"P_mean_recovery_normalized_EMA": "P_mean_recovery",

"P_mean_late_normalized_EMA": "P_mean_late",

"Weighted_similarity_late_fixed_physical": "Weighted_similarity_late",

"Weighted_similarity_late_normalized_EMA": "Weighted_similarity_late",

"Retention_contrast_late_fixed_physical": "Retention_contrast_late",

"Retention_contrast_late_normalized_EMA": "Retention_contrast_late",

"Corr_Rpast_Plate_fixed_physical": "Corr_Rpast_Plate",

"Corr_Rpast_Plate_normalized_EMA": "Corr_Rpast_Plate",

"Corr_Rpast_Pfull_fixed_physical": "Corr_Rpast_Pfull",

"Corr_Rpast_Pfull_normalized_EMA": "Corr_Rpast_Pfull",

"r_mean_late_fixed_physical": "r_mean_late",

"r_mean_late_normalized_EMA": "r_mean_late",

"Retention_selected_late_fixed_physical": "Retention_selected_late",

"Retention_selected_late_normalized_EMA": "Retention_selected_late",

"Emergence_unselected_late_fixed_physical": "Emergence_unselected_late",

"Emergence_unselected_late_normalized_EMA": "Emergence_unselected_late",

"Jaccard_final_fixed_physical": "Jaccard_final",

}
Validate no duplicates in extended scalar keys

if len(EXTENDED_BRANCH_SCALAR_KEYS) !=
len(set(EXTENDED_BRANCH_SCALAR_KEYS)):

text

duplicates = [k for k in EXTENDED_BRANCH_SCALAR_KEYS if
list(EXTENDED_BRANCH_SCALAR_KEYS).count(k) > 1]

raise RuntimeError(f"Duplicate extended scalar schema keys: {sorted(set(duplicates))}")

Validate schema-map consistency

if set(EXTENDED_BRANCH_SCALAR_KEYS) !=
set(EXTENDED_SCALAR_EQUIVALENCE_MAP.keys()):

text

missing_map = set(EXTENDED_BRANCH_SCALAR_KEYS) -
set(EXTENDED_SCALAR_EQUIVALENCE_MAP.keys())

extra_map = set(EXTENDED_SCALAR_EQUIVALENCE_MAP.keys()) - set(EXTENDED_BRANCH_SCALAR_KEYS)

raise RuntimeError(

f"Extended scalar map mismatch: "

f"missing mappings={sorted(missing_map)}, extra mappings={sorted(extra_map)}"

)

All required scalar keys (frozen + extended)

REQUIRED_BRANCH_SCALAR_KEYS = FROZEN_BRANCH_SCALAR_KEYS + EXTENDED_BRANCH_SCALAR_KEYS
Extended matrix equivalence map for internal consistency validation at rho=1
Maps extended matrix keys to their frozen equivalents

EXTENDED_MATRIX_EQUIVALENCE_MAP = {

text

"B_mean_late_fixed_physical": "B_mean_late",

"P_full_fixed_physical": "P_full",

"P_immediate_fixed_physical": "P_immediate",

"P_recovery_fixed_physical": "P_recovery",

"P_late_fixed_physical": "P_late",

"B_mean_late_normalized_EMA": "B_mean_late",

"P_full_normalized_EMA": "P_full",

"P_immediate_normalized_EMA": "P_immediate",

"P_recovery_normalized_EMA": "P_recovery",

"P_late_normalized_EMA": "P_late",

}
==========
Baseline Equivalence Test
==========

def exact_array_equal_with_nan(a: np.ndarray, b: np.ndarray) -> bool:

text

"""Compare arrays with NaN-aware exact equality."""

a = np.asarray(a)

b = np.asarray(b)

if a.shape != b.shape or a.dtype != b.dtype:

return False

if np.issubdtype(a.dtype, np.floating) or np.issubdtype(a.dtype, np.complexfloating):

a_nan = np.isnan(a)

b_nan = np.isnan(b)

if not np.array_equal(a_nan, b_nan):

return False

finite = ~a_nan

return np.array_equal(a[finite], b[finite])

return np.array_equal(a, b)

def run_baseline_equivalence(

text

seeds: list[int],

config: ModelConfig,

protocol: ProtocolConfig,

frozen_manifest_path: str,

frozen_npz_path: str,

branch_order: tuple[str, ...],

frozen_csv_path: str | None = None,

) -> dict[str, Any]:

text

"""Run baseline equivalence test at rho=1 (T_window = T_*).

MANDATORY: Must reproduce frozen run exactly for all seeds.

This function validates that the engine produces EXACTLY the same outputs

as the frozen production run for all required keys.

Requires NaN-aware exact equality for ALL required schema keys:

- All prefix keys

- All branch matrix keys

- All branch scalar keys

Returns:

status: "passed" | "failed" | "not_run"

seed_results: per-seed validation details

engine_source_sha256: hash of this engine source

frozen_manifest_hash: hash of frozen manifest

frozen_npz_hash: hash of frozen NPZ

"""

import numpy as np

```
from pathlib import Path

import hashlib



# Preflight validation: validate config and protocol before any simulation

try:

config.validate()

except Exception as e:

return {

"status": "failed",

"error": f"Config validation failed: {e}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()) if frozen_manifest_path else None,

"frozen_npz": str(Path(frozen_npz_path).resolve()) if frozen_npz_path else None,

"frozen_csv_path": frozen_csv_path,

}



try:

protocol.validate(config)
```

except Exception as e:

return {

"status": "failed",

"error": f"Protocol validation failed: {e}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()) if frozen_manifest_path else None,

"frozen_npz": str(Path(frozen_npz_path).resolve()) if frozen_npz_path else None,

"frozen_csv_path": frozen_csv_path,

}

Helper function to compute source hash

def *source*sha256() -> str | None:

try:

return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

return None

P1 #6: Preflight all authority files - check existence before opening

```
# Check frozen manifest exists

if not os.path.isfile(frozen_manifest_path):

return {

"status": "failed",

"error": f"Frozen manifest not found: {frozen_manifest_path}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()) if frozen_manifest_path else None,

"frozen_npz": str(Path(frozen_npz_path).resolve()) if frozen_npz_path else None,

"frozen_csv_path": frozen_csv_path,

}



# Check frozen NPZ exists

if not os.path.isfile(frozen_npz_path):

return {

"status": "failed",

"error": f"Frozen NPZ not found: {frozen_npz_path}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()) if frozen_manifest_path else None,

"frozen_npz": str(Path(frozen_npz_path).resolve()) if frozen_npz_path else None,
```

```
"frozen_csv_path": frozen_csv_path,

}



# Early failure: frozen scalar CSV is required for scalar equivalence

if frozen_csv_path is None:

return {

"status": "failed",

"error": "frozen scalar CSV is required for scalar equivalence",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": None,

}



# Check frozen CSV exists (if provided)

if frozen_csv_path is not None and not os.path.isfile(frozen_csv_path):

return {

"status": "failed",
```

```
"error": f"Frozen scalar CSV not found: {frozen_csv_path}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# Record engine source hash

engine_source_hash = *source*sha256()



results = {

"status": "not_run",

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"mean_omega": config.mean_omega,

"T_window": config.T_window,
```

"T_star": config.T_star,

"alpha": config.alpha,

"a": config.a,

"q": config.q,

"delta": config.delta,

"eta": config.eta,

"mu": config.mu,

"gamma": config.gamma,

"B_c": config.B_c,

"B_init_scale": config.B_init_scale,

"scale_epsilon": config.scale_epsilon,

"capacity": config.capacity,

"delay_steps": config.delay_steps,

},

"protocol": {

"T_sim": protocol.T_sim,

"burn_end_time": protocol.burn_end_time,

```
"split_time": protocol.split_time,
"future_duration": protocol.future_duration,
"diagnostic_stride": protocol.diagnostic_stride,
"branches": list(protocol.branches),
},
"n_seeds": len(seeds),
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"engine_source_sha256": engine_source_hash,
"seed_results": {},
}


# Load frozen manifest
with open(frozen_manifest_path, "r") as f:
frozen_manifest = json.load(f)


# P0 #4: Validate seed/branch authority - seeds and branches must match manifest
```

```
manifest_seeds = [int(x) for x in frozen_manifest["randomization"]["paired_seeds"]]

manifest_branches = tuple(frozen_manifest["protocol"]["branches"])

if list(seeds) != manifest_seeds:

return {

"status": "failed",

"error": f"Seed authority mismatch: seeds={seeds}, manifest={manifest_seeds}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

if tuple(branch_order) != manifest_branches:

return {

"status": "failed",

"error": f"Branch authority mismatch: branch_order={branch_order}, manifest={manifest_branches}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}
```

```
# P0 #3: Validate baseline horizon metadata

# At rho=1 (T_window = T_star), horizon metadata must match frozen values

if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):

expected_metadata = BASELINE_HORIZON_METADATA

actual_window_steps = int(round(config.T_window / config.dt))

actual_recovery_end_ema = min(5 * actual_window_steps, protocol.future_steps(config))

actual_fixed_immediate_end = int(round(17.5 / config.dt))

actual_fixed_recovery_end = int(round(87.5 / config.dt))


if actual_window_steps != expected_metadata["window_steps"]:

return {

"status": "failed",

"error": f"window_steps mismatch: actual={actual_window_steps},
expected={expected_metadata['window_steps']}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,
```

```
}

if actual_recovery_end_ema != expected_metadata["recovery_end_ema"]:

return {

"status": "failed",

"error": f"recovery_end_ema mismatch: actual={actual_recovery_end_ema},
expected={expected_metadata['recovery_end_ema']}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

if actual_fixed_immediate_end != expected_metadata["fixed_immediate_end"]:

return {

"status": "failed",

"error": f"fixed_immediate_end mismatch: actual={actual_fixed_immediate_end},
expected={expected_metadata['fixed_immediate_end']}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,
```

```
}

if actual_fixed_recovery_end != expected_metadata["fixed_recovery_end"]:

return {

"status": "failed",

"error": f"fixed_recovery_end mismatch: actual={actual_fixed_recovery_end},
expected={expected_metadata['fixed_recovery_end']}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}




# Load frozen data

with np.load(frozen_npz_path, allow_pickle=False) as frozen_data:

frozen_files = dict(frozen_data)




# P1 #7: Preflight NPZ schema - validate expected keys exist

# Check prefix keys for all seeds

for seed in seeds:
```

```
seed_str = str(seed)

for key in REQUIRED_PREFIX_KEYS:

frozen_key = f"prefix_seed_{seed_str}_{key}"

if frozen_key not in frozen_files:

return {

"status": "failed",

"error": f"Missing NPZ key: {frozen_key}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# Check branch matrix keys for all seeds and branches

for seed in seeds:

seed_str = str(seed)

for branch in branch_order:

for key in FROZEN_BRANCH_MATRIX_KEYS:
```

```
frozen_key = f"{branch}_seed_{seed_str}_{key}"

if frozen_key not in frozen_files:

return {

"status": "failed",

"error": f"Missing NPZ key: {frozen_key}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# Load frozen scalar CSV - REQUIRED for scalar equivalence

# Fail immediately if CSV is missing or invalid

if frozen_csv_path is None or not os.path.isfile(frozen_csv_path):

return {

"status": "failed",

"error": f"Frozen scalar CSV not found: {frozen_csv_path}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
```

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

frozen_scalar_csv: dict[str, dict[str, Any]] = {}

try:

import csv

with open(frozen_csv_path, "r", newline="", encoding="utf-8") as f:

reader = csv.DictReader(f)

for row in reader:

seed_id = row.get("seed", "")

branch = row.get("branch", "")

if seed_id and branch:

key = f"{branch}_seed_{seed_id}"

P1 #5: CSV duplicate detection - fail if duplicate (seed, branch) rows

if key in frozen_scalar_csv:

return {

```
"status": "failed",

"error": f"Duplicate CSV row detected for {key}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

frozen_scalar_csv[key] = {}

for k, v in row.items():

if k not in ("seed", "seed_index", "branch", "rho", "T_window"):

try:

frozen_scalar_csv[key][k] = float(v)

except (ValueError, TypeError):

frozen_scalar_csv[key][k] = v

except Exception as e:

return {

"status": "failed",

"error": f"Could not load frozen CSV {frozen_csv_path}: {e}",
```

```
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# Validate frozen CSV has all expected seeds and branches

# Expected: 40 seeds × 7 branches = 280 rows

expected_csv_rows = len(seeds) * len(branch_order)

actual_csv_rows = len(frozen_scalar_csv)

if actual_csv_rows != expected_csv_rows:

return {

"status": "failed",

"error": f"Frozen CSV has {actual_csv_rows} rows, expected {expected_csv_rows} (40 seeds × 7 branches)",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}
```

```
# Validate all expected scalar columns exist in frozen CSV

# Check a sample row to see what columns are available

sample_key = None

for key in frozen_scalar_csv:

sample_key = key

break



if sample_key and frozen_scalar_csv[sample_key]:

available_columns = set(frozen_scalar_csv[sample_key].keys())

missing_columns = set(FROZEN_BRANCH_SCALAR_KEYS) - available_columns

if missing_columns:

return {

"status": "failed",

"error": f"Frozen CSV missing {len(missing_columns)} required scalar columns:
{sorted(missing_columns)[:5]}",

"missing_columns": sorted(missing_columns),

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
```

```
"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# Record frozen hashes - use actual file hashes, not canonical JSON hashes

# Record frozen hashes - use actual file hashes, not canonical JSON hashes

results["frozen_manifest_file_sha256"] = file_sha256(frozen_manifest_path)

results["frozen_npz_file_sha256"] = file_sha256(frozen_npz_path)

if frozen_csv_path is not None and os.path.exists(frozen_csv_path):

results["frozen_scalar_csv_file_sha256"] = file_sha256(frozen_csv_path)



# Also record engine source hash

results["engine_source_sha256"] = engine_source_hash



# Also keep canonical JSON hash for reference

results["frozen_manifest_hash"] = hashlib.sha256(

json.dumps(frozen_manifest, sort_keys=True).encode()
```

).hexdigest()

Validate config matches frozen manifest model - use EXACT equality

frozen_model = frozen_manifest["model"]

Check N

if frozen_model["N"] != config.N:

results["status"] = "failed"

results["error"] = f"N mismatch: config={config.N}, frozen={frozen_model['N']}"

return results

Check omega - use exact equality

frozen_omega_arr = np.array(frozen_model["omega"], dtype=float)

config_omega_arr = np.array(list(config.omega), dtype=float)

if not np.array_equal(frozen_omega_arr, config_omega_arr):

results["status"] = "failed"

results["error"] = f"omega mismatch: frozen={frozen_model['omega']}, config={list(config.omega)}"

return results

Check all other model params with EXACT equality (rtol=0, atol=0)

model_checks = [

("dt", frozen_model["dt"], config.dt),

("tau", frozen_model["tau"], config.tau),

("a", frozen_model["a"], config.a),

("q", frozen_model["q"], config.q),

("delta", frozen_model["delta"], config.delta),

("eta", frozen_model["eta"], config.eta),

("mu", frozen_model["mu"], config.mu),

("gamma", frozen_model["gamma"], config.gamma),

("B_c", frozen_model["B_c"], config.B_c),

("B_init_scale", frozen_model.get("B_init_scale", 0.0), config.B_init_scale),

("T_window", frozen_model["T_window"], config.T_window),

]

```
for name, frozen_val, config_val in model_checks:

if not np.isclose(frozen_val, config_val, rtol=0.0, atol=0.0):

results["status"] = "failed"

results["error"] = f"{name} mismatch: frozen={frozen_val}, config={config_val}"

return results



# Check derived values

if not np.isclose(frozen_model.get("alpha", 0.0), config.alpha, rtol=0.0, atol=1e-15):

results["status"] = "failed"

results["error"] = f"alpha mismatch: frozen={frozen_model.get('alpha')}, config={config.alpha}"

return results



# Now run actual equivalence test for each seed

print("Running baseline equivalence test...")

all_passed = True
```

for seed in seeds:

seed_str = str(seed)

seed_results = {"status": "passed", "checks": {}, "max_diffs": {}, "errors": {}}

try:

Simulate with our engine

split_state, prefix = simulate_prefix(seed, config, protocol)

branch_rngs = make_branch_rngs(seed, branch_order)

Simulate each branch

engine_results = {}

for branch in protocol.branches:

rng = branch_rngs[branch]

engine_results[branch] = simulate_future(

split_state.clone(),

prefix,

branch,

config,

```
protocol,

rng,

)


# CRITICAL: Compute historical cross-branch metrics (the 8 time-based fields)

# These must be computed AFTER all branches are simulated

if "continue" in engine_results:

finalize_historical_branch_metrics(engine_results)


# Check prefix state - iterate over REQUIRED_PREFIX_KEYS

for key in REQUIRED_PREFIX_KEYS:

engine_val = prefix.get(key)

frozen_key = f"prefix_seed_{seed_str}_{key}"

frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"prefix_{key}_present_in_frozen"] = False

all_passed = False
```

continue

else:

seed_results["checks"][f"prefix_{key}_present_in_frozen"] = True

if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"prefix_{key}_exact"] = True

else:

seed_results["checks"][f"prefix_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):

if engine_val.shape != frozen_val.shape:

seed_results["errors"][f"prefix_{key}_shape_mismatch"] = f"engine shape {engine_val.shape} != frozen shape {frozen_val.shape}"

seed_results["max_diffs"][f"prefix_{key}"] = None

else:

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(frozen_val), 0.0, frozen_val)

```
seed_results["max_diffs"][f"prefix_{key}"] = float(np.max(np.abs(e - f)))

else:

seed_results["max_diffs"][f"prefix_{key}"] = None


# Check branch outputs - separate FROZEN vs EXTENDED keys

for branch in protocol.branches:

engine_branch = engine_results[branch]

frozen_prefix = f"{branch}_seed_{seed_str}"


# === FROZEN MATRIX KEYS: Must exist in historical NPZ ===

for key in FROZEN_BRANCH_MATRIX_KEYS:

engine_val = engine_branch.get(key)

frozen_key = f"{frozen_prefix}_{key}"

frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = False

all_passed = False
```

```
continue

else:

seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = True


if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False

all_passed = False

continue

else:

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = True


if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"{branch}_{key}_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False
```

```
if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):

if engine_val.shape != frozen_val.shape:

seed_results["errors"][f"{branch}_{key}_shape_mismatch"] = f"engine shape {engine_val.shape} != frozen shape {frozen_val.shape}"

seed_results["max_diffs"][f"{branch}_{key}"] = None

else:

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(frozen_val), 0.0, frozen_val)

seed_results["max_diffs"][f"{branch}_{key}"] = float(np.max(np.abs(e - f)))

else:

seed_results["max_diffs"][f"{branch}_{key}"] = None


# === EXTENDED MATRIX KEYS: Validate internally at rho=1 ===

# These do NOT exist in frozen NPZ. At rho=1, both horizon systems

# should produce identical results (1750 and 8750 steps match).

if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):

for key in EXTENDED_BRANCH_MATRIX_KEYS:

engine_val = engine_branch.get(key)
```

```
if engine_val is None:
seed_results["status"] = "failed"
seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False
all_passed = False
continue
else:
seed_results["checks"][f"{branch}_{key}_present_in_engine"] = True

# Use module-level map for consistency - direct indexing
frozen_equiv_key = EXTENDED_MATRIX_EQUIVALENCE_MAP[key]
if frozen_equiv_key:
engine_frozen_equiv = engine_branch.get(frozen_equiv_key)
if engine_frozen_equiv is not None:
if exact_array_equal_with_nan(engine_val, engine_frozen_equiv):
seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = True
else:
seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = False
seed_results["status"] = "failed"
```

```
all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(engine_frozen_equiv, np.ndarray):

if engine_val.shape != engine_frozen_equiv.shape:

seed_results["errors"][f"{branch}_{key}_shape_mismatch"] = f"engine shape {engine_val.shape} != equiv shape {engine_frozen_equiv.shape}"

seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = None

else:

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(engine_frozen_equiv), 0.0, engine_frozen_equiv)

seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = float(np.max(np.abs(e - f)))

else:

seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = None


# === SCALAR KEYS ===

# Use module-level constants for consistency

# FROZEN_BRANCH_SCALAR_KEYS: All 42 historical fields from frozen production

# EXTENDED_BRANCH_SCALAR_KEYS: New fields in v1 for dual-horizon support


for key in FROZEN_BRANCH_SCALAR_KEYS:
```

```
engine_val = engine_branch.get(key)


# P1 #8: Use CSV as authority, not NPZ fallback
# Get from CSV only (production stores scalars in CSV)
frozen_val = None
csv_key = f"{branch}_seed_{seed_str}"
if csv_key in frozen_scalar_csv and key in frozen_scalar_csv[csv_key]:
frozen_val = frozen_scalar_csv[csv_key][key]


# FROZEN scalars MUST exist in historical data
if frozen_val is None:
seed_results["status"] = "failed"
seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = False
all_passed = False
continue
else:
seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = True


if engine_val is None:
```

```
seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False

all_passed = False

continue

else:

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = True


# For scalars, compare directly - DO NOT silently skip

# Normalize to scalar value

try:

engine_scalar = float(engine_val)

except (TypeError, ValueError) as e:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_is_scalar"] = False

seed_results["errors"][f"{branch}_{key}_type_error"] = f"Cannot convert to float:
{type(engine_val).__name__}: {e}"

all_passed = False

continue
```

try:

frozen_scalar = float(frozen_val)

except (TypeError, ValueError) as e:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_frozen_is_scalar"] = False

seed_results["errors"][f"{branch}_{key}_frozen_type_error"] = f"Cannot convert frozen to float: {type(frozen_val).__name__}: {e}"

all_passed = False

continue

Compare with exact equality (rtol=0, atol=0)

engine_is_nan = bool(np.isnan(engine_scalar))

frozen_is_nan = bool(np.isnan(frozen_scalar))

if engine_is_nan and frozen_is_nan:

seed_results["checks"][f"{branch}_{key}_exact"] = True

elif np.isclose(engine_scalar, frozen_scalar, rtol=0.0, atol=0.0):

seed_results["checks"][f"{branch}_{key}_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

seed_results["max_diffs"][f"{branch}_{key}"] = float(abs(engine_scalar - frozen_scalar)) if not (engine_is_nan or frozen_is_nan) else None

=== EXTENDED SCALAR KEYS: Validate internally at rho=1 ===

if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):

for key in EXTENDED_BRANCH_SCALAR_KEYS:

engine_val = engine_branch.get(key)

if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False

all_passed = False

continue

else:

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = True

```
# Use module-level map for consistency - direct indexing

frozen_equiv = EXTENDED_SCALAR_EQUIVALENCE_MAP[key]

if frozen_equiv:

# Get the engine's own frozen-equivalent value

engine_frozen_equiv = engine_branch.get(frozen_equiv)


if engine_frozen_equiv is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{frozen_equiv}_present_in_engine"] = False

all_passed = False

continue


# Validate internal consistency: extended should equal frozen equivalent

try:

engine_scalar = float(engine_val)

equiv_scalar = float(engine_frozen_equiv)

except (TypeError, ValueError) as e:

seed_results["status"] = "failed"
```

```
seed_results["checks"][f"{branch}_{key}_internal_consistency"] = False

seed_results["errors"][f"{branch}_{key}_internal_type_error"] = str(e)

all_passed = False

continue


engine_is_nan = bool(np.isnan(engine_scalar))

equiv_is_nan = bool(np.isnan(equiv_scalar))


if engine_is_nan and equiv_is_nan:

seed_results["checks"][f"{branch}_{key}_internal_exact"] = True

elif np.isclose(engine_scalar, equiv_scalar, rtol=0.0, atol=0.0):

seed_results["checks"][f"{branch}_{key}_internal_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_internal_exact"] = False

seed_results["status"] = "failed"

all_passed = False

seed_results["max_diffs"][f"{branch}_{key}_internal"] = float(abs(engine_scalar - equiv_scalar)) if
not (engine_is_nan or equiv_is_nan) else None
```

```
except Exception as e:

seed_results["status"] = "failed"

seed_results["error"] = str(e)

import traceback

seed_results["traceback"] = traceback.format_exc()

all_passed = False


results["seed_results"][seed_str] = seed_results

print(f" Seed {seed_str}: {seed_results['status']}")



if all_passed:

results["status"] = "passed"

print("Baseline equivalence: PASSED")

else:

results["status"] = "failed"

print("Baseline equivalence: FAILED")

# Print details of failures

for seed_str, seed_results in results["seed_results"].items():
```

```
if seed_results.get("status") != "passed":

print(f" Seed {seed_str} failures:")

for check, val in seed_results.get("checks", {}).items():

if not val:

print(f" CHECK FAILED: {check}")

for key, val in seed_results.get("max_diffs", {}).items():

print(f" MAX DIFF: {key} = {val}")



return results

================================================================================
Main API
================================================================================
Known branches - allowlist for validation

KNOWN_BRANCHES = frozenset({

text

"continue",

"reset_ema",

"reset_B",
```

"reset_B_ema",

"B_carrier_only",

"full_reset",

"future_shuffle",

})

def run_seed(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

branch_order: tuple[str, ...],

) -> dict[str, Any]:

text

"""Run a single seed through prefix + all branches.

CRITICAL: branch_order defines the FULL frozen RNG namespace (7 branches).

protocol.branches defines which branches are actually executed.

This ensures future_shuffle gets the 7th RNG sequence (not the 4th)

as in production, even when only 4 study branches are executed.

Returns complete seed results with all branches.

"""

Preflight validation: validate config and protocol

config.validate()

protocol.validate(config)

Validate all protocol branches are known

for branch in protocol.branches:

if branch not in KNOWN_BRANCHES:

raise ValueError(

f"Unknown branch '{branch}'. Known branches: {sorted(KNOWN_BRANCHES)}"

)

```
# Validate all protocol branches are in branch_order (for RNG namespace)

for branch in protocol.branches:

if branch not in branch_order:

raise ValueError(

f"Branch '{branch}' in protocol.branches but not in branch_order. "

f"branch_order must include all protocol branches for correct RNG namespace."

)



# Validate branch_order has no duplicates and all are known

if len(branch_order) != len(set(branch_order)):

raise ValueError(f"branch_order has duplicate branches: {branch_order}")

for branch in branch_order:

if branch not in KNOWN_BRANCHES:

raise ValueError(f"Unknown branch '{branch}' in branch_order. Known branches:
{sorted(KNOWN_BRANCHES)}")



# Simulate prefix
```

```
split_state, prefix = simulate_prefix(seed, config, protocol)



# Create branch RNGs using frozen algorithm with FULL branch_order

branch_rngs = make_branch_rngs(seed, branch_order)



# Execute only the branches in protocol.branches, but use RNGs from branch_order

branch_results = {}

for branch in protocol.branches:

if branch not in branch_rngs:

raise ValueError(

f"Branch '{branch}' not found in branch_rngs. "

f"This indicates branch_order={branch_order} does not match the RNG namespace. "

f"Registered branches: {list(branch_rngs.keys())}"

)

rng = branch_rngs[branch]



branch_results[branch] = simulate_future(

split_state.clone(),
```

```
prefix,

branch,

config,

protocol,

rng,

)



# Compute historical cross-branch metrics (the 8 time-based fields)

# These must be computed AFTER all branches are simulated

if "continue" in branch_results:

finalize_historical_branch_metrics(branch_results)



return {

"seed": int(seed),

"config": {

"N": config.N,

"dt": config.dt,
```

"tau": config.tau,

"T_window": config.T_window,

"alpha": config.alpha,

"mean_omega": config.mean_omega,

"T_star": config.T_star,

},

"protocol": {

"T_sim": protocol.T_sim,

"burn_end_time": protocol.burn_end_time,

"split_time": protocol.split_time,

"future_duration": protocol.future_duration,

},

"prefix": prefix,

"branches": branch_results,

}

if name == "main":

text

```
# Quick validation

config = DEFAULT_MODEL_CONFIG

config.validate()

protocol = DEFAULT_PROTOCOL_CONFIG

protocol.validate(config)



print(f"Engine v1 loaded successfully")

print(f" N={config.N}, dt={config.dt}, tau={config.tau}")

print(f" T_window={config.T_window}, alpha={config.alpha:.6f}")

print(f" mean_omega={config.mean_omega:.6f}, T_*={config.T_star:.6f}")

print(f" delay_steps={config.delay_steps}")

print(f" Protocol: T_sim={protocol.T_sim}, burn={protocol.burn_end_time}, "

f"split={protocol.split_time}, future={protocol.future_duration}")
```

Ignoring paste corruption, this version is materially improved. The extended scalar map is now actually used, duplicate scalar keys are gone, historical metrics are finalized, authority files and NPZ keys are preflighted, and scalar comparison correctly uses the CSV authority.
Verdict

Core numerical step map: approximately 9.5/10
Frozen RNG fidelity: approximately 9.5/10
Dual-horizon implementation: approximately 9.5/10
Baseline certificate: approximately 9/10
Compile/import: GO

Mandatory baseline: make the P0 corrections below first

No saved replay has been produced yet; this is still a static assessment.
P0 corrections
1. Baseline does not require protocol.branches to equal all frozen branches

You validate:

Python

tuple(branch_order) == manifest_branches

but do not validate:

Python

tuple(protocol.branches) == manifest_branches

This creates a genuine false-pass path. A caller could supply:

Python

branch_order = all seven frozen branches
protocol.branches = only four study branches

The NPZ/CSV preflight would see all seven frozen branches, but the engine would simulate and compare only the four protocol branches. The function could still return "passed" without testing all seven.
Fix

Immediately after validating branch_order:

Python

if tuple(protocol.branches) != manifest_branches:
return {

"status": "failed",
"error": (
"Executed baseline branch mismatch: "
f"protocol.branches={protocol.branches}, "
f"manifest={manifest_branches}"
),
"frozen_manifest": str(
Path(frozen_manifest_path).resolve()
),
"frozen_npz": str(
Path(frozen_npz_path).resolve()
),
"frozen_csv_path": frozen_csv_path,
}

For the mandatory gate, branch order and executed branch order should both exactly equal the frozen manifest.
2. Horizon metadata is derived preflight but not checked in branch outputs

The preflight correctly verifies that the configuration would produce:

text

window_steps = 1750
recovery_end_ema = 8750
fixed_immediate_end = 1750
fixed_recovery_end = 8750

But it never verifies that each returned engine_branch actually contains those metadata values.

A bug in simulate_future() metadata could therefore pass.
Fix

Inside the branch-output loop, before matrix/scalar checks:

Python

for key, expected in BASELINE_HORIZON_METADATA.items():

observed = engine_branch.get(key)

passed = (
isinstance(observed, (int, np.integer))
and int(observed) == int(expected)
)

seed_results["checks"][
f"{branch}_{key}_baseline_exact"
] = passed

if not passed:
seed_results["status"] = "failed"
seed_results["errors"][
f"{branch}_{key}_baseline_mismatch"
] = (
f"observed={observed!r}, "
f"expected={expected!r}"
)
all_passed = False

This turns the metadata into a certified output rather than merely a duplicated calculation.
3. Exact CSV key coverage is still not preflighted

Duplicate detection and row-count validation are now present, which is good. But a CSV containing 280 unique, incorrect (seed, branch) keys could pass the row-count check.

It would eventually fail after simulation, but only after wasting the full baseline runtime.
Fix

After parsing:

Python

expected_csv_keys = {
f"{branch}_seed_{seed}"
for seed in seeds
for branch in branch_order

}

actual_csv_keys = set(frozen_scalar_csv)

if actual_csv_keys != expected_csv_keys:
missing = sorted(expected_csv_keys - actual_csv_keys)
extra = sorted(actual_csv_keys - expected_csv_keys)

return {
"status": "failed",
"error": (
"Frozen CSV condition-key mismatch: "
f"{len(missing)} missing, "
f"{len(extra)} extra"
),
"missing_csv_keys": missing,
"extra_csv_keys": extra,
...
}

Also validate required scalar columns in every row, not only sample_key:

Python

for csv_key, row_values in frozen_scalar_csv.items():
missing = (
set(FROZEN_BRANCH_SCALAR_KEYS)
- set(row_values)
)
if missing:
return {
"status": "failed",
"error": (
f"Frozen CSV row {csv_key} missing "
f"{len(missing)} required fields"
),
"missing_columns": sorted(missing),
...
}

4. Explicitly validate the frozen protocol and safeguard

The model parameters are checked against the manifest, but the baseline does not directly verify several protocol quantities or the fixed safeguard:

text

T_sim
burn_end
hist_end
future_steps
diagnostic_stride
scale_epsilon = 1e-12

Most deviations would eventually fail matrix comparison, but these are implementation identities and should fail before simulation.

Add:

Python

protocol_checks = {
"T_sim": (
protocol.T_sim,
FROZEN_T_SIM,
),
"n_steps": (
protocol.n_steps(config),
FROZEN_N_STEPS,
),
"burn_end": (
protocol.burn_end(config),
FROZEN_BURN_END,
),
"hist_end": (
protocol.hist_end(config),
FROZEN_HIST_END,
),
"future_steps": (

protocol.future_steps(config),
FROZEN_FUTURE_STEPS,
),
"diagnostic_stride": (
protocol.diagnostic_stride,
50,
),
"scale_epsilon": (
config.scale_epsilon,
FROZEN_ZERO_TOL,
),
}

for name, (observed, expected) in protocol_checks.items():
if observed != expected:
return failed_result(...)

The frozen manifest contains several of these fields, so compare against both the manifest and frozen constants where available.
P1 validation hardening
5. ModelConfig.N is still not required to be an integer

The comment says “positive integer,” but:

Python

N = 12.0

passes the current validation and later fails when used as an array dimension.

Use:

Python

if (
isinstance(self.N, (bool, np.bool_))
or not isinstance(self.N, (int, np.integer))
or self.N <= 0

):
raise ValueError("N must be a positive integer")

The initial np.isfinite(self.N) check then becomes unnecessary.

Similarly, permit NumPy integer diagnostic strides but reject booleans:

Python

```
if (
isinstance(self.diagnostic_stride, (bool, np.bool_))
or not isinstance(
self.diagnostic_stride,
(int, np.integer),
)
or self.diagnostic_stride <= 0
):
raise ValueError(
"diagnostic_stride must be a positive integer"
)
```

6. Validate protocol values are finite

Add explicit finiteness validation for:

text

```
T_sim
burn_end_time
split_time
future_duration
```

Otherwise infinities can reach round() and raise less informative errors.

Also validate:

Python

```
if not self.branches:
raise ValueError("branches must be nonempty")


if len(self.branches) != len(set(self.branches)):
raise ValueError("protocol branches contain duplicates")
```

Known-branch validation can remain in run_seed() and baseline preflight.
7. Matrix schema-map integrity should mirror scalar integrity

You correctly added duplicate and mapping checks for extended scalars. Do the same for matrices:

Python

```
if len(EXTENDED_BRANCH_MATRIX_KEYS) != len(
set(EXTENDED_BRANCH_MATRIX_KEYS)
):
raise RuntimeError(
"Duplicate extended matrix schema keys"
)


if set(EXTENDED_BRANCH_MATRIX_KEYS) != set(
EXTENDED_MATRIX_EQUIVALENCE_MAP
):
missing = (
set(EXTENDED_BRANCH_MATRIX_KEYS)
- set(EXTENDED_MATRIX_EQUIVALENCE_MAP)
)
extra = (
set(EXTENDED_MATRIX_EQUIVALENCE_MAP)
- set(EXTENDED_BRANCH_MATRIX_KEYS)
)
raise RuntimeError(
f"Extended matrix map mismatch: "
f"missing={sorted(missing)}, "
f"extra={sorted(extra)}"
)
```

The direct dictionary indexing already makes an omission fail at runtime, but import-time validation is cleaner.

8. Restore literal frozen persistence-ratio logic

The frozen production implementation uses:

Python

```
if abs(reference_late_persistence) > ZERO_TOL
```

The current engine uses:

Python

```
if (
np.isfinite(reference_late_persistence)
and abs(reference_late_persistence) > FROZEN_ZERO_TOL
)
```

Saved finite values should be unaffected, but the docstring says exact reproduction. Use the original condition literally:

Python

```
result["Late_persistence_ratio_to_continue"] = (
float(
branch_late_persistence
/ reference_late_persistence
)
if abs(reference_late_persistence)
> FROZEN_ZERO_TOL
else float("nan")
)
```

Keep the explicit np.isfinite() check for the similarity ratio, because the frozen source uses it there.
9. Catch malformed manifest and NPZ files inside the gate

Existence checks are now present, but malformed JSON or corrupt NPZ still escapes as an exception rather than returning a structured failure result.

Wrap:

Python

```
json.load(...)
np.load(...)
```

in try/except, and validate the required manifest structure before indexing:

text

```
model
protocol
schema
randomization.paired_seeds
protocol.branches
```

The driver may catch an exception, but the engine gate itself promises a structured certificate.
10. Compare manifest schema to the expected frozen schema

Require:

Python

```
manifest_prefix = tuple(
frozen_manifest["schema"]["prefix_keys"]
)
manifest_matrices = tuple(
frozen_manifest["schema"]["branch_matrix_keys"]
)
manifest_scalars = tuple(
frozen_manifest["schema"]["branch_scalar_keys"]
)
```

Then verify uniqueness and set equality with:

text

REQUIRED_PREFIX_KEYS
FROZEN_BRANCH_MATRIX_KEYS
FROZEN_BRANCH_SCALAR_KEYS

The scalar ordering in this engine differs slightly from the historical declaration, so either:

restore the exact frozen order and compare tuples; or
compare unique sets while separately recording that ordering is not semantically used.

State validation assessment

The new transition-boundary checks are safe and should not change trajectories. A few refinements:

Validate immediately after initialization:

Python

state.validate(config, step=-1)

Validate immediately after intervention, before the first future step.
Validate once after the final future step.
Use rtol=0.0 in state symmetry and diagonal checks:

Python

np.allclose(
self.B,
self.B.T,
rtol=0.0,
atol=1e-12,
)

The current checks at selected step indices are harmless, but the checks at window_steps and horizon boundaries occur after that indexed update. Since they are only integrity checks, this does not alter horizon membership.
Performance before the full grid

The two major caches—edge indices and omega arrays—are present. Remaining improvements should ideally happen before the successful baseline so the passing source hash is the grid source hash:
Cache alpha

config.alpha calls np.exp twice per timestep in:

Python

```
config.alpha * state.coherence_acc
+ (1.0 - config.alpha) * z_inst
```

At minimum:

Python

```
alpha = config.alpha
state.coherence_acc[:] = (
alpha * state.coherence_acc
+ (1.0 - alpha) * z_inst
)
```

Better: cache derived immutable configuration values once.
Cache delay steps

config.delay_steps is recomputed during every history update. Cache it per configuration.
Use a bounded deque

Frozen production uses a deque(maxlen=delay_steps + 1). The current list and pop(0) are numerically equivalent but less direct and slower.
Protect cached arrays

After creating cached arrays:

Python

```
edges[0].setflags(write=False)
edges[1].setflags(write=False)
```

omega_array.setflags(write=False)

This prevents accidental global cache mutation.

These are operational improvements, not correctness blockers.
Smaller cleanup

alpha comparison still permits atol=1e-15; now that construction matches frozen np.exp, use exact zero tolerance.
The hardcoded message "40 seeds × 7 branches" should use dynamic values.
simulate_prefix() computes edges but does not use it.
compute_R() assigns N but does not use it.
Duplicate “Record frozen hashes” and “Compute time-based metrics” comments remain.
Unused imports include copy, time, and field.
make_seed_sequence() and array_sha256() appear unused.
The top docstring should say:

text

T_WINDOW = 17.502917152858814... ≈ 17.5

rather than equality to 17.5.

What has been successfully corrected

Since the previous version, the following are now sound:

all 42 historical scalar fields are represented;
historical metrics are finalized inside baseline equivalence;
scalar CSV is the sole scalar authority;
duplicate CSV rows are rejected;
frozen NPZ keys are preflighted;
seed and RNG branch-order authority are checked;
all authority file paths are checked before simulation;
extended scalar coverage is complete;
the full module-level extended scalar map is used;
duplicate extended scalar fields are removed;
fixed and normalized matrices use a module-level equivalence map;

undefined scalar differences are represented by None;
baseline horizon configuration is preflighted;
state dimensions, shapes, finiteness, and admissibility are checked at selected boundaries;
run-time branch subset and RNG namespace are separated correctly.

Bottom line

I do not see a new numerical-dynamics defect. The most important remaining certificate bug is:

branch_order is required to contain all seven branches, but protocol.branches is not.

Fix that before accepting any baseline certificate. Also add actual returned horizon-metadata checks and exact CSV key coverage. After those corrections, this engine is ready for compile → import → baseline-only testing.

Next file:

#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Driver.

Parameterized study varying T_window = rho T_ where T_* = 25 / mean(OMEGA) = 17.5

and rho ∈ {0.5, 0.75, 1, 1.5, 2}.

Study Design:

For each seed s ∈ seeds (40 seeds)

For each rho ∈ {0.5, 0.75, 1, 1.5, 2}

Set T_window = rho * 17.5

Set alpha = exp(-dt / T_window)

Full regeneration for each (s, rho) - NO split state reuse

Branches: continue, reset_ema, reset_B, future_shuffle

Fixed physical horizons: immediate=0-17.5, recovery=17.5-87.5, late=87.5-320

EMA-normalized horizons: immediate=0-1 T_window, recovery=1-5 T_window

Output:

Scalars in CSV: per (seed, rho, branch) scalar outcomes

Arrays in NPZ: per (seed, rho, branch) trajectory and matrix outcomes

MANDATORY: Baseline equivalence test at rho=1 must pass against frozen production.

"""

```
from future import annotations

import argparse

import csv

import hashlib

import json

import os

import platform

import shutil

import sys
```

```
import time

import tempfile

from pathlib import Path

from typing import Any

try:

text

import pandas as pd

except ImportError:

text

print("ERROR: pandas is required for production T_window study")

print("Install with: pip install pandas")

sys.exit(1)

import numpy as np
Import the experiment engine

import aief_experiment_engine_v1 as engine
================================================================================
Study Configuration
================================================================================

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"
```

WORKFLOW_VERSION = "v1.0"
Frozen production values (from AIEF_1_4_memory_ablation_updated_REAL.py)

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA))

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.5
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_STAR / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
Rho values for T_window sensitivity

RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)
Frozen branch order from AIEF_1_4_memory_ablation_updated_REAL.py

FROZEN_BRANCH_ORDER = (

text

"continue",

"reset_ema",

"reset_B",

"reset_B_ema",

"B_carrier_only",

"full_reset",

"future_shuffle",

)
Study subset of branches

STUDY_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Fixed physical horizon times (in time units, not steps)

PHYSICAL_IMMEDIATE_END = 17.5

PHYSICAL_RECOVERY_END = 87.5 # Also PHYSICAL_LATE_START

PHYSICAL_FUTURE_END = 320.0
Defaults

DEFAULT_MASTER_SEED = 20260714

DEFAULT_N_SEEDS = 40

DEFAULT_RUN_ROOT = "runs"
==========
REQUIRED OUTPUT SCHEMA
==========
These are the MINIMUM required outputs that must be present in engine results
If any are missing, the study should fail with a clear error

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)

REQUIRED_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
Primary outcomes (must be present)
NOW USING FIXED-PHYSICAL HORIZONS as the primary estimands

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"P_mean_late_fixed_physical",

)
Secondary outcomes for comparison (normalized EMA)

SECONDARY_OUTCOMES = (

text

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"P_mean_late_normalized_EMA",

)
Backward compatible names (map to normalized EMA for legacy compatibility)

LEGACY_OUTCOMES = (

text

"Weighted_similarity_late",

"Retention_contrast_late",

"P_mean_late",

)
Separate FROZEN vs EXTENDED schemas
FROZEN keys must exist in historical frozen data

FROZEN_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

Projection diagnostics (in frozen data)

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

)
P1 #7: Separate study metadata from frozen scalar output
T_window and rho are study metadata, not part of historical frozen branch scalar schema

STUDY_METADATA_KEYS = (

text

"T_window",

"rho",

)
EXTENDED keys are new in v1 (dual-horizon)

EXTENDED_BRANCH_SCALAR_KEYS = (

text

Fixed physical horizon scalars (PRIMARY ESTIMANDS)

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

Normalized EMA horizon scalars (explicit)

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

)
All required scalar keys (frozen + extended)

REQUIRED_BRANCH_SCALAR_KEYS = FROZEN_BRANCH_SCALAR_KEYS + EXTENDED_BRANCH_SCALAR_KEYS
P1 #6: Removed DESIRED_BRANCH_SCALAR_KEYS as they are always NaN

These fields require finalize_historical_branch_metrics() to be called, which is not done for individual (seed, rho) runs. The new analysis script computes richer condition-specific recovery estimands from trajectories, so these legacy fields are not needed.
All scalar keys for CSV output (required + study metadata)

BRANCH_SCALAR_KEYS = REQUIRED_BRANCH_SCALAR_KEYS + STUDY_METADATA_KEYS
All matrix keys

BRANCH_MATRIX_KEYS = REQUIRED_BRANCH_MATRIX_KEYS

def is_finite_number(value: Any) -> bool:

text

"""Check if a value is a finite number.

Handles int, float, numpy scalar types, and excludes NaN, inf, -inf, None, strings.

"""

return (

isinstance(

value,

(

int,

float,

np.integer,

np.floating,

),

)

and np.isfinite(value)

)

def validate_branch_outputs(branch: str, branch_result: dict[str, Any]) -> None:

text

"""Validate that a branch result contains all required outputs.

P1 #8: Use engine's REQUIRED_BRANCH_MATRIX_KEYS for matrix validation.

Also validates array shapes and metadata consistency.

Raises RuntimeError if required keys are missing or have wrong shapes.

"""

Validate required scalars

missing_scalars = set(REQUIRED_BRANCH_SCALAR_KEYS) - set(branch_result.keys())

```
# P1 #8: Use engine's matrix keys for validation

missing_matrices = set(engine.REQUIRED_BRANCH_MATRIX_KEYS) - set(branch_result.keys())



# P1 #8: Validate basic shapes for key arrays

shape_errors = []

if "B_final" in branch_result:

expected_shape = (FROZEN_N, FROZEN_N)

actual_shape = branch_result["B_final"].shape

if actual_shape != expected_shape:

shape_errors.append(f"B_final shape {actual_shape} != {expected_shape}")



# Validate trajectory lengths

expected_samples = (FROZEN_FUTURE_STEPS + 50 - 1) // 50 # diagnostic_stride=50

for key in ("sample_times", "density_trajectory", "similarity_trajectory"):

if key in branch_result:

actual_len = len(branch_result[key])
```

```
if actual_len != expected_samples:

shape_errors.append(f"{key} length {actual_len} != {expected_samples}")



# Validate horizon metadata consistency

metadata_errors = []

if "rho" in branch_result:

# Note: rho is added by simulate_seed_rho, not by engine

pass

if "T_window" in branch_result:

# T_window is added by simulate_seed_rho

pass

if "window_steps" in branch_result:

expected_window_steps = int(round(branch_result["T_window"] / FROZEN_DT))

if branch_result["window_steps"] != expected_window_steps:

metadata_errors.append(f"window_steps {branch_result['window_steps']} !=
{expected_window_steps}")

if "fixed_immediate_end" in branch_result:

if branch_result["fixed_immediate_end"] != 1750:
```

```
metadata_errors.append(f"fixed_immediate_end {branch_result['fixed_immediate_end']} != 1750")

if "fixed_recovery_end" in branch_result:

if branch_result["fixed_recovery_end"] != 8750:

metadata_errors.append(f"fixed_recovery_end {branch_result['fixed_recovery_end']} != 8750")



# Combine all errors

all_errors = []

if missing_scalars:

all_errors.extend([f"missing scalar: {k}" for k in sorted(missing_scalars)[:10]])

if missing_matrices:

all_errors.extend([f"missing matrix: {k}" for k in sorted(missing_matrices)[:10]])

if shape_errors:

all_errors.extend(shape_errors[:10])

if metadata_errors:

all_errors.extend(metadata_errors[:10])



if all_errors:
```

raise RuntimeError(

f"Branch '{branch}' has {len(all_errors)} validation errors: "

f"{all_errors[:5]}... ({len(all_errors)} total)"

)

==========
Utility Functions
==========
Note: master-seed is now type=int, no longer supports 'random' for production

def generate_seed_set(master_seed: int, count: int) -> list[int]:

text

"""Generate deterministic seed set from master seed."""

if count <= 0:

raise ValueError("seed count must be positive")

children = np.random.SeedSequence(master_seed).spawn(count)

seeds = [

int(child.generate_state(1, dtype=np.uint64)[0])

for child in children

]

if len(set(seeds)) != len(seeds):

raise RuntimeError("duplicate generated seeds")

return seeds

def file_sha256(path: str | os.PathLike[str]) -> str:

text

"""Compute SHA-256 hash of a file.

Local helper for file hashing - does not depend on engine utilities.

"""

digest = hashlib.sha256()

with open(path, "rb") as handle:

for block in iter(lambda: handle.read(1024 * 1024), b""):

digest.update(block)

return digest.hexdigest()

def source_sha256() -> str | None:

text

"""Compute SHA-256 of this source file."""

try:

```
return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

return None

def atomic_write_text(path: str | Path, text: str) -> None:

text

"""Write text atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

temporary = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temporary, "w", encoding="utf-8", newline="") as handle:

handle.write(text)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(temporary), str(path))

finally:

if temporary.exists():
```

```
temporary.unlink()

def atomic_write_json(path: str | Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically with fsync."""

text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"

atomic_write_text(path, text)

def save_npz_atomic(path: str | Path, payload: dict[str, Any]) -> None:

text

"""Save NPZ atomically with fsync.



Uses a temporary file with .npz suffix so np.savez_compressed doesn't add extra suffix.

"""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

# Use tempfile.mkstemp to create a temporary file in the same directory

fd, temporary = tempfile.mkstemp(

dir=str(path.parent),
```

```
prefix=f".{[path.name](http://path.name)}.",

suffix=".tmp.npz",

)

try:

with os.fdopen(fd, "wb") as handle:

np.savez_compressed(handle, **payload)

handle.flush()

os.fsync(handle.fileno())

os.replace(temporary, str(path))

finally:

if os.path.exists(temporary):

os.unlink(temporary)

def load_npz(path: str | Path) -> dict[str, Any]:

text

"""Load NPZ file."""

payload: dict[str, Any] = {}

with np.load(str(path), allow_pickle=False) as data:

for key in data.files:
```

```
value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

return payload

def save_csv_atomic(path: str | Path, dataframe) -> None:

text

"""Save CSV atomically with fsync."""

if pd is None:

raise RuntimeError("pandas is required for CSV output but is not installed")



path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

fd, temporary = tempfile.mkstemp(

dir=str(path.parent),

prefix=f".{[path.name](http://path.name)}.",

suffix=".tmp",

)

try:
```

with os.fdopen(fd, "w", encoding="utf-8", newline="") as handle:

dataframe.to_csv(handle, index=False)

handle.flush()

os.fsync(handle.fileno())

os.replace(temporary, str(path))

finally:

if os.path.exists(temporary):

os.unlink(temporary)

==========
Configuration and Validation
==========

def validate_engine_contract() -> None:

text

"""P1 #12: Validate that driver constants agree with engine.

Ensures driver and engine have consistent frozen production values.

"""

```
checks = {

"N": (FROZEN_N, engine.FROZEN_N),

"dt": (FROZEN_DT, engine.FROZEN_DT),

"tau": (FROZEN_TAU, engine.FROZEN_TAU),

"T_star": (FROZEN_T_STAR, engine.FROZEN_T_STAR),

"B_c": (FROZEN_B_C, engine.FROZEN_B_C),

"B_init_scale": (FROZEN_B_INIT_SCALE, engine.FROZEN_B_INIT_SCALE),

}



for name, (driver_value, engine_value) in checks.items():

if driver_value != engine_value:

raise RuntimeError(

f"Driver/engine {name} mismatch: "

f"driver={driver_value}, engine={engine_value}"

)



if not np.array_equal(FROZEN_OMEGA, engine.FROZEN_OMEGA):
```

```
raise RuntimeError(

f"Driver/engine omega mismatch: "

f"driver={FROZEN_OMEGA.tolist()}, engine={engine.FROZEN_OMEGA.tolist()}"

)



# Also check other important constants

if FROZEN_T_SIM != engine.FROZEN_T_SIM:

raise RuntimeError(f"Driver/engine T_SIM mismatch: {FROZEN_T_SIM} !=
{engine.FROZEN_T_SIM}")

if FROZEN_DELTA != engine.FROZEN_DELTA:

raise RuntimeError(f"Driver/engine DELTA mismatch: {FROZEN_DELTA} !=
{engine.FROZEN_DELTA}")

if FROZEN_ETA != engine.FROZEN_ETA:

raise RuntimeError(f"Driver/engine ETA mismatch: {FROZEN_ETA} != {engine.FROZEN_ETA}")

if FROZEN_GAMMA != engine.FROZEN_GAMMA:

raise RuntimeError(f"Driver/engine GAMMA mismatch: {FROZEN_GAMMA} !=
{engine.FROZEN_GAMMA}")

if FROZEN_MU != engine.FROZEN_MU:

raise RuntimeError(f"Driver/engine MU mismatch: {FROZEN_MU} != {engine.FROZEN_MU}")
```

```
if FROZEN_A != engine.FROZEN_A:

raise RuntimeError(f"Driver/engine A mismatch: {FROZEN_A} != {engine.FROZEN_A}")

if FROZEN_Q != engine.FROZEN_Q:

raise RuntimeError(f"Driver/engine Q mismatch: {FROZEN_Q} != {engine.FROZEN_Q}")

def create_model_config(rho: float) -> engine.ModelConfig:

text

"""Create ModelConfig for a given rho value."""

T_window = rho * FROZEN_T_STAR

return engine.ModelConfig(

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,
```

```
gamma=FROZEN_GAMMA,

T_window=T_window,

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=1e-12,

)

def create_protocol_config() -> engine.ProtocolConfig:

text

"""Create ProtocolConfig matching frozen production."""

return engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=STUDY_BRANCHES,

)
```

```
================================================================================
Simulation Functions
================================================================================

def simulate_seed_rho(

text

seed: int,

rho: float,

config: engine.ModelConfig,

protocol: engine.ProtocolConfig,

study_branches: tuple[str, ...],

) -> dict[str, Any]:

text

"""Simulate a single (seed, rho) combination.



CRITICAL: Uses FROZEN_BRANCH_ORDER for RNG namespace, but only simulates

study_branches subset. This ensures future_shuffle gets the 7th RNG sequence

(not the 4th) as in production.
```

```
Returns complete results for all branches at this (seed, rho).

"""

# Create branch RNGs using FULL frozen branch order for correct namespace

branch_rngs = engine.make_branch_rngs(seed, FROZEN_BRANCH_ORDER)



# Create a protocol with just the study branches

study_protocol = engine.ProtocolConfig(

T_sim=protocol.T_sim,

burn_end_time=protocol.burn_end_time,

split_time=protocol.split_time,

future_duration=protocol.future_duration,

diagnostic_stride=protocol.diagnostic_stride,

branches=study_branches,

)



# Simulate prefix

split_state, prefix = engine.simulate_prefix(seed, config, study_protocol)
```

```
# Simulate only study branches, using RNGs from frozen namespace

branch_results = {}

for branch in study_branches:

if branch not in branch_rngs:

raise RuntimeError(

f"Branch '{branch}' not in FROZEN_BRANCH_ORDER. "

f"Cannot map to production RNG namespace."

)

rng = branch_rngs[branch]

branch_results[branch] = engine.simulate_future(

split_state.clone(),

prefix,

branch,

config,

study_protocol,

rng,
```

```
)

# P1 #9: Validate branch outputs immediately after simulation
# Check both primary and secondary outcomes for finiteness
for key in PRIMARY_OUTCOMES + SECONDARY_OUTCOMES:
if key not in branch_results[branch]:
raise RuntimeError(
f"schema: {branch} missing {key}"
)
if not is_finite_number(branch_results[branch][key]):
raise RuntimeError(
f"schema: {branch} has non-finite {key}={branch_results[branch][key]}"
)

# Add metadata
branch_results[branch]["T_window"] = config.T_window
branch_results[branch]["rho"] = rho
```

```
return {

"seed": int(seed),

"rho": rho,

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"T_window": config.T_window,

"alpha": config.alpha,

"mean_omega": config.mean_omega,

"T_star": config.T_star,

},

"protocol": {

"T_sim": study_protocol.T_sim,

"burn_end_time": study_protocol.burn_end_time,

"split_time": study_protocol.split_time,

"future_duration": study_protocol.future_duration,
```

},

"prefix": prefix,

"branches": branch_results,

}

def run_baseline_equivalence(

text

seeds: list[int],

frozen_manifest_path: str,

frozen_npz_path: str,

frozen_csv_path: str,

) -> dict[str, Any]:

text

"""Run baseline equivalence test at rho=1 (T_window = T_*).

MANDATORY: Must reproduce frozen run exactly for ALL seeds.

Verifies:

1. Seed list matches frozen manifest exactly (same seeds, same order)

2. All 40 seeds pass equivalence (if 40 seeds provided)

3. Engine produces all required outputs

This helper ONLY performs equivalence and returns a result.

Main handles manifests and output paths.

"""

import json

Load frozen manifest to compare seeds

try:

with open(frozen_manifest_path, 'r', encoding='utf-8') as f:

frozen_manifest = json.load(f)

except FileNotFoundError:

return {

"status": "failed",

"error": f"Frozen manifest not found: {frozen_manifest_path}",

"seeds_match": False,

}

except json.JSONDecodeError as e:

return {

"status": "failed",

"error": f"Frozen manifest is not valid JSON: {frozen_manifest_path} ({e})",

"seeds_match": False,

}

Validate manifest has required structure

if "randomization" not in frozen_manifest or "paired_seeds" not in frozen_manifest["randomization"]:

return {

"status": "failed",

"error": f"Frozen manifest missing required structure: {frozen_manifest_path}",

"seeds_match": False,

}

```
frozen_seeds = frozen_manifest["randomization"]["paired_seeds"]



# Verify seed list matches frozen manifest EXACTLY

# This is critical: baseline equivalence tests a specific cohort

if len(seeds) != len(frozen_seeds):

return {

"status": "failed",

"error": f"Seed count mismatch: generated={len(seeds)}, frozen={len(frozen_seeds)}",

"frozen_seeds": [int(s) for s in frozen_seeds],

"generated_seeds": [int(s) for s in seeds],

"seeds_match": False,

}



for i, (gen_seed, frozen_seed) in enumerate(zip(seeds, frozen_seeds)):

if int(gen_seed) != int(frozen_seed):

return {

"status": "failed",
```

```
"error": f"Seed mismatch at index {i}: generated={gen_seed}, frozen={frozen_seed}",

"frozen_seeds": [int(s) for s in frozen_seeds],

"generated_seeds": [int(s) for s in seeds],

"seeds_match": False,

}



# Verify master seed matches

frozen_master_seed = frozen_manifest["randomization"]["master_seed"]

# Note: We can't check the generated master seed here as it's not passed,

# but the seed generation method should match



# Validate frozen CSV path exists

if not os.path.exists(frozen_csv_path):

return {

"status": "failed",

"error": f"Frozen scalar CSV not found: {frozen_csv_path}",

"seeds_match": True,
```

```
}



# Create config for rho=1

config = create_model_config(rho=1.0)



# For baseline equivalence, we need to validate ALL 7 branches

# to certify the engine as reusable infrastructure

baseline_protocol = engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,

burn_end_time=160.0,

split_time=480.0,

future_duration=320.0,

diagnostic_stride=50,

branches=FROZEN_BRANCH_ORDER, # All 7 branches for engine certification

)



# Run the engine's baseline equivalence test with ALL seeds and ALL 7 branches
```

```
results = [engine.run](http://engine.run)_baseline_equivalence(
seeds=seeds,
config=config,
protocol=baseline_protocol,
frozen_manifest_path=frozen_manifest_path,
frozen_npz_path=frozen_npz_path,
frozen_csv_path=frozen_csv_path,
branch_order=FROZEN_BRANCH_ORDER, # Use FULL branch order for RNG namespace
)


# Add seed comparison info
results["seed_comparison"] = {
"generated_seeds": [int(s) for s in seeds],
"frozen_seeds": [int(s) for s in frozen_seeds],
"seeds_match": True,
}
```

```
return results

# ============================================================================
# Manifest and Output
# ============================================================================

def build_manifest(

text

master_seed: int,

seeds: list[int],

rho_values: tuple[float, ...],

) -> dict[str, Any]:

text

"""Build study manifest."""

# Compute config hash from model parameters

model_params = {

"N": FROZEN_N,

"dt": FROZEN_DT,

"tau": FROZEN_TAU,

"omega": FROZEN_OMEGA.tolist(),
```

"a": FROZEN_A,

"q": FROZEN_Q,

"delta": FROZEN_DELTA,

"eta": FROZEN_ETA,

"gamma": FROZEN_GAMMA,

"mu": FROZEN_MU,

"B_c": FROZEN_B_C,

"B_init_scale": FROZEN_B_INIT_SCALE,

"T_star": FROZEN_T_STAR,

}

protocol_params = {

"T_sim": FROZEN_T_SIM,

"burn_end_time": 160.0,

"split_time": 480.0,

"future_duration": 320.0,

"diagnostic_stride": 50,

```
"branches": list(STUDY_BRANCHES),

}



study_params = {

"rho_values": list(rho_values),

"T_window_values": [rho * FROZEN_T_STAR for rho in rho_values],

"n_seeds": len(seeds),

"statistical_unit": "40 seed/graphs",

}



# Add dependencies - engine and driver source hashes

dependencies = {}



# Engine source hash

try:

engine_path = Path(engine.__file__).resolve()

engine_hash = file_sha256(engine_path)
```

```
dependencies["aief_experiment_engine_v1"] = {
"path": str(engine_path),
"sha256": engine_hash,
}
except Exception as e:
dependencies["aief_experiment_engine_v1"] = {
"path": "NOT_FOUND",
"sha256": "NOT_AVAILABLE",
"error": str(e),
}


# Driver source hash (this file)
try:
driver_path = Path(__file__).resolve()
driver_hash = file_sha256(driver_path)
dependencies["AIEF_1_4_Twindow_sensitivity"] = {
"path": str(driver_path),
```

```
"sha256": driver_hash,

}

except Exception as e:

dependencies["AIEF_1_4_Twindow_sensitivity"] = {

"path": "NOT_FOUND",

"sha256": "NOT_AVAILABLE",

"error": str(e),

}



manifest = {

"study_name": STUDY_NAME,

"workflow_version": WORKFLOW_VERSION,

"source_sha256": source_sha256(),

"dependencies": dependencies,

"model": model_params,

"protocol": protocol_params,

"study": study_params,
```

```
"randomization": {
"master_seed": int(master_seed),
"paired_seeds": [int(seed) for seed in seeds],
"seed_generation": "SeedSequence(master_seed).spawn(n_seeds)",
"branch_generation": "second per-seed child sequence spawns named uint64 branch seeds",
"bit_generator": np.random.default_rng(0).bit_generator.__class__.__name__,
# P1 #14: Record the full RNG namespace explicitly
"branch_rng_namespace": list(FROZEN_BRANCH_ORDER),
"executed_branches": list(STUDY_BRANCHES),
},
"environment": {
"python": platform.python_version(),
"numpy": np.__version__,
"platform": platform.platform(),
},
"schema": {
"prefix_keys": list(REQUIRED_PREFIX_KEYS),
"branch_scalar_keys": list(BRANCH_SCALAR_KEYS),
```

"branch_matrix_keys": list(BRANCH_MATRIX_KEYS),

"required_branch_scalar_keys": list(REQUIRED_BRANCH_SCALAR_KEYS),

"required_branch_matrix_keys": list(REQUIRED_BRANCH_MATRIX_KEYS),

},

"notes": {

"purpose": "T_window sensitivity study - varying T_window = rho *T_*",

"T_*": FROZEN_T_STAR,

"fixed_physical_horizons": {

"immediate": f"0-{PHYSICAL_IMMEDIATE_END} time units",

"recovery": f"{PHYSICAL_IMMEDIATE_END}-{PHYSICAL_RECOVERY_END} time units",

"late": f"{PHYSICAL_RECOVERY_END}-{PHYSICAL_FUTURE_END} time units",

},

"ema_normalized_horizons": {

"immediate": "0-1 T_window",

"recovery": "1-5 T_window",

"late": "5 T_window to end",

},

"full_regeneration": "Each (seed, rho) pair is independently initialized",

},

}

return manifest

def flatten_result(

text

seed: int,

rho: float,

result: dict[str, Any],

) -> dict[str, Any]:

text

"""Flatten a (seed, rho) result into a flat dict for NPZ storage."""

payload: dict[str, Any] = {

f"seed_{seed}_rho_{rho}_config_T_window": result["config"]["T_window"],

f"seed_{seed}_rho_{rho}_config_alpha": result["config"]["alpha"],

}

```
# Add prefix keys

for key, value in result["prefix"].items():

payload[f"prefix_seed_{seed}_rho_{rho}_{key}"] = value



# Add branch results

for branch, branch_result in result["branches"].items():

for key, value in branch_result.items():

if isinstance(value, np.ndarray):

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value

else:

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value



return payload

def extract_scalar_row(

text

seed: int,
```

```
seed_index: int,

rho: float,

branch: str,

branch_result: dict[str, Any],

) -> dict[str, Any]:

text

"""Extract a scalar row for CSV output.



Validates that all required outputs are present before extraction.

"""

# Validate required outputs

validate_branch_outputs(branch, branch_result)



row = {

"seed_index": seed_index,

"seed": seed,

"rho": rho,
```

"T_window": branch_result.get("T_window", float('nan')),

"branch": branch,

}

for key in BRANCH_SCALAR_KEYS:

if key in branch_result:

row[key] = branch_result[key]

else:

row[key] = float('nan')

return row

==
Main Execution
==

def parse_arguments() -> argparse.Namespace:

text

"""Parse command line arguments."""

parser = argparse.ArgumentParser(

```
description=__doc__,
formatter_class=argparse.RawDescriptionHelpFormatter,
)
parser.add_argument(
"--master-seed",
type=int,
default=DEFAULT_MASTER_SEED,
help="fixed integer master seed (default: 20260714). For production, must match frozen data.",
)
parser.add_argument(
"--n-seeds",
type=int,
default=DEFAULT_N_SEEDS,
help=f"number of seeds (default: {DEFAULT_N_SEEDS})",
)
parser.add_argument(
"--run-root",
```

default=DEFAULT_RUN_ROOT,

help=f"output root directory (default: {DEFAULT_RUN_ROOT})",

)

parser.add_argument(

"--frozen-manifest",

default=None,

help="path to frozen manifest for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-npz",

default=None,

help="path to frozen NPZ for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-scalar",

default=None,

help="path to frozen scalar CSV for baseline equivalence test. "

"MANDATORY for scalar validation. Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--overwrite",

action="store_true",

help="overwrite existing run directory. Use with caution - will delete existing data.",

)

NOTE: --skip-baseline removed for production safety

Baseline equivalence is MANDATORY and cannot be skipped

For debugging, use a separate nonproduction namespace

parser.add_argument(

"--dry-run",

action="store_true",

help="validate configuration and exit without running",

```
)

parser.add_argument(

"--baseline-only",

action="store_true",

help=(

"run and save the mandatory frozen-baseline "

"certificate, then exit without launching the grid"

),

)

return parser.parse_args()

def main() -> None:

text

"""Main entry point."""

args = parse_arguments()



# Generate seeds

seeds = generate_seed_set(int(args.master_seed), args.n_seeds)
```

```
# Build manifest

manifest = build_manifest(

master_seed=int(args.master_seed),

seeds=seeds,

rho_values=RHO_VALUES,

)



# Compute config hash

manifest_text = json.dumps(

manifest, sort_keys=True, separators=(",", ":"), allow_nan=False

)

config_hash = hashlib.sha256(manifest_text.encode()).hexdigest()[:12]



# P1 #12: Validate driver/engine contract

try:

validate_engine_contract()
```

```
except RuntimeError as e:

print(f"\nFATAL: Driver/engine contract violation: {e}")

sys.exit(1)



# P1 #10: Validate mandatory input paths before creating/deleting run directory

# This prevents contaminating a production root after a simple path error

for path_name, path_value in [

("frozen-manifest", args.frozen_manifest),

("frozen-npz", args.frozen_npz),

("frozen-scalar", args.frozen_scalar),

]:

if path_value is None:

print(f"\n{path_name} path not provided!")

print("All frozen baseline artifacts are required for production runs.")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then provide all paths with --frozen-manifest, --frozen-npz, --frozen-scalar")
```

```
sys.exit(1)

if not os.path.isfile(path_value):

print(f"\n{path_name} file not found: {path_value}")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path.")

sys.exit(1)



# Check dry-run mode FIRST to avoid contaminating production directory

if args.dry_run:

# P1 #12: Validate driver/engine contract first

try:

validate_engine_contract()

except RuntimeError as e:

print(f"\nDry run: Driver/engine contract violation: {e}")

sys.exit(1)
```

P1 #11: Dry-run should actually validate configuration

Validate all levels

protocol_config = create_protocol_config()

try:

Validate base config (rho=1.0)

base_config = create_model_config(1.0)

base_config.validate()

protocol_config.validate(base_config)

Validate all rho configs

for rho in RHO_VALUES:

config = create_model_config(rho)

config.validate()

protocol_config.validate(config)

Validate study design

if not set(STUDY_BRANCHES) <= set(FROZEN_BRANCH_ORDER):

raise ValueError(f"STUDY_BRANCHES {STUDY_BRANCHES} not subset of FROZEN_BRANCH_ORDER {FROZEN_BRANCH_ORDER}")

```
if len(STUDY_BRANCHES) != len(set(STUDY_BRANCHES)):

raise ValueError(f"STUDY_BRANCHES has duplicates: {STUDY_BRANCHES}")

if len(RHO_VALUES) != len(set(RHO_VALUES)):

raise ValueError(f"RHO_VALUES has duplicates: {RHO_VALUES}")

if 1.0 not in RHO_VALUES:

raise ValueError("RHO_VALUES must include 1.0 for baseline equivalence")


print("Dry run: configuration validated successfully.")

print(f" Validated {len(RHO_VALUES)} rho values: {RHO_VALUES}")

print(f" Validated {len(STUDY_BRANCHES)} study branches: {STUDY_BRANCHES}")

print(f" Validated {args.n_seeds} seeds from master seed {args.master_seed}")

print(f"Configuration hash: {config_hash}")

print("To execute, run without --dry-run")

print(f"Expected run directory: {Path([args.run](http://args.run)_root) / config_hash}")

except Exception as e:

print(f"Dry run: configuration validation FAILED: {e}")

sys.exit(1)

return
```

Create run directory

run_dir = Path(args.run_root) / config_hash

Check if run directory already exists

if run_dir.exists():

if args.overwrite:

Overwrite mode: remove existing directory

print(f"Overwriting existing run directory: {run_dir}")

try:

shutil.rmtree(run_dir)

except Exception as e:

print(f"Cannot overwrite: error removing directory: {e}")

sys.exit(1)

run_dir.mkdir(parents=True, exist_ok=True)

else:

Default: fail if directory exists

```
print(f"Run directory {run_dir} already exists")

print("Use --overwrite to replace")

sys.exit(1)

else:

# New run directory

run_dir.mkdir(parents=True, exist_ok=True)



# Save manifest

manifest["config_hash"] = config_hash

manifest_path = run_dir / "manifest.json"

atomic_write_json(manifest_path, manifest)



print(f"{STUDY_NAME}")

print(f" master seed: {args.master_seed}")

print(f" n_seeds: {args.n_seeds}")

print(f" rho values: {RHO_VALUES}")

print(f" configuration: {config_hash}")
```

```
print(f" output: {run_dir}")

print()



# Create protocol config (same for all rho)

protocol = create_protocol_config()



# MANDATORY: Run baseline equivalence test at rho=1

# This cannot be skipped - baseline equivalence is required for freezeability

print("=" * 80)

print("Running MANDATORY baseline equivalence test at rho=1...")

print("=" * 80)



# Validate all frozen paths are provided (MANDATORY)

for path_name, path_value in [

("frozen-manifest", args.frozen_manifest),

("frozen-npz", args.frozen_npz),

("frozen-scalar", args.frozen_scalar),
```

]:

if path_value is None:

print(f"\n{path_name} path not provided!")

print("All frozen baseline artifacts are required for production runs.")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40")

print("Then provide all paths with --frozen-manifest, --frozen-npz, --frozen-scalar")

sys.exit(1)

if not os.path.isfile(path_value):

print(f"\n{path_name} file not found: {path_value}")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path.")

sys.exit(1)

Run baseline equivalence - MANDATORY for all seeds

```
try:
baseline_results = run_baseline_equivalence(
seeds=seeds,
frozen_manifest_path=args.frozen_manifest,
frozen_npz_path=args.frozen_npz,
frozen_csv_path=args.frozen_scalar,
)
except Exception as exc:
# Handle baseline equivalence exception - save failure manifest
failure_manifest = {
**manifest,
"config_hash": config_hash,
"status": "FAILED",
"failure_reason": "baseline_equivalence_exception",
"exception_type": type(exc).__name__,
"exception_message": str(exc),
"freezeable": False,
}
```

```
failure_manifest_path = run_dir / "FAILED_manifest.json"

atomic_write_json(failure_manifest_path, failure_manifest)

print(f"\nBaseline equivalence exception occurred!")

print(f"Failure manifest saved to {failure_manifest_path}")

print(f"Exception: {type(exc).__name__}: {exc}")

sys.exit(1)



if baseline_results["status"] != "passed":

error_msg = baseline_results.get('error', 'unknown')

print(f"\nBaseline equivalence test FAILED: {error_msg}")


# Check if it's a configuration mismatch

if "mismatch" in error_msg.lower():

print("\nConfiguration mismatch detected!")

print("The frozen data does not match the expected production configuration.")

print("Expected configuration (from
AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py)):")

print(f" N={FROZEN_N}, DT={FROZEN_DT}, TAU={FROZEN_TAU}")
```

```
print(f" OMEGA={FROZEN_OMEGA.tolist()}")

print(f" T_WINDOW={FROZEN_T_STAR:.6f}")

print("\nTo generate correct frozen data:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path with --frozen-manifest and --frozen-npz")



# Save failure manifest with detailed results

failure_manifest = {

**manifest,

"config_hash": config_hash,

"status": "FAILED",

"failure_reason": "baseline_equivalence_failed",

"failure_details": baseline_results,

"freezeable": False,

}

failure_manifest_path = run_dir / "FAILED_manifest.json"

atomic_write_json(failure_manifest_path, failure_manifest)
```

```
# Also save detailed per-seed results

if "seed_results" in baseline_results:

baseline_eq_path = run_dir / "baseline_equivalence_by_seed.json"

atomic_write_json(baseline_eq_path, baseline_results["seed_results"])

print(f" Per-seed results saved to {baseline_eq_path}")


print(f"\nFailure manifest saved to {failure_manifest_path}")


sys.exit(1)

else:

print("\nBaseline equivalence test PASSED")

print("=" * 80)


# Save detailed baseline equivalence certificate

baseline_cert_path = run_dir / "baseline_equivalence_certificate.json"

atomic_write_json(baseline_cert_path, baseline_results)

print(f" Detailed certificate saved to {baseline_cert_path}")


# Save per-seed baseline results separately for easier audit
```

```
if "seed_results" in baseline_results:

baseline_seed_path = run_dir / "baseline_equivalence_by_seed.json"

atomic_write_json(baseline_seed_path, baseline_results["seed_results"])

print(f" Per-seed results saved to {baseline_seed_path}")


# P0 #3: Update manifest with baseline equivalence certificate

# Record all frozen input hashes using CORRECT engine key names

# Engine returns: frozen_manifest_file_sha256, frozen_npz_file_sha256,
frozen_scalar_csv_file_sha256

# Engine also returns: frozen_manifest, frozen_npz, frozen_manifest_hash


frozen_input_hashes = {}


# Get frozen manifest file hash (engine key: frozen_manifest_file_sha256)

if "frozen_manifest_file_sha256" in baseline_results:

frozen_input_hashes["manifest_file_sha256"] = baseline_results["frozen_manifest_file_sha256"]

if "frozen_manifest" in baseline_results:

frozen_input_hashes["manifest_path"] = baseline_results["frozen_manifest"]
```

```
# Get frozen NPZ file hash (engine key: frozen_npz_file_sha256)
if "frozen_npz_file_sha256" in baseline_results:
frozen_input_hashes["npz_file_sha256"] = baseline_results["frozen_npz_file_sha256"]
if "frozen_npz" in baseline_results:
frozen_input_hashes["npz_path"] = baseline_results["frozen_npz"]

# Get frozen scalar CSV file hash (engine key: frozen_scalar_csv_file_sha256)
if "frozen_scalar_csv_file_sha256" in baseline_results:
frozen_input_hashes["scalar_csv_file_sha256"] = baseline_results["frozen_scalar_csv_file_sha256"]

# Get canonical JSON hash of frozen manifest (engine key: frozen_manifest_hash)
if "frozen_manifest_hash" in baseline_results:
frozen_input_hashes["manifest_canonical_json_sha256"] = baseline_results["frozen_manifest_hash"]

# Read frozen production source hash from frozen manifest itself
# This is the proper way - don't search for production source files
try:
with open(args.frozen_manifest, 'r', encoding='utf-8') as f:
frozen_manifest_data = json.load(f)
```

```
if "source_sha256" in frozen_manifest_data:
frozen_input_hashes["frozen_production_source"] = frozen_manifest_data["source_sha256"]
except Exception:
pass

# Also include scalar CSV path for completeness
if args.frozen_scalar:
frozen_input_hashes["scalar_csv_path"] = str(Path(args.frozen_scalar).resolve())

manifest["baseline_equivalence"] = {
"status": "passed",
"n_seeds_tested": baseline_results["n_seeds"],
"n_seeds_passed": baseline_results["n_seeds"],
"engine_source_sha256": baseline_results.get("engine_source_sha256"),
"frozen_manifest_hash": baseline_results.get("frozen_manifest_hash"),
"frozen_npz_hash": baseline_results.get("frozen_npz_file_sha256"),
"frozen_csv_hash": baseline_results.get("frozen_scalar_csv_file_sha256", "N/A"),
"frozen_production_source_hash": frozen_input_hashes.get("frozen_production_source", "N/A"),
```

```
"frozen_input_hashes": frozen_input_hashes,

"certificate_path": str(baseline_cert_path),

"seed_results_path": str(baseline_seed_path) if "seed_results" in baseline_results else "N/A",

"branches_validated": list(FROZEN_BRANCH_ORDER), # All 7 branches

}


# Re-save manifest with baseline certificate

atomic_write_json(manifest_path, manifest)


# P0 #2: Handle --baseline-only mode

if args.baseline_only:

manifest["execution_mode"] = "baseline_only"

manifest["baseline_certified"] = True

manifest["freezeable"] = False

manifest["completion"] = {

"status": "baseline_only_completed",

"study_grid_started": False,

"baseline_passed": True,
```

```
}

manifest["output_hashes"] = {

"baseline_certificate": file_sha256(baseline_cert_path),

"baseline_seed_results": file_sha256(baseline_seed_path) if "seed_results" in baseline_results else "N/A",

}

atomic_write_json(manifest_path, manifest)

print("\nBaseline-only run completed successfully.")

print("The T_window grid was not started.")

return



# Run the study

print("\nRunning T_window sensitivity study...")

print(f" rho values: {RHO_VALUES}")

print(f" branches: {STUDY_BRANCHES}")

print(f" seeds: {len(seeds)}")
```

all_scalar_rows: list[dict[str, Any]] = []

all_matrix_payload: dict[str, Any] = {} # Full payload in memory; checkpoints also written per-condition

Track condition status

condition_status = []

successful_conditions = 0

failed_conditions = 0

started = time.time()

total_combinations = len(seeds) * len(RHO_VALUES)

Fatal error flag for breaking outer loop on schema/config mismatch

fatal_error = False

for seed_index, seed in enumerate(seeds):

Check for fatal error from previous seed

if fatal_error:

```
print(f" Skipping seed {seed} due to fatal schema/config error")

break


for rho in RHO_VALUES:

condition_start = time.time()


# Create config for this rho

config = create_model_config(rho)


# Checkpoint path for this condition (saved for diagnostic recovery, not resume)

checkpoint_path = run_dir / f"condition_seed_{seed}_rho_{rho:.2f}.npz"


try:

# Validate config before simulation

config.validate()

protocol.validate(config)


# Run simulation

result = simulate_seed_rho(
```

```
seed=seed,

rho=rho,

config=config,

protocol=protocol,

study_branches=STUDY_BRANCHES,

)


# Extract scalar rows

for branch in STUDY_BRANCHES:

branch_result = result["branches"][branch]

row = extract_scalar_row(

seed=seed,

seed_index=seed_index,

rho=rho,

branch=branch,

branch_result=branch_result,

)

all_scalar_rows.append(row)
```

```
# Extract matrix payload

matrix_payload = flatten_result(seed, rho, result)

all_matrix_payload.update(matrix_payload)


# Save checkpoint atomically

save_npz_atomic(checkpoint_path, matrix_payload)


condition_status.append({

"seed": seed,

"rho": rho,

"status": "success",

"elapsed_seconds": time.time() - condition_start,

})

successful_conditions += 1


except Exception as e:

error_str = str(e)

condition_status.append({
```

```
"seed": seed,

"rho": rho,

"status": "failed",

"error": error_str,

"elapsed_seconds": time.time() - condition_start,

})

failed_conditions += 1

print(f" ERROR: seed={seed}, rho={rho:.2f}: {e}")


# Stop immediately on schema/config mismatch - use fatal flag to break outer loop

if "schema" in error_str.lower() or "config" in error_str.lower() or "missing" in error_str.lower():

print(f"\n FATAL: Schema/config mismatch detected - stopping immediately")

print(f" This error will affect all remaining conditions")

# Set fatal error flag to break outer loop after inner loop

fatal_error = True

break


# Status
```

```
elapsed = time.time() - started

current_combination = seed_index * len(RHO_VALUES) + list(RHO_VALUES).index(rho) + 1

status_str = f"[{current_combination}/{total_combinations}] seed={seed}, rho={rho:.2f}
({elapsed:.1f}s)"

if successful_conditions + failed_conditions == current_combination:

print(f" {status_str}")



# Save condition status table

# P1 #16: pandas is mandatory (script exits at import without it), so remove dead fallback

if condition_status:

# pandas is guaranteed to be available due to import-time check

status_df = pd.DataFrame(condition_status)

status_path = run_dir / "twindow_condition_status.csv"

save_csv_atomic(status_path, status_df)

print(f"\nCondition status saved to {status_path}")



# Record condition status in manifest

manifest["condition_status"] = {
```

"path": str(status_path),

"sha256": file_sha256(status_path),

"expected_rows": total_combinations,

"successful_rows": successful_conditions,

"failed_rows": failed_conditions,

}

Also record pandas version in environment

manifest["environment"]["pandas"] = pd.__version__

print(f" Successful: {successful_conditions}/{total_combinations}")

print(f" Failed: {failed_conditions}/{total_combinations}")

If any conditions failed, mark as non-freezeable

if failed_conditions > 0:

manifest["freezeable"] = False

manifest["failed_conditions"] = failed_conditions

atomic_write_json(manifest_path, manifest)

```
print(f"\nWARNING: {failed_conditions} conditions failed - run is NOT freezeable")



# Validate we have the expected number of scalar rows

expected_rows = total_combinations * len(STUDY_BRANCHES)

actual_rows = len(all_scalar_rows)

if actual_rows != expected_rows:

print(f"\nERROR: Expected {expected_rows} scalar rows, got {actual_rows}")

print("Run is INCOMPLETE and NOT freezeable")



# Validate primary outcomes are present and finite

missing_primary = []

for row in all_scalar_rows:

for key in PRIMARY_OUTCOMES:

if key not in row or not is_finite_number(row[key]):

missing_primary.append(f"seed={row['seed']}, rho={row['rho']}, branch={row['branch']},
key={key}")



# P1 #5: Validate secondary outcomes are present and finite
```

```
missing_secondary = []

for row in all_scalar_rows:

for key in SECONDARY_OUTCOMES:

if key not in row or not is_finite_number(row[key]):

missing_secondary.append(f"seed={row['seed']}, rho={row['rho']}, branch={row['branch']},
key={key}")



if missing_primary:

print(f"\nERROR: {len(missing_primary)} primary outcomes are missing or non-finite!")

print(f" First 5 missing: {missing_primary[:5]}")

print(" This indicates engine is not producing required outputs.")

print("Run is INCOMPLETE and NOT freezeable")



if missing_secondary:

print(f"\nERROR: {len(missing_secondary)} secondary outcomes are missing or non-finite!")

print(f" First 5 missing: {missing_secondary[:5]}")

print(" This indicates engine is not producing required outputs.")

print("Run is INCOMPLETE and NOT freezeable")
```

```
# Save outputs

csv_path = run_dir / f"{STUDY_NAME}_{config_hash}_scalar_summary.csv"

npz_path = run_dir / f"{STUDY_NAME}_{config_hash}_branch_diagnostics.npz"



# Write CSV

if all_scalar_rows:

# Sort by seed_index, rho, branch for consistent ordering

all_scalar_rows.sort(key=lambda r: (r["seed_index"], r["rho"], r["branch"]))


# Get all fieldnames

fieldnames = list(all_scalar_rows[0].keys())


temporary_csv = csv_path.with_suffix(csv_path.suffix + ".tmp")

try:

with open(temporary_csv, "w", newline="", encoding="utf-8") as handle:

writer = csv.DictWriter(handle, fieldnames=fieldnames)

writer.writeheader()
```

```
writer.writerows(all_scalar_rows)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(temporary_csv), str(csv_path))

finally:

if temporary_csv.exists():

temporary_csv.unlink()



# Write NPZ

if all_matrix_payload:

save_npz_atomic(npz_path, all_matrix_payload)



# Determine completion status

# P1 #5: Require both primary and secondary outcomes to be complete

complete = (

failed_conditions == 0

and successful_conditions == total_combinations
```

```
and len(all_scalar_rows) == expected_rows

and len(missing_primary) == 0

and len(missing_secondary) == 0

)



# Top-level freezeable flag

baseline_passed = baseline_results.get("status") == "passed"

manifest["freezeable"] = complete and baseline_passed



# Final manifest update with completion status

manifest["completion"] = {

"status": "completed" if complete else "incomplete",

"n_conditions_requested": total_combinations,

"n_conditions_successful": successful_conditions,

"n_conditions_failed": failed_conditions,

"n_scalar_rows": len(all_scalar_rows),

"n_matrix_entries": len(all_matrix_payload),
```

```
"n_missing_primary_outcomes": len(missing_primary),

"n_missing_secondary_outcomes": len(missing_secondary),

"elapsed_seconds": time.time() - started,

"freezeable": manifest["freezeable"],

}



# Add output hashes - hash ALL final outputs for complete provenance

output_hashes = {}



# Main outputs

if csv_path.exists():

output_hashes["scalar_csv"] = file_sha256(csv_path)

if npz_path.exists():

output_hashes["branch_npz"] = file_sha256(npz_path)



# Condition status

condition_status_path = run_dir / "twindow_condition_status.csv"
```

```
if condition_status_path.exists():

output_hashes["condition_status_csv"] = file_sha256(condition_status_path)



# Baseline equivalence certificate

baseline_cert_path = run_dir / "baseline_equivalence_certificate.json"

if baseline_cert_path.exists():

output_hashes["baseline_certificate"] = file_sha256(baseline_cert_path)



# Baseline per-seed results (if saved separately)

baseline_seed_path = run_dir / "baseline_equivalence_by_seed.json"

if baseline_seed_path.exists():

output_hashes["baseline_seed_results"] = file_sha256(baseline_seed_path)



if output_hashes:

manifest["output_hashes"] = output_hashes
```

```
# P1 #15: Condition checkpoints are diagnostic, non-authoritative artifacts
# The consolidated NPZ is the authority
manifest["checkpoints"] = {
"role": "diagnostic, non-authoritative",
"count": successful_conditions,
"authoritative_consolidation": str(npz_path),
}


# Re-save final manifest
atomic_write_json(manifest_path, manifest)


# Print completion status
if complete and baseline_passed:
print(f"\nRun COMPLETED successfully and is FREEZEABLE")
elif complete and not baseline_passed:
print(f"\nRun COMPLETED but baseline equivalence FAILED - NOT freezeable")
# P0 #4: Exit nonzero for non-freezeable runs
```

```
sys.exit(1)

else:

print(f"\nRun INCOMPLETE - NOT freezeable")

if failed_conditions &gt; 0:

print(f" Failed conditions: {failed_conditions}/{total_combinations}")

if len(all_scalar_rows) != expected_rows:

print(f" Missing scalar rows: {expected_rows - len(all_scalar_rows)}")

if len(missing_primary) &gt; 0:

print(f" Missing primary outcomes: {len(missing_primary)}")

# P0 #4: Exit nonzero for non-freezeable runs

sys.exit(1)



# Print summary

elapsed = time.time() - started

print(f"\nCompleted in {elapsed:.1f}s")

# P1 #17: Print conditionally - only if files exist

if csv_path.exists():
```

```
print(f"Saved {csv_path}")

if npz_path.exists():

print(f"Saved {npz_path}")

print(f"Manifest: {manifest_path}")

# Print top-level freezeable flag

freezeable_status = manifest.get("freezeable", False)

print(f"Freezeable: {freezeable_status}")



# Quick summary statistics

print("\n" + "=" * 80)

print("Summary Statistics by (rho, branch)")

print("=" * 80)



# Use the missing_primary count already computed earlier

if missing_primary:

print(f"WARNING: {len(missing_primary)} primary outcomes are non-finite or missing!")

print(f" First 5 missing: {missing_primary[:5]}")
```

```
print(" This indicates engine is not producing required outputs.")



# Group by rho and branch

for rho in RHO_VALUES:

for branch in STUDY_BRANCHES:

rows = [r for r in all_scalar_rows if r["rho"] == rho and r["branch"] == branch]

if rows:

# Only show keys that are actually present and not all NaN

available_keys = [k for k in PRIMARY_OUTCOMES if any(

k in r and not (isinstance(r[k], float) and np.isnan(r[k]))

for r in rows

)]


if not available_keys:

print(f" rho={rho:.2f}, branch={branch:&lt;15} | ALL PRIMARY OUTCOMES MISSING/NaN!")

continue


stats = {}
```

for key in available_keys:

values = [r[key] for r in rows if key in r and isinstance(r[key], (int, float)) and not np.isnan(r[key])]

if values:

Use sample SD (ddof=1) for descriptive statistics

stats[key] = f"{np.mean(values):.4f}±{np.std(values, ddof=1):.4f}"

else:

stats[key] = "N/A"

print(f" rho={rho:.2f}, branch={branch:<15} | " +

" | ".join(f"{k}={stats.get(k, 'N/A')}" for k in available_keys))

if name == "main":

text

main()

Ignoring paste corruption, this driver is much improved. The major prior blockers are fixed:

file_sha256() now exists;
--baseline-only is restored;
frozen input hash names now match the engine;
normalized secondary outcomes are checked;
incomplete runs exit nonzero;
always-NaN desired fields were removed;
inputs are checked before run-directory creation;
the RNG namespace is recorded;
checkpoints are explicitly non-authoritative.

Verdict
Mandatory baseline-only run

GO, contingent on the remaining engine corrections from the previous review.

I do not see a driver defect that should deterministically prevent the correct 40-seed, seven-branch baseline-only run.
Full five-level grid

Near-GO, but I recommend correcting the schema/provenance issues below first, so the source hash that passes the baseline is the source used for the grid.
P0 before the full grid
1. Dry-run incorrectly requires all frozen artifact paths

The driver validates:

text

--frozen-manifest
--frozen-npz
--frozen-scalar

before reaching:

Python

if args.dry_run:

Therefore:

Bash

python AIEF_1_4_Twindow_sensitivity.py --dry-run

fails unless all baseline paths are supplied, even though the help text says dry-run validates configuration.
Fix

Move the dry-run block before mandatory frozen-path validation. The order should be:

parse arguments;
generate seeds;
validate driver/engine contract;
construct and validate every model/protocol configuration;
if --dry-run, print and return;
otherwise require the three frozen files;
create or delete the run directory.

The current dry-run also calls validate_engine_contract() twice. Keep only one call.

If you intentionally want dry-run to validate baseline paths, rename or document it as a production preflight. The current behavior and help text disagree.
2. The manifest matrix schema contradicts actual validation and output

validate_branch_outputs() correctly requires:

Python

engine.REQUIRED_BRANCH_MATRIX_KEYS

but the manifest still records:

Python

BRANCH_MATRIX_KEYS = REQUIRED_BRANCH_MATRIX_KEYS

where the local tuple contains only the eleven historical fields.

The actual NPZ contains the extended matrices because flatten_result() writes every branch-result item, but the manifest claims a smaller schema.
Fix

Separate frozen and study schemas:

Python

```
FROZEN_BRANCH_MATRIX_KEYS = (
"B_final",
"B_mean_late",
"P_full",
"P_immediate",
"P_recovery",
"P_late",
"R_physical_future",
"R_driver_future",
"sample_times",
"density_trajectory",
"similarity_trajectory",
)

REQUIRED_BRANCH_MATRIX_KEYS = tuple(
engine.REQUIRED_BRANCH_MATRIX_KEYS
)

BRANCH_MATRIX_KEYS = REQUIRED_BRANCH_MATRIX_KEYS
```

Then the manifest truthfully records the validated output contract.

Also validate at import/preflight that the frozen eleven-key set is a subset of the engine schema.
3. Metadata validation does not actually validate rho or T_window

This code does nothing:

Python

```
if "rho" in branch_result:
pass
if "T_window" in branch_result:
pass
```

The function cannot validate them because it is not given their expected values.

Change the signature:

Python

```
def validate_branch_outputs(
branch: str,
branch_result: dict[str, Any],
expected_rho: float,
expected_T_window: float,
) -> None:
```

Then require:

Python

```
if "rho" not in branch_result:
metadata_errors.append("missing rho")
elif float(branch_result["rho"]) != float(expected_rho):
metadata_errors.append(
f"rho={branch_result['rho']} != {expected_rho}"
)

if "T_window" not in branch_result:
metadata_errors.append("missing T_window")
elif float(branch_result["T_window"]) != float(
expected_T_window
):
metadata_errors.append(
f"T_window={branch_result['T_window']} "
f"!= {expected_T_window}"
)
```

Call it from extract_scalar_row() with rho and the expected condition value, or validate before extraction.
4. recovery_end_ema is not validated

The driver validates:

text

window_steps
fixed_immediate_end
fixed_recovery_end

but not:

text

recovery_end_ema

Add:

Python

expected_recovery_end = min(
5 * expected_window_steps,
FROZEN_FUTURE_STEPS,
)

if branch_result.get("recovery_end_ema") != expected_recovery_end:
metadata_errors.append(
f"recovery_end_ema="
f"{branch_result.get('recovery_end_ema')} "
f"!= {expected_recovery_end}"
)

This metadata is central to the normalized-EMA horizon definition.
5. Prefix outputs are declared but never validated

REQUIRED_PREFIX_KEYS appears in the manifest, but simulate_seed_rho() does not verify the returned prefix.

Add:

Python

```
def validate_prefix_outputs(
prefix: dict[str, Any],
) -> None:
missing = set(REQUIRED_PREFIX_KEYS) - set(prefix)
if missing:
raise RuntimeError(
f"schema: prefix missing {sorted(missing)}"
)


expected_shapes = {
"B_at_split": (FROZEN_N, FROZEN_N),
"R_physical_past": (FROZEN_N, FROZEN_N),
"theta_at_split": (FROZEN_N,),
"coherence_acc_at_split": (
FROZEN_N,
FROZEN_N,
),
"history_at_split": (
FROZEN_DELAY_STEPS + 1,
FROZEN_N,
),
"selected_at_split": (
FROZEN_N,
FROZEN_N,
),
}


for key, expected in expected_shapes.items():
observed = np.asarray(prefix[key])
if observed.shape != expected:
raise RuntimeError(
f"schema: prefix {key} shape "
f"{observed.shape} != {expected}"
)
```

Call it immediately after engine.simulate_prefix().
P1 validation hardening
6. Matrix validation checks only B_final shape

The engine matrix contract includes many 12×1212×12 matrices. Validate all matrix-valued outputs:

Python

```
MATRIX_12X12_KEYS = (
"B_final",
"B_mean_late",
"P_full",
"P_immediate",
"P_recovery",
"P_late",
"R_physical_future",
"R_driver_future",
"B_mean_late_fixed_physical",
"P_full_fixed_physical",
"P_immediate_fixed_physical",
"P_recovery_fixed_physical",
"P_late_fixed_physical",
"B_mean_late_normalized_EMA",
"P_full_normalized_EMA",
"P_immediate_normalized_EMA",
"P_recovery_normalized_EMA",
"P_late_normalized_EMA",
)
```

Require each shape to be (12, 12) and each to be finite. The trajectory similarity array may legitimately contain NaNs after zero-weight resets, but the matrices should be finite.

Also validate:

Python

```
sample_times
density_trajectory
similarity_trajectory
```

all have the same length.

For density:

Python

```
np.all(np.isfinite(density))
np.all((density >= 0) & (density <= 1))
```

For finite similarity entries:

Python

```
finite = similarity[np.isfinite(similarity)]
np.all((finite >= -1 - tol) & (finite <= 1 + tol))
```

7. Validate the exact diagnostic-time schedule

Length 640 is necessary but not sufficient. Require:

Python

```
expected_times = (
np.arange(
0,
FROZEN_FUTURE_STEPS,
50,
dtype=float,
)
+ 1.0
) * FROZEN_DT
```

Then:

Python

```
if not np.array_equal(
np.asarray(branch_result["sample_times"]),
expected_times,
):
```

raise RuntimeError(
f"schema: {branch} sample-time schedule mismatch"
)

This certifies the 0.01 through 319.51 saved-time convention used by the recovery analysis.
8. Prevent silent NPZ key collisions

Before:

Python

all_matrix_payload.update(matrix_payload)

add:

Python

overlap = set(all_matrix_payload) & set(matrix_payload)
if overlap:
raise RuntimeError(
f"NPZ key collision: {sorted(overlap)[:10]}"
)

The current rho grid produces distinct keys, but explicit collision detection protects against future formatting or grid changes.
9. Validate the study design during normal execution too

The subset/duplicate/rho checks are currently only in the dry-run block. They should be part of a shared preflight invoked in both dry-run and normal modes.

For example:

Python

def validate_study_design() -> None:
if not set(STUDY_BRANCHES) <= set(
FROZEN_BRANCH_ORDER

):
raise ValueError(...)

if len(STUDY_BRANCHES) != len(set(STUDY_BRANCHES)):
raise ValueError(...)

if len(RHO_VALUES) != len(set(RHO_VALUES)):
raise ValueError(...)

if 1.0 not in RHO_VALUES:
raise ValueError(...)

Also validate study_protocol inside simulate_seed_rho() before simulation.
10. Strengthen the driver/engine contract

The current contract check is useful. Add:

Python

"scale_epsilon": (
1e-12,
engine.FROZEN_ZERO_TOL,
),
"delay_steps": (
FROZEN_DELAY_STEPS,
engine.FROZEN_DELAY_STEPS,
),
"window_steps": (
FROZEN_WINDOW_STEPS,
engine.FROZEN_WINDOW_STEPS,
),
"recovery_end": (
FROZEN_RECOVERY_END,
engine.FROZEN_RECOVERY_END,
),

Also require:

Python

tuple(engine.DEFAULT_PROTOCOL_CONFIG.branches)
== FROZEN_BRANCH_ORDER

This directly protects the branch-child RNG namespace.
Manifest/configuration identity
11. The configuration hash still includes absolute paths and environment

The hash is computed from the complete manifest, which contains:

text

absolute engine path
absolute driver path
Python version
NumPy version
platform string

Therefore, identical source and scientific configuration in a different directory or environment receives a different configuration hash.

Prefer a canonical identity payload:

Python

config_identity = {
"study_name": STUDY_NAME,
"workflow_version": WORKFLOW_VERSION,
"model": model_params,
"protocol": protocol_params,
"study": study_params,
"randomization": {
"master_seed": int(master_seed),
"paired_seeds": [int(s) for s in seeds],
"branch_rng_namespace": list(
FROZEN_BRANCH_ORDER
),

"executed_branches": list(STUDY_BRANCHES),
},
"source_hashes": {
"engine": engine_hash,
"driver": driver_hash,
},
}

Hash that payload only. Keep absolute paths and environment in the full provenance manifest, outside the configuration identity.

At minimum, add:

Python

"scale_epsilon": 1e-12

to model_params.

This is not a blocker for a single-machine baseline test, but it should be settled before freezing lineage.
12. Record pandas version from the start

pandas is imported and mandatory even in baseline-only mode, but its version is added only after the full condition-status table is written.

Put:

Python

"pandas": pd.__version__,

in build_manifest() immediately. That gives baseline-only manifests complete environment provenance too.
Smaller issues
13. Repeated frozen-path validation

The three frozen paths are validated before run-directory creation and then validated again immediately before baseline execution.

The second block is redundant. Remove it to simplify the control flow.
14. is_finite_number() accepts booleans

Because bool is a subclass of int, True is accepted as a finite number. Tighten it:

Python

if isinstance(value, (bool, np.bool_)):
return False

Not expected in current outputs, but easy to correct.
15. The condition failure record lacks traceback/context

Save at least:

Python

"exception_type": type(e).__name__,
"error": str(e),

Optionally include a traceback in a separate diagnostic JSON. Avoid placing extremely long tracebacks in the status CSV.
16. Incomplete summary omits secondary count

The incomplete branch prints missing primary outcomes but not missing secondary outcomes. Add:

Python

if missing_secondary:
print(
f" Missing secondary outcomes: "
f"{len(missing_secondary)}"
)

The manifest already records the count correctly.
17. Local frozen scalar naming is slightly misleading

The local FROZEN_BRANCH_SCALAR_KEYS contains the 34 condition-local historical fields, not all 42 frozen production fields. The remaining eight require cross-branch finalization.

Consider renaming it:

Python

CONDITION_LOCAL_LEGACY_SCALAR_KEYS

This avoids confusion with the engine's complete 42-field frozen schema.

Removing the legacy eight from the study CSV is scientifically defensible because the analysis regenerates richer recovery estimands from trajectories.
What now looks correct

The following important items are now properly implemented:

--baseline-only does not launch the grid;
baseline-only manifests are explicitly non-freezeable study artifacts;
all three frozen files are mandatory for real execution;
baseline checks all 40 matched seeds and all seven frozen branches;
dependency and output hashes use a defined local hash function;
actual frozen manifest/NPZ/CSV hashes are recorded under the correct engine keys;
the frozen production source hash comes from the frozen manifest;
full seven-branch RNG namespace is retained;
only four study branches are executed;
every (s,ρ)(s,ρ) prefix is regenerated independently;
primary fixed-physical outcomes are checked immediately and globally;
normalized-EMA secondary outcomes are checked immediately and globally;
partial and failed runs are non-freezeable and exit nonzero;
condition checkpoints are atomic and explicitly non-authoritative;
consolidated outputs and status files are hashed;
stale run directories are rejected unless explicitly overwritten;
always-NaN desired columns have been removed;
study metadata is separated from historical scalar fields.

Bottom line

The driver is now suitable for the planned baseline-only test, assuming the corrected engine is used. Before the full grid, align the manifest matrix schema with the engine, validate exact condition metadata and prefix outputs, and preferably stabilize the configuration hash.

The intended first execution remains:

Bash

```
python AIEF_1_4_Twindow_sensitivity.py \
--frozen-manifest runs/5a9f093a2fe0/manifest.json \
--frozen-npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--frozen-scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--baseline-only \
--run-root twindow_baseline_tests
```

Only an exact saved certificate should authorize the full grid.

And the last File:

```
#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Analysis Driver v3.

Correct implementation of the prespecified estimands:

Regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}

Branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]

Cross-T topology: S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Recovery scaling: t_rec / T_window for both horizon systems

Output:
```

twindow_paired_regime_contrasts.csv

twindow_branch_interactions.csv

twindow_cross_condition_topology.csv

twindow_recovery_summary.csv

twindow_sensitivity_report.txt

analysis_manifest.json

This script implements the 5-stage analysis structure as specified.

"""

```
from future import annotations

import argparse

import csv

import hashlib

import json

import os

import sys

import time

from pathlib import Path
```

```
from typing import Any

import numpy as np

import pandas as pd
================================================================================
Configuration
================================================================================

ANALYSIS_VERSION = "v3.0"

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"
Expected values (for validation)

EXPECTED_N_SEEDS = 40

EXPECTED_RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)

EXPECTED_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Primary outcomes - FIXED-PHYSICAL HORIZONS (corrected from legacy)

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"P_mean_late_fixed_physical",

)
Secondary outcomes - NORMALIZED EMA HORIZONS
```

SECONDARY_NORMALIZED_OUTCOMES = (

text

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"P_mean_late_normalized_EMA",

)
Expected counts

EXPECTED_N_CONDITIONS = EXPECTED_N_SEEDS * len(EXPECTED_RHO_VALUES)

EXPECTED_N_ROWS = EXPECTED_N_CONDITIONS * len(EXPECTED_BRANCHES)

EXPECTED_N_REGIME_CONTRASTS = EXPECTED_N_SEEDS
(len(EXPECTED_RHO_VALUES) - 1) len(PRIMARY_OUTCOMES)

EXPECTED_N_DID = EXPECTED_N_SEEDS (len(EXPECTED_RHO_VALUES) - 1)
(len(EXPECTED_BRANCHES) - 1) * len(PRIMARY_OUTCOMES)

EXPECTED_N_TOPOLOGY = EXPECTED_N_SEEDS * (len(EXPECTED_RHO_VALUES) - 1)
Recovery DID expected count: 40 seeds 4 non-baseline rho 3 non-continue branches 2 systems 2 metrics * 13 outcomes = 24960

EXPECTED_N_RECOVERY_DID = EXPECTED_N_SEEDS (len(EXPECTED_RHO_VALUES) - 1) len(NON_CONTINUE_BRANCHES) 2 2 * 13
Recovery groups: 5 rho 4 branches 2 systems * 2 metrics = 80 groups

EXPECTED_N_RECOVERY_GROUPS = len(EXPECTED_RHO_VALUES)
len(EXPECTED_BRANCHES) 2 * 2
Recovery DID groups: 4 rho 3 branches 2 systems 2 metrics 13 outcomes = 624 groups

EXPECTED_N_RECOVERY_DID_GROUPS = (len(EXPECTED_RHO_VALUES) - 1)
len(NON_CONTINUE_BRANCHES) 2 2 13

Censoring-aware fields that must have 40 finite values per group

RESTRICTED_RECOVERY_FIELDS = (

text

"restricted_first_crossing_time",

"restricted_sustained_time",

"restricted_held_time",

"restricted_first_over_Twindow",

"restricted_sustained_over_Twindow",

"restricted_held_over_Twindow",

"auc_deficit_vs_continue",

"first_crossing_observed",

"sustained_observed",

"held_observed",

)
Non-continue branches for DID

NON_CONTINUE_BRANCHES = ("reset_ema", "reset_B", "future_shuffle")
Matrix keys needed for cross-T topology

MATRIX_KEYS = ("B_final",)
==========
Utility Functions

```
================================================================================
====

def load_csv_atomic(path: str | Path) -> pd.DataFrame:

text

"""Load CSV file atomically with validation."""

path = Path(path)



# Use pandas with explicit dtypes

df = [pd.read](http://pd.read)_csv(

path,

dtype={"seed": str, "branch": str}, # Keep seeds as strings

)



# Convert numeric columns, coercing errors to NaN

for col in df.columns:

if col not in ("seed", "seed_index", "branch", "rho"):

try:

df[col] = [pd.to](http://pd.to)_numeric(df[col], errors="coerce")
```

except Exception:

pass

return df

def load_npz_selective(path: str | Path, keys: list[str] | None = None) -> dict[str, Any]:

text

"""Load only specified keys from NPZ file to save memory."""

path = Path(path)

payload: dict[str, Any] = {}

if not path.exists():

print(f"Warning: NPZ file not found: {path}")

return payload

try:

with np.load(str(path), allow_pickle=False) as data:

if keys is None:

keys = list(data.files)

for key in keys:

if key in data:

value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

except Exception as e:

print(f"Warning: Could not load NPZ {path}: {e}")

return payload

def save_csv_atomic(path: str | Path, df: pd.DataFrame) -> None:

text

"""Save CSV atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

df.to_csv(temp_path, index=False)

temp_path.replace(path)

fsync the directory to ensure the replace is visible

dir_fd = os.open(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

def save_text_atomic(path: str | Path, text: str) -> None:

text

"""Save text file atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

```
temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")
try:
with open(temp_path, "w", encoding="utf-8") as f:
f.write(text)
f.flush()
os.fsync(f.fileno())
temp_path.replace(path)
# fsync the directory
dir_fd = [os.open](http://os.open)(str(path.parent), os.O_RDONLY)
try:
os.fsync(dir_fd)
finally:
os.close(dir_fd)
finally:
if temp_path.exists():
temp_path.unlink()
def save_json_atomic(path: str | Path, data: dict[str, Any]) -> None:
```

text

"""Save JSON atomically with fsync, handling NaN values."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

Convert NaN and infinity to None for JSON serialization

Recursively handles nested structures including numpy arrays

def sanitize(obj: Any) -> Any:

if isinstance(obj, dict):

return {k: sanitize(v) for k, v in obj.items()}

elif isinstance(obj, list):

return [sanitize(v) for v in obj]

elif isinstance(obj, tuple):

return tuple(sanitize(v) for v in obj)

elif isinstance(obj, float):

if np.isnan(obj) or np.isinf(obj):

return None

```
return obj

elif isinstance(obj, (np.integer, np.floating)):

val = obj.item()

if isinstance(val, float) and (np.isnan(val) or np.isinf(val)):

return None

return val

elif isinstance(obj, np.ndarray):

# Convert to list and recursively sanitize to preserve structure

return sanitize(obj.tolist())

else:

return obj



temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temp_path, "w", encoding="utf-8") as f:

json.dump(sanitize(data), f, indent=2, sort_keys=True, allow_nan=False)

f.write("\n")
```

```
f.flush()

os.fsync(f.fileno())

temp_path.replace(path)

# fsync the directory

dir_fd = [os.open](http://os.open)(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

def file_sha256(path: str | Path) -> str:

text

"""Compute SHA-256 hash of a file."""

import hashlib as hl

digest = hl.sha256()

path = Path(path)
```

if not path.exists():

return "FILE_NOT_FOUND"

with open(path, "rb") as f:

for block in iter(lambda: f.read(1024 * 1024), b""):

digest.update(block)

return digest.hexdigest()

```
================================================================================
Metric Functions
================================================================================
```

def first_crossing_time(times: np.ndarray, values: np.ndarray, threshold: float) -> tuple[float, bool]:

text

"""Find first time when values cross above threshold.

Returns (first_crossing_time, observed).

"""

times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)

Handle NaN values by treating them as below threshold

above_threshold = (np.isfinite(values) & (values >= threshold))

crossing_indices = np.flatnonzero(above_threshold)

if len(crossing_indices) > 0:

return float(times[crossing_indices[0]]), True

else:

return float('nan'), False

def first_time_sustained_for_rest(times: np.ndarray, values: np.ndarray, threshold: float) -> tuple[float, bool]:

text

"""Find earliest time after which values stay above threshold for rest of trajectory.

Returns (sustained_time, observed).

"""

times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)

Handle NaN values by treating them as below threshold

condition = (np.isfinite(values) & (values >= threshold))

Check from the end backwards: find first index where all subsequent values are above threshold

suffix_all = np.logical_and.accumulate(condition[::-1])[::-1]

sustained_indices = np.flatnonzero(suffix_all)

if len(sustained_indices) > 0:

return float(times[sustained_indices[0]]), True

else:

return float('nan'), False

def first_time_held_for_duration(times: np.ndarray, values: np.ndarray, threshold: float, duration: float) -> tuple[float, bool]:

text

"""Find earliest time after which values stay above threshold for at least the specified duration.

Returns (held_time, observed).

"""

times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)

if len(times) < 2:

return float('nan'), False

Calculate sample step

sample_step = float(times[1] - times[0])

if sample_step <= 0:

return float('nan'), False

Required number of consecutive samples

required_samples = max(1, int(np.ceil(duration / sample_step)))

```
# Handle NaN values by treating them as below threshold

condition = (np.isfinite(values) &amp;amp; (values &amp;gt;= threshold))



# Use sliding window to find runs of sufficient length

held_time = float('nan')

observed = False



for i in range(len(condition) - required_samples + 1):

window = condition[i:i + required_samples]

if np.all(window):

held_time = float(times[i])

observed = True

break



return held_time, observed

def trapezoidal_auc_deficit(times: np.ndarray, reference: np.ndarray, target: np.ndarray) -> float:
```

text

"""Calculate one-sided AUC deficit: integral of max(0, reference - target) using trapezoidal rule.

Handles NaN values by treating them as zero for the deficit calculation.

Uses np.trapezoid if available, falls back to np.trapz for older NumPy versions.

NaN cosine values (from zero-vector weights) are imputed as zero for recovery

and AUC calculations only - this is a prespecified recovery-only convention.

"""

times = np.asarray(times, dtype=float)

reference = np.asarray(reference, dtype=float)

target = np.asarray(target, dtype=float)

if len(times) < 2:

return float('nan')

Calculate one-sided deficit: max(0, reference - target)

Handle NaN by replacing with 0 (prespecified recovery-only convention for undefined zero-vector cosine)

ref_safe = np.where(np.isfinite(reference), reference, 0.0)

tgt_safe = np.where(np.isfinite(target), target, 0.0)

deficit = np.maximum(0.0, ref_safe - tgt_safe)

Trapezoidal integration - use np.trapezoid if available, else np.trapz

if hasattr(np, "trapezoid"):

return float(np.trapezoid(deficit, x=times))

return float(np.trapz(deficit, x=times))

def cosine_similarity(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute cosine similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]

norm_x = np.linalg.norm(x)

norm_y = np.linalg.norm(y)

if norm_x <= 1e-12 or norm_y <= 1e-12:

return float('nan')

return float(np.dot(x, y) / (norm_x * norm_y))

def jaccard_similarity(B1: np.ndarray, B2: np.ndarray, threshold: float) -> float:

text

"""Compute Jaccard similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges] > threshold

y = B2[edges] > threshold

union = np.count_nonzero(x | y)

if union == 0:

return float('nan')

return float(np.count_nonzero(x & y) / union)

def normalized_distance(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute normalized L2 distance between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]

diff = x - y

norm_diff = np.linalg.norm(diff)

norm_y = np.linalg.norm(y)

if norm_y <= 1e-12:

return float('nan')

return float(norm_diff / norm_y)

def compute_recovery_scaling(

text

df: pd.DataFrame,

npz_path: Path,

b_c: float,

) -> pd.DataFrame:

text

"""Compute recovery scaling metrics for both horizon systems.

For each seed, rho, branch, and horizon system:

1. Load: sample_times, density_trajectory, similarity_trajectory

2. Use the same seed/rho continue branch as reference

3. Calculate condition-specific targets:

- Density target: 0.9 * D_B_final_continue (fraction of edges above B_c)

- Similarity target: 0.9 * S_late_continue (for appropriate horizon system)

4. Calculate:

- first_crossing_time: first time metric > target

- sustained_time: earliest time after which metric stays above target for rest of trajectory

- held_time: earliest time after which metric stays above target for at least T_window duration

- censoring indicators for each event type

- restricted_time: horizon for censored events, event_time for observed events

- auc_deficit: one-sided integral of (continue_trajectory - branch_trajectory) using trapezoidal rule

- t_rec / T_window: normalized recovery times

5. Output separate rows for fixed_physical and normalized_EMA horizon systems

Returns DataFrame with complete recovery metrics.

"""

Get unique seeds, rho values, and branches from the data

seeds = sorted(df["seed"].unique())

rho_values = sorted([float(r) for r in df["rho"].unique()])

branches = sorted(df["branch"].unique())

Build complete list of required keys for all conditions

We need: sample_times, density_trajectory, similarity_trajectory for all seed/rho/branch

all_keys = set()

```
for seed in seeds:

for rho in rho_values:

for branch in branches:

prefix = f"{branch}_seed_{seed}_rho_{rho}"

all_keys.update([

f"{prefix}_sample_times",

f"{prefix}_density_trajectory",

f"{prefix}_similarity_trajectory",

])



# Load all required data in one operation

print(f" Loading {len(all_keys)} trajectory arrays from NPZ...")

npz_data = load_npz_selective(npz_path, keys=list(all_keys))



# Check for missing keys - this should be fatal for production

missing_keys = [k for k in all_keys if k not in npz_data]

if missing_keys:
```

```
raise RuntimeError(

f"Missing trajectory keys in NPZ: {missing_keys[:10]}... "

f"({len(missing_keys)} total missing). "

f"This indicates schema mismatch or incomplete simulation."

)



recovery_rows = []



for seed in seeds:

for rho in rho_values:

# Get T_window for this rho from the data

t_window = float(df[(df["seed"] == seed) &amp;amp; (df["rho"] == rho)]["T_window"].iloc[0])



# Get continue branch reference values for this seed/rho

cont_row = df[(df["seed"] == seed) &amp;amp; (df["rho"] == rho) &amp;amp; (df["branch"] ==
"continue")].iloc[0]



# Get reference targets for both horizon systems
```

```
# Density target: 0.9 * D_B_final_continue (fraction of edges above B_c)

density_target_continue = 0.9 * float(cont_row["D_B_final"])


# Similarity targets for both horizon systems

similarity_target_fixed = 0.9 * float(cont_row["Weighted_similarity_late_fixed_physical"])

similarity_target_normalized = 0.9 * float(cont_row["Weighted_similarity_late_normalized_EMA"])


for branch in branches:

# INCLUDE continue branch for DID calculation

# For continue: compare trajectory with itself, AUC deficit = 0

# This makes recovery DID well-defined: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]


prefix = f"{branch}_seed_{seed}_rho_{rho}"

cont_prefix = f"continue_seed_{seed}_rho_{rho}"


# Load trajectory data

try:

sample_times = np.asarray(npz_data[f"{prefix}_sample_times"], dtype=float)

density_trajectory = np.asarray(npz_data[f"{prefix}_density_trajectory"], dtype=float)
```

```
similarity_trajectory = np.asarray(npz_data[f"{prefix}_similarity_trajectory"], dtype=float)


# Load continue (reference) trajectory data

cont_sample_times = np.asarray(npz_data[f"{cont_prefix}_sample_times"], dtype=float)

cont_density_trajectory = np.asarray(npz_data[f"{cont_prefix}_density_trajectory"], dtype=float)

cont_similarity_trajectory = np.asarray(npz_data[f"{cont_prefix}_similarity_trajectory"], dtype=float)


except Exception as e:

raise RuntimeError(f"Error loading trajectory data for seed={seed}, rho={rho}, branch={branch}:
{e}")


# Validate that sample times match between branch and continue

# Use exact equality since all trajectories use the same diagnostic schedule

if not np.array_equal(sample_times, cont_sample_times):

raise RuntimeError(

f"Sample times mismatch for seed={seed}, rho={rho}, branch={branch}: "

f"branch has {len(sample_times)} times, continue has {len(cont_sample_times)}"

)


# Also validate that trajectory lengths match
```

```
if len(density_trajectory) != len(cont_density_trajectory):
raise RuntimeError(
f"Density trajectory length mismatch for seed={seed}, rho={rho}, branch={branch}: "
f"branch has {len(density_trajectory)} points, continue has {len(cont_density_trajectory)}"
)
if len(similarity_trajectory) != len(cont_similarity_trajectory):
raise RuntimeError(
f"Similarity trajectory length mismatch for seed={seed}, rho={rho}, branch={branch}: "
f"branch has {len(similarity_trajectory)} points, continue has {len(cont_similarity_trajectory)}"
)

# Process each horizon system
for horizon_system, similarity_target in [
("fixed_physical", similarity_target_fixed),
("normalized_EMA", similarity_target_normalized),
]:
# Process density recovery
```

```
density_values = density_trajectory

density_target = density_target_continue


# Calculate first crossing for density

density_first_crossing, density_first_observed = first_crossing_time(

sample_times, density_values, density_target

)


# Calculate sustained recovery for density (stays above target for rest of trajectory)

density_sustained, density_sustained_observed = first_time_sustained_for_rest(

sample_times, density_values, density_target

)


# Calculate held recovery for density (stays above target for T_window duration)

density_held, density_held_observed = first_time_held_for_duration(

sample_times, density_values, density_target, t_window

)


# Calculate AUC deficit for density (one-sided: continue - branch)
```

```
density_auc_deficit = trapezoidal_auc_deficit(
sample_times, cont_density_trajectory, density_values
)

# Process similarity recovery
similarity_values = similarity_trajectory
similarity_target = similarity_target

# Calculate first crossing for similarity
sim_first_crossing, sim_first_observed = first_crossing_time(
sample_times, similarity_values, similarity_target
)

# Calculate sustained recovery for similarity
sim_sustained, sim_sustained_observed = first_time_sustained_for_rest(
sample_times, similarity_values, similarity_target
)

# Calculate held recovery for similarity
```

```
sim_held, sim_held_observed = first_time_held_for_duration(
sample_times, similarity_values, similarity_target, t_window
)

# Calculate AUC deficit for similarity
sim_auc_deficit = trapezoidal_auc_deficit(
sample_times, cont_similarity_trajectory, similarity_values
)

# Calculate restricted times (horizon for censored, event_time for observed)
# Use the last sample time as the horizon (not the elapsed difference)
horizon_time = float(sample_times[-1]) if len(sample_times) &gt; 0 else float('nan')

# For density metrics
density_restricted_first = density_first_crossing if density_first_observed else horizon_time
density_restricted_sustained = density_sustained if density_sustained_observed else horizon_time
density_restricted_held = density_held if density_held_observed else horizon_time

# For similarity metrics
```

sim_restricted_first = sim_first_crossing if sim_first_observed else horizon_time

sim_restricted_sustained = sim_sustained if sim_sustained_observed else horizon_time

sim_restricted_held = sim_held if sim_held_observed else horizon_time

Calculate normalized times (t_rec / T_window)

Raw observed times (conditional on recovery)

density_first_over_Twindow = density_first_crossing / t_window if np.isfinite(density_first_crossing) and t_window > 0 else float('nan')

density_sustained_over_Twindow = density_sustained / t_window if np.isfinite(density_sustained) and t_window > 0 else float('nan')

density_held_over_Twindow = density_held / t_window if np.isfinite(density_held) and t_window > 0 else float('nan')

sim_first_over_Twindow = sim_first_crossing / t_window if np.isfinite(sim_first_crossing) and t_window > 0 else float('nan')

sim_sustained_over_Twindow = sim_sustained / t_window if np.isfinite(sim_sustained) and t_window > 0 else float('nan')

sim_held_over_Twindow = sim_held / t_window if np.isfinite(sim_held) and t_window > 0 else float('nan')

Censoring-aware restricted normalized times (cohort-level estimates)

These use restricted times (horizon for censored, event_time for observed) divided by T_window

density_restricted_first_over_Twindow = density_restricted_first / t_window if np.isfinite(density_restricted_first) and t_window > 0 else float('nan')

density_restricted_sustained_over_Twindow = density_restricted_sustained / t_window if np.isfinite(density_restricted_sustained) and t_window > 0 else float('nan')

density_restricted_held_over_Twindow = density_restricted_held / t_window if np.isfinite(density_restricted_held) and t_window > 0 else float('nan')

sim_restricted_first_over_Twindow = sim_restricted_first / t_window if np.isfinite(sim_restricted_first) and t_window > 0 else float('nan')

sim_restricted_sustained_over_Twindow = sim_restricted_sustained / t_window if np.isfinite(sim_restricted_sustained) and t_window > 0 else float('nan')

sim_restricted_held_over_Twindow = sim_restricted_held / t_window if np.isfinite(sim_restricted_held) and t_window > 0 else float('nan')

Add rows for density metrics

Note: For continue branch, AUC deficit = 0 (comparing trajectory with itself)

Held duration uses T_window for both horizon systems (Option B: documented below)

Density targets are horizon-independent (same 0.9 * D_B_final_continue for both systems)

recovery_rows.append({

"seed": seed,

"rho": rho,

"T_window": t_window,

"branch": branch,

"horizon_system": horizon_system,

"metric": "density",

"target_value": density_target,

"target_source": "0.9 * D_B_final_continue",

"target_horizon_dependence": "horizon_independent",

"held_duration_definition": "one_current_T_window",

"auc_horizon": "full_future",

"first_crossing_time": density_first_crossing,

"first_crossing_observed": density_first_observed,

"sustained_time": density_sustained,

"sustained_observed": density_sustained_observed,

"held_time": density_held,

"held_observed": density_held_observed,

"restricted_first_crossing_time": density_restricted_first,

"restricted_sustained_time": density_restricted_sustained,

"restricted_held_time": density_restricted_held,

"auc_deficit_vs_continue": density_auc_deficit,

```
"first_crossing_over_Twindow": density_first_over_Twindow,

"sustained_time_over_Twindow": density_sustained_over_Twindow,

"held_time_over_Twindow": density_held_over_Twindow,

"restricted_first_over_Twindow": density_restricted_first_over_Twindow,

"restricted_sustained_over_Twindow": density_restricted_sustained_over_Twindow,

"restricted_held_over_Twindow": density_restricted_held_over_Twindow,

})


# Add rows for similarity metrics

# Similarity targets are horizon-dependent (different for fixed_physical vs normalized_EMA)

recovery_rows.append({

"seed": seed,

"rho": rho,

"T_window": t_window,

"branch": branch,

"horizon_system": horizon_system,

"metric": "similarity",
```

"target_value": similarity_target,

"target_source": f"0.9 * Weighted_similarity_late_{horizon_system}",

"target_horizon_dependence": "horizon_dependent",

"held_duration_definition": "one_current_T_window",

"auc_horizon": "full_future",

"first_crossing_time": sim_first_crossing,

"first_crossing_observed": sim_first_observed,

"sustained_time": sim_sustained,

"sustained_observed": sim_sustained_observed,

"held_time": sim_held,

"held_observed": sim_held_observed,

"restricted_first_crossing_time": sim_restricted_first,

"restricted_sustained_time": sim_restricted_sustained,

"restricted_held_time": sim_restricted_held,

"auc_deficit_vs_continue": sim_auc_deficit,

"first_crossing_over_Twindow": sim_first_over_Twindow,

"sustained_time_over_Twindow": sim_sustained_over_Twindow,

"held_time_over_Twindow": sim_held_over_Twindow,

```
"restricted_first_over_Twindow": sim_restricted_first_over_Twindow,

"restricted_sustained_over_Twindow": sim_restricted_sustained_over_Twindow,

"restricted_held_over_Twindow": sim_restricted_held_over_Twindow,

})



if not recovery_rows:

raise RuntimeError("No recovery scaling data computed - check NPZ contents and data structure")



# Validate expected count: 40 seeds × 5 rho × 4 branches × 2 metrics × 2 horizon systems = 3200

# Now includes continue branch for DID calculation

expected_count = len(seeds) *len(rho_values)* len(branches) *2* 2

if len(recovery_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} recovery rows, got {len(recovery_rows)}. "

f"This may indicate a logic error in recovery calculation."

)
```

```
recovery_df = pd.DataFrame(recovery_rows)



print(f"✅ Stage 4b: Recovery Scaling - {len(recovery_rows)} recovery metrics computed")

print(f" Seeds: {recovery_df['seed'].nunique()}, Rho: {recovery_df['rho'].nunique()}, "

f"Branches: {recovery_df['branch'].nunique()}, Metrics: {recovery_df['metric'].nunique()}, "

f"Systems: {recovery_df['horizon_system'].nunique()}")



return recovery_df

================================================================================
Stage 1: Validation
================================================================================

def validate_run(

text

csv_path: Path,

npz_path: Path,

manifest_path: Path,

) -> dict[str, Any]:
```

text

"""Validate the run before analysis.

Checks:

- 800 scalar rows (40 seeds × 5 rho × 4 branches)

- 40 unique seeds

- 5 rho values per seed

- 4 branches per seed/rho

- baseline equivalence passed

- run is freezeable

- no duplicate condition keys

- all primary outcomes are present and numeric

"""

results = {

"status": "passed",

"errors": [],

"warnings": [],

```
}



# Load manifest

try:

with open(manifest_path, "r") as f:

manifest = json.load(f)

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Cannot load manifest: {e}")

return results



# Validate condition status - MANDATORY for production

condition_status_path = Path(csv_path.parent) / "twindow_condition_status.csv"

if not condition_status_path.exists():

results["status"] = "failed"

results["errors"].append("Condition status file not found - REQUIRED for production analysis")

return results
```

try:

condition_df = pd.read_csv(condition_status_path)

Check for duplicate condition-status rows

if condition_df.duplicated(subset=["seed", "rho"]).any():

results["status"] = "failed"

results["errors"].append("Condition status file has duplicate (seed, rho) rows")

return results

Validate status values

unique_statuses = set(condition_df["status"].unique())

valid_statuses = {"success", "failed"}

invalid_statuses = unique_statuses - valid_statuses

if invalid_statuses:

results["status"] = "failed"

results["errors"].append(

f"Condition status file has invalid status values: {invalid_statuses}"

```
)

return results


successful_conditions = (condition_df["status"] == "success").sum()

failed_conditions = (condition_df["status"] == "failed").sum()

total_conditions = len(condition_df)


expected_conditions = EXPECTED_N_CONDITIONS

if total_conditions != expected_conditions:

results["status"] = "failed"

results["errors"].append(

f"Condition status: expected {expected_conditions} conditions, "

f"got {total_conditions}"

)

return results


if failed_conditions &gt; 0:

results["status"] = "failed"

results["errors"].append(
```

```
f"Condition status: {failed_conditions} conditions failed, "
f"{successful_conditions} succeeded"
)
return results

# Validate exact condition coverage matches manifest
condition_df["seed"] = condition_df["seed"].astype(str)
condition_df["rho"] = [pd.to](http://pd.to)_numeric(condition_df["rho"], errors="coerce")

observed_conditions = set(
zip(condition_df["seed"], condition_df["rho"])
)

# Get expected conditions from manifest
try:
manifest_seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]
manifest_rho_values = [float(r) for r in manifest["study"]["rho_values"]]
expected_conditions_from_manifest = {
```

```
(str(seed), float(rho))

for seed in manifest_seeds

for rho in manifest_rho_values

}


if observed_conditions != expected_conditions_from_manifest:

missing = expected_conditions_from_manifest - observed_conditions

extra = observed_conditions - expected_conditions_from_manifest

if missing:

results["status"] = "failed"

results["errors"].append(

f"Condition status missing {len(missing)} conditions: {list(missing)[:5]}"

)

return results

if extra:

results["status"] = "failed"

results["errors"].append(

f"Condition status has {len(extra)} extra conditions: {list(extra)[:5]}"
```

```
)

return results

except Exception as e:

results["warnings"].append(f"Could not validate condition coverage against manifest: {e}")


# Hash the condition status file

condition_status_hash = file_sha256(condition_status_path)

results["condition_status_hash"] = condition_status_hash


except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Could not validate condition status: {e}")

return results



# Check baseline equivalence

baseline_eq = manifest.get("baseline_equivalence", {})

if baseline_eq.get("status") != "passed":

results["status"] = "failed"
```

```
results["errors"].append(
f"Baseline equivalence not passed: {baseline_eq.get('status', 'unknown')}"
)
return results


# Check freezeable status - FAIL if not freezeable
# Check both top-level and completion.freezeable, default to False
completion = manifest.get("completion", {})
freezeable = manifest.get("freezeable", completion.get("freezeable", False))


# Also check completion status
completion_status = completion.get("status", "")
n_successful = completion.get("n_conditions_successful", 0)
n_failed = completion.get("n_conditions_failed", 0)
n_expected = EXPECTED_N_CONDITIONS


# Must have all conditions successful and explicit freezeable=true
```

```
if not freezeable:

results["status"] = "failed"

results["errors"].append("Run is marked as non-freezeable")

return results



if completion_status != "completed":

results["status"] = "failed"

results["errors"].append(f"Run not completed: status={completion_status}")

return results



if n_successful != n_expected or n_failed != 0:

results["status"] = "failed"

results["errors"].append(

f"Incomplete run: {n_successful}/{n_expected} successful, {n_failed} failed"

)

return results
```

```
# Load CSV

try:

df = load_csv_atomic(csv_path)

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Cannot load CSV: {e}")

return results



# Validate row count

expected_rows = EXPECTED_N_ROWS

actual_rows = len(df)

if actual_rows != expected_rows:

results["status"] = "failed"

results["errors"].append(

f"Row count mismatch: expected {expected_rows}, got {actual_rows}"

)
```

```
# Validate unique seeds
unique_seeds = df["seed"].unique()
if len(unique_seeds) != EXPECTED_N_SEEDS:
results["status"] = "failed"
results["errors"].append(
f"Seed count mismatch: expected {EXPECTED_N_SEEDS}, got {len(unique_seeds)}"
)


# Validate rho values
unique_rho = sorted([float(r) for r in df["rho"].unique()])
expected_rho = sorted([float(r) for r in EXPECTED_RHO_VALUES])
if unique_rho != expected_rho:
results["status"] = "failed"
results["errors"].append(
f"Rho values mismatch: expected {expected_rho}, got {unique_rho}"
)
```

```
# Validate branches

unique_branches = sorted(df["branch"].unique())

expected_branches = sorted(EXPECTED_BRANCHES)

if unique_branches != expected_branches:

results["status"] = "failed"

results["errors"].append(

f"Branches mismatch: expected {expected_branches}, got {unique_branches}"

)



# Validate completeness: each seed/rho/branch combination should exist

missing_combinations = []

for seed in unique_seeds:

seed_data = df[df["seed"] == seed]

for rho in [float(r) for r in df["rho"].unique()]:

rho_data = seed_data[seed_data["rho"] == rho]

for branch in unique_branches:

if not ((rho_data["branch"] == branch).any()):
```

```
missing_combinations.append(f"seed={seed}, rho={rho}, branch={branch}")



if missing_combinations:

results["status"] = "failed"

results["errors"].append(

f"Missing {len(missing_combinations)} combinations: {missing_combinations[:5]}"

)



# Check for duplicates

duplicate_rows = df.duplicated(subset=["seed", "rho", "branch"])

if duplicate_rows.any():

results["status"] = "failed"

results["errors"].append(

f"Duplicate rows found: {duplicate_rows.sum()} duplicates"

)



# Validate exact cohort coverage in CSV matches manifest
```

```
try:
manifest_seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]
manifest_rho_values = [float(r) for r in manifest["study"]["rho_values"]]
manifest_branches = manifest["protocol"]["branches"]

expected_csv_conditions = {
(str(seed), float(rho), branch)
for seed in manifest_seeds
for rho in manifest_rho_values
for branch in manifest_branches
}

observed_csv_conditions = set(
zip(df["seed"], df["rho"], df["branch"])
)

if observed_csv_conditions != expected_csv_conditions:
missing = expected_csv_conditions - observed_csv_conditions
extra = observed_csv_conditions - expected_csv_conditions
```

```
if missing:
results["status"] = "failed"
results["errors"].append(
f"CSV missing {len(missing)} conditions: {list(missing)[:5]}"
)
if extra:
results["status"] = "failed"
results["errors"].append(
f"CSV has {len(extra)} extra conditions: {list(extra)[:5]}"
)
if missing or extra:
return results
except Exception as e:
results["warnings"].append(f"Could not validate CSV cohort against manifest: {e}")



# Validate primary and secondary outcomes are present and finite
all_outcomes = PRIMARY_OUTCOMES + SECONDARY_NORMALIZED_OUTCOMES
```

```
for outcome in all_outcomes:

if outcome not in df.columns:

results["status"] = "failed"

results["errors"].append(f"Outcome '{outcome}' missing from CSV")

else:

# Check that values are numeric

non_numeric = df[outcome].apply(lambda x: not isinstance(x, (int, float, np.number)))

if non_numeric.any():

results["status"] = "failed"

results["errors"].append(

f"Outcome '{outcome}' has {non_numeric.sum()} non-numeric values"

)


# Check that values are finite (not NaN, inf, or -inf)

non_finite = ~np.isfinite(df[outcome])

if non_finite.any():

results["status"] = "failed"
```

```
results["errors"].append(

f"Outcome '{outcome}' has {non_finite.sum()} non-finite values"

)



# Get baseline equivalence certificate hash - MANDATORY for production

baseline_cert_path = Path(csv_path.parent) / "baseline_equivalence_certificate.json"

if not baseline_cert_path.exists():

results["status"] = "failed"

results["errors"].append("Baseline equivalence certificate not found - REQUIRED for production analysis")

return results

baseline_cert_hash = file_sha256(baseline_cert_path)



# Validate baseline certificate contents

try:

with open(baseline_cert_path, "r") as f:

certificate = json.load(f)

cert_status = certificate.get("status", "unknown")
```

```
cert_n_seeds = certificate.get("n_seeds", 0)

cert_seed_results = certificate.get("seed_results", {})


if cert_status != "passed":

results["status"] = "failed"

results["errors"].append(

f"Baseline certificate status is '{cert_status}', expected 'passed'"

)

return results


if cert_n_seeds != EXPECTED_N_SEEDS:

results["status"] = "failed"

results["errors"].append(

f"Baseline certificate has {cert_n_seeds} seeds, expected {EXPECTED_N_SEEDS}"

)

return results


if len(cert_seed_results) != EXPECTED_N_SEEDS:
```

```
results["status"] = "failed"

results["errors"].append(

f"Baseline certificate has {len(cert_seed_results)} seed results, expected {EXPECTED_N_SEEDS}"

)

return results


for seed_id, seed_result in cert_seed_results.items():

if seed_result.get("status") != "passed":

results["status"] = "failed"

results["errors"].append(

f"Baseline certificate seed {seed_id} status is '{seed_result.get('status')}', expected 'passed'"

)

return results

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Could not validate baseline certificate contents: {e}")

return results
```

```
# Record validation info
results["validation"] = {
"n_rows": actual_rows,
"n_seeds": len(unique_seeds),
"n_rho": len(unique_rho),
"n_branches": len(unique_branches),
"n_duplicates": int(duplicate_rows.sum()),
"n_missing_combinations": len(missing_combinations),
"baseline_equivalence": baseline_eq.get("status"),
"freezeable": manifest.get("freezeable", False),
"baseline_certificate_hash": baseline_cert_hash,
}

# Also expose at top level for manifest generation
results["baseline_certificate_hash"] = baseline_cert_hash

if results["status"] == "passed":
```

```
print("✅ Stage 1: Validation PASSED")

print(f" {actual_rows} rows, {len(unique_seeds)} seeds, {len(unique_rho)} rho,
{len(unique_branches)} branches")

else:

print("❌ Stage 1: Validation FAILED")

for error in results["errors"]:

print(f" {error}")

for warning in results["warnings"]:

print(f" WARNING: {warning}")



return results

================================================================================
Stage 2: Regime Contrasts
================================================================================

def compute_regime_contrasts(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,
```

) -> pd.DataFrame:

text

"""Compute regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}

Uses direct merge instead of pivot to avoid MultiIndex column issues.

For each seed and non-baseline rho, compute the difference between

the outcome at that rho and the outcome at rho=1 (baseline).

"""

Filter to continue branch only

cont = df.loc[

df["branch"] == "continue",

["seed", "rho", *outcomes],

].copy()

Get baseline (rho=1.0) values

baseline = (

```
cont.loc[cont["rho"] == 1.0]

.drop(columns="rho")

.rename(

columns={

outcome: f"{outcome}_baseline"

for outcome in outcomes

}

)

)



# Merge with all rho values

merged = cont.merge(

baseline,

on="seed",

how="inner",

validate="many_to_one",

)
```

```
# Filter to non-baseline rho

merged = merged.loc[merged["rho"] != 1.0]



# Compute contrasts

rows = []

for _, row in merged.iterrows():

for outcome in outcomes:

value = row[outcome]

reference = row[f"{outcome}_baseline"]



# Validate that both values are numeric

if not isinstance(value, (int, float, np.number)) or not isinstance(reference, (int, float, np.number)):

raise RuntimeError(f"Non-numeric value for {outcome} at seed={row['seed']}, rho={row['rho']}")



rows.append({

"seed": row["seed"],

"rho": row["rho"],
```

```
"outcome": outcome,

"Y_rho_continue": float(value),

"Y_baseline_continue": float(reference),

"paired_regime_contrast": float(value - reference),

})



if not rows:

raise RuntimeError("No regime contrasts computed - check input data")



# Validate expected count

expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)* len(outcomes)

if len(rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} regime contrasts, got {len(rows)}"

)



contrasts_df = pd.DataFrame(rows)
```

```
print(f"✅ Stage 2: Regime Contrasts - {len(rows)} contrasts computed")

print(f" Seeds: {contrasts_df['seed'].nunique()}, Rho values: {contrasts_df['rho'].nunique()}, Outcomes: {contrasts_df['outcome'].nunique()}")




return contrasts_df

================================================================================
Stage 3: Branch Difference-in-Differences
================================================================================

def compute_branch_did(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:

text

"""Compute branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]
```

```
For each seed, non-baseline rho, and non-continue branch, compute the

difference-in-differences relative to continue branch.



Skips rho=1.0 (baseline) as DID should be zero by construction.

"""

did_rows = []



for seed in df["seed"].unique():

seed_data = df[df["seed"] == seed]



for rho in sorted([float(r) for r in df["rho"].unique()]):

# Skip baseline rho=1.0

if rho == 1.0:

continue



rho_data = seed_data[seed_data["rho"] == rho]



# Get baseline (rho=1.0) data for this seed

baseline_data = seed_data[seed_data["rho"] == 1.0]
```

```
for branch in NON_CONTINUE_BRANCHES:

for outcome in outcomes:

# Get Y_{s,ρ,b}

y_rho_b_match = rho_data[rho_data["branch"] == branch][outcome]

if len(y_rho_b_match) == 0:

raise RuntimeError(f"Missing data: seed={seed}, rho={rho}, branch={branch}, outcome={outcome}")

y_rho_b = float(y_rho_b_match.iloc[0])


# Get Y_{s,ρ,continue}

y_rho_c_match = rho_data[rho_data["branch"] == "continue"][outcome]

if len(y_rho_c_match) == 0:

raise RuntimeError(f"Missing continue data: seed={seed}, rho={rho}, outcome={outcome}")

y_rho_c = float(y_rho_c_match.iloc[0])


# Get Y_{s,1,b}

y_base_b_match = baseline_data[baseline_data["branch"] == branch][outcome]

if len(y_base_b_match) == 0:
```

```
raise RuntimeError(f"Missing baseline data: seed={seed}, branch={branch}, outcome={outcome}")

y_base_b = float(y_base_b_match.iloc[0])


# Get Y_{s,1,continue}

y_base_c_match = baseline_data[baseline_data["branch"] == "continue"][outcome]

if len(y_base_c_match) == 0:

raise RuntimeError(f"Missing baseline continue data: seed={seed}, outcome={outcome}")

y_base_c = float(y_base_c_match.iloc[0])


# Compute DID

branch_effect_rho = y_rho_b - y_rho_c

branch_effect_baseline = y_base_b - y_base_c

did = branch_effect_rho - branch_effect_baseline


did_rows.append({

"seed": seed,

"rho": rho,

"branch": branch,

"outcome": outcome,
```

"Y_rho_branch": y_rho_b,

"Y_rho_continue": y_rho_c,

"branch_effect_rho": branch_effect_rho,

"Y_baseline_branch": y_base_b,

"Y_baseline_continue": y_base_c,

"branch_effect_baseline": branch_effect_baseline,

"difference_in_differences": did,

})

Validate expected count

expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)*
len(NON_CONTINUE_BRANCHES) * len(outcomes)

if len(did_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} DID values, got {len(did_rows)}"

)

did_df = pd.DataFrame(did_rows)

print(f"✅ Stage 3: Branch DID - {len(did_rows)} DID values computed")

print(f" Seeds: {did_df['seed'].nunique()}, Rho: {did_df['rho'].nunique()}, Branches: {did_df['branch'].nunique()}")

return did_df

==
Stage 4: Cross-T Topology and Recovery
==

def build_continue_b_final_key(seed: str, rho: float) -> str:

text

"""Build canonical key for B_final matrix."""

return f"continue_seed_{seed}_rho_{rho}_B_final"

def compute_cross_t_topology(

text

npz_path: Path,

df: pd.DataFrame,

seeds: list[str],

rho_values: list[float],

b_c: float,

) -> pd.DataFrame:

text

"""Compute cross-T topology metrics.

For each seed and non-baseline rho, compute:

- S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- J_{s,ρ} = Jaccard(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- D_{s,ρ} = normalized_L2_distance(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Requires exactly EXPECTED_N_TOPOLOGY rows (40 seeds × 4 non-baseline rho = 160).

Any missing matrix is a schema failure.

"""

Build list of exact keys to load

keys_to_load = []

```
for seed in seeds:

for rho in rho_values:

keys_to_load.append(build_continue_b_final_key(seed, rho))



# Load only the required B_final matrices

npz_data = load_npz_selective(npz_path, keys=keys_to_load)



topology_rows = []

missing_keys = []



for seed in seeds:

for rho in rho_values:

if rho == 1.0:

continue # Skip baseline



# Get B_final for this (seed, rho, continue)

rho_key = build_continue_b_final_key(seed, rho)

baseline_key = build_continue_b_final_key(seed, 1.0)
```

```
if rho_key not in npz_data:
missing_keys.append(rho_key)
continue
if baseline_key not in npz_data:
missing_keys.append(baseline_key)
continue

B_rho = npz_data[rho_key]
B_baseline = npz_data[baseline_key]

# Compute metrics
cos_sim = cosine_similarity(B_rho, B_baseline)
jaccard = jaccard_similarity(B_rho, B_baseline, threshold=b_c)
norm_dist = normalized_distance(B_rho, B_baseline)

topology_rows.append({
"seed": seed,
"rho": rho,
```

```
"cross_T_cosine": cos_sim,

"cross_T_Jaccard": jaccard,

"cross_T_normalized_distance": norm_dist,

})



# Validate we have all expected comparisons

expected_count = EXPECTED_N_TOPOLOGY

if len(topology_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} topology comparisons, got {len(topology_rows)}. "

f"Missing keys: {missing_keys[:10]}"

)



topology_df = pd.DataFrame(topology_rows)



print(f"✅ Stage 4: Cross-T Topology - {len(topology_rows)} topology comparisons computed")

print(f" Seeds: {topology_df['seed'].nunique()}, Rho: {topology_df['rho'].nunique()}")
```

return topology_df

def compute_recovery_did(

text

recovery: pd.DataFrame,

) -> pd.DataFrame:

text

"""Compute recovery interaction DID: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]

For each seed, non-baseline rho, and NON-CONTINUE branch only, compute the

difference-in-differences for recovery outcomes relative to continue branch.

Uses restricted times (censoring-aware) and normalized t/T_window as primary outcomes.

Skips rho=1.0 (baseline) as DID should be zero by construction.

Skips continue branch as it generates trivial zero-DID rows.

```
Expected count: 40 seeds *4 non-baseline rho* 3 non-continue branches *2 systems* 2 metrics * 13 outcomes = 24960

"""

if recovery.empty:

return pd.DataFrame()



did_rows = []



seeds = sorted(recovery["seed"].unique())

rho_values = sorted([float(r) for r in recovery["rho"].unique()])

horizon_systems = sorted(recovery["horizon_system"].unique())

metrics = sorted(recovery["metric"].unique())



# Recovery outcomes to compute DID for

# Prioritize censoring-aware restricted outcomes as primary

recovery_outcomes = [

"restricted_first_crossing_time",
```

"restricted_sustained_time",

"restricted_held_time",

"restricted_first_over_Twindow",

"restricted_sustained_over_Twindow",

"restricted_held_over_Twindow",

"auc_deficit_vs_continue",

"first_crossing_over_Twindow",

"sustained_time_over_Twindow",

"held_time_over_Twindow",

]

Also compute DID for recovery indicators

indicator_outcomes = [

"first_crossing_observed",

"sustained_observed",

"held_observed",

]

for seed in seeds:

seed_data = recovery[recovery["seed"] == seed]

for rho in rho_values:

Skip baseline rho=1.0

if rho == 1.0:

continue

rho_data = seed_data[seed_data["rho"] == rho]

Get baseline (rho=1.0) data for this seed

baseline_data = seed_data[seed_data["rho"] == 1.0]

for branch in NON_CONTINUE_BRANCHES:

EXCLUDE continue branch - it generates trivial zero-DID rows

DID is only defined for non-continue branches: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]

for horizon_system in horizon_systems:

for metric in metrics:

```
# Compute DID for each recovery outcome

for outcome in recovery_outcomes:

# Get values: R_{s,ρ,b}, R_{s,ρ,c}, R_{s,1,b}, R_{s,1,c}


# R_{s,ρ,b}

r_rho_b_match = rho_data[

(rho_data["branch"] == branch) &amp;amp;

(rho_data["horizon_system"] == horizon_system) &amp;amp;

(rho_data["metric"] == metric)

][outcome]


# R_{s,ρ,c} (continue branch at same rho)

r_rho_c_match = rho_data[

(rho_data["branch"] == "continue") &amp;amp;

(rho_data["horizon_system"] == horizon_system) &amp;amp;

(rho_data["metric"] == metric)

][outcome]
```

```
# R_{s,1,b} (baseline rho=1.0, same branch)

r_base_b_match = baseline_data[

(baseline_data["branch"] == branch) &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]


# R_{s,1,c} (baseline rho=1.0, continue)

r_base_c_match = baseline_data[

(baseline_data["branch"] == "continue") &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]


# Validate exactly one match for each cell - missing cells indicate schema error

matches = {

"rho_branch": r_rho_b_match,

"rho_continue": r_rho_c_match,
```

```
"baseline_branch": r_base_b_match,

"baseline_continue": r_base_c_match,

}

for label, match in matches.items():

if len(match) != 1:

raise RuntimeError(

f"Recovery DID requires exactly one {label} row; "

f"seed={seed}, rho={rho}, branch={branch}, "

f"horizon={horizon_system}, metric={metric}, "

f"outcome={outcome}, found={len(match)}"

)


r_rho_b = float(r_rho_b_match.iloc[0])

r_rho_c = float(r_rho_c_match.iloc[0])

r_base_b = float(r_base_b_match.iloc[0])

r_base_c = float(r_base_c_match.iloc[0])


# Compute DID: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]
```

```
branch_effect_rho = r_rho_b - r_rho_c

branch_effect_baseline = r_base_b - r_base_c

did = branch_effect_rho - branch_effect_baseline


did_rows.append({

"seed": seed,

"rho": rho,

"branch": branch,

"horizon_system": horizon_system,

"metric": metric,

"outcome": outcome,

"R_rho_branch": r_rho_b,

"R_rho_continue": r_rho_c,

"branch_effect_rho": branch_effect_rho,

"R_baseline_branch": r_base_b,

"R_baseline_continue": r_base_c,

"branch_effect_baseline": branch_effect_baseline,

"recovery_difference_in_differences": did,
```

```
})

# Compute DID for recovery indicators (binary outcomes)
for outcome in indicator_outcomes:
# Get values
r_rho_b_match = rho_data[
(rho_data["branch"] == branch) &amp;amp;
(rho_data["horizon_system"] == horizon_system) &amp;amp;
(rho_data["metric"] == metric)
][outcome]

r_rho_c_match = rho_data[
(rho_data["branch"] == "continue") &amp;amp;
(rho_data["horizon_system"] == horizon_system) &amp;amp;
(rho_data["metric"] == metric)
][outcome]

r_base_b_match = baseline_data[
(baseline_data["branch"] == branch) &amp;amp;
```

```
(baseline_data["horizon_system"] == horizon_system) &amp;amp;
(baseline_data["metric"] == metric)
][outcome]

r_base_c_match = baseline_data[
(baseline_data["branch"] == "continue") &amp;amp;
(baseline_data["horizon_system"] == horizon_system) &amp;amp;
(baseline_data["metric"] == metric)
][outcome]

# Validate exactly one match for each cell - missing cells indicate schema error
matches = {
"rho_branch": r_rho_b_match,
"rho_continue": r_rho_c_match,
"baseline_branch": r_base_b_match,
"baseline_continue": r_base_c_match,
}
for label, match in matches.items():
```

```
if len(match) != 1:

raise RuntimeError(

f"Recovery DID requires exactly one {label} row; "

f"seed={seed}, rho={rho}, branch={branch}, "

f"horizon={horizon_system}, metric={metric}, "

f"outcome={outcome}, found={len(match)}"

)


r_rho_b = float(r_rho_b_match.iloc[0])

r_rho_c = float(r_rho_c_match.iloc[0])

r_base_b = float(r_base_b_match.iloc[0])

r_base_c = float(r_base_c_match.iloc[0])


# For binary outcomes, use difference in proportions

branch_effect_rho = r_rho_b - r_rho_c

branch_effect_baseline = r_base_b - r_base_c

did = branch_effect_rho - branch_effect_baseline
```

```
did_rows.append({

"seed": seed,

"rho": rho,

"branch": branch,

"horizon_system": horizon_system,

"metric": metric,

"outcome": outcome,

"R_rho_branch": r_rho_b,

"R_rho_continue": r_rho_c,

"branch_effect_rho": branch_effect_rho,

"R_baseline_branch": r_base_b,

"R_baseline_continue": r_base_c,

"branch_effect_baseline": branch_effect_baseline,

"recovery_difference_in_differences": did,

})



if not did_rows:
```

```
return pd.DataFrame()



# Validate exact count: 40 seeds *4 non-baseline rho* 3 non-continue branches *2 systems* 2 metrics * 13 outcomes = 24960

if len(did_rows) != EXPECTED_N_RECOVERY_DID:

raise RuntimeError(

f"Expected exactly {EXPECTED_N_RECOVERY_DID} recovery DID rows, got {len(did_rows)}. "

f"This indicates a logic error - continue branch should be excluded."

)



did_df = pd.DataFrame(did_rows)



print(f"✅ Stage 4c: Recovery DID - {len(did_rows)} recovery DID values computed")

print(f" Seeds: {did_df['seed'].nunique()}, Rho: {did_df['rho'].nunique()}, "

f"Branches: {did_df['branch'].nunique()}, Systems: {did_df['horizon_system'].nunique()}")



return did_df

def compute_seed_level_summaries(
```

```
text

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

recovery: pd.DataFrame | None = None,

n_bootstrap: int = 1000,

bootstrap_seed: int | None = None,

) -> dict[str, pd.DataFrame]:

text

"""Compute seed-level summaries with bootstrap confidence intervals.



For each outcome and rho, compute:

- Mean contrast

- Std error

- 95% bootstrap CI



Uses seeded bootstrap for reproducibility.
```

"""

summaries = {}

Create seeded RNG for bootstrap

if bootstrap_seed is None:

raise ValueError("--bootstrap-seed is required for reproducible analysis")

rng = np.random.default_rng(bootstrap_seed)

Regime contrast summaries

if not regime_contrasts.empty:

Use named aggregation to avoid MultiIndex columns

regime_summary = regime_contrasts.groupby(

["rho", "outcome"],

as_index=False,

).agg(

mean=("paired_regime_contrast", "mean"),

std=("paired_regime_contrast", "std"),

```
n=("paired_regime_contrast", "count"),

)


# Add bootstrap CI

for (rho, outcome), group in regime_contrasts.groupby(["rho", "outcome"]):

values = group["paired_regime_contrast"].dropna().values

if len(values) &gt; 0:

# Seeded bootstrap

indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)


ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = regime_summary[(regime_summary["rho"] == rho) &amp;amp; (regime_summary["outcome"] ==
outcome)].index

regime_summary.loc[idx, "ci_lower"] = ci_lower

regime_summary.loc[idx, "ci_upper"] = ci_upper
```

summaries["regime_contrasts"] = regime_summary

print(f"✅ Stage 5: Regime Contrast Summaries - {len(regime_summary)} groups")

Branch DID summaries

if not branch_did.empty:

did_summary = branch_did.groupby(

["rho", "branch", "outcome"],

as_index=False,

).agg(

mean=("difference_in_differences", "mean"),

std=("difference_in_differences", "std"),

n=("difference_in_differences", "count"),

)

Add bootstrap CI

for (rho, branch, outcome), group in branch_did.groupby(["rho", "branch", "outcome"]):

values = group["difference_in_differences"].dropna().values

if len(values) > 0:

```
indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)


ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = did_summary[(did_summary["rho"] == rho) &amp;amp; (did_summary["branch"] == branch) &amp;amp;
(did_summary["outcome"] == outcome)].index

did_summary.loc[idx, "ci_lower"] = ci_lower

did_summary.loc[idx, "ci_upper"] = ci_upper


summaries["branch_did"] = did_summary

print(f"✅ Stage 5: Branch DID Summaries - {len(did_summary)} groups")



# Topology summaries

if not topology.empty:

# Flatten the aggregation to avoid MultiIndex columns

topology_summary = topology.groupby("rho", as_index=False).agg(

cosine_mean=("cross_T_cosine", "mean"),
```

```
cosine_std=("cross_T_cosine", "std"),

cosine_n=("cross_T_cosine", "count"),

jaccard_mean=("cross_T_Jaccard", "mean"),

jaccard_std=("cross_T_Jaccard", "std"),

jaccard_n=("cross_T_Jaccard", "count"),

distance_mean=("cross_T_normalized_distance", "mean"),

distance_std=("cross_T_normalized_distance", "std"),

distance_n=("cross_T_normalized_distance", "count"),

)


# Add bootstrap CI for topology metrics (seed-level bootstrap by rho)

for rho, group in topology.groupby("rho"):

# Bootstrap cosine similarity

cosine_values = group["cross_T_cosine"].dropna().values

if len(cosine_values) &gt; 0:

indices = rng.integers(0, len(cosine_values), size=(n_bootstrap, len(cosine_values)))

bootstrap_means = np.mean(cosine_values[indices], axis=1)
```

```
cosine_ci_lower = float(np.percentile(bootstrap_means, 2.5))

cosine_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "cosine_ci_lower"] = cosine_ci_lower

topology_summary.loc[idx, "cosine_ci_upper"] = cosine_ci_upper


# Bootstrap Jaccard similarity

jaccard_values = group["cross_T_Jaccard"].dropna().values

if len(jaccard_values) &gt; 0:

indices = rng.integers(0, len(jaccard_values), size=(n_bootstrap, len(jaccard_values)))

bootstrap_means = np.mean(jaccard_values[indices], axis=1)

jaccard_ci_lower = float(np.percentile(bootstrap_means, 2.5))

jaccard_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "jaccard_ci_lower"] = jaccard_ci_lower

topology_summary.loc[idx, "jaccard_ci_upper"] = jaccard_ci_upper
```

```
# Bootstrap normalized distance

distance_values = group["cross_T_normalized_distance"].dropna().values

if len(distance_values) &gt; 0:

indices = rng.integers(0, len(distance_values), size=(n_bootstrap, len(distance_values)))

bootstrap_means = np.mean(distance_values[indices], axis=1)

distance_ci_lower = float(np.percentile(bootstrap_means, 2.5))

distance_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "distance_ci_lower"] = distance_ci_lower

topology_summary.loc[idx, "distance_ci_upper"] = distance_ci_upper


summaries["topology"] = topology_summary

print(f"✅ Stage 5: Topology Summaries - {len(topology_summary)} rho values")



# Recovery scaling summaries with bootstrap CI

if recovery is not None and not recovery.empty:

# Group by metric, horizon_system, branch, rho and compute summaries
```

```
# Include both raw event times AND censoring-aware restricted times
recovery_summary = recovery.groupby(
["rho", "branch", "horizon_system", "metric"],
as_index=False,
).agg(
# Raw event times (conditional on observed recovery)
mean_first_crossing=("first_crossing_time", "mean"),
std_first_crossing=("first_crossing_time", "std"),
n_first_crossing=("first_crossing_time", "count"),
mean_sustained=("sustained_time", "mean"),
std_sustained=("sustained_time", "std"),
n_sustained=("sustained_time", "count"),
mean_held=("held_time", "mean"),
std_held=("held_time", "std"),
n_held=("held_time", "count"),
mean_auc_deficit=("auc_deficit_vs_continue", "mean"),
std_auc_deficit=("auc_deficit_vs_continue", "std"),
n_auc_deficit=("auc_deficit_vs_continue", "count"),
```

```
# Censoring-aware restricted times (cohort-level estimates)
mean_restricted_first=("restricted_first_crossing_time", "mean"),
std_restricted_first=("restricted_first_crossing_time", "std"),
n_restricted_first=("restricted_first_crossing_time", "count"),
mean_restricted_sustained=("restricted_sustained_time", "mean"),
std_restricted_sustained=("restricted_sustained_time", "std"),
n_restricted_sustained=("restricted_sustained_time", "count"),
mean_restricted_held=("restricted_held_time", "mean"),
std_restricted_held=("restricted_held_time", "std"),
n_restricted_held=("restricted_held_time", "count"),

# Normalized times (t/T_window) - raw observed (conditional on recovery)
mean_first_over_Twindow=("first_crossing_over_Twindow", "mean"),
std_first_over_Twindow=("first_crossing_over_Twindow", "std"),
n_first_over_Twindow=("first_crossing_over_Twindow", "count"),
mean_sustained_over_Twindow=("sustained_time_over_Twindow", "mean"),
std_sustained_over_Twindow=("sustained_time_over_Twindow", "std"),
```

```
n_sustained_over_Twindow=("sustained_time_over_Twindow", "count"),

mean_held_over_Twindow=("held_time_over_Twindow", "mean"),

std_held_over_Twindow=("held_time_over_Twindow", "std"),

n_held_over_Twindow=("held_time_over_Twindow", "count"),


# Censoring-aware restricted normalized times (cohort-level estimates)

mean_restricted_first_over_Twindow=("restricted_first_over_Twindow", "mean"),

std_restricted_first_over_Twindow=("restricted_first_over_Twindow", "std"),

n_restricted_first_over_Twindow=("restricted_first_over_Twindow", "count"),

mean_restricted_sustained_over_Twindow=("restricted_sustained_over_Twindow", "mean"),

std_restricted_sustained_over_Twindow=("restricted_sustained_over_Twindow", "std"),

n_restricted_sustained_over_Twindow=("restricted_sustained_over_Twindow", "count"),

mean_restricted_held_over_Twindow=("restricted_held_over_Twindow", "mean"),

std_restricted_held_over_Twindow=("restricted_held_over_Twindow", "std"),

n_restricted_held_over_Twindow=("restricted_held_over_Twindow", "count"),


# Recovery probabilities

observed_rate_first=("first_crossing_observed", "mean"),
```

```
observed_rate_sustained=("sustained_observed", "mean"),

observed_rate_held=("held_observed", "mean"),

)


# Add bootstrap CI for recovery metrics

for (rho, branch, horizon_system, metric), group in recovery.groupby(["rho", "branch",
"horizon_system", "metric"]):

# Bootstrap first crossing time (conditional on observed)

first_crossing_values = group["first_crossing_time"].dropna().values

if len(first_crossing_values) &gt; 0:

indices = rng.integers(0, len(first_crossing_values), size=(n_bootstrap, len(first_crossing_values)))

bootstrap_means = np.nanmean(first_crossing_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;

(recovery_summary["branch"] == branch) &amp;
```

```
(recovery_summary["horizon_system"] == horizon_system) &amp;amp;
(recovery_summary["metric"] == metric)
].index
recovery_summary.loc[idx, "ci_lower_first_crossing"] = ci_lower
recovery_summary.loc[idx, "ci_upper_first_crossing"] = ci_upper

# Bootstrap restricted first crossing time (cohort-level, includes censored)
restricted_first_values = group["restricted_first_crossing_time"].dropna().values
if len(restricted_first_values) &gt; 0:
indices = rng.integers(0, len(restricted_first_values), size=(n_bootstrap, len(restricted_first_values)))
bootstrap_means = np.nanmean(restricted_first_values[indices], axis=1)
ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))
ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))

idx = recovery_summary[
(recovery_summary["rho"] == rho) &amp;amp;
(recovery_summary["branch"] == branch) &amp;amp;
(recovery_summary["horizon_system"] == horizon_system) &amp;amp;
```

```
(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_first"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_first"] = ci_upper


# Bootstrap restricted sustained time

restricted_sustained_values = group["restricted_sustained_time"].dropna().values

if len(restricted_sustained_values) &gt; 0:

indices = rng.integers(0, len(restricted_sustained_values), size=(n_bootstrap,
len(restricted_sustained_values)))

bootstrap_means = np.nanmean(restricted_sustained_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)
```

```
].index

recovery_summary.loc[idx, "ci_lower_restricted_sustained"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_sustained"] = ci_upper


# Bootstrap restricted held time

restricted_held_values = group["restricted_held_time"].dropna().values

if len(restricted_held_values) &gt; 0:

indices = rng.integers(0, len(restricted_held_values), size=(n_bootstrap, len(restricted_held_values)))

bootstrap_means = np.nanmean(restricted_held_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;

(recovery_summary["branch"] == branch) &amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;

(recovery_summary["metric"] == metric)

].index
```

```
recovery_summary.loc[idx, "ci_lower_restricted_held"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_held"] = ci_upper


# Bootstrap normalized first crossing time over T_window

first_over_Twindow_values = group["first_crossing_over_Twindow"].dropna().values

if len(first_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(first_over_Twindow_values), size=(n_bootstrap,
len(first_over_Twindow_values)))

bootstrap_means = np.nanmean(first_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_first_over_Twindow"] = ci_lower
```

```
recovery_summary.loc[idx, "ci_upper_first_over_Twindow"] = ci_upper


# Bootstrap normalized sustained time over T_window

sustained_over_Twindow_values = group["sustained_time_over_Twindow"].dropna().values

if len(sustained_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(sustained_over_Twindow_values), size=(n_bootstrap,
len(sustained_over_Twindow_values)))

bootstrap_means = np.nanmean(sustained_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;

(recovery_summary["branch"] == branch) &amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_sustained_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_sustained_over_Twindow"] = ci_upper
```

```
# Bootstrap normalized held time over T_window

held_over_Twindow_values = group["held_time_over_Twindow"].dropna().values

if len(held_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(held_over_Twindow_values), size=(n_bootstrap,
len(held_over_Twindow_values)))

bootstrap_means = np.nanmean(held_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))



idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_held_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_held_over_Twindow"] = ci_upper
```

```
# Bootstrap AUC deficit

auc_values = group["auc_deficit_vs_continue"].dropna().values

if len(auc_values) &gt; 0:

indices = rng.integers(0, len(auc_values), size=(n_bootstrap, len(auc_values)))

bootstrap_means = np.nanmean(auc_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_auc_deficit"] = ci_lower

recovery_summary.loc[idx, "ci_upper_auc_deficit"] = ci_upper


# Bootstrap restricted normalized first crossing time over T_window
```

```
restricted_first_over_Twindow_values = group["restricted_first_over_Twindow"].dropna().values

if len(restricted_first_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(restricted_first_over_Twindow_values), size=(n_bootstrap, len(restricted_first_over_Twindow_values)))

bootstrap_means = np.nanmean(restricted_first_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_first_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_first_over_Twindow"] = ci_upper


# Bootstrap restricted normalized sustained time over T_window

restricted_sustained_over_Twindow_values = group["restricted_sustained_over_Twindow"].dropna().values
```

if len(restricted_sustained_over_Twindow_values) > 0:

indices = rng.integers(0, len(restricted_sustained_over_Twindow_values), size=(n_bootstrap, len(restricted_sustained_over_Twindow_values)))

bootstrap_means = np.nanmean(restricted_sustained_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))

idx = recovery_summary[

(recovery_summary["rho"] == rho) &

(recovery_summary["branch"] == branch) &

(recovery_summary["horizon_system"] == horizon_system) &

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_sustained_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_sustained_over_Twindow"] = ci_upper

Bootstrap restricted normalized held time over T_window

restricted_held_over_Twindow_values = group["restricted_held_over_Twindow"].dropna().values

if len(restricted_held_over_Twindow_values) > 0:

indices = rng.integers(0, len(restricted_held_over_Twindow_values), size=(n_bootstrap, len(restricted_held_over_Twindow_values)))

bootstrap_means = np.nanmean(restricted_held_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))

idx = recovery_summary[

(recovery_summary["rho"] == rho) &

(recovery_summary["branch"] == branch) &

(recovery_summary["horizon_system"] == horizon_system) &

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_held_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_held_over_Twindow"] = ci_upper

Bootstrap recovery probabilities (observed rates)

First crossing observed rate

first_observed_values = group["first_crossing_observed"].dropna().values

```
if len(first_observed_values) &gt; 0:

indices = rng.integers(0, len(first_observed_values), size=(n_bootstrap, len(first_observed_values)))

bootstrap_means = np.nanmean(first_observed_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_observed_rate_first"] = ci_lower

recovery_summary.loc[idx, "ci_upper_observed_rate_first"] = ci_upper


# Sustained observed rate

sustained_observed_values = group["sustained_observed"].dropna().values

if len(sustained_observed_values) &gt; 0:
```

```
indices = rng.integers(0, len(sustained_observed_values), size=(n_bootstrap,
len(sustained_observed_values)))

bootstrap_means = np.nanmean(sustained_observed_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_observed_rate_sustained"] = ci_lower

recovery_summary.loc[idx, "ci_upper_observed_rate_sustained"] = ci_upper


# Held observed rate

held_observed_values = group["held_observed"].dropna().values

if len(held_observed_values) &gt; 0:

indices = rng.integers(0, len(held_observed_values), size=(n_bootstrap, len(held_observed_values)))
```

```
bootstrap_means = np.nanmean(held_observed_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_observed_rate_held"] = ci_lower

recovery_summary.loc[idx, "ci_upper_observed_rate_held"] = ci_upper


summaries["recovery"] = recovery_summary

print(f"✅ Stage 5: Recovery Summaries - {len(recovery_summary)} groups")



return summaries

================================================================================
```

Report Generation

==========

def generate_text_report(

text

validation: dict[str, Any],

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

regime_summaries: pd.DataFrame,

did_summaries: pd.DataFrame,

topology_summaries: pd.DataFrame,

recovery_summaries: pd.DataFrame | None = None,

regime_summaries_secondary: pd.DataFrame | None = None,

did_summaries_secondary: pd.DataFrame | None = None,

recovery_did_summary: pd.DataFrame | None = None,

) -> str:

text

"""Generate human-readable text report."""

```
lines = []
lines.append("=" * 80)
lines.append("AIEF-1.4 T_window Sensitivity Study - Analysis Report")
lines.append("=" * 80)
lines.append("")


# Validation summary
lines.append("VALIDATION SUMMARY")
lines.append("-" * 40)
val = validation.get("validation", {})
lines.append(f" Status: {validation.get('status', 'unknown')}")
lines.append(f" Rows: {val.get('n_rows', 0)} (expected: {EXPECTED_N_ROWS})")
lines.append(f" Seeds: {val.get('n_seeds', 0)} (expected: {EXPECTED_N_SEEDS})")
lines.append(f" Rho values: {val.get('n_rho', 0)} (expected: {len(EXPECTED_RHO_VALUES)})")
lines.append(f" Branches: {val.get('n_branches', 0)} (expected: {len(EXPECTED_BRANCHES)})")
lines.append(f" Duplicates: {val.get('n_duplicates', 0)}")
lines.append(f" Missing combinations: {val.get('n_missing_combinations', 0)}")
```

```
lines.append(f" Baseline equivalence: {val.get('baseline_equivalence', 'unknown')}")

lines.append(f" Freezeable: {val.get('freezeable', False)}")

lines.append("")



# Regime contrasts (Primary - fixed_physical horizons)

lines.append("REGIME CONTRASTS (Δ = Y_{s,ρ,continue} - Y_{s,1,continue})")

lines.append("-" * 40)

lines.append("Primary outcomes: fixed_physical horizons")

lines.append("")

if not regime_summaries.empty:

for rho in sorted(regime_summaries["rho"].unique()):

rho_data = regime_summaries[regime_summaries["rho"] == rho]

for outcome in PRIMARY_OUTCOMES:

out_data = rho_data[rho_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])
```

```
n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No regime contrast data")

lines.append("")



# Secondary regime contrasts (normalized_EMA horizons)

lines.append("SECONDARY REGIME CONTRASTS (Normalized EMA horizons)")

lines.append("-" * 40)

if regime_summaries_secondary is not None and not regime_summaries_secondary.empty:

for rho in sorted(regime_summaries_secondary["rho"].unique()):

rho_data = regime_summaries_secondary[regime_summaries_secondary["rho"] == rho]

for outcome in SECONDARY_NORMALIZED_OUTCOMES:
```

```
out_data = rho_data[rho_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No secondary regime contrast data")

lines.append("")



# Branch DID (Primary - fixed_physical horizons)

lines.append("BRANCH DIFFERENCE-IN-DIFFERENCES")

lines.append("-" * 40)

lines.append("I_{s,ρ,b} = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]")
```

```
lines.append("Primary outcomes: fixed_physical horizons")

lines.append("")

if not did_summaries.empty:

for rho in sorted(did_summaries["rho"].unique()):

rho_data = did_summaries[did_summaries["rho"] == rho]

for branch in sorted(rho_data["branch"].unique()):

branch_data = rho_data[rho_data["branch"] == branch]

for outcome in PRIMARY_OUTCOMES:

out_data = branch_data[branch_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"
```

```
lines.append(f" rho={rho:.2f}, branch={branch}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No branch DID data")

lines.append("")



# Secondary Branch DID (normalized_EMA horizons)

lines.append("SECONDARY BRANCH DIFFERENCE-IN-DIFFERENCES (Normalized EMA)")

lines.append("-" * 40)

if did_summaries_secondary is not None and not did_summaries_secondary.empty:

for rho in sorted(did_summaries_secondary["rho"].unique()):

rho_data = did_summaries_secondary[did_summaries_secondary["rho"] == rho]

for branch in sorted(rho_data["branch"].unique()):

branch_data = rho_data[rho_data["branch"] == branch]

for outcome in SECONDARY_NORMALIZED_OUTCOMES:

out_data = branch_data[branch_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])
```

```
std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, branch={branch}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str}
(n={n_val})")

else:

lines.append(" No secondary branch DID data")

lines.append("")



# Cross-T topology

lines.append("CROSS-T TOPOLOGY METRICS")

lines.append("-" * 40)

if not topology_summaries.empty:

for *, row in topology*summaries.iterrows():

rho = float(row["rho"])
```

```
lines.append(f" rho={rho:.2f}:")

lines.append(f" Cosine similarity: {float(row['cosine_mean']):.4f} ± {float(row['cosine_std']):.4f}
(n={int(row['cosine_n'])})")

if "cosine_ci_lower" in row and not np.isnan(float(row.get("cosine_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['cosine_ci_lower']):.4f}, {float(row['cosine_ci_upper']):.4f}]")

lines.append(f" Jaccard similarity: {float(row['jaccard_mean']):.4f} ± {float(row['jaccard_std']):.4f}
(n={int(row['jaccard_n'])})")

if "jaccard_ci_lower" in row and not np.isnan(float(row.get("jaccard_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['jaccard_ci_lower']):.4f}, {float(row['jaccard_ci_upper']):.4f}]")

lines.append(f" Normalized distance: {float(row['distance_mean']):.4f} ±
{float(row['distance_std']):.4f} (n={int(row['distance_n'])})")

if "distance_ci_lower" in row and not np.isnan(float(row.get("distance_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['distance_ci_lower']):.4f}, {float(row['distance_ci_upper']):.4f}]")

else:

lines.append(" No cross-T topology data")

lines.append("")



# Recovery scaling metrics

lines.append("RECOVERY SCALING METRICS")
```

```
lines.append("-" * 40)

lines.append("Cohort-level estimates (censoring-aware) and conditional-on-recovery estimates")

lines.append("")

if recovery_summaries is not None and not recovery_summaries.empty:

for *, row in recovery*summaries.iterrows():

rho = float(row["rho"])

branch = str(row["branch"])

horizon_system = str(row["horizon_system"])

metric = str(row["metric"])

lines.append(f" rho={rho:.2f}, branch={branch}, {horizon_system}, {metric}:")



# Cohort-level estimates (includes censored) - restricted times

lines.append(f" Cohort-level estimates (all seeds, censoring-aware):")

lines.append(f" Mean restricted first crossing: {float(row['mean_restricted_first']):.4f} ±
{float(row['std_restricted_first']):.4f} (n={int(row['n_restricted_first'])})")

if "ci_lower_restricted_first" in row and not np.isnan(float(row.get("ci_lower_restricted_first",
float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_first']):.4f},
{float(row['ci_upper_restricted_first']):.4f}]")
```

```
lines.append(f" Mean restricted sustained: {float(row['mean_restricted_sustained']):.4f} ±
{float(row['std_restricted_sustained']):.4f} (n={int(row['n_restricted_sustained'])})")

if "ci_lower_restricted_sustained" in row and not
np.isnan(float(row.get("ci_lower_restricted_sustained", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_sustained']):.4f},
{float(row['ci_upper_restricted_sustained']):.4f}]")

lines.append(f" Mean restricted held: {float(row['mean_restricted_held']):.4f} ±
{float(row['std_restricted_held']):.4f} (n={int(row['n_restricted_held'])})")

if "ci_lower_restricted_held" in row and not np.isnan(float(row.get("ci_lower_restricted_held",
float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_held']):.4f},
{float(row['ci_upper_restricted_held']):.4f}]")

lines.append(f" AUC deficit: {float(row['mean_auc_deficit']):.4f} ± {float(row['std_auc_deficit']):.4f}
(n={int(row['n_auc_deficit'])})")

if "ci_lower_auc_deficit" in row and not np.isnan(float(row.get("ci_lower_auc_deficit", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_auc_deficit']):.4f},
{float(row['ci_upper_auc_deficit']):.4f}]")



# Cohort-level normalized times (restricted t/T_window)

lines.append(f" Cohort-level normalized times (restricted, all seeds):")

lines.append(f" Mean restricted first/t_window: {float(row.get('mean_restricted_first_over_Twindow',
float('nan'))):.4f} ± {float(row.get('std_restricted_first_over_Twindow', float('nan'))):.4f}
(n={int(row.get('n_restricted_first_over_Twindow', 0))})")

if "ci_lower_restricted_first_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_restricted_first_over_Twindow", float('nan')))):
```

```
lines.append(f" 95% CI: [{float(row['ci_lower_restricted_first_over_Twindow']):.4f},
{float(row['ci_upper_restricted_first_over_Twindow']):.4f}]")

lines.append(f" Mean restricted sustained/t_window:
{float(row.get('mean_restricted_sustained_over_Twindow', float('nan'))):.4f} ±
{float(row.get('std_restricted_sustained_over_Twindow', float('nan'))):.4f}
(n={int(row.get('n_restricted_sustained_over_Twindow', 0))})")

if "ci_lower_restricted_sustained_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_restricted_sustained_over_Twindow", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_sustained_over_Twindow']):.4f},
{float(row['ci_upper_restricted_sustained_over_Twindow']):.4f}]")

lines.append(f" Mean restricted held/t_window: {float(row.get('mean_restricted_held_over_Twindow',
float('nan'))):.4f} ± {float(row.get('std_restricted_held_over_Twindow', float('nan'))):.4f}
(n={int(row.get('n_restricted_held_over_Twindow', 0))})")

if "ci_lower_restricted_held_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_restricted_held_over_Twindow", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_held_over_Twindow']):.4f},
{float(row['ci_upper_restricted_held_over_Twindow']):.4f}]")



# Recovery probabilities with CI

lines.append(f" Recovery probabilities (with 95% bootstrap CI):")

lines.append(f" First crossing observed: {float(row['observed_rate_first']):.4f}")

if "ci_lower_observed_rate_first" in row and not
np.isnan(float(row.get("ci_lower_observed_rate_first", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_observed_rate_first']):.4f},
{float(row['ci_upper_observed_rate_first']):.4f}]")
```

```
lines.append(f" Sustained recovery observed: {float(row['observed_rate_sustained']):.4f}")

if "ci_lower_observed_rate_sustained" in row and not
np.isnan(float(row.get("ci_lower_observed_rate_sustained", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_observed_rate_sustained']):.4f},
{float(row['ci_upper_observed_rate_sustained']):.4f}]")

lines.append(f" Held recovery observed: {float(row['observed_rate_held']):.4f}")

if "ci_lower_observed_rate_held" in row and not
np.isnan(float(row.get("ci_lower_observed_rate_held", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_observed_rate_held']):.4f},
{float(row['ci_upper_observed_rate_held']):.4f}]")



# Conditional-on-recovery estimates (observed seeds only)

lines.append(f" Conditional-on-recovery estimates (observed seeds only):")

lines.append(f" First crossing: {float(row['mean_first_crossing']):.4f} ±
{float(row['std_first_crossing']):.4f} (n={int(row['n_first_crossing'])})")

if "ci_lower_first_crossing" in row and not np.isnan(float(row.get("ci_lower_first_crossing",
float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_first_crossing']):.4f},
{float(row['ci_upper_first_crossing']):.4f}]")

lines.append(f" Sustained: {float(row['mean_sustained']):.4f} ± {float(row['std_sustained']):.4f}
(n={int(row['n_sustained'])})")

if "ci_lower_sustained" in row and not np.isnan(float(row.get("ci_lower_sustained", float('nan')))):
```

```
lines.append(f" 95% CI (conditional): [{float(row['ci_lower_sustained']):.4f},
{float(row['ci_upper_sustained']):.4f}]")

lines.append(f" Held: {float(row['mean_held']):.4f} ± {float(row['std_held']):.4f}
(n={int(row['n_held'])})")

if "ci_lower_held" in row and not np.isnan(float(row.get("ci_lower_held", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_held']):.4f},
{float(row['ci_upper_held']):.4f}]")



# Conditional-on-recovery normalized times (observed seeds only)

lines.append(f" Conditional-on-recovery normalized times (observed seeds only):")

lines.append(f" First crossing/t_window: {float(row['mean_first_over_Twindow']):.4f} ±
{float(row['std_first_over_Twindow']):.4f} (n={int(row['n_first_over_Twindow'])})")

if "ci_lower_first_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_first_over_Twindow", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_first_over_Twindow']):.4f},
{float(row['ci_upper_first_over_Twindow']):.4f}]")

lines.append(f" Sustained/t_window: {float(row['mean_sustained_over_Twindow']):.4f} ±
{float(row['std_sustained_over_Twindow']):.4f} (n={int(row['n_sustained_over_Twindow'])})")

if "ci_lower_sustained_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_sustained_over_Twindow", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_sustained_over_Twindow']):.4f},
{float(row['ci_upper_sustained_over_Twindow']):.4f}]")

lines.append(f" Held/t_window: {float(row['mean_held_over_Twindow']):.4f} ±
{float(row['std_held_over_Twindow']):.4f} (n={int(row['n_held_over_Twindow'])})")
```

```
if "ci_lower_held_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_held_over_Twindow", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_held_over_Twindow']):.4f},
{float(row['ci_upper_held_over_Twindow']):.4f}]")

else:

lines.append(" No recovery scaling data")

lines.append("")



# Recovery interaction DID

lines.append("RECOVERY INTERACTION DID")

lines.append("-" * 40)

lines.append("I_{s,ρ,b}^recovery = [R_{s,ρ,b} - R_{s,ρ,continue}] - [R_{s,1,b} - R_{s,1,continue}]")

lines.append("")

if recovery_did_summary is not None and not recovery_did_summary.empty:

for *, row in recovery*did_summary.iterrows():

rho = float(row["rho"])

branch = str(row["branch"])

horizon_system = str(row["horizon_system"])

metric = str(row["metric"])
```

```
outcome = str(row["outcome"])

lines.append(f" rho={rho:.2f}, branch={branch}, {horizon_system}, {metric}, {outcome}:")

mean_val = float(row["mean"])

std_val = float(row["std"])

n_val = int(row["n"])

ci_lower = float(row.get("ci_lower", float('nan')))

ci_upper = float(row.get("ci_upper", float('nan')))


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" DID: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No recovery DID data")

lines.append("")



lines.append("=" * 80)

lines.append("END OF REPORT")

lines.append("=" * 80)
```

```
return "\n".join(lines)

def generate_analysis_manifest(

text

csv_path: Path,

npz_path: Path,

manifest_path: Path,

validation: dict[str, Any],

output_files: dict[str, Path],

source_path: Path,

bootstrap_seed: int,

n_bootstrap: int,

command_line: str,

) -> dict[str, Any]:

text

"""Generate analysis manifest with complete provenance."""

# Compute hashes
```

```
csv_hash = file_sha256(csv_path)

npz_hash = file_sha256(npz_path)

manifest_hash = file_sha256(manifest_path)

source_hash = file_sha256(source_path)



output_hashes = {}

for name, path in output_files.items():

output_hashes[name] = file_sha256(path)



manifest = {

"analysis_version": ANALYSIS_VERSION,

"timestamp": int(time.time()),

"study_name": STUDY_NAME,


"source": {

"path": str(source_path),

"sha256": source_hash,

},
```

"command_line": command_line,

"bootstrap": {

"seed": bootstrap_seed,

"n_iterations": n_bootstrap,

},

"input_hashes": {

"scalar_csv": csv_hash,

"branch_npz": npz_hash,

"simulation_manifest": manifest_hash,

"condition_status": validation.get("condition_status_hash", "NOT_FOUND"),

"baseline_certificate": validation.get("baseline_certificate_hash", "NOT_FOUND"),

},

"validation": validation,

"output_hashes": output_hashes,

"environment": {

"python": sys.version,

"numpy": np.__version__,

"pandas": pd.__version__,

},

"notes": {

"primary_outcomes": list(PRIMARY_OUTCOMES),

"secondary_outcomes": list(SECONDARY_NORMALIZED_OUTCOMES),

"expected_n_seeds": EXPECTED_N_SEEDS,

"expected_rho_values": [float(r) for r in EXPECTED_RHO_VALUES],

"expected_branches": list(EXPECTED_BRANCHES),

"expected_n_regime_contrasts": EXPECTED_N_REGIME_CONTRASTS,

"expected_n_did": EXPECTED_N_DID,

"expected_n_topology": EXPECTED_N_TOPOLOGY,

"horizon_definitions": {

"fixed_physical": {

"immediate": "0-17.5 time units",

"recovery": "17.5-87.5 time units",

"late": "87.5-320 time units"

},

"normalized_EMA": {

"immediate": "0-1×T_window",

"recovery": "1-5×T_window",

"late": "5×T_window to end"

}

},

"interpretation_constraints": {

"primary_estimand": "fixed_physical horizons",

"secondary_estimand": "normalized_EMA horizons",

"recovery_metrics": "t_rec/T_window for both horizon systems"

}

},

}

return manifest